Linux - Initiation 2025

Une formation présentée par Ascent et Andromed.

Appuyez sur espace pour la page suivante →

Ascent

MINISTÈRE
DES ARMÉES
Liberté
Égalité
Fraternité

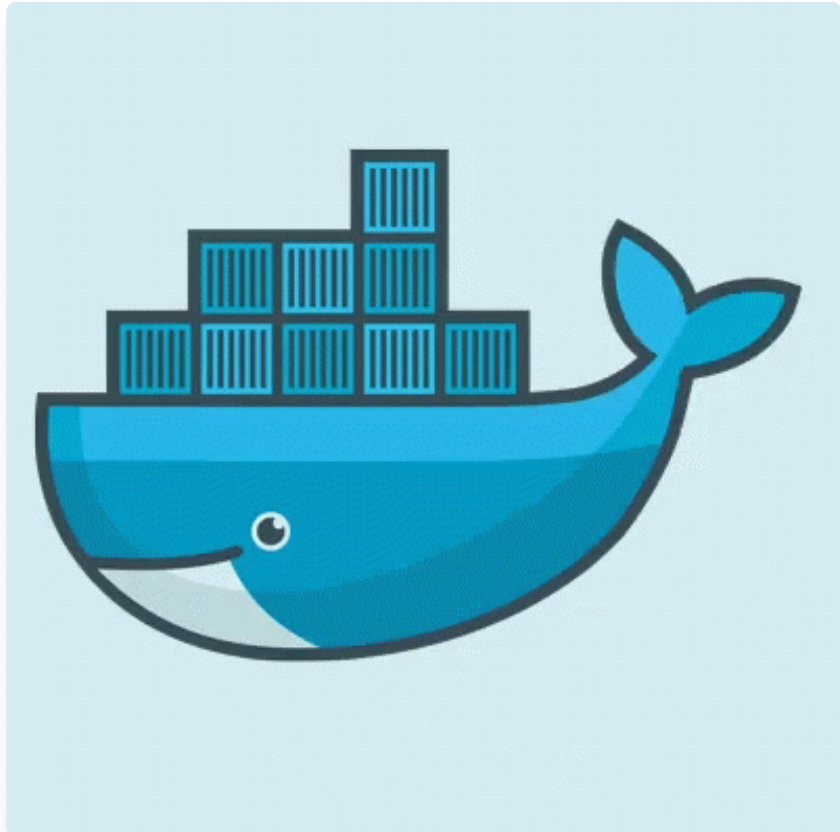
Jimmylan Surquin

Fondateur Andromed

- Lille, France 🇫🇷
- Création de contenu sur jimmylansrq
- Blog & Portfolio jimmylan.fr

DISCLAIMER 🐧

Dans cette formation nous allons voir les concepts fondamentaux d'Unix/Linux en 2025.



SOMMAIRE UNIX/LINUX – INITIATION 📜

🗃️ 1. Introduction et historique Unix/Linux

🔧 2. Commandes de base essentielles

📦 3. Gestion des paquets et maintenance

👥 4. Gestion des utilisateurs et permissions

💾 5. Stockage et système de fichiers

⚙️ 6. Gestion des processus et services

📡 7. Configuration réseau

🤖 8. Automatisation et scripts

🔐 9. Sécurité système

☁️ 10. Services serveurs

📊 11. Supervision & Analyse Système

🚨 12. Supervision Avancée (Zabbix)

⚡ 13. Optimisation & Performances (BONUS)

🌐 14. Optimisation Réseau Linux (BONUS)

✅ QCM de validation finale

Note : Chaque module contient :

- 📖 Cours théorique
- ✅ QCM de validation
- 🎯 Exercices pratiques

PROGRAMME INITIATION UNIX/LINUX 📅

Module 1 - Introduction

- Historique Unix, GNU/Linux
- Philosophie Unix
- Architecture système
- Noyau, shell

Module 2 - Commandes de base 🆕

- Navigation (cd, ls, pwd)
- Manipulation fichiers (cp, mv, rm)
- Recherche (find, grep)
- Visualisation (cat, less, head, tail)
- Redirections et pipes

Module 3 - Paquets et maintenance

- apt/dnf : installation
- Mise à jour système
- Gestion dépendances

Module 4 - Utilisateurs et permissions

- Comptes et groupes
- Permissions (chmod, chown)
- sudo et sudoers
- Quotas disque

Module 5 - Stockage

- Disques et partitions
- Montage (mount, fstab)
- Liens symboliques
- ACL

PROGRAMME INITIATION UNIX/LINUX 17

Module 6 - Processus et services

- PID, signaux
- nice, kill
- systemd
- journalctl

Module 7 - Réseau

- Configuration IP
- SSH sécurisé
- Pare-feu (ufw)
- Diagnostic (ping, traceroute)

Module 8 - Automatisation

- Scripts Bash
- Cron et at
- Logs système

Module 9 - Sécurité

- Durcissement SSH
- fail2ban
- Audit (auditd)
- Chiffrement (LUKS)

Module 10 - Services serveurs

- Nginx
- MySQL
- SSL/TLS
- Monitoring

PROGRAMME INITIATION UNIX/LINUX 📆

Module 11 - Supervision & Analyse

- Monitoring (top, htop, iotop)
- Analyse des logs
- Métriques système
- Tableaux de bord

Module 12 - Supervision Avancée

- Zabbix serveur et agent
- Alertes et notifications
- Templates personnalisés
- Dashboard avancés

Module 13 - Optimisation (BONUS) ⚡

- Tuning kernel (sysctl)
- Optimisation mémoire
- Optimisation serveurs
- Performance des services

Module 14 - Optimisation Réseau (BONUS) 🌐

- Pile TCP/IP et sysctl
- Carte réseau (NIC)
- Files d'attente & bufferbloat
- Pare-feu & conteneurs

QCM Final ✅

- Validation des connaissances
- Tous les modules
- Certification de la formation

Module 1

Introduction et rappels Unix/Linux

Découverte de l'écosystème Unix/Linux

Revenir au sommaire

Qu'est-ce qu'Unix ? 🖥️

Unix est un système d'exploitation né en 1969 aux laboratoires Bell (AT&T).

Créateurs principaux :

- **Ken Thompson** et **Dennis Ritchie**
- Développé initialement en assembleur, puis réécrit en C (1973)

Caractéristiques révolutionnaires :

- Multi-utilisateurs et multi-tâches
- Portabilité grâce au langage C
- Architecture modulaire et simple

Analogie : Unix, c'est comme...

Imaginez Unix comme **les fondations d'une maison** :

- **Solides** : éprouvées depuis plus de 50 ans
- **Universelles** : utilisées dans presque toutes les constructions modernes
- **Invisibles** : on ne les voit pas, mais tout repose dessus

Aujourd'hui, Unix et ses descendants sont partout :

- Serveurs web (Google, Facebook, Amazon...)
- Smartphones (Android, iOS)
- Supercalculateurs
- Objets connectés

De Unix à Linux

Problème des années 80-90 :

- Unix était propriétaire et coûteux
- Les universités voulaient un système libre pour l'enseignement

La révolution GNU (1983) :

- Richard Stallman lance le projet GNU (GNU's Not Unix)
- Objectif : créer un système d'exploitation libre compatible Unix
- Développement des outils : shell bash, compilateur gcc, éditeurs...

La naissance de Linux (1991) 🎂

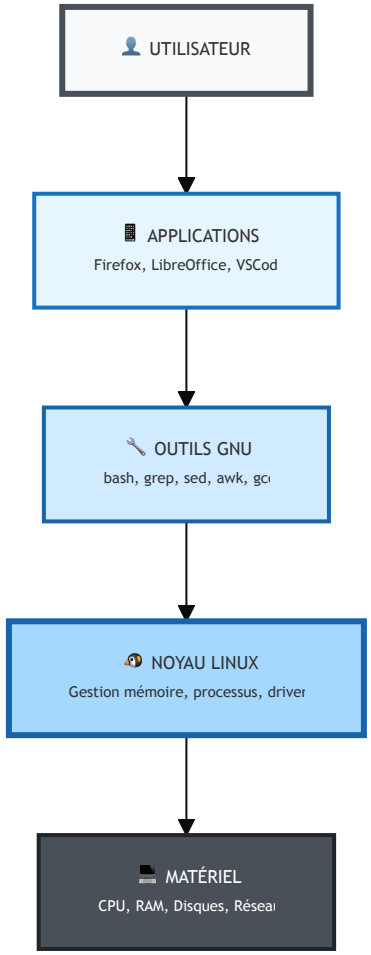
Linus Torvalds, étudiant finlandais de 21 ans :

> "I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu)"

Le noyau Linux :

- Développé initialement pour son ordinateur personnel
- Combiné avec les outils GNU → GNU/Linux
- Publié sous licence GPL (libre et open source)

Architecture en couches



Les deux composants essentiels

🔧 GNU (Projet GNU)

- Outils en ligne de commande
- Compilateurs et bibliothèques
- Shell (bash)
- Utilitaires système

Linux (Noyau)

- Interface avec le matériel
- Gestion de la mémoire
- Ordonnancement des processus
- Pilotes de périphériques

= GNU/Linux 🎯 Un système d'exploitation complet et fonctionnel

Revenir au sommaire

Les distributions Linux 📦

Une **distribution** = Linux + outils GNU + logiciels + gestionnaire de paquets

Principales familles :

Debian/Ubuntu (apt)

- Stable, grande communauté
- Parfait pour débuter et serveurs
- Ubuntu, Debian, Linux Mint, Pop!_OS

Red Hat/Fedora (dnf/yum)

- Entreprise, certifications
- RHEL, CentOS, AlmaLinux, Rocky Linux, Fedora

Les distributions Linux (suite) 📦

Arch (pacman)

- Rolling release, minimaliste
- Pour utilisateurs avancés
- Arch Linux, Manjaro, EndeavourOS

SUSE (zypper)

- Entreprise européenne
- openSUSE, SLES

Autres

- Gentoo (compilation source)
- Alpine (ultra-léger, conteneurs)
- NixOS (reproductibilité)

Pourquoi tant de distributions ? 🤔

Analogie : les restaurants 🍽️

Tous servent de la nourriture (Linux), mais :

- **Fast-food** (Ubuntu) : rapide, facile, populaire
- **Restaurant gastronomique** (Arch) : personnalisable, pour connaisseurs
- **Cantine d'entreprise** (RHEL) : fiable, support professionnel
- **Food truck** (Alpine) : léger, mobile, efficace

Principe : le cœur (noyau Linux) est le même, seul l'emballage change !

La philosophie Unix 🧘

"Tout est fichier"

- Les périphériques sont des fichiers (`/dev/sda` , `/dev/tty`)
- La configuration = fichiers texte
- Les processus = fichiers dans `/proc`

Principe KISS (Keep It Simple, Stupid)

- Programmes simples qui font une seule chose bien
- Combiner les outils avec des pipes `|`

Exemples de philosophie Unix 🔧

Un programme = une fonction

```
# Mauvais : un programme qui fait tout
super_outil --search --sort --count fichier.txt

# Unix way : combiner des outils simples
cat fichier.txt | grep "motif" | sort | wc -l
```

Avantage : flexibilité et réutilisabilité

Architecture d'un système Linux 🏛️

4 couches principales :

1. **Matériel (Hardware)**
   - Processeur, RAM, disques, cartes réseau
2. **Noyau (Kernel)**
   - Gestion du matériel
   - Gestion de la mémoire et des processus
   - Pilotes de périphériques
3. **Shell**
   - Interface entre l'utilisateur et le noyau
   - Interprète les commandes
4. **Applications utilisateur**
   - Programmes que vous utilisez

Le noyau Linux en détail 🎯

Rôles principaux :

Gestion des processus

- Création, ordonnancement, terminaison
- Communication inter-processus (IPC)

Gestion de la mémoire

- Allocation/libération mémoire
- Mémoire virtuelle (swap)
- Cache disque

Le noyau Linux en détail (suite) 🎯

Gestion du système de fichiers

- VFS (Virtual File System)
- Support ext4, xfs, btrfs, NFS, etc.

Gestion des périphériques

- Pilotes de périphériques (drivers)
- Accès aux périphériques via `/dev`

Gestion réseau

- Pile TCP/IP
- Interfaces réseau
- Pare-feu (netfilter/iptables)

Le Shell : votre interface avec le système 🐚

Qu'est-ce qu'un shell ?

- Programme qui interprète vos commandes
- Interface entre vous et le noyau

Shells populaires :

- **bash** (Bourne Again Shell) : le plus courant
- **zsh** (Z Shell) : moderne, plugins
- **fish** (Friendly Interactive Shell) : convivial
- **sh** (Bourne Shell) : le shell historique original

Le Shell : analogie 💬

Le shell, c'est comme un **interprète dans une réunion internationale** :

- Vous parlez en "langage humain" (commandes)
- L'interprète traduit au noyau en "langage machine"
- Le noyau exécute et renvoie la réponse
- L'interprète vous traduit le résultat

Exemple :

```
$ ls -la
# Vous : "montre-moi les fichiers"
# Shell : traduit en appels système
# Noyau : liste les fichiers
# Shell : affiche le résultat formaté
```

Le système de fichiers Unix/Linux 📁

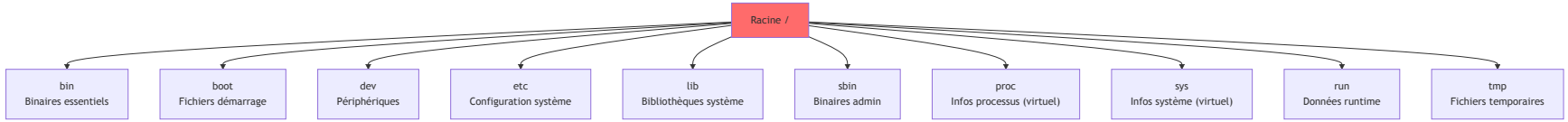
Hiérarchie unifiée :

- Tout part de la racine /
- Pas de lettre de lecteur (C:, D:) comme Windows
- Les disques sont "montés" dans l'arborescence

Principe : une arborescence, un point d'entrée

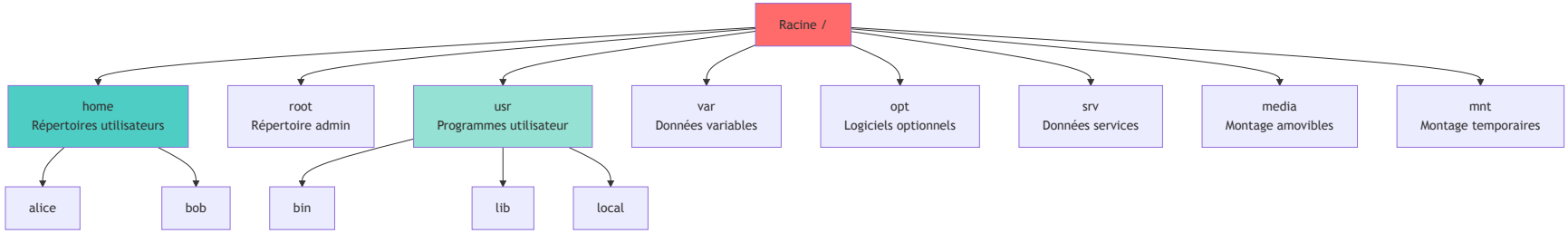
Structure des répertoires principaux (1/2)

Répertoires système essentiels



Structure des répertoires principaux (2/2)

Répertoires utilisateur et données



Revenir au sommaire

Rappels : commandes essentielles (1/4) 🔧

Navigation :

```
pwd                 # Afficher le répertoire courant
cd /chemin          # Changer de répertoire
cd ..               # Remonter d'un niveau
cd ~                # Aller dans votre home
cd -                # Retourner au répertoire précédent
```

Rappels : commandes essentielles (2/4) 🔧

Lister et afficher :

```
ls                 # Lister les fichiers
ls -l              # Liste détaillée
ls -la             # Inclure les fichiers cachés
ls -lh             # Tailles lisibles (K, M, G)
cat fichier.txt    # Afficher le contenu
less fichier.txt   # Parcourir un fichier (q pour quitter)
head fichier.txt   # 10 premières lignes
tail fichier.txt   # 10 dernières lignes
tail -f fichier.log # Suivre un fichier en temps réel
```

Rappels : commandes essentielles (3/4) 🔧

Manipuler fichiers et répertoires :

```
cp source dest      # Copier
cp -r dir1 dir2     # Copier un répertoire
mv source dest      # Déplacer/renommer
rm fichier          # Supprimer un fichier
rm -r dossier       # Supprimer un répertoire
rm -rf dossier      # Force (ATTENTION : dangereux!)
mkdir nouveau_dir   # Créer un répertoire
mkdir -p a/b/c      # Créer une arborescence
touch fichier.txt   # Créer un fichier vide
```

Rappels : commandes essentielles (4/4) 🔧

Recherche et filtres :

```
find /chemin -name "*.txt"        # Trouver des fichiers
find /home -type f -size +100M    # Fichiers > 100 Mo
grep "motif" fichier.txt          # Rechercher dans un fichier
grep -r "motif" /repertoire       # Recherche récursive
grep -i "motif" fichier           # Insensible à la casse
which commande                    # Localiser un exécutable
whereis commande                  # Trouver binaire, source, man
```

Les pipes : combiner les commandes 🔗

Le pipe | **:** connecte la sortie d'une commande à l'entrée d'une autre

```
# Compter les lignes d'un fichier
cat fichier.txt | wc -l

# Trouver un processus
ps aux | grep "nginx"

# Les 10 plus gros fichiers du répertoire
ls -lh | sort -k5 -h | tail -10

# Extraire et trier des données
cat /etc/passwd | cut -d: -f1 | sort
```

Analogie : comme une chaîne de production en usine, chaque outil fait sa partie du travail.

Redirections : entrées/sorties

Rediriger la sortie :

```
ls -la > liste.txt           # Écraser le fichier
ls -la >> liste.txt          # Ajouter à la fin
commande 2> erreurs.txt      # Rediriger les erreurs
commande &> tout.txt         # Rediriger tout (stdout + stderr)
commande > /dev/null 2>&1    # Ignorer toutes les sorties
```

Redirections : entrées/sorties (suite) 📥📤

Rediriger l'entrée :

```
sort < liste.txt            # Lire depuis un fichier
wc -l < fichier.txt         # Compter les lignes
```

Permissions : rappels de base 🔒

Chaque fichier a :

- Un **propriétaire** (owner)
- Un **groupe**
- Des **permissions** pour : owner, group, others

Types de permissions :

- `r` (read) : lecture
- `w` (write) : écriture
- `x` (execute) : exécution

```
-rw-r--r--  1 alice  users  1024 Jan 10 10:30 fichier.txt
drwxr-xr-x  2 alice  users  4096 Jan 10 10:31 dossier/
```

Changer les permissions 🔐

```
chmod 755 script.sh          # rwxr-xr-x
chmod u+x script.sh          # Ajouter exécution pour user
chmod g-w fichier.txt        # Retirer écriture pour group
chmod o-r fichier.txt        # Retirer lecture pour others

chown alice:users fichier    # Changer propriétaire et groupe
chown alice fichier          # Changer seulement propriétaire
chgrp users fichier          # Changer seulement groupe
```

Les processus : bases 🏃

Qu'est-ce qu'un processus ?

- Un programme en cours d'exécution
- Chaque processus a un PID (Process ID) unique

```
ps                          # Processus de votre session
ps aux                      # Tous les processus
top                         # Vue en temps réel
htop                        # Version améliorée (à installer)
pgrep nginx                 # Trouver PID d'un processus
```

Gérer les processus 🎮

```
command &                    # Lancer en arrière-plan
jobs                         # Lister les tâches en cours
fg                           # Ramener en avant-plan
bg                           # Mettre en arrière-plan
Ctrl+Z                       # Suspendre un processus
Ctrl+C                       # Interrompre un processus

kill 1234                    # Terminer processus PID 1234
kill -9 1234                 # Tuer de force (SIGKILL)
killall nginx                # Tuer tous les processus nginx
pkill -f "nom_pattern"       # Tuer par nom/motif
```

L'éditeur de texte : nano/vim

nano : simple et intuitif

```
nano fichier.txt
# Ctrl+O : sauvegarder
# Ctrl+X : quitter
# Ctrl+K : couper une ligne
# Ctrl+U : coller
```

vim : puissant mais courbe d'apprentissage

```
vim fichier.txt
# i : passer en mode insertion
# Esc : revenir en mode commande
# :w : sauvegarder
# :q : quitter
# :wq : sauvegarder et quitter
# :q! : quitter sans sauvegarder
```

Variables d'environnement 🌍

Variables système importantes :

```
echo $HOME                  # Répertoire personnel
echo $USER                  # Nom d'utilisateur
echo $PATH                  # Chemins des exécutables
echo $SHELL                 # Shell par défaut
echo $PWD                   # Répertoire actuel

export MA_VAR="valeur"      # Définir une variable
env                         # Lister toutes les variables
printenv                    # Idem
```

Le PATH : où sont les commandes ? 🔍

Le PATH : liste des répertoires où le shell cherche les commandes

```
echo $PATH
# Résultat typique :
# /usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin

# Ajouter un répertoire au PATH
export PATH=$PATH:/mon/nouveau/chemin

# Rendre permanent (dans ~/.bashrc ou ~/.zshrc)
echo 'export PATH=$PATH:/mon/nouveau/chemin' >> ~/.bashrc
source ~/.bashrc
```

Fichiers de configuration du shell 🔧

Pourquoi 2 fichiers différents ?

Fichier	Quand il est lu ?	On met quoi dedans ?
`~/.bash_profile` `~/.zprofile`	À la **connexion** (SSH, login)	Variables d'environnement (PATH)
`~/.bashrc` `~/.zshrc`	À **chaque nouveau terminal**	Alias, fonctions, prompt

💡 **En gros :**

- `.bash_profile` = 1 fois quand tu te connectes
- `.bashrc` = à chaque fois que tu ouvres un terminal

Configuration du shell : Exemples 🛠️

Dans `~/.bash_profile` (ou `~/.zprofile` pour Zsh) :

```
# Variables d'environnement
export PATH=$PATH:~/bin
export EDITOR=vim
```

Dans `~/.bashrc` (ou `~/.zshrc` pour Zsh) :

```
# Alias
alias ll='ls -lah'
alias update='sudo apt update && sudo apt upgrade'

# Fonction personnalisée
mkcd() {
    mkdir -p "$1" && cd "$1"
}
```

Appliquer les changements :

```
source ~/.bashrc     # Pour Bash
source ~/.zshrc      # Pour Zsh
```

Aide et documentation 📚

Obtenir de l'aide :

```
man ls                    # Manuel de la commande ls
man -k "keyword"          # Rechercher dans les manuels
info ls                   # Info pages (alternative à man)
ls --help                 # Aide rapide
whatis ls                 # Description courte
apropos "keyword"         # Rechercher par mot-clé
```

Sections du manuel :

- Section 1 : Commandes utilisateur
- Section 5 : Formats de fichiers
- Section 8 : Commandes d'administration

Raccourcis clavier Bash ⌨️

Navigation :

```
Ctrl+A          # Début de ligne
Ctrl+E          # Fin de ligne
Ctrl+U          # Effacer jusqu'au début
Ctrl+K          # Effacer jusqu'à la fin
Ctrl+W          # Effacer le mot précédent
Ctrl+R          # Rechercher dans l'historique
Ctrl+L          # Effacer l'écran (comme clear)
```

Historique des commandes

```
history                       # Afficher l'historique
history | grep "ssh"          # Chercher dans l'historique
!123                          # Exécuter la commande #123
!!                            # Répéter la dernière commande
!$                            # Dernier argument de la commande précédente
sudo !!                       # Relancer la dernière commande avec sudo

# Recherche interactive
Ctrl+R                        # Puis taper des lettres
```

Alias : raccourcis personnalisés

```
# Définir un alias temporaire
alias ll='ls -lah'
alias ..='cd ..'
alias ...='cd ../..'
alias update='sudo apt update && sudo apt upgrade'

# Rendre permanent (dans ~/.bashrc)
echo "alias ll='ls -lah'" >> ~/.bashrc
source ~/.bashrc

# Lister les alias
alias

# Supprimer un alias
unalias ll
```

Quelques conseils de débutant 💡

1. Lisez les messages d'erreur

- Ils contiennent souvent la solution

2. Utilisez la complétion automatique (Tab)

- Gagne du temps et évite les erreurs

3. Attention avec `rm -rf`

- Pas de corbeille en ligne de commande !
- Vérifiez toujours avant de supprimer

4. Sauvegardez vos données

- Avant toute manipulation importante

Quelques conseils de débutant (suite) 💡

5. Testez dans un environnement sûr

- Machine virtuelle ou conteneur Docker

6. Prenez des notes

- Créez votre propre "cheat sheet"

7. Pratiquez régulièrement

- La ligne de commande devient naturelle avec la pratique

8. Consultez la documentation

- `man` est votre meilleur ami

Pour aller plus loin 🚀

Ressources utiles :

- **Linux Journey** : tutoriels interactifs
- **OverTheWire** : jeux pour apprendre Linux
- **ExplainShell** : explique les commandes shell
- **TLDR pages** : alternatives simplifiées à man

Communautés :

- Forums Ubuntu/Debian/Arch
- Stack Overflow
- Reddit : r/linux, r/linuxquestions
- IRC : #linux sur libera.chat

Récapitulatif du Module 1 ✅

Ce que vous avez appris :

- ✅ Historique d'Unix et naissance de Linux
- ✅ Philosophie Unix ("tout est fichier", modularité)
- ✅ Architecture d'un système Linux (noyau, shell, applications)
- ✅ Structure du système de fichiers
- ✅ Commandes essentielles de base
- ✅ Pipes et redirections
- ✅ Permissions de base
- ✅ Processus et leur gestion
- ✅ Variables d'environnement
- ✅ Aide et documentation

Environnement de travail 🐳

Pour la suite, on a besoin d'un terminal Linux !

Option simple : Docker (pas besoin de VM lourde)

1. **Installer Docker Desktop :**
   - Windows : https://docs.docker.com/desktop/install/windows-install/
   - Mac : https://docs.docker.com/desktop/install/mac-install/
   - Linux : https://docs.docker.com/desktop/install/linux-install/
2. **Créer un fichier** `docker-compose.yml` **:**

```yaml
services:
  ubuntu:
    image: ubuntu:24.04
    container_name: formation-unix
    stdin_open: true
    tty: true
    command: /bin/bash
```

Lancer l'environnement 🚀

3 commandes, c'est tout :

```
# Démarrer le conteneur Ubuntu
docker-compose up -d

# Se connecter au terminal Linux
docker exec -it formation-unix bash

# Quitter : Ctrl+D ou
exit
```

Arrêter/supprimer :

```
docker-compose down       # Arrêter
docker-compose down -v    # Arrêter et tout supprimer
```

💡 **C'est tout !** On va passer 90% du temps dans le terminal, pas besoin de se prendre la tête avec une VM graphique.

Prochaine étape 🎯

Module 2 : Commandes de base essentielles

- Navigation dans le système de fichiers
- Manipulation de fichiers et répertoires
- Recherche avec find et grep
- Visualisation de fichiers
- Redirections et pipes avancés

Questions ? 🤔

N’hésitez pas à poser vos questions maintenant !

Le prochain module approfondira la gestion des utilisateurs et des permissions.

QCM Module 1 : Introduction Unix/Linux ✅

10 questions théoriques sur Unix/Linux

Revenir au sommaire

Question 1

Qui sont les créateurs d'Unix ?

A) Linus Torvalds et Richard Stallman

B) Ken Thompson et Dennis Ritchie

C) Bill Gates et Steve Jobs

D) Larry Page et Sergey Brin

Question 2

Que signifie GNU dans GNU/Linux ?

A) General New Unix

B) GNU’s Not Unix

C) Global Network Unix

D) Great Native Unix

Question 3

Quel est le noyau de Linux ?

A) Un programme graphique

B) Le cœur du système qui gère le matériel

C) Un éditeur de texte

D) Un navigateur web

Revenir au sommaire

Question 4

Que signifie le principe Unix "tout est fichier" ?

A) Tous les programmes sont stockés dans des fichiers

B) Les périphériques et processus sont accessibles comme des fichiers

C) On ne peut travailler qu'avec des fichiers texte

D) Il faut tout sauvegarder dans des fichiers

Question 5

Quelle distribution Linux est connue pour sa stabilité en serveur ?

A) Windows Server

B) macOS

C) Debian

D) MS-DOS

Question 6

Qu’est-ce qu’un shell ?

A) Un type de disque dur

B) L’interface entre l’utilisateur et le noyau

C) Un navigateur web

D) Un éditeur de code

Revenir au sommaire

Question 7

Quelle est la philosophie Unix principale ?

A) Faire des programmes complexes qui font tout

B) Faire des programmes simples qui font une chose bien

C) Toujours utiliser une interface graphique

D) Éviter la ligne de commande

Question 8

Quelle entreprise a créé Unix à l'origine ?

A) Microsoft

B) Apple

C) AT&T Bell Labs

D) Google

Revenir au sommaire

Question 9

Qu’est-ce qu’une distribution Linux ?

A) Un système d’exploitation complet basé sur le noyau Linux

B) Seulement le noyau Linux

C) Un type de disque dur

D) Un langage de programmation

Question 10

Quel est l'avantage principal de Linux ?

A) Il coûte très cher

B) Il est fermé et propriétaire

C) Il est open source et gratuit

D) Il ne fonctionne que sur Apple

Revenir au sommaire

Réponses du Module 1

Réponse 1 : B) Ken Thompson et Dennis Ritchie

- Unix a été créé en 1969 aux laboratoires Bell (AT&T)

Réponse 2 : B) GNU’s Not Unix

- C’est un acronyme récursif créé par Richard Stallman

Réponse 3 : B) Le cœur du système qui gère le matériel

- Le noyau (kernel) est l’interface entre le matériel et les logiciels

Réponse 4 : B) Les périphériques et processus sont accessibles comme des fichiers

- Unix utilise l’abstraction de fichier pour accéder aux périphériques (`/dev`), aux processus (`/proc`), etc.

Réponse 5 : C) Debian

- Debian est réputée pour sa stabilité, notamment en environnement serveur

Réponses du Module 1 (suite)

Réponse 6 : B) L'interface entre l'utilisateur et le noyau

- Le shell (bash, zsh, etc.) interprète les commandes

Réponse 7 : B) Faire des programmes simples qui font une chose bien

- Philosophie Unix : programmes modulaires et combinables avec pipes

Réponse 8 : C) AT&T Bell Labs

- Unix a été développé aux laboratoires Bell en 1969

Réponse 9 : A) Un système d'exploitation complet basé sur le noyau Linux

- Une distribution inclut le noyau + GNU + outils + gestionnaire de paquets

Réponse 10 : C) Il est open source et gratuit

- L'open source permet la transparence, la sécurité et la liberté

Revenir au sommaire

Score du Module 1 📊

Comptez vos bonnes réponses :

- **9-10** : Excellent ! ⭐⭐⭐
- **7-8** : Bien ! ⭐⭐
- **5-6** : À revoir ⭐
- **< 5** : Relire le module 1

Revenir au sommaire

ommandes de base 🔧

Les commandes essentielles d'Unix/Linux

Maîtrisons les commandes de base →

Ascent

MINISTÈRE
DES ARMÉES
Liberté
Égalité
Fraternité

Commandes de navigation 🧭

Se déplacer dans le système

pwd (Print Working Directory)

```
pwd                     # Afficher le répertoire courant
pwd -P                  # Chemin physique (sans liens)
```

cd (Change Directory)

```
cd                      # Aller au répertoire home
cd /chemin/absolu       # Chemin absolu
cd chemin/relatif       # Chemin relatif
cd ..                   # Remonter d'un niveau
cd -                    # Retourner au répertoire précédent
cd ~                    # Aller au répertoire home
```

ls (List)

```
ls                      # Liste simple
ls -la                  # Liste détaillée avec fichiers cachés
ls -lh                  # Tailles humaines
ls -R                   # Récursif
ls -t                   # Trié par date
ls -S                   # Trié par taille
```

Commandes de manipulation 📁

Créer, copier, déplacer, supprimer

mkdir (Make Directory)

```
mkdir nom_repertoire       # Créer un répertoire
mkdir -p chemin/complet    # Créer les parents si nécessaire
mkdir dir1 dir2 dir3       # Créer plusieurs répertoires
```

cp (Copy)

```
cp source destination      # Copier un fichier
cp -r source destination   # Copier un répertoire
cp -v source destination   # Verbose (afficher les opérations)
cp -i source destination   # Interactive (confirmation)
cp -p source destination   # Préserver les attributs
```

mv (Move)

```
mv ancien nouveau          # Renommer
mv fichier destination/    # Déplacer
mv -i fichier destination/ # Interactive
mv -v fichier destination/ # Verbose
```

Commandes de suppression

Supprimer fichiers et répertoires

rm (Remove)

```
rm fichier                # Supprimer un fichier
rm -f fichier             # Forcer (sans confirmation)
rm -i fichier             # Interactive
rm -v fichier             # Verbose
rm fichier1 fichier2      # Plusieurs fichiers
```

rmdir (Remove Directory)

```
rmdir repertoire          # Supprimer un répertoire vide
rmdir -p chemin/complet   # Supprimer les parents vides
```

rm avec options avancées

```
rm -r repertoire          # Récursif (supprimer répertoire et contenu)
rm -rf repertoire         # Forcer récursif (dangereux !)
rm --preserve-root        # Protéger la racine
```

Commandes de visualisation

Afficher le contenu des fichiers

cat (Concatenate)

```
cat fichier                # Afficher le contenu complet
cat fichier1 fichier2      # Concaténer plusieurs fichiers
cat > nouveau_fichier      # Créer un fichier (Ctrl+D pour terminer)
cat >> fichier             # Ajouter à la fin
```

less et more

```
less fichier               # Navigation interactive
more fichier               # Navigation page par page
# Navigation dans less : espace (page suivante), b (page précédente), q (quitter)
```

head et tail

```
head fichier               # 10 premières lignes
head -n 20 fichier         # 20 premières lignes
tail fichier               # 10 dernières lignes
tail -n 20 fichier         # 20 dernières lignes
tail -f fichier            # Suivre en temps réel
```

Éditeurs de texte

Modifier des fichiers en ligne de commande

nano (débutant-friendly)

```
nano fichier.txt              # Ouvrir/créer un fichier
# Ctrl+O : Sauvegarder
# Ctrl+X : Quitter
# Ctrl+K : Couper une ligne
# Ctrl+U : Coller
```

vim/vi (avancé)

```
vim fichier.txt               # Ouvrir avec vim
# i : Mode insertion (pour écrire)
# Esc : Mode commande
# :w : Sauvegarder
# :q : Quitter
# :wq : Sauvegarder et quitter
# :q! : Quitter sans sauvegarder
```

Édition rapide avec echo

```
echo "contenu" > fichier.txt  # Écraser
echo "ajout" >> fichier.txt   # Ajouter
```

Commandes de recherche 🔍

Trouver des informations

grep (Global Regular Expression Print)

```
grep "motif" fichier       # Rechercher un motif
grep -i "motif" fichier    # Insensible à la casse
grep -v "motif" fichier    # Inverser (lignes sans motif)
grep -r "motif" repertoire # Récursif
grep -n "motif" fichier    # Afficher les numéros de ligne
grep -l "motif" *          # Afficher seulement les noms de fichiers
```

find

```
find . -name "fichier.txt" # Rechercher par nom
find . -type f             # Seulement les fichiers
find . -type d             # Seulement les répertoires
find . -size +100M         # Plus de 100MB
find . -mtime -7           # Modifiés dans les 7 jours
find . -user utilisateur   # Appartenant à un utilisateur
```

Commandes de traitement de texte

Manipuler le texte

echo

```
echo "texte"              # Afficher du texte
echo -n "texte"           # Sans retour à la ligne
echo $VARIABLE            # Afficher une variable
echo -e "texte\nligne"    # Interpréter les caractères spéciaux
```

sed (Stream Editor)

```
sed 's/ancien/nouveau/g' fichier  # Remplacer
sed '1,5d' fichier                # Supprimer les lignes 1-5
sed 's/^/prefixe/' fichier        # Ajouter un préfixe
sed 's/$/suffixe/' fichier        # Ajouter un suffixe
```

awk

```
awk '{print $1}' fichier          # Afficher la première colonne
awk '{print $1, $3}' fichier      # Afficher les colonnes 1 et 3
awk '/motif/' fichier             # Lignes contenant un motif
awk '{sum += $1} END {print sum}' fichier  # Somme de la première colonne
```

Commandes de tri et filtrage

Organiser les données

sort

```
sort fichier              # Trier alphabétiquement
sort -n fichier           # Trier numériquement
sort -r fichier           # Trier en ordre inverse
sort -k2 fichier          # Trier par la 2ème colonne
sort -u fichier           # Supprimer les doublons
```

uniq

```
uniq fichier              # Supprimer les lignes consécutives identiques
uniq -c fichier           # Compter les occurrences
uniq -d fichier           # Afficher seulement les doublons
sort fichier | uniq       # Combinaison classique
```

wc (Word Count)

```
wc fichier                # Lignes, mots, caractères
wc -l fichier             # Seulement les lignes
wc -w fichier             # Seulement les mots
wc -c fichier             # Seulement les caractères
```

Commandes de compression 📦

Compresser et décompresser

gzip/gunzip

```
gzip fichier              # Compresser (supprime l'original)
gzip -k fichier           # Garder l'original
gunzip fichier.gz         # Décompresser
gzip -d fichier.gz        # Alternative pour décompresser
```

tar (Tape Archive)

```
tar -czf archive.tar.gz repertoire/  # Créer une archive compressée
tar -xzf archive.tar.gz              # Extraire
tar -cjf archive.tar.bz2 repertoire/ # Compresser avec bzip2
tar -xjf archive.tar.bz2             # Extraire bzip2
tar -tf archive.tar.gz               # Lister le contenu
```

zip/unzip

```
zip archive.zip fichier    # Créer une archive zip
zip -r archive.zip repertoire/  # Récursif
unzip archive.zip          # Extraire
unzip -l archive.zip       # Lister le contenu
```

Commandes de réseau

Configuration et test réseau

ping

```
ping google.com            # Test de connectivité
ping -c 4 google.com       # 4 pings seulement
ping -i 2 google.com       # Intervalle de 2 secondes
ping -s 1000 google.com    # Taille de paquet de 1000 bytes
```

netstat

```
netstat -tuln              # Ports en écoute
netstat -i                 # Interfaces réseau
netstat -r                 # Table de routage
netstat -s                 # Statistiques
```

ss (Socket Statistics)

```
ss -tuln                   # Ports en écoute (moderne)
ss -i                      # Informations détaillées
ss -s                      # Statistiques
```

Commandes système ⚙️

Informations sur le système

uname

```
uname                    # Nom du système
uname -a                 # Toutes les informations
uname -r                 # Version du noyau
uname -m                 # Architecture
```

who et w

```
who                      # Utilisateurs connectés
who am i                 # Mon utilisateur
w                        # Utilisateurs et leurs processus
```

date et time

```
date                     # Date et heure actuelles
date +%Y-%m-%d           # Format personnalisé
time commande            # Temps d'exécution d'une commande
```

Commandes de processus ⚡

Gérer les processus

ps (Process Status)

```
ps                      # Mes processus
ps aux                  # Tous les processus
ps -ef                  # Format étendu
ps -p PID               # Processus spécifique
ps -u utilisateur       # Processus d'un utilisateur
```

top et htop

```
top                     # Processus en temps réel
htop                    # Interface améliorée
# Navigation : q (quitter), k (tuer), r (renice), h (aide)
```

kill et pkill

```
kill PID                # Terminer un processus
kill -9 PID             # Forcer la terminaison
killall nom_processus   # Tuer par nom
pkill nom_processus     # Alternative moderne
```

Commandes de monitoring

Surveiller le système

df (Disk Free)

```
df                      # Espace disque
df -h                   # Format humain
df -i                   # Utilisation des inodes
df -T                   # Type de système de fichiers
```

du (Disk Usage)

```
du                      # Utilisation du répertoire courant
du -h                   # Format humain
du -sh *                # Taille de tous les éléments
du -d 1                 # Profondeur 1
```

free

```
free                    # Utilisation mémoire
free -h                 # Format humain
free -s 5               # Actualisation toutes les 5 secondes
```

Commandes de redirection 🔄

Contrôler les entrées/sorties

Redirection de sortie

```
commande > fichier          # Écraser
commande >> fichier         # Ajouter
commande 2> fichier         # Rediriger les erreurs
commande &> fichier         # Tout (sortie + erreurs)
```

Redirection d'entrée

```
commande < fichier          # Lire depuis un fichier
cat < fichier               # Afficher le contenu
```

Pipes (tuyaux)

```
commande1 | commande2       # Sortie de 1 vers entrée de 2
ls -la | grep "fichier"     # Filtrer la sortie
ps aux | grep "processus"   # Chercher un processus
cat fichier | sort | uniq   # Chaîner plusieurs commandes
```

Commandes de variables 🌍

Gérer les variables d'environnement

Définir des variables

```
VARIABLE=valeur             # Variable locale
export VARIABLE=valeur      # Variable d'environnement
export PATH=$PATH:/nouveau/chemin  # Ajouter au PATH
```

Variables importantes

```
echo $HOME                  # Répertoire utilisateur
echo $USER                  # Nom utilisateur
echo $PWD                   # Répertoire courant
echo $SHELL                 # Shell utilisé
echo $PATH                  # Chemin de recherche
```

Variables de position

```
echo $1 $2 $3               # Arguments de ligne de commande
echo $#                     # Nombre d'arguments
echo $@                     # Tous les arguments
echo $?                     # Code de retour de la dernière commande
```

Commandes de scriptage

Automatisation de base

Créer un script

```
#!/bin/bash
# Mon premier script
echo "Hello World!"
echo "Date: $(date)"
echo "Utilisateur: $USER"
```

Exécuter un script

```
chmod +x script.sh          # Rendre exécutable
./script.sh                 # Exécuter
bash script.sh              # Exécuter avec bash
source script.sh            # Exécuter dans le shell actuel
```

Variables dans les scripts

```
#!/bin/bash
NOM="John"
echo "Bonjour $NOM"
echo "Arguments: $1 $2 $3"
echo "Nombre d'arguments: $#"
```

Exercices pratiques 🎯

Mise en pratique des commandes

Exercice 1 : Navigation et manipulation

```
# Créer une structure de test
mkdir -p test/{docs,src,backup}
cd test
echo "contenu1" > docs/fichier1.txt
echo "contenu2" > src/script.py
ls -la
tree
```

Exercice 2 : Recherche et filtrage

```
# Créer des fichiers de test
echo "ligne1" > test.txt
echo "ligne2" >> test.txt
echo "ligne3" >> test.txt
grep "ligne" test.txt
wc -l test.txt
sort test.txt
```

Exercice 3 : Redirection et pipes

```
# Utiliser les pipes
ls -la | grep "fichier"
ps aux | grep "bash"
cat test.txt | sort | uniq
```

Bonnes pratiques 💡

Sécurité

- Toujours vérifier avant de supprimer
- Utiliser `-i` pour les commandes destructives
- Faire des sauvegardes régulières
- Ne pas exécuter de commandes inconnues

Productivité

- Utiliser la complétion automatique (Tab)
- Créer des alias pour les commandes fréquentes
- Utiliser l'historique efficacement
- Combiner les commandes avec des pipes

Organisation

- Structurer les répertoires logiquement
- Utiliser des noms de fichiers descriptifs
- Documenter les scripts
- Maintenir un environnement propre

Prochaine étape 🎯

Module 3 : Gestion des paquets et maintenance

- Gestionnaires de paquets (apt, dnf, yum)
- Installation et mise à jour de logiciels
- Gestion des dépendances
- Nettoyage et maintenance du système
- Snapshots et rollbacks

QCM Module 2 : Commandes de base ✅

10 questions sur les commandes essentielles

Revenir au sommaire

Question 1

Quelle commande permet de voir le répertoire actuel ?

A) `dir`

B) `pwd`

C) `whereami`

D) `current`

Question 2

Que fait la commande `ls -la` ?

A) Liste seulement les fichiers

B) Liste avec détails et fichiers cachés

C) Liste seulement les répertoires

D) Liste les liens symboliques

Question 3

Comment créer un répertoire et ses parents ?

A) `mkdir -p /chemin/complet`

B) `mkdir -r /chemin/complet`

C) `mkdir --all /chemin/complet`

D) `mkdir -a /chemin/complet`

Question 4

Quelle commande copie un répertoire avec son contenu ?

A) `cp source dest`

B) `cp -r source dest`

C) `copy source dest`

D) `cp -a source dest`

Revenir au sommaire

Question 5

Que fait la commande `cat fichier.txt` ?

A) Crée le fichier

B) Supprime le fichier

C) Affiche le contenu du fichier

D) Édite le fichier

Revenir au sommaire

Question 6

Comment afficher les 10 dernières lignes d'un fichier ?

A) `head fichier`

B) `tail fichier`

C) `end fichier`

D) `bottom fichier`

Question 7

Que signifie le symbole `>` en shell ?

A) Redirection d'entrée

B) Redirection de sortie (écrase)

C) Pipe

D) Commentaire

Question 8

Comment chercher "erreur" dans tous les fichiers .log ?

A) `find erreur *.log`

B) `search erreur *.log`

C) `grep erreur *.log`

D) `locate erreur *.log`

Question 9

Que fait la commande `find / -name "*.txt"` ?

A) Recherche tous les fichiers .txt depuis la racine

B) Crée des fichiers .txt

C) Supprime les fichiers .txt

D) Liste le contenu des fichiers .txt

Revenir au sommaire

Question 10

Comment chaîner deux commandes avec un pipe ?

A) `cmd1 & cmd2`

B) `cmd1 | cmd2`

C) `cmd1 > cmd2`

D) `cmd1 >> cmd2`

Revenir au sommaire

Réponses du Module 2

Réponse 1 : B) `pwd`

- pwd = Print Working Directory

Réponse 2 : B) Liste avec détails et fichiers cachés

- `-l` = format long, `-a` = all (fichiers cachés inclus)

Réponse 3 : A) `mkdir -p /chemin/complet`

- `-p` = parents, crée les répertoires intermédiaires

Réponse 4 : B) `cp -r source dest`

- `-r` = récursif, pour copier les répertoires

Réponse 5 : C) Affiche le contenu du fichier

- cat = concatenate, affiche le contenu

Revenir au sommaire

Réponses du Module 2 (suite)

Réponse 6 : B) `tail fichier`

- tail affiche la fin, head affiche le début

Réponse 7 : B) Redirection de sortie (écrase)

- `>` redirige et écrase, `>>` ajoute à la fin

Réponse 8 : C) `grep erreur *.log`

- grep recherche des motifs dans des fichiers

Réponse 9 : A) Recherche tous les fichiers .txt depuis la racine

- find cherche des fichiers selon des critères

Réponse 10 : B) `cmd1 | cmd2`

- Le pipe `|` envoie la sortie de cmd1 vers l'entrée de cmd2

Revenir au sommaire

Score du Module 2

Comptez vos bonnes réponses :

- **9-10** : Excellent ! ⭐⭐⭐
- **7-8** : Bien ! ⭐⭐
- **5-6** : À revoir ⭐
- **< 5** : Relire le module 2

Revenir au sommaire

🎯 Exercices pratiques - Module 2

Maîtriser les commandes de base

Exercice 1 : Navigation

Objectif : Se déplacer dans le système

Consignes :

1. Affichez votre répertoire actuel
2. Listez le contenu de `/usr/bin`
3. Allez dans `/tmp`
4. Revenez à votre répertoire personnel
5. Affichez le chemin complet de `/tmp`
6. Retournez au répertoire précédent avec `cd -`

Solution Exercice 1 💡

```
# 1. Répertoire actuel
pwd

# 2. Contenu de /usr/bin
ls /usr/bin

# Avec détails
ls -l /usr/bin

# Seulement les 20 premiers
ls /usr/bin | head -20

# 3. Aller dans /tmp
cd /tmp

# 4. Retour au home
cd
# ou
cd ~

# 5. Chemin complet de /tmp
realpath /tmp
# ou
readlink -f /tmp

# 6. Retour au répertoire précédent
cd -
```

Exercice 2 : Manipulation de fichiers 📁

Objectif : Créer, copier, déplacer, renommer

Consignes :

1. Créez un répertoire `test` dans votre home
2. Dans `test`, créez 5 fichiers : `file1.txt` à `file5.txt`
3. Copiez `file1.txt` en `file1_backup.txt`
4. Renommez `file2.txt` en `document.txt`
5. Déplacez `file3.txt` dans votre home
6. Créez un sous-répertoire `archive`
7. Déplacez tous les fichiers restants dans `archive`

Solution Exercice 2 💡

```bash
# 1. Créer le répertoire
mkdir ~/test
cd ~/test

# 2. Créer 5 fichiers
touch file{1..5}.txt
# ou
touch file1.txt file2.txt file3.txt file4.txt file5.txt

# Vérifier
ls -l

# 3. Copier
cp file1.txt file1_backup.txt

# 4. Renommer
mv file2.txt document.txt

# 5. Déplacer
mv file3.txt ~/

# 6. Créer sous-répertoire
mkdir archive

# 7. Déplacer tous les fichiers
mv *.txt archive/

# Vérifier
ls -la
ls -la archive/
```

Exercice 3 : Affichage et recherche 🔍

Objectif : Visualiser et chercher du contenu

Consignes :

1. Créez un fichier `nombres.txt` avec les nombres de 1 à 100
2. Affichez tout le contenu
3. Affichez seulement les 10 premières lignes
4. Affichez seulement les 10 dernières lignes
5. Cherchez le nombre "42"
6. Comptez le nombre de lignes du fichier

Solution Exercice 3 💡

```
# 1. Créer le fichier avec nombres
seq 1 100 > nombres.txt

# Vérifier
ls -l nombres.txt

# 2. Afficher tout
cat nombres.txt

# 3. 10 premières lignes
head nombres.txt
# ou
head -n 10 nombres.txt

# 4. 10 dernières lignes
tail nombres.txt
# ou
tail -n 10 nombres.txt

# 5. Chercher 42
grep "42" nombres.txt
# Résultat: 42 (et aussi 142 si on va au-delà de 100)

# Pour match exact
grep "^42$" nombres.txt

# 6. Compter les lignes
wc -l nombres.txt
# ou
cat nombres.txt | wc -l
```

Exercice 4 : Redirections et pipes 🔄

Objectif : Maîtriser les redirections

Consignes :

1. Listez `/etc` et sauvegardez dans `liste_etc.txt`
2. Ajoutez la date à la fin de ce fichier
3. Créez un fichier avec "Hello" puis ajoutez "World"
4. Comptez combien de fichiers `.conf` dans `/etc`
5. Trouvez les 5 plus gros fichiers de `/var/log`

Solution Exercice 4 💡

```
# 1. Lister et sauvegarder
ls /etc > liste_etc.txt

# Vérifier
wc -l liste_etc.txt

# 2. Ajouter la date
date >> liste_etc.txt

# Vérifier
tail liste_etc.txt

# 3. Fichier avec contenu progressif
echo "Hello" > message.txt
echo "World" >> message.txt

# Vérifier
cat message.txt

# 4. Compter fichiers .conf
ls /etc/*.conf 2>/dev/null | wc -l
# ou
find /etc -name "*.conf" 2>/dev/null | wc -l

# 5. Plus gros fichiers de /var/log
sudo du -h /var/log/* 2>/dev/null | sort -hr | head -5
# ou
sudo find /var/log -type f -exec du -h {} + 2>/dev/null | \
    sort -hr | head -5
```

Exercice 5 : Recherche avancée 🎯

Objectif : Utiliser find et grep efficacement

Consignes :

1. Trouvez tous les fichiers `.txt` dans votre home
2. Trouvez les fichiers modifiés dans les dernières 24h
3. Trouvez les fichiers de plus de 1 Mo
4. Créez un fichier `log.txt` avec différents niveaux (INFO, ERROR, WARN)
5. Extrayez seulement les lignes ERROR

Solution Exercice 5 💡

```
# 1. Tous les .txt dans home
find ~ -name "*.txt" 2>/dev/null

# Avec détails
find ~ -name "*.txt" -ls 2>/dev/null

# 2. Modifiés dernières 24h
find ~ -mtime -1 2>/dev/null

# 3. Fichiers > 1 Mo
find ~ -size +1M 2>/dev/null

# Avec tailles
find ~ -size +1M -exec ls -lh {} \; 2>/dev/null

# 4. Créer fichier log
cat > log.txt << EOF
2025-11-24 10:00:00 INFO Application démarrée
2025-11-24 10:05:00 ERROR Connexion échouée
2025-11-24 10:10:00 WARN Mémoire basse
2025-11-24 10:15:00 INFO Traitement en cours
2025-11-24 10:20:00 ERROR Base de données inaccessible
2025-11-24 10:25:00 INFO Traitement terminé
EOF

# 5. Extraire les ERROR
grep ERROR log.txt

# Avec couleur
grep --color ERROR log.txt

# Compter les erreurs
grep -c ERROR log.txt

# Avec numéros de ligne
grep -n ERROR log.txt
```

Exercice 6 : Combinaison de commandes 🔗

Objectif : Chaîner plusieurs commandes

Consignes :

1. Listez les processus, cherchez "bash", comptez-les
2. Affichez le contenu de `/etc/passwd`, triez-le, prenez les 10 premiers
3. Trouvez votre utilisation disque et triez par taille
4. Listez `/bin`, comptez combien commencent par "s"
5. Créez une ligne de commande qui affiche l'heure toutes les 5 secondes

Solution Exercice 6 💡

```bash
# 1. Processus bash
ps aux | grep bash | wc -l

# 2. /etc/passwd trié
cat /etc/passwd | sort | head -10
# ou directement
sort /etc/passwd | head -10

# 3. Usage disque trié
du -h ~ | sort -hr | head -20

# 4. Commandes commençant par "s"
ls /bin | grep "^s" | wc -l

# 5. Affichage heure répété
while true; do clear; date; sleep 5; done
# Ctrl+C pour arrêter

# Alternative avec watch
watch -n 5 date
```

Exercice bonus : Script de nettoyage 🧹

Objectif : Créer un script utile

Créez `cleanup.sh` **qui :**

- Trouve les fichiers temporaires (`.tmp` , `.bak`)
- Les liste
- Demande confirmation
- Les supprime
- Affiche un résumé

Solution Exercice bonus 💡

```bash
#!/bin/bash
# Script de nettoyage

echo "===== Script de nettoyage ====="
echo

# Chercher fichiers temporaires
echo "Recherche de fichiers temporaires..."
TMP_FILES=$(find ~ -name "*.tmp" -o -name "*.bak" 2>/dev/null)

if [ -z "$TMP_FILES" ]; then
    echo "✅ Aucun fichier temporaire trouvé"
    exit 0
fi

# Lister les fichiers
echo "Fichiers trouvés :"
echo "$TMP_FILES"
echo

# Compter
COUNT=$(echo "$TMP_FILES" | wc -l)
echo "Total : $COUNT fichier(s)"

# Confirmation
read -p "Voulez-vous les supprimer ? (o/N) " -n 1 -r
echo

if [[ $REPLY =~ ^[Oo]$ ]]; then
    # Supprimer
    echo "$TMP_FILES" | while read file; do
        rm "$file"
        echo "  ✓ Supprimé: $file"
    done
    echo
    echo "✅ Nettoyage terminé : $COUNT fichier(s) supprimé(s)"
else
    echo "❌ Nettoyage annulé"
fi
```

Points clés à retenir 📌

Navigation :

- `pwd` , `cd` , `ls`
- Chemins absolus vs relatifs
- `cd -` pour revenir

Manipulation :

- `mkdir` , `touch`
- `cp` , `mv` , `rm`
- Option `-r` pour récursif

Visualisation :

- `cat` , `less` , `more`
- `head` , `tail`
- `grep` pour chercher

Redirections :

- `>` écrase
- ajoute

Revenir au sommaire

📦 Module 3

Gestion des paquets et maintenance

Installation, mise à jour et maintenance des logiciels

Revenir au sommaire

Qu'est-ce qu'un paquet ? 📦

Paquet : archive contenant un logiciel et ses métadonnées

Analogie : un meuble IKEA 🪑

- **Paquet** = carton avec pièces et instructions
- **Gestionnaire de paquets** = outils pour assembler
- **Dépôt** = magasin IKEA (catalogue de paquets)
- **Dépendances** = autres meubles nécessaires (étagères pour certains objets)

Contenu d'un paquet :

- Fichiers binaires
- Fichiers de configuration
- Documentation
- Scripts d'installation/désinstallation
- Liste des dépendances

Systèmes de paquets Linux 📚

Deux grandes familles :

Debian/Ubuntu (.deb)

- Gestionnaire : `dpkg` (bas niveau)
- Frontend : `apt` , `apt-get` , `aptitude`
- Distributions : Debian, Ubuntu, Mint, Pop!_OS

Red Hat/Fedora (.rpm)

- Gestionnaire : `rpm` (bas niveau)
- Frontend : `dnf` (ancien : `yum`)
- Distributions : RHEL, CentOS, AlmaLinux, Rocky, Fedora

Pourquoi des gestionnaires de paquets ? 🤔

Sans gestionnaire :

- Télécharger manuellement
- Résoudre les dépendances à la main
- Pas de mises à jour automatiques
- Risque de conflits

Avec gestionnaire :

- Installation en une commande
- Dépendances résolues automatiquement
- Mises à jour centralisées
- Désinstallation propre

APT : Advanced Package Tool 🔧

apt : gestionnaire pour Debian/Ubuntu

Commandes essentielles :

```
# Mettre à jour la liste des paquets
sudo apt update

# Mettre à jour les paquets installés
sudo apt upgrade

# Rechercher un paquet
apt search nginx

# Voir les infos d'un paquet
apt show nginx
```

Installer des paquets avec apt 📩

```
# Installer un paquet
sudo apt install nginx

# Installer plusieurs paquets
sudo apt install nginx mysql-server php

# Installer sans confirmation
sudo apt install -y nginx

# Installer une version spécifique
sudo apt install nginx=1.18.0-0ubuntu1

# Réinstaller un paquet
sudo apt install --reinstall nginx
```

Désinstaller des paquets avec apt 🗑️

```
# Désinstaller un paquet (garde la config)
sudo apt remove nginx

# Désinstaller avec la config
sudo apt purge nginx

# Supprimer les dépendances inutilisées
sudo apt autoremove

# Purge + autoremove
sudo apt purge nginx && sudo apt autoremove
```

Mettre à jour le système avec apt 🔄

```
# Mettre à jour la liste des paquets
sudo apt update

# Mettre à jour les paquets (sans supprimer)
sudo apt upgrade

# Mise à jour complète (peut supprimer des paquets)
sudo apt full-upgrade

# Distribution upgrade (changement de version)
sudo apt dist-upgrade

# Combo classique
sudo apt update && sudo apt upgrade -y
```

Différence update vs upgrade 🔀

apt update : met à jour la **liste** des paquets disponibles

- Comme consulter le catalogue du magasin
- Aucune installation
- À faire avant upgrade

apt upgrade : met à jour les paquets **installés**

- Comme acheter les nouvelles versions
- Installe vraiment les mises à jour
- Nécessite `update` avant

Analogie : Netflix 📺

- **apt update** = Rafraîchir le catalogue Netflix
  - Voir les nouveaux films disponibles
  - Ne télécharge rien
  - Rapide (quelques secondes)
- **apt upgrade** = Télécharger les épisodes
  - Installe vraiment les nouvelles versions
  - Peut prendre du temps
  - Utilise de la bande passante

Exemple pratique :

```
# Scénario : votre serveur web
sudo apt update          # Consulte si nouvelle version de nginx
# → "nginx 1.20 disponible (vous avez 1.18)"

sudo apt upgrade -y      # Installe nginx 1.20
# → Télécharge, installe, redémarre si nécessaire
```

Rechercher des paquets 🔍

```
# Recherche simple
apt search nginx

# Recherche avec description
apt search --names-only nginx

# Lister tous les paquets disponibles
apt list

# Lister les paquets installés
apt list --installed

# Lister les paquets à mettre à jour
apt list --upgradable

# Informations détaillées
apt show nginx
```

Dépendances 🔗

Dépendance : paquet nécessaire au fonctionnement

```
# Voir les dépendances d'un paquet
apt depends nginx

# Voir les dépendances inverses (qui dépend de ce paquet)
apt rdepends nginx

# Simuler une installation (voir ce qui sera installé)
apt install --simulate nginx

# Installer les dépendances d'un paquet sans l'installer
sudo apt build-dep nginx
```

Dépôts APT 📚

Dépôts : sources des paquets

Fichier principal : `/etc/apt/sources.list`

```
cat /etc/apt/sources.list
```

```
deb http://fr.archive.ubuntu.com/ubuntu/ jammy main restricted
deb http://fr.archive.ubuntu.com/ubuntu/ jammy-updates main restricted
deb http://security.ubuntu.com/ubuntu/ jammy-security main restricted
```

Format des sources APT

```
deb http://archive.ubuntu.com/ubuntu/ jammy main
│   └─ URL                           │     └─ Composants
└─ Type (deb = binaire, deb-src = source)
                                     └─ Version
```

Composants Ubuntu/Debian :

- **main** : logiciels libres supportés
- **restricted** : logiciels propriétaires supportés
- **universe** : logiciels libres communautaires
- **multiverse** : logiciels non-libres

Ajouter un dépôt PPA (Ubuntu) +

PPA : Personal Package Archive

```
# Ajouter un PPA
sudo add-apt-repository ppa:ondrej/php
sudo apt update

# Supprimer un PPA
sudo add-apt-repository --remove ppa:ondrej/php

# Ou manuellement
sudo nano /etc/apt/sources.list.d/ondrej-ubuntu-php-jammy.list
```

⚠️ **Attention :** les PPA ne sont pas officiels, vérifier la source !

Ajouter un dépôt externe

Exemple : Docker

```
# Installer les prérequis
sudo apt install ca-certificates curl gnupg

# Ajouter la clé GPG
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Ajouter le dépôt
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Mettre à jour et installer
sudo apt update
sudo apt install docker-ce
```

Nettoyer le système APT 🧹

```
# Supprimer les paquets téléchargés
sudo apt clean

# Supprimer les anciens paquets téléchargés
sudo apt autoclean

# Supprimer les dépendances inutilisées
sudo apt autoremove

# Combo nettoyage complet
sudo apt autoremove --purge && sudo apt clean
```

dpkg : gestionnaire de bas niveau 🔩

dpkg : gestion directe de paquets .deb

```
# Installer un paquet .deb
sudo dpkg -i package.deb

# Si dépendances manquantes
sudo apt install -f

# Lister les paquets installés
dpkg -l

# Voir les fichiers d'un paquet
dpkg -L nginx

# Trouver à quel paquet appartient un fichier
dpkg -S /usr/sbin/nginx
```

dpkg : informations et maintenance 📊

```
# Informations sur un paquet installé
dpkg -s nginx

# Lister les fichiers d'un paquet .deb
dpkg -c package.deb

# Extraire un paquet .deb sans installer
dpkg-deb -x package.deb /tmp/extracted

# Reconfigurer un paquet
sudo dpkg-reconfigure package-name

# Réparer les paquets cassés
sudo dpkg --configure -a
```

DNF : gestionnaire Red Hat/Fedora 🎩

dnf : remplace yum (depuis Fedora 22, RHEL 8)

```
# Mettre à jour la liste des paquets et installer les mises à jour
sudo dnf upgrade

# Rechercher un paquet
dnf search nginx

# Informations sur un paquet
dnf info nginx

# Installer un paquet
sudo dnf install nginx
```

Installer avec DNF 📥

```
# Installer un paquet
sudo dnf install nginx

# Installer plusieurs paquets
sudo dnf install nginx mariadb-server php

# Installer sans confirmation
sudo dnf install -y nginx

# Installer un groupe de paquets
sudo dnf groupinstall "Development Tools"

# Réinstaller
sudo dnf reinstall nginx
```

Désinstaller avec DNF 🗑️

```
# Désinstaller un paquet
sudo dnf remove nginx

# Supprimer avec les dépendances inutilisées
sudo dnf autoremove nginx

# Supprimer un groupe
sudo dnf groupremove "Development Tools"
```

Mettre à jour avec DNF 🔄

```
# Vérifier les mises à jour disponibles
dnf check-update

# Mettre à jour tous les paquets
sudo dnf upgrade

# Mettre à jour un paquet spécifique
sudo dnf upgrade nginx

# Mettre à jour vers une nouvelle version de Fedora
sudo dnf system-upgrade download --releasever=39
sudo dnf system-upgrade reboot
```

Rechercher avec DNF 🔍

```
# Recherche simple
dnf search nginx

# Lister tous les paquets
dnf list

# Lister les paquets installés
dnf list installed

# Lister les paquets disponibles
dnf list available

# Informations détaillées
dnf info nginx

# Voir l'historique des transactions
dnf history

# Annuler une transaction
sudo dnf history undo last
```

Dépôts DNF

Fichiers de configuration : `/etc/yum.repos.d/`

```
ls /etc/yum.repos.d/
```

Exemple de fichier de dépôt :

```
[epel]
name=Extra Packages for Enterprise Linux $releasever - $basearch
baseurl=https://download.fedoraproject.org/pub/epel/$releasever/Everything/$basearch/
enabled=1
gpgcheck=1
gpgkey=https://download.fedoraproject.org/pub/epel/RPM-GPG-KEY-EPEL-$releasever
```

Gérer les dépôts DNF 🔧

```
# Lister les dépôts
dnf repolist

# Lister tous les dépôts (y compris désactivés)
dnf repolist --all

# Activer un dépôt
sudo dnf config-manager --enable epel

# Désactiver un dépôt
sudo dnf config-manager --disable epel

# Ajouter un dépôt
sudo dnf config-manager --add-repo https://example.com/repo/example.repo
```

EPEL : paquets supplémentaires RHEL

EPEL : Extra Packages for Enterprise Linux

```
# Installer EPEL sur RHEL/AlmaLinux/Rocky
sudo dnf install epel-release

# Ou manuellement
sudo dnf install https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm

# Vérifier
dnf repolist | grep epel
```

EPEL apporte des milliers de paquets additionnels !

Nettoyer le système DNF 🧹

```
# Nettoyer le cache
sudo dnf clean all

# Supprimer les paquets orphelins
sudo dnf autoremove

# Supprimer les anciens noyaux (garder 2 derniers)
sudo dnf remove $(dnf repoquery --installonly --latest-limit=-2 -q)
```

RPM : gestionnaire de bas niveau 🔩

rpm : gestion directe de paquets .rpm

```
# Installer un paquet .rpm
sudo rpm -ivh package.rpm

# Mettre à jour
sudo rpm -Uvh package.rpm

# Désinstaller
sudo rpm -e package-name

# Lister les paquets installés
rpm -qa

# Voir les fichiers d'un paquet
rpm -ql nginx
```

RPM : informations

```
# Informations sur un paquet installé
rpm -qi nginx

# Informations sur un fichier .rpm
rpm -qip package.rpm

# Trouver à quel paquet appartient un fichier
rpm -qf /usr/sbin/nginx

# Vérifier l'intégrité d'un paquet
rpm -V nginx

# Lister les scripts du paquet
rpm -q --scripts nginx
```

Snap : paquets universels 📦

Snap : format de paquet universel (Canonical/Ubuntu)

```
# Installer snapd
sudo apt install snapd        # Ubuntu/Debian
sudo dnf install snapd        # Fedora

# Rechercher un snap
snap find vlc

# Installer un snap
sudo snap install vlc

# Lister les snaps installés
snap list

# Mettre à jour
sudo snap refresh vlc
sudo snap refresh   # Tous

# Désinstaller
sudo snap remove vlc
```

Snap : avantages et inconvénients ⚖️

Avantages ✅

- Isolation (conteneurisé)
- Mises à jour automatiques
- Même paquet pour toutes les distributions
- Plusieurs versions en parallèle

Inconvénients ❌

- Plus volumineux (dépendances incluses)
- Démarrage plus lent
- Consomme plus de mémoire
- Accès limité au système (sandboxing)

Flatpak : alternative à Snap 📦

Flatpak : format universel (projet libre)

```
# Installer Flatpak
sudo apt install flatpak              # Ubuntu/Debian
sudo dnf install flatpak              # Fedora

# Ajouter Flathub (dépôt principal)
flatpak remote-add --if-not-exists flathub \
  https://flathub.org/repo/flathub.flatpakrepo

# Rechercher une application
flatpak search firefox

# Installer
flatpak install flathub org.mozilla.firefox

# Lancer
flatpak run org.mozilla.firefox
```

Flatpak : gestion 🔧

```
# Lister les applications installées
flatpak list

# Mettre à jour tout
flatpak update

# Désinstaller
flatpak uninstall org.mozilla.firefox

# Nettoyer les runtimes inutilisés
flatpak uninstall --unused

# Voir les infos
flatpak info org.mozilla.firefox
```

AppImage : exécutables portables 🎒

AppImage : un seul fichier exécutable

```
# Télécharger une AppImage
wget https://example.com/app.AppImage

# Rendre exécutable
chmod +x app.AppImage

# Lancer
./app.AppImage

# C'est tout ! Aucune installation nécessaire
```

Avantages :

- Pas d'installation
- Portable (USB)
- Pas de privilèges admin

Inconvénients :

- Pas de mises à jour automatiques
- Pas d'intégration système

Revenir au sommaire

Compiler depuis les sources 🔨

Quand compiler ?

- Paquet pas disponible
- Version très récente nécessaire
- Options de compilation spécifiques

```
# Installer les outils de compilation
sudo apt install build-essential    # Debian/Ubuntu
sudo dnf groupinstall "Development Tools"  # RHEL/Fedora

# Télécharger les sources
wget https://example.com/software-1.0.tar.gz
tar -xzf software-1.0.tar.gz
cd software-1.0
```

Compiler depuis les sources (suite) 🔨

```
# Configuration
./configure --prefix=/usr/local

# Compilation
make

# Installation
sudo make install

# Désinstallation (si supporté)
sudo make uninstall
```

Sécurité : mises à jour automatiques 🔐

Pourquoi automatiser ?

- Correctifs de sécurité critiques
- Pas d'oubli
- Système toujours à jour
- Réduction de la surface d'attaque
- Conformité aux politiques de sécurité

Solutions disponibles :

- **Ubuntu/Debian** : unattended-upgrades
- **RHEL/Fedora** : dnf-automatic

Qu'est-ce que unattended-upgrades ? 🤖

unattended-upgrades : paquet pour automatiser les mises à jour Ubuntu/Debian

Fonctionnement :

- Télécharge et installe automatiquement les mises à jour
- Sans intervention manuelle
- Peut être configuré pour différents types de mises à jour

Cas d'usage idéaux :

- Serveurs de production
- Machines qu'on ne gère pas quotidiennement
- Environnements nécessitant une sécurité renforcée
- Parc de machines important

Installation de unattended-upgrades 📥

```
# Installer le paquet
sudo apt install unattended-upgrades
```

Activation de unattended-upgrades

```
# Activer le service (mode interactif)
sudo dpkg-reconfigure unattended-upgrades
```

Activation de unattended-upgrades (suite)

Cette commande vous demandera :

- **Yes** : Active les mises à jour automatiques
- **No** : Désactive les mises à jour automatiques

Alternative : configuration manuelle

```
# Éditer le fichier de configuration
sudo nano /etc/apt/apt.conf.d/20auto-upgrades
```

Configuration auto-upgrades

Fichier : `/etc/apt/apt.conf.d/20auto-upgrades`

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
APT::Periodic::Download-Upgradeable-Packages "1";
APT::Periodic::AutocleanInterval "7";
```

Configuration auto-upgrades (explication)

Signification des paramètres :

- **Update-Package-Lists "1"** : Met à jour la liste des paquets quotidiennement
- **Unattended-Upgrade "1"** : Active les mises à jour automatiques quotidiennes
- **Download-Upgradeable-Packages "1"** : Télécharge les mises à jour disponibles
- **AutocleanInterval "7"** : Nettoie le cache tous les 7 jours

Valeurs possibles :

- **"0"** = Désactivé
- **"1"** = Tous les jours
- **"7"** = Tous les 7 jours

Configuration avancée : 50unattended-upgrades 🔧

Fichier principal : `/etc/apt/apt.conf.d/50unattended-upgrades`

Ce fichier contrôle :

- Quelles mises à jour installer
- Si le système doit redémarrer
- Notifications par email
- Gestion des paquets

```
# Éditer la configuration
sudo nano /etc/apt/apt.conf.d/50unattended-upgrades
```

Configuration : sources de mises à jour 📚

```
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
    "${distro_id}:${distro_codename}-updates";
    // "${distro_id}:${distro_codename}-proposed";
    // "${distro_id}:${distro_codename}-backports";
};
```

Configuration : sources (explication) 📚

Options disponibles :

- `-security` : Correctifs de sécurité uniquement (recommandé ✅)
- `-updates` : Mises à jour stables
- `-proposed` : Paquets proposés (tests) ⚠️
- `-backports` : Nouvelles versions backportées ⚠️

Recommandation : n'activer que `-security` pour les serveurs critiques

Configuration : redémarrage automatique 🔄

```
# Autoriser le redémarrage automatique ?
Unattended-Upgrade::Automatic-Reboot "false";

# Si oui, à quelle heure ?
Unattended-Upgrade::Automatic-Reboot-Time "03:00";

# Redémarrer même si des utilisateurs sont connectés ?
Unattended-Upgrade::Automatic-Reboot-WithUsers "false";
```

Configuration : notifications email

```
# Adresse email pour les notifications
Unattended-Upgrade::Mail "admin@example.com";

# Quand envoyer un email ?
// "always" : toujours
// "only-on-error" : seulement en cas d'erreur
// "on-change" : quand des mises à jour sont installées
Unattended-Upgrade::MailReport "on-change";
```

Configuration : gestion des paquets 📦

```
# Supprimer les dépendances inutilisées ?
Unattended-Upgrade::Remove-Unused-Dependencies "true";

# Supprimer les nouveaux paquets inutilisés ?
Unattended-Upgrade::Remove-New-Unused-Dependencies "true";

# Mise à jour automatique des paquets en liste noire ?
Unattended-Upgrade::Remove-Unused-Kernel-Packages "true";
```

Configuration : liste noire 🚫

```
# Paquets à ne JAMAIS mettre à jour automatiquement
Unattended-Upgrade::Package-Blacklist {
    "nginx";
    "mysql-server";
    "postgresql";
};
```

Configuration : liste noire (pourquoi ?) 🤔

Cas d'usage :

- Paquets critiques nécessitant supervision
- Applications avec configurations spécifiques
- Services nécessitant des tests avant mise à jour
- Paquets pouvant casser des dépendances

Exemple : serveur web avec configuration custom

- Mise à jour de nginx pourrait écraser la config
- Préférable de gérer manuellement

Exemple de configuration complète (sécurité seule)

```
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
};

Unattended-Upgrade::Automatic-Reboot "false";
Unattended-Upgrade::Mail "admin@example.com";
Unattended-Upgrade::MailReport "on-change";

Unattended-Upgrade::Remove-Unused-Dependencies "true";
Unattended-Upgrade::Remove-New-Unused-Dependencies "true";
```

Exemple de configuration complète (avec redémarrage) 🔄

```
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
    "${distro_id}:${distro_codename}-updates";
};

Unattended-Upgrade::Automatic-Reboot "true";
Unattended-Upgrade::Automatic-Reboot-Time "03:00";
Unattended-Upgrade::Automatic-Reboot-WithUsers "false";

Unattended-Upgrade::Mail "admin@example.com";
Unattended-Upgrade::MailReport "always";
```

Avantages de unattended-upgrades ✅

Sécurité renforcée 🔐

- Correctifs critiques appliqués immédiatement
- Pas de délai d'intervention humaine
- Réduction de la fenêtre de vulnérabilité

Gain de temps ⏰

- Plus besoin de lancer manuellement `apt update && apt upgrade`
- Gestion automatique de plusieurs serveurs
- Concentration sur les tâches importantes

Avantages de unattended-upgrades (suite) ✅

Contrôle total 🎛️

- Choix précis des catégories de mises à jour
- Liste noire pour les paquets sensibles
- Configuration des horaires de redémarrage
- Notifications personnalisables

Fiabilité 🛡️

- Process testé et éprouvé
- Gestion intelligente des dépendances
- Rollback possible en cas de problème

Points à savoir ⚠️

Redémarrages nécessaires :

- Les mises à jour du noyau nécessitent un redémarrage
- Certaines bibliothèques système également
- Vérifier `/var/run/reboot-required` pour savoir si un redémarrage est nécessaire

```
# Vérifier si un redémarrage est nécessaire
test -f /var/run/reboot-required && echo "Redémarrage nécessaire"

# Voir pourquoi
cat /var/run/reboot-required.pkgs
```

Logs et surveillance 📋

Localisation des logs :

```
# Logs principaux
/var/log/unattended-upgrades/

# Journal détaillé
/var/log/unattended-upgrades/unattended-upgrades.log

# Journal de dpkg
/var/log/unattended-upgrades/unattended-upgrades-dpkg.log
```

Consulter les logs 🔍

```
# Voir les dernières mises à jour
sudo tail -f /var/log/unattended-upgrades/unattended-upgrades.log

# Voir les erreurs
sudo grep -i error /var/log/unattended-upgrades/unattended-upgrades.log

# Voir les mises à jour d'aujourd'hui
sudo grep "$(date +%Y-%m-%d)" \
  /var/log/unattended-upgrades/unattended-upgrades.log
```

Tester la configuration ✅

```
# Tester en mode debug (simulation)
sudo unattended-upgrade --debug --dry-run

# Forcer l'exécution immédiate
sudo unattended-upgrade --debug

# Vérifier le statut du service
sudo systemctl status unattended-upgrades
```

Désactiver temporairement 🛑

```
# Arrêter le service
sudo systemctl stop unattended-upgrades

# Désactiver au démarrage
sudo systemctl disable unattended-upgrades

# Réactiver
sudo systemctl enable --now unattended-upgrades
```

DNF Automatic (RHEL/Fedora) 🤖

```
# Installer
sudo dnf install dnf-automatic

# Configuration
sudo nano /etc/dnf/automatic.conf
```

```
[commands]
upgrade_type = security
download_updates = yes
apply_updates = yes

[email]
email_to = admin@example.com
```

```
# Activer
sudo systemctl enable --now dnf-automatic.timer
```

Vérifier les mises à jour de sécurité 🔒

Ubuntu/Debian :

```
# Lister les mises à jour de sécurité
sudo apt list --upgradable | grep security

# Installer seulement les mises à jour de sécurité
sudo unattended-upgrade -d
```

RHEL/Fedora :

```
# Lister les mises à jour de sécurité
sudo dnf updateinfo list security

# Installer seulement les correctifs de sécurité
sudo dnf upgrade --security
```

Gestion des versions de noyau 🧠

```
# Voir la version actuelle
uname -r

# Lister les noyaux installés (Debian/Ubuntu)
dpkg -l | grep linux-image

# Supprimer un ancien noyau
sudo apt remove linux-image-5.15.0-25-generic

# Ou autoremove pour nettoyer automatiquement
sudo apt autoremove
```

Gestion des versions de noyau (RHEL/Fedora) 🧠

```
# Lister les noyaux installés
rpm -q kernel

# Configuration : nombre de noyaux à garder
sudo nano /etc/dnf/dnf.conf
```

```
installonly_limit=3
```

```
# Supprimer les anciens noyaux
sudo dnf remove --oldinstallonly --setopt installonly_limit=2 kernel
```

Dépannage : paquets cassés 🔧

Debian/Ubuntu :

```
# Réparer les paquets cassés
sudo apt install -f

# Reconfigurer tous les paquets non configurés
sudo dpkg --configure -a

# Forcer la suppression d'un paquet bloqué
sudo dpkg --remove --force-remove-reinstreq package-name
```

Dépannage : paquets cassés (RHEL/Fedora) 🔧

```
# Vérifier les problèmes
sudo dnf check

# Réparer
sudo dnf distro-sync

# Réinitialiser un paquet
sudo dnf reinstall package-name

# En cas de problème avec la base de données RPM
sudo rpm --rebuilddb
```

Verrouiller une version de paquet 🔒

Debian/Ubuntu : apt-mark

```
# Empêcher la mise à jour
sudo apt-mark hold nginx

# Voir les paquets verrouillés
apt-mark showhold

# Déverrouiller
sudo apt-mark unhold nginx
```

Verrouiller une version de paquet (DNF) 🔒

RHEL/Fedora : versionlock

```
# Installer le plugin
sudo dnf install python3-dnf-plugin-versionlock

# Verrouiller un paquet
sudo dnf versionlock add nginx

# Lister les verrous
sudo dnf versionlock list

# Supprimer un verrou
sudo dnf versionlock delete nginx
```

Alternatives : plusieurs versions 🔀

update-alternatives : gérer plusieurs versions d'un outil

```
# Lister les alternatives pour un outil
update-alternatives --list editor

# Voir la configuration
update-alternatives --display editor

# Changer l'alternative
sudo update-alternatives --config editor

# Ajouter une alternative
sudo update-alternatives --install /usr/bin/editor editor \
  /usr/bin/vim 100
```

Exemples d'alternatives courantes

```
# Éditeur par défaut
sudo update-alternatives --config editor

# Java
sudo update-alternatives --config java

# Navigateur par défaut
sudo update-alternatives --config x-www-browser

# Python
sudo update-alternatives --config python
```

Logs et historique 📜

Logs APT :

```
# Historique des installations
cat /var/log/apt/history.log

# Journal détaillé
cat /var/log/apt/term.log

# Historique complet (y compris archives)
zcat /var/log/apt/history.log.*.gz | less
```

Revenir au sommaire

Logs et historique DNF

```
# Historique DNF
dnf history

# Détails d'une transaction
dnf history info 5

# Annuler une transaction
sudo dnf history undo 5

# Refaire une transaction
sudo dnf history redo 5

# Logs
cat /var/log/dnf.log
```

Bonnes pratiques 📋

1. **Toujours faire** `update` **avant** `upgrade`

```
sudo apt update && sudo apt upgrade
```

2. **Lire les changements avant de confirmer**
   - Vérifier ce qui sera installé/supprimé
3. **Sauvegarder avant les mises à jour majeures**
   - Snapshot VM ou backup
4. **Activer les mises à jour de sécurité automatiques**
   - unattended-upgrades ou dnf-automatic

Bonnes pratiques (suite) 📋

5. **Nettoyer régulièrement**

```
sudo apt autoremove && sudo apt clean
```

6. **Ne pas mélanger les sources**

   - Éviter de mélanger Debian et Ubuntu
   - Éviter les PPA non vérifiés

7. **Surveiller les obsolescences**

```
apt list --upgradable
```

8. **Documenter les modifications**

   - Noter les dépôts ajoutés
   - Noter les versions verrouillées

Cas pratiques 💼

Installer LAMP (Linux, Apache, MySQL, PHP) :

```
# Debian/Ubuntu
sudo apt update
sudo apt install apache2 mysql-server php libapache2-mod-php php-mysql

# RHEL/Fedora
sudo dnf install httpd mariadb-server php php-mysqlnd
sudo systemctl enable --now httpd mariadb
```

Cas pratiques (suite) 💼

Installer Docker depuis le dépôt officiel :

```
# Debian/Ubuntu
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# RHEL/Fedora
sudo dnf config-manager --add-repo \
  https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install docker-ce docker-ce-cli containerd.io
sudo systemctl enable --now docker
```

Récapitulatif du Module 3 ✅

Ce que vous avez appris :

- ✅ Concepts de paquets et dépendances
- ✅ APT (Debian/Ubuntu) : install, remove, update, upgrade
- ✅ DNF/YUM (RHEL/Fedora) : install, remove, upgrade
- ✅ Gestion des dépôts
- ✅ Formats universels (Snap, Flatpak, AppImage)
- ✅ Mises à jour automatiques de sécurité
- ✅ Compilation depuis les sources
- ✅ Dépannage de paquets cassés
- ✅ Bonnes pratiques de maintenance

Prochaine étape 🎯

Module 4 : Gestion des utilisateurs et permissions

- Création et gestion des comptes utilisateurs
- Groupes et appartenance
- Permissions avancées (chmod, chown, umask)
- Sudo et sudoers
- Quotas disque
- Logs et journalctl

Questions ? 🤔

N’hésitez pas à poser vos questions maintenant !

Le prochain module explorera l’automatisation avec les scripts et les tâches planifiées.

QCM Module 3 : Paquets et maintenance ✅

10 questions sur les gestionnaires de paquets

Revenir au sommaire

Question 1

Quel gestionnaire de paquets est utilisé sur Ubuntu ?

A) `yum`

B) `dnf`

C) `apt`

D) `pacman`

Revenir au sommaire

Question 2

Quelle commande met à jour la liste des paquets ?

A) `apt upgrade`

B) `apt update`

C) `apt refresh`

D) `apt sync`

Revenir au sommaire

Question 3

Comment installer un paquet ?

A) `apt install paquet`

B) `apt get paquet`

C) `apt add paquet`

D) `apt download paquet`

Question 4

Quelle commande recherche un paquet ?

A) `apt find`

B) `apt search`

C) `apt query`

D) `apt locate`

Question 5

Comment supprimer un paquet avec sa config ?

A) `apt remove`

B) `apt delete`

C) `apt purge`

D) `apt uninstall`

Revenir au sommaire

Question 6

Quel gestionnaire de paquets est utilisé sur Fedora/RHEL ?

A) `yum`

B) `dnf`

C) `apt`

D) A et B (dnf remplace yum)

Revenir au sommaire

Question 7

Que fait `apt autoremove` ?

A) Supprime tous les paquets

B) Supprime les paquets inutilisés

C) Supprime le cache

D) Supprime les anciennes versions

Revenir au sommaire

Question 8

Qu’est-ce qu’un PPA ?

A) Un dépôt personnel (Personal Package Archive)

B) Un type de paquet

C) Un gestionnaire de paquets

D) Un format de fichier

Question 9

Comment nettoyer le cache de paquets téléchargés ?

A) `apt clean`

B) `apt clear`

C) `apt flush`

D) `apt delete-cache`

Question 10

Qu’est-ce que Snap ?

A) Un gestionnaire de paquets traditionnel

B) Un format de paquet universel conteneurisé

C) Un outil de sauvegarde

D) Un éditeur de texte

Réponses du Module 3

Réponse 1 : C) `apt`

- apt = Advanced Package Tool (Debian/Ubuntu)

Réponse 2 : B) `apt update`

- Toujours faire `update` avant `upgrade`

Réponse 3 : A) `apt install paquet`

- `sudo apt install nginx` par exemple

Réponse 4 : B) `apt search`

- `apt search nginx` cherche tous les paquets contenant "nginx"

Réponse 5 : C) `apt purge`

- `remove` garde la config, `purge` supprime tout

Revenir au sommaire

Réponses du Module 3 (suite) 📝

Réponse 6 : D) A et B (dnf remplace yum)

- `dnf` est le successeur moderne de `yum`

Réponse 7 : B) Supprime les paquets inutilisés

- Nettoie les dépendances orphelines

Réponse 8 : A) Un dépôt personnel (Personal Package Archive)

- Spécifique à Ubuntu, permet d'ajouter des dépôts tiers

Réponse 9 : A) `apt clean`

- Libère de l'espace disque en supprimant les paquets téléchargés

Réponse 10 : B) Un format de paquet universel conteneurisé

- Alternative : Flatpak et AppImage

MINISTÈRE DES ARMÉES
Liberté
Égalité
Fraternité

Revenir au sommaire

Score du Module 3 📊

Comptez vos bonnes réponses :

- **9-10** : Excellent ! ⭐⭐⭐
- **7-8** : Bien ! ⭐⭐
- **5-6** : À revoir ⭐
- **< 5** : Relire le module 6

Revenir au sommaire

🎯 Exercices pratiques - Module 3

Gestion des paquets et maintenance

Exercice 1 : Gestion de base avec apt 📦

Objectif : Maîtriser apt

Consignes :

1. Mettez à jour la liste des paquets
2. Recherchez le paquet `htop`
3. Affichez les informations détaillées de `htop`
4. Installez `htop` et `tree`
5. Listez les paquets installés contenant "net"
6. Mettez à jour tous les paquets

Solution Exercice 1 💡

```
# 1. Mise à jour de la liste
sudo apt update

# 2. Rechercher htop
apt search htop

# 3. Informations
apt show htop

# 4. Installer
sudo apt install htop tree -y

# 5. Paquets installés avec "net"
apt list --installed | grep net

# 6. Mise à jour complète
sudo apt upgrade -y
# ou pour upgrade complet
sudo apt full-upgrade -y

# Vérifier
htop
tree /etc
```

Exercice 2 : Nettoyage et maintenance 🧹

Objectif : Nettoyer le système

Consignes :

1. Affichez l'espace utilisé par le cache apt
2. Nettoyez le cache
3. Supprimez les paquets orphelins
4. Listez les paquets qui ne sont plus nécessaires
5. Supprimez-les
6. Vérifiez l'espace libéré

Solution Exercice 2 💡

```
# 1. Espace du cache
du -sh /var/cache/apt/archives/

# 2. Nettoyer le cache
sudo apt clean
# Vérifier
du -sh /var/cache/apt/archives/

# 3 & 4 & 5. Paquets orphelins
# Voir la liste
apt autoremove --dry-run

# Supprimer
sudo apt autoremove -y

# Purger complètement
sudo apt autoclean

# 6. Vérifier l'espace disque
df -h /

# Combo complet de nettoyage
sudo apt clean && sudo apt autoremove -y && sudo apt autoclean
```

Exercice 3 : Gestion des versions 🔢

Objectif : Installer des versions spécifiques

Consignes :

1. Listez les versions disponibles de `nginx`
2. Installez une version spécifique
3. Verrouillez le paquet pour éviter les mises à jour
4. Tentez de mettre à jour (devrait échouer)
5. Déverrouillez le paquet
6. Désinstallez sans purger la configuration

Solution Exercice 3 💡

```
# 1. Versions disponibles
apt-cache policy nginx

# 2. Installer version spécifique (exemple)
sudo apt install nginx=1.18.0-0ubuntu1

# 3. Verrouiller (hold)
sudo apt-mark hold nginx

# Vérifier
apt-mark showhold

# 4. Tenter mise à jour
sudo apt upgrade
# nginx ne sera PAS mis à jour

# 5. Déverrouiller
sudo apt-mark unhold nginx

# 6. Désinstaller sans purge
sudo apt remove nginx
# La config reste dans /etc/nginx

# Pour purger complètement
sudo apt purge nginx
```

Exercice 4 : Ajout de dépôts PPA 📚

Objectif : Ajouter des sources tierces

Consignes :

1. Ajoutez le PPA de `git-core` (dernière version de Git)
2. Mettez à jour et installez git depuis ce PPA
3. Listez tous les PPA actifs
4. Supprimez le PPA
5. Revenez à la version officielle

⚠️ **Note :** PPA uniquement sur Ubuntu/dérivés

Solution Exercice 4 💡

```
# 1. Ajouter PPA
sudo add-apt-repository ppa:git-core/ppa -y

# 2. Mettre à jour et installer
sudo apt update
sudo apt install git -y

# Vérifier version
git --version

# 3. Lister les PPA
grep -r "ppa.launchpad.net" /etc/apt/sources.list.d/

# 4. Supprimer le PPA
sudo add-apt-repository --remove ppa:git-core/ppa -y

# 5. Revenir à la version officielle
sudo apt update
sudo apt install --reinstall git -y

# Alternative: installer depuis source
sudo apt install build-essential
wget https://github.com/git/git/archive/refs/tags/v2.43.0.tar.gz
tar -xzf v2.43.0.tar.gz
cd git-2.43.0
make prefix=/usr/local all
sudo make prefix=/usr/local install
```

Exercice 5 : Paquets Snap 📦

Objectif : Utiliser Snap

Consignes :

1. Installez `snapd` si nécessaire
2. Listez les snaps disponibles populaires
3. Installez un snap (ex: `hello-world`)
4. Listez les snaps installés
5. Mettez à jour les snaps
6. Supprimez le snap

Solution Exercice 5 💡

```
# 1. Installer snapd
sudo apt install snapd -y

# 2. Rechercher des snaps
snap find "editor"
snap find "browser"

# 3. Installer
sudo snap install hello-world

# Tester
hello-world

# 4. Lister installés
snap list

# 5. Mettre à jour
sudo snap refresh
# ou un snap spécifique
sudo snap refresh hello-world

# 6. Supprimer
sudo snap remove hello-world

# Voir les révisions
snap list --all

# Infos sur un snap
snap info firefox
```

Exercice bonus : Script de mise à jour automatique 🤖

Objectif : Automatiser les mises à jour

Créez un script qui :

- Met à jour la liste des paquets
- Installe les mises à jour
- Nettoie automatiquement
- Envoie un rapport par email
- Log toutes les actions

Solution Exercice bonus 💡

```
#!/bin/bash
# Script de mise à jour automatique

LOG_FILE="/var/log/auto_update.log"
ADMIN_EMAIL="admin@example.com"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a $LOG_FILE
}

log "===== Début mise à jour automatique ====="

# Mettre à jour la liste
log "Mise à jour de la liste des paquets..."
sudo apt update >> $LOG_FILE 2>&1

# Compter les mises à jour disponibles
UPDATES=$(apt list --upgradable 2>/dev/null | grep -c upgradable)
log "Mises à jour disponibles : $UPDATES"

if [ $UPDATES -gt 0 ]; then
    # Installer les mises à jour
    log "Installation des mises à jour..."
    sudo DEBIAN_FRONTEND=noninteractive apt upgrade -y >> $LOG_FILE 2>&1

    if [ $? -eq 0 ]; then
        log "✅ Mises à jour installées avec succès"
    else
        log "❌ Erreur lors des mises à jour"
        echo "Erreur de mise à jour sur $(hostname)" | \
            mail -s "Erreur Mise à Jour" $ADMIN_EMAIL
    fi
else
    log "Système à jour"
fi

# Nettoyage
log "Nettoyage..."
sudo apt autoremove -y >> $LOG_FILE 2>&1
sudo apt autoclean >> $LOG_FILE 2>&1

# Vérifier si redémarrage nécessaire
i    /var/run/reboot-required ]; then
    "⚠️  Redémarrage requis"
    o "Redémarrage nécessaire sur $(hostname)" | \
        mail -s "Redémarrage Requis" $ADMIN_EMAIL
```

Revenir au sommaire

Points clés à retenir 📌

APT :

- `update` : liste des paquets
- `upgrade` : installer mises à jour
- `install` : installer un paquet
- `remove` / `purge` : désinstaller

Maintenance :

- `autoremove` : supprimer orphelins
- `clean` : nettoyer cache
- `apt-mark hold` : verrouiller version

Alternatives :

- Snap : paquets universels
- Flatpak : alternative à Snap
- AppImage : applications portables

👥 Module 4

Gestion du système et des utilisateurs

Maîtriser les comptes, groupes et permissions

Revenir au sommaire

Pourquoi gérer les utilisateurs ? 👥

Unix/Linux est multi-utilisateurs par conception

Imaginez un immeuble :

- Chaque locataire (utilisateur) a sa clé (mot de passe)
- Chacun a son appartement (home directory)
- Certaines zones sont communes (dossiers partagés)
- Le gardien (root) a accès à tout

Objectifs :

- Sécurité : qui peut faire quoi ?
- Isolation : chacun ses données
- Organisation : groupes de travail

L'utilisateur root : le super-admin 👑

root : l'utilisateur tout-puissant (UID 0)

Pouvoirs :

- Accès à tous les fichiers
- Modification de toute configuration
- Installation de logiciels
- Gestion des utilisateurs
- Redémarrage, arrêt du système

⚠️ **Avec de grands pouvoirs viennent de grandes responsabilités**

```
# Danger ! Peut détruire le système
rm -rf /
```

Ne jamais travailler en root ! 🚨

Pourquoi ?

1. **Erreur humaine** : une faute de frappe peut être fatale
2. **Sécurité** : un malware aurait tous les droits
3. **Audit** : difficile de tracer qui a fait quoi
4. **Bonnes pratiques** : utiliser `sudo` pour les tâches admin

Analogie : Utiliser root en permanence, c'est comme conduire une voiture de sport à 200 km/h en ville : possible, mais dangereux !

Les fichiers d'authentification

Fichiers clés du système :

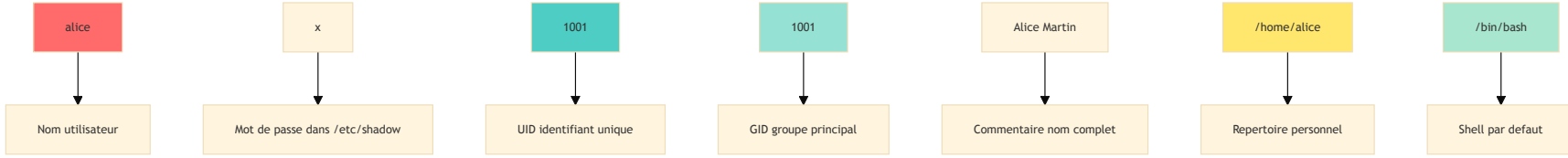
`/etc/passwd` : informations des utilisateurs

```
alice:x:1001:1001:Alice Martin:/home/alice:/bin/bash
bob:x:1002:1002:Bob Dupont:/home/bob:/bin/bash
```

Format : `username:x:UID:GID:comment:home:shell`

Décryptage de /etc/passwd 🔍

Format : `alice:x:1001:1001:Alice Martin:/home/alice:/bin/bash`



UID remarquables :

- 0 : root
- 1-999 : comptes système
- 1000+ : utilisateurs normaux

Le fichier /etc/shadow 🔒

Contient les mots de passe chiffrés (accessible uniquement par root)

```
alice:$6$rounds=5000$salt$hashedpassword:19000:0:99999:7:::
```

Format :

```
username:password_hash:last_change:min:max:warn:inactive:expire
```

Paramètres :

- `last_change` : jours depuis 1970 du dernier changement
- `min` : jours minimum entre changements
- `max` : jours maximum avant expiration
- `warn` : jours d'avertissement avant expiration

Le fichier /etc/group 👥

Définit les groupes et leurs membres

```
users:x:100:
developers:x:1010:alice,bob,charlie
admins:x:1020:alice
docker:x:999:alice,bob
```

Format : `group_name:x:GID:member_list`

Pourquoi des groupes ?

- Simplifier la gestion des permissions
- Organiser les utilisateurs par projet/département
- Partager l'accès à des ressources

Créer un utilisateur : useradd 🆕

```
# Méthode simple
sudo useradd alice

# Méthode complète
sudo useradd -m -s /bin/bash -c "Alice Martin" -G sudo,developers alice

# Options importantes :
# -m : créer le home directory
# -s : définir le shell
# -c : commentaire (nom complet)
# -G : groupes supplémentaires
# -d : spécifier le home directory
# -u : spécifier l'UID
```

Créer un utilisateur : adduser 🆕

Alternative interactive (Debian/Ubuntu) :

```
sudo adduser alice
```

Cette commande :

- Crée l'utilisateur
- Crée le home directory
- Demande le mot de passe
- Demande les informations (nom, téléphone, etc.)
- Configure les permissions

Plus simple pour les débutants !

Définir un mot de passe 🔑

```
# Définir le mot de passe d'un utilisateur
sudo passwd alice

# Changer son propre mot de passe
passwd

# Forcer le changement au prochain login
sudo passwd -e alice

# Verrouiller un compte
sudo passwd -l alice

# Déverrouiller un compte
sudo passwd -u alice
```

Politique de mot de passe fort 💪

Bonnes pratiques :

- ✅ Au moins 12 caractères
- ✅ Mélange majuscules/minuscules
- ✅ Chiffres et caractères spéciaux
- ✅ Pas de mots du dictionnaire
- ✅ Pas d'informations personnelles

Outils :

```
# Générer un mot de passe aléatoire
pwgen 16 1

# Ou avec openssl
openssl rand -base64 16
```

Modifier un utilisateur : usermod 🔧

```
# Changer le shell
sudo usermod -s /bin/zsh alice

# Changer le home directory
sudo usermod -d /home/newdir alice

# Ajouter à des groupes supplémentaires
sudo usermod -aG docker,sudo alice

# Changer l'UID
sudo usermod -u 2000 alice

# Changer le nom d'utilisateur
sudo usermod -l newalice alice

# Verrouiller le compte
sudo usermod -L alice

# Déverrouiller
sudo usermod -U alice
```

⚠️ Option -aG vs -G

Différence CRITIQUE :

```
# MAUVAIS : remplace TOUS les groupes
sudo usermod -G docker alice
# → alice n'est plus que dans 'docker'

# BON : ajoute aux groupes existants
sudo usermod -aG docker alice
# → alice garde ses groupes ET gagne 'docker'
```

Pensez-y comme :

- `-G` : réinitialiser la liste
- `-aG` : ajouter (append)

Supprimer un utilisateur : userdel 🗑️

```
# Supprimer seulement le compte
sudo userdel alice

# Supprimer le compte ET le home directory
sudo userdel -r alice

# Vérifier avant de supprimer
ls -la /home/alice
ps -u alice  # Vérifier si des processus tournent
```

⚠️ **Attention :**

- Les fichiers appartenant à l'utilisateur ailleurs restent
- Les processus en cours doivent être arrêtés

Gérer les groupes 👥

Créer un groupe :

```
sudo groupadd developers
sudo groupadd -g 2000 project_team  # Avec GID spécifique
```

Modifier un groupe :

```
sudo groupmod -n newname oldname     # Renommer
sudo groupmod -g 3000 developers     # Changer GID
```

Supprimer un groupe :

```
sudo groupdel developers
```

Gérer l'appartenance aux groupes

```
# Ajouter un utilisateur à un groupe
sudo usermod -aG developers alice
sudo gpasswd -a alice developers     # Alternative

# Retirer un utilisateur d'un groupe
sudo gpasswd -d alice developers

# Lister les groupes d'un utilisateur
groups alice
id alice

# Changer temporairement de groupe principal
newgrp developers
```

Groupes primaires vs secondaires 🔀

Groupe primaire (primary group) :

- Un seul par utilisateur
- Défini dans `/etc/passwd`
- Les nouveaux fichiers créés lui appartiennent

Groupes secondaires (supplementary groups) :

- Multiples possibles
- Listés dans `/etc/group`
- Donnent des accès supplémentaires

```
id alice
# uid=1001(alice) gid=1001(alice) groups=1001(alice),27(sudo),999(docker)
#                    ↑ primaire          ↑ secondaires
```

Revenir au sommaire

Permissions Unix : les bases 🔐

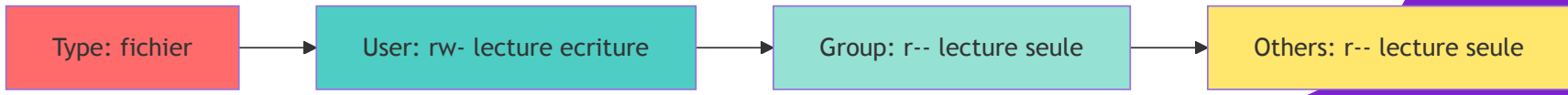
3 types d'actions :

- **r** (read) : lire
- **w** (write) : écrire
- **x** (execute) : exécuter

3 catégories d'utilisateurs :

- **u** (user) : propriétaire
- **g** (group) : groupe
- **o** (others) : autres

Exemple : `-rw-r--r-- 1 alice developers 1024 Jan 10 10:30 fichier.txt`



Permissions en notation octale 🔢

Chaque trio = un chiffre :

```
r (4) + w (2) + x (1) = 7
r (4) + w (2)         = 6
r (4)         + x (1) = 5
r (4)                 = 4
        w (2) + x (1) = 3
        w (2)         = 2
                x (1) = 1
                      = 0
```

Exemples courants :

- `644` : rw-r–r-- (fichier texte)
- `755` : rwxr-xr-x (script exécutable)
- `700` : rwx------ (fichier privé)
- `777` : rwxrwxrwx (tout le monde peut tout faire - DANGEREUX)

Changer les permissions : chmod 🔧

Notation symbolique :

```
chmod u+x script.sh          # Ajouter exécution pour user
chmod g-w fichier.txt        # Retirer écriture pour group
chmod o-r secret.txt         # Retirer lecture pour others
chmod a+r public.txt         # Ajouter lecture pour all
chmod u=rwx,g=rx,o=r file    # Définir explicitement
```

Changer les permissions : chmod (suite) 🔧

Notation octale :

```
chmod 644 fichier.txt        # rw-r--r--
chmod 755 script.sh          # rwxr-xr-x
chmod 700 private.txt        # rwx------
chmod 600 ~/.ssh/id_rsa      # rw------- (clé SSH privée)
```

Récursif :

```
chmod -R 755 /var/www/html   # Applique à tous les fichiers
```

Permissions sur les répertoires 📁

Particularités des répertoires :

- **r** : lister le contenu (ls)
- **w** : créer/supprimer des fichiers dedans
- **x** : traverser (cd) le répertoire

Exemple pratique :

```
# Répertoire accessible mais non listable
chmod 711 /home/alice/secret
# → On peut aller dedans si on connaît le nom,
#   mais on ne peut pas lister son contenu
```

Changer propriétaire et groupe : chown/chgrp

```
# Changer le propriétaire
sudo chown alice fichier.txt

# Changer propriétaire et groupe
sudo chown alice:developers fichier.txt

# Seulement le groupe
sudo chgrp developers fichier.txt

# Récursif
sudo chown -R alice:developers /projet

# Préserver le groupe actuel avec :
sudo chown alice: fichier.txt
```

umask : permissions par défaut 🎭

umask : masque qui définit les permissions des nouveaux fichiers/dossiers

Le problème : quand on crée un fichier, quelles permissions lui donner ?

La solution : umask définit automatiquement les permissions par défaut

Important : umask **retire** des permissions, il ne les ajoute pas !

Analogie : le moule à gâteau 🧁

Imaginez umask comme un moule à gâteau :

Sans moule (umask 0000) :

- Le gâteau déborde partout
- Fichiers : 666 (rw-rw-rw-) → tout le monde peut écrire ⚠️
- Dangereux !

Avec un moule (umask 0022) :

- Le moule bloque certaines parties
- Fichiers : 644 (rw-r–r–) → seul le propriétaire peut écrire ✅
- Sécurisé !

Le moule définit la forme finale en bloquant ce qu'on ne veut pas

Autre analogie : le filtre de confidentialité 🔒

Pensez à un filtre de confidentialité sur un écran :

Écran sans filtre (umask 0000) :

- Tout le monde voit tout
- Permissions maximales

Écran avec filtre (umask 0022) :

- Certaines personnes ne peuvent que regarder, pas toucher
- Permissions contrôlées

Filtre opaque (umask 0077) :

- Personne d'autre ne peut rien voir ni toucher
- Permissions minimales

Comment fonctionne umask ?

umask fonctionne par SOUSTRACTION :

```
Permissions maximales - umask = Permissions finales
```

Permissions maximales par défaut :

- **Fichiers** : 666 (rw-rw-rw-)
- **Dossiers** : 777 (rwxrwxrwx)

umask retire des permissions à ces valeurs maximales

Voir le umask actuel 👀

```
# Afficher le umask actuel
umask
```

Résultat de umask

```
# Résultat typique :
0022
```

Lecture du nombre : 0022

- Premier 0 : permissions spéciales (on l'ignore généralement)
- 0 : permissions retirées au **propriétaire** (user)
- 2 : permissions retirées au **groupe** (group)
- 2 : permissions retirées aux **autres** (others)

Calcul détaillé : fichiers

Exemple avec umask 0022 :

```
 Fichier : 666 (rw-rw-rw-)
umask :  -022
         ----
Résultat: 644 (rw-r--r--)
```

Détail du calcul :

- **User** : 6 - 0 = 6 (rw-)
- **Group** : 6 - 2 = 4 (r–)
- **Others** : 6 - 2 = 4 (r–)

Résultat : rw-r–r– = Le propriétaire peut lire/écrire, les autres seulement lire

Calcul détaillé : dossiers 📁

Exemple avec umask 0022 :

```
 Dossier : 777 (rwxrwxrwx)
umask :  -022
         ----
Résultat: 755 (rwxr-xr-x)
```

Détail du calcul :

- **User** : 7 - 0 = 7 (rwx)
- **Group** : 7 - 2 = 5 (r-x)
- **Others** : 7 - 2 = 5 (r-x)

Résultat : rwxr-xr-x = Le propriétaire a tous les droits, les autres peuvent lire et entrer

Tableau de conversion umask 📊

umask	Fichiers (666-umask)	Dossiers (777-umask)	Description
0000	666 (rw-rw-rw-)	777 (rwxrwxrwx)	Tout permis ⚠️
0002	664 (rw-rw-r–)	775 (rwxrwxr-x)	Groupe = user
0022	644 (rw-r–r–)	755 (rwxr-xr-x)	Standard ✅
0027	640 (rw-r-----)	750 (rwxr-x—)	Restrictif
0077	600 (rw-------)	700 (rwx------)	Privé 🔒

Test pratique : umask 0022 🧪

```
# Vérifier le umask actuel
umask
```

Résultat du test (umask 0022)

```
# Résultat
0022
```

Créer un fichier test

```
# Créer un fichier
touch test1.txt

# Voir ses permissions
ls -l test1.txt
```

Permissions obtenues

```
-rw-r--r-- 1 user group 0 Dec  2 10:00 test1.txt
```

Explication :

- **rw-** : propriétaire peut lire et écrire
- **r–** : groupe peut seulement lire
- **r–** : autres peuvent seulement lire

C’est exactement 644 = 666 - 022 ✅

Test pratique : umask 0077 🔐

```
# Changer le umask (très restrictif)
umask 0077

# Créer un nouveau fichier
touch test2.txt

# Voir ses permissions
ls -l test2.txt
```

Résultat avec umask 0077

```
-rw------- 1 user group 0 Dec  2 10:00 test2.txt
```

Explication :

- **rw-** : propriétaire peut lire et écrire
- **—** : groupe ne peut RIEN faire
- **—** : autres ne peuvent RIEN faire

C'est exactement 600 = 666 - 077 ✅

Fichier totalement privé ! 🔒

Revenir au sommaire

Cas d'usage : umask 0022 (défaut) 🏢

Situation : environnement de bureau normal

```
umask 0022
```

Résultat :

- Fichiers : 644 (rw-r–r–)
- Dossiers : 755 (rwxr-xr-x)

Avantages :

- ✅ Vous pouvez modifier vos fichiers
- ✅ Les autres peuvent les lire (partage facile)
- ✅ Les autres ne peuvent pas les modifier (sécurité)

Idéal pour : postes de travail, serveurs web

Cas d'usage : umask 0027 (restrictif) 🔐

Situation : travail en équipe avec groupes

```
umask 0027
```

Résultat :

- Fichiers : 640 (rw-r-----)
- Dossiers : 750 (rwxr-x—)

Avantages :

- ✅ Vous avez tous les droits
- ✅ Votre groupe peut lire
- ✅ Les autres ne voient RIEN

Idéal pour : projets d'équipe, données sensibles partagées

Cas d'usage : umask 0077 (privé) 🚫

Situation : données personnelles sensibles

```
umask 0077
```

Résultat :

- Fichiers : 600 (rw-------)
- Dossiers : 700 (rwx------)

Avantages :

- ✅ Vous seul pouvez accéder
- ✅ Sécurité maximale
- ❌ Aucun partage possible

Idéal pour : clés SSH, mots de passe, données bancaires

Modifier le umask temporairement ⏱️

```
# Changer pour la session actuelle
umask 0027
```

Vérifier le changement

```
# Vérifier
umask
```

Résultat de la vérification

```
# Résultat
0027
```

Créer un fichier avec le nouveau umask

```
# Créer un fichier
touch test3.txt

# Vérifier les permissions
ls -l test3.txt
```

Permissions avec umask 0027

```
-rw-r----- 1 user group 0 Dec  2 10:00 test3.txt
```

Le groupe peut lire, mais pas les autres !

⚠️ **ATTENTION :** ce changement est temporaire

- Valable uniquement pour cette session
- Disparaît quand vous fermez le terminal

Modifier le umask de façon permanente 💾

Pour votre utilisateur uniquement :

```
# Ouvrir votre fichier de configuration
nano ~/.bashrc
```

Ajouter la ligne umask

```
# Ajouter à la fin du fichier
umask 0027
```

Activer les changements

```
# Recharger la configuration
source ~/.bashrc

# Vérifier
umask
```

Modifier le umask système 🌍

Pour tous les utilisateurs :

```
# Éditer le fichier système
sudo nano /etc/profile
```

Ajouter umask dans /etc/profile

```
# Ajouter à la fin
umask 0022
```

Cas pratiques complets 💼

Scénario 1 : Développeur freelance

```
# Fichiers privés par défaut
umask 0077

# Créer un projet
mkdir mon-projet
touch mon-projet/code.py

# Résultat : personne d'autre ne peut voir
ls -ld mon-projet        # drwx------
ls -l mon-projet/code.py # -rw-------
```

Cas pratiques (suite) 💼

Scénario 2 : Équipe de développement

```
# Partage avec le groupe
umask 0027

# Créer un projet d'équipe
mkdir projet-equipe
touch projet-equipe/README.md

# Résultat : le groupe peut collaborer
ls -ld projet-equipe          # drwxr-x---
ls -l projet-equipe/README.md # -rw-r-----
```

Cas pratiques (suite) 💼

Scénario 3 : Serveur web public

```
# Fichiers lisibles par tous
umask 0022

# Créer un site web
mkdir /var/www/html/site
touch /var/www/html/site/index.html

# Résultat : tout le monde peut lire
ls -ld /var/www/html/site            # drwxr-xr-x
ls -l /var/www/html/site/index.html  # -rw-r--r--
```

Comprendre les valeurs en octal 🔢

Rappel des valeurs :

- **4** = r (read)
- **2** = w (write)
- **1** = x (execute)

Quand umask retire 2 :

- Retire uniquement le droit d'écriture (w)
- 6 - 2 = 4 : reste seulement la lecture (r–)
- 7 - 2 = 5 : reste lecture + exécution (r-x)

Exemples de calcul pas à pas 🧮

umask 0002 :

```
 Fichier : 666 - 002 = 664
User   : 6 - 0 = 6 (rw-)
Group  : 6 - 0 = 6 (rw-)
Others : 6 - 2 = 4 (r--)
→ Groupe a les mêmes droits que le propriétaire
```

Exemples de calcul pas à pas (suite)

umask 0077 :

```
 Fichier : 666 - 077 = 600
User   : 6 - 0 = 6 (rw-)
Group  : 6 - 7 = 0 (---)  [7 retire tout]
Others : 6 - 7 = 0 (---)  [7 retire tout]
→ Fichier totalement privé
```

Erreurs courantes ❌

Erreur 1 : Penser que umask AJOUTE des permissions

```
# FAUX : umask 0777 ne donne pas tous les droits
# VRAI : umask 0777 retire tout !
umask 0777  # Fichiers = 666-777 = ???  (invalide)
```

Erreur 2 : Confondre umask et chmod

- **umask** : définit les permissions PAR DÉFAUT des NOUVEAUX fichiers
- **chmod** : modifie les permissions de fichiers EXISTANTS

Vérifier l'impact du umask 🔍

```
# Méthode complète pour tester
echo "Mon umask actuel :"
umask

echo -e "\nTest fichier :"
touch /tmp/test_umask.txt
ls -l /tmp/test_umask.txt

echo -e "\nTest dossier :"
mkdir /tmp/test_umask_dir
ls -ld /tmp/test_umask_dir

# Nettoyer
rm /tmp/test_umask.txt
rmdir /tmp/test_umask_dir
```

Récapitulatif umask

À retenir :

- ✅ umask **retire** des permissions (c'est un masque)
- ✅ Calcul : `666 - umask` pour fichiers, `777 - umask` pour dossiers
- ✅ umask 0022 : standard (propriétaire écrit, autres lisent)
- ✅ umask 0077 : privé (seul le propriétaire accède)
- ✅ Temporaire : `umask 0027` dans le terminal
- ✅ Permanent : ajouter `umask 0027` dans `~/.bashrc`

Revenir au sommaire

sudo : exécuter en tant que root 🔑

sudo : "**S**uper**U**ser **DO**"

Le problème :

- Certaines commandes nécessitent les droits root (administrateur)
- Se connecter en root est dangereux (une erreur = catastrophe)

La solution :

- `sudo` permet d'exécuter **une commande** avec les droits root
- Traçabilité : on sait qui a fait quoi
- Sécurité : pas besoin du mot de passe root

Pourquoi sudo plutôt que root ? 🤔

Se connecter en root directement :

```
su -        # Mauvaise pratique ❌
```

Problèmes :

- ❌ Vous êtes root pour TOUT (dangereux)
- ❌ Pas de trace de qui a fait quoi
- ❌ Facile d'oublier qu'on est root
- ❌ Une erreur peut détruire le système

Avantages de sudo ✅

Utiliser sudo :

```
sudo apt update    # Bonne pratique ✅
```

Avantages :

- ✅ Root uniquement pour cette commande
- ✅ Traçabilité complète dans `/var/log/auth.log`
- ✅ On reste utilisateur normal après
- ✅ Peut limiter les commandes autorisées
- ✅ Timeout de session (15 min par défaut)

Analogie : le badge d'accès 🏢

Imaginez un immeuble de bureaux :

Root permanent (su -) :

- Badge administrateur H24
- Accès à TOUTES les pièces en permanence
- Si vous perdez le badge = catastrophe

Sudo :

- Vous demandez l'accès temporaire à une pièce
- Le gardien note votre passage
- Badge rendu automatiquement après 15 min
- Si problème, on sait qui a fait quoi

Utilisation basique de sudo

```
# Exécuter une commande en root
sudo apt update
```

Commandes sudo avancées 🚀

```
# Ouvrir un shell root temporaire (déconseillé mais parfois utile)
sudo -i        # Shell root avec environnement root
sudo -s        # Shell root avec votre environnement
```

Commandes sudo avancées (suite) 🚀

```
# Exécuter en tant qu'un autre utilisateur
sudo -u alice cat /home/alice/fichier.txt
sudo -u www-data ls /var/www/html
```

Commandes sudo avancées (suite) 🚀

```
# Éditer un fichier de façon sécurisée
sudoedit /etc/hosts    # Préférable à "sudo nano"
```

Commandes sudo avancées (suite) 🚀

```
# Relancer sudo sans retaper le mot de passe
sudo -v     # Prolonge la session sudo de 15 min
```

Commandes sudo avancées (suite) 🚀

```
# Voir ce qu'on peut faire avec sudo
sudo -l
```

Comment donner les droits sudo à un utilisateur ? 👤

Deux méthodes principales :

1. **Méthode SIMPLE** : ajouter au groupe `sudo` ou `wheel` ✅
   - Recommandé pour la majorité des cas
   - Rapide et sûr
   - L'utilisateur peut tout faire avec sudo
2. **Méthode AVANCÉE** : éditer `/etc/sudoers` 🔧
   - Pour des permissions spécifiques
   - Limiter certaines commandes
   - Configuration fine

On commence par la méthode simple !

Méthode 1 : Ajouter au groupe sudo (Ubuntu/Debian) 👥

Sur Ubuntu/Debian, le groupe magique est `sudo`

```
# Ajouter alice au groupe sudo
sudo usermod -aG sudo alice
```

Vérifier l'ajout au groupe

```
# Vérifier que alice est dans le groupe sudo
groups alice
```

Résultat de la vérification

```
alice : alice sudo
```

Alice doit se déconnecter/reconnecter pour que ça prenne effet !

Test des droits sudo

```
# Alice peut maintenant utiliser sudo
su - alice
sudo apt update
# → Demande le mot de passe d'alice
# → Fonctionne ! ✅
```

Méthode 1 : Ajouter au groupe wheel (RHEL/Fedora)

Sur RHEL/Fedora/AlmaLinux, le groupe magique est `wheel`

```
# Ajouter bob au groupe wheel
sudo usermod -aG wheel bob
```

Vérifier l'ajout au groupe wheel

```
# Vérifier que bob est dans le groupe wheel
groups bob
```

Résultat de la vérification (wheel)

```
bob : bob wheel
```

Bob doit se déconnecter/reconnecter !

Différences entre distributions

Distribution	Groupe sudo
Ubuntu	`sudo`
Debian	`sudo`
Linux Mint	`sudo`
RHEL	`wheel`
AlmaLinux	`wheel`
Rocky Linux	`wheel`
Fedora	`wheel`

Attention : chercher dans `/etc/sudoers` pour voir quel groupe est configuré

Revenir au sommaire

Retirer les droits sudo 🚫

```
# Retirer alice du groupe sudo (Ubuntu/Debian)
sudo gpasswd -d alice sudo
```

Retirer les droits sudo (RHEL/Fedora) 🚫

```
# Retirer bob du groupe wheel (RHEL/Fedora)
sudo gpasswd -d bob wheel
```

Méthode 2 : Éditer /etc/sudoers (avancé) 🔧

Quand utiliser cette méthode ?

✅ Limiter les commandes autorisées ✅ Donner sudo sans mot de passe (pour l'automatisation) ✅ Donner sudo seulement pour certains programmes ✅ Configuration plus fine et granulaire

⚠️ **Attention : toujours utiliser** `visudo` **pour éditer !**

Pourquoi visudo ? 🤔

Fichier : `/etc/sudoers`

Si vous faites une erreur de syntaxe dans ce fichier :

- 💥 PLUS PERSONNE ne peut utiliser sudo
- 💥 Vous êtes bloqué hors du système
- 💥 Impossible de réparer (vous ne pouvez plus éditer le fichier !)

visudo = éditeur sécurisé :

- ✅ Vérifie la syntaxe avant de sauvegarder
- ✅ Bloque l'enregistrement si erreur
- ✅ Évite les catastrophes

Ouvrir /etc/sudoers avec visudo

```
# TOUJOURS utiliser visudo
sudo visudo
```

Ne JAMAIS faire ça ❌

```
# DANGEREUX - Ne JAMAIS faire ça !
sudo nano /etc/sudoers    ❌
sudo vim /etc/sudoers     ❌

# Toujours utiliser visudo
sudo visudo               ✅
```

Format de base de /etc/sudoers 📋

```
utilisateur HOSTS=(UTILISATEURS_CIBLES) COMMANDES
```

Décomposition :

- **utilisateur** : qui peut utiliser sudo
- **HOSTS** : depuis quelles machines (généralement ALL)
- **UTILISATEURS_CIBLES** : en tant que qui (généralement ALL = root)
- **COMMANDES** : quelles commandes (ALL = tout)

Exemple 1 : Tous les droits sudo 👑

```
# Alice peut tout faire avec sudo
alice ALL=(ALL:ALL) ALL
```

Explication de la syntaxe

```
alice ALL=(ALL:ALL) ALL
  ↓     ↓    ↓   ↓   ↓
  |     |    |   |   └─ Toutes les commandes
  |     |    |   └──── En tant que n'importe quel groupe
  |     |    └──────── En tant que n'importe quel utilisateur
  |     └───────────── Depuis n'importe quelle machine
  └─────────────────── L'utilisateur alice
```

Résultat : Alice peut utiliser sudo exactement comme les membres du groupe sudo

Exemple 2 : sudo sans mot de passe 🔓

```
# Bob peut tout faire SANS mot de passe (automatisation)
bob ALL=(ALL) NOPASSWD: ALL
```

Exemple 2 : Cas d'usage

Quand utiliser NOPASSWD ?

✅ Scripts d'automatisation ✅ Tâches cron qui nécessitent sudo ✅ Déploiement CI/CD ✅ Ansible/Puppet/Chef

⚠️ **Attention : très risqué pour un humain !**

Exemple 3 : Limiter à certaines commandes 🎯

```
# Charlie peut SEULEMENT installer des paquets
charlie ALL=(ALL) /usr/bin/apt, /usr/bin/apt-get
```

Exemple 3 : Utilisation

```
# Charlie peut faire ça :
sudo apt install nginx          ✅
sudo apt update                 ✅

# Mais PAS ça :
sudo systemctl restart nginx    ❌
sudo rm -rf /                   ❌
sudo useradd hacker             ❌
```

Sécurité : Charlie ne peut faire QUE ce qui est autorisé

Exemple 4 : Redémarrer des services 🔄

```
# David peut redémarrer apache et nginx
david ALL=(ALL) /bin/systemctl restart apache2, \
                /bin/systemctl restart nginx, \
                /bin/systemctl status *
```

Exemple 4 : Utilisation

```
# David peut faire ça :
sudo systemctl restart apache2   ✅
sudo systemctl restart nginx     ✅
sudo systemctl status mysql      ✅

# Mais PAS ça :
sudo systemctl stop apache2      ❌
sudo systemctl disable nginx     ❌
```

Exemple 5 : Groupe avec permissions 👥

```
# Tous les membres du groupe "ops" peuvent tout faire
%ops ALL=(ALL:ALL) ALL
```

Exemple 5 : Explication

Le symbole % indique un groupe

```
%ops        # Le groupe "ops"
%sudo       # Le groupe "sudo"
%wheel      # Le groupe "wheel"
```

Tous les membres du groupe héritent des permissions

Exemple 6 : Groupe sans mot de passe 🚀

```
# Le groupe "deployers" peut tout sans mot de passe
%deployers ALL=(ALL) NOPASSWD: ALL
```

Exemple 6 : Cas d'usage

Créer un groupe pour l'automatisation :

```
# Créer le groupe
sudo groupadd deployers

# Ajouter l'utilisateur de déploiement
sudo usermod -aG deployers jenkins

# Configurer dans /etc/sudoers avec visudo
%deployers ALL=(ALL) NOPASSWD: ALL
```

Exemple 7 : Exécuter en tant qu'un autre utilisateur 🎭

```
# Emma peut exécuter des commandes en tant que www-data
emma ALL=(www-data) ALL
```

Exemple 7 : Utilisation

```
# Emma peut faire ça :
sudo -u www-data ls /var/www/html          ✅
sudo -u www-data cat /var/www/html/index.php ✅

# Mais PAS ça (nécessite root, pas www-data) :
sudo systemctl restart apache2             ❌
```

Configuration par fichiers séparés 📁

Bonne pratique : ne pas éditer `/etc/sudoers` **directement**

Utiliser le dossier `/etc/sudoers.d/` **:**

```
# Créer un fichier pour l'utilisateur alice
sudo visudo -f /etc/sudoers.d/alice
```

Contenu du fichier séparé

```
# Fichier: /etc/sudoers.d/alice
alice ALL=(ALL:ALL) ALL
```

Avantages des fichiers séparés ✅

Pourquoi utiliser `/etc/sudoers.d/` **?**

✅ Organisation : un fichier par utilisateur/groupe ✅ Facilite la gestion avec Ansible/Puppet ✅ Pas de risque de casser le fichier principal ✅ Facile à supprimer un utilisateur ✅ Meilleure traçabilité des changements

Exemple de fichier pour un groupe 👥

```
# Créer un fichier pour le groupe ops
sudo visudo -f /etc/sudoers.d/ops
```

Contenu du fichier groupe

```
# Fichier: /etc/sudoers.d/ops
%ops ALL=(ALL) /bin/systemctl restart *, \
               /bin/systemctl status *, \
               /usr/bin/journalctl
```

Supprimer les droits sudo (fichier séparé) 🗑️

```
# Retirer les droits de alice
sudo rm /etc/sudoers.d/alice

# Vérifier
sudo visudo -c    # Vérifie la syntaxe de tous les fichiers
```

Voir ses permissions sudo 🔍

```
# Voir ce que VOUS pouvez faire avec sudo
sudo -l
```

Exemple de sortie de sudo -l

```
User alice may run the following commands on server:
    (ALL : ALL) ALL
```

Alice peut tout faire !

Voir les permissions d'un autre utilisateur 👀

```
# Voir ce que bob peut faire (en tant que root)
sudo -l -U bob
```

Exemple de sortie pour bob

```
User bob may run the following commands on server:
    (ALL) /usr/bin/apt, /usr/bin/apt-get
```

Bob peut seulement utiliser apt !

Logs sudo : traçabilité 📜

Tous les usages de sudo sont enregistrés !

```
# Voir les logs sudo (Ubuntu/Debian)
sudo cat /var/log/auth.log | grep sudo
```

Logs sudo (RHEL/Fedora) 📜

```
# Voir les logs sudo (RHEL/Fedora)
sudo journalctl | grep sudo
```

Exemple de log sudo

```
Dec  2 10:30:15 server sudo: alice : TTY=pts/0 ; PWD=/home/alice ;
  USER=root ; COMMAND=/usr/bin/apt update

Dec  2 10:32:45 server sudo: bob : command not allowed ;
  COMMAND=/bin/systemctl restart apache2
```

Traçabilité complète : qui, quand, quoi, succès ou échec

Sécurité : bonnes pratiques 🔐

✅ À FAIRE :

1. Toujours utiliser `visudo` pour éditer la configuration
2. Ajouter au groupe sudo/wheel pour les admins de confiance
3. Limiter les commandes pour les utilisateurs spécifiques
4. Surveiller les logs régulièrement
5. Utiliser `/etc/sudoers.d/` pour l'organisation

Sécurité : bonnes pratiques (suite) 🔐

❌ À ÉVITER :

1. Ne JAMAIS éditer `/etc/sudoers` avec nano/vim directement
2. Éviter `NOPASSWD: ALL` pour des humains
3. Ne pas donner `ALL=(ALL:ALL) ALL` sans réfléchir
4. Ne pas oublier de tester la configuration
5. Ne pas laisser trainer des sessions sudo ouvertes

Revenir au sommaire

Récapitulatif : quand utiliser quelle méthode ? 🤔

Méthode 1 : Ajouter au groupe sudo/wheel

```
sudo usermod -aG sudo alice
```

✅ Pour donner tous les droits sudo ✅ Rapide et simple ✅ Recommandé pour les admins système

Récapitulatif : quand utiliser quelle méthode ? (suite) 🤔

Méthode 2 : Éditer /etc/sudoers

```
sudo visudo
```

✅ Pour limiter à certaines commandes ✅ Pour des configurations spécifiques ✅ Pour l'automatisation (NOPASSWD) ✅ Pour des besoins particuliers

Cas pratique complet : nouvel admin 💼

```
# Créer l'utilisateur
sudo useradd -m -s /bin/bash john

# Définir son mot de passe
sudo passwd john

# Lui donner les droits sudo (méthode simple)
sudo usermod -aG sudo john    # Ubuntu/Debian
# OU
sudo usermod -aG wheel john   # RHEL/Fedora

# Vérifier
groups john
sudo -l -U john
```

Cas pratique complet : utilisateur limité 💼

```
# Créer un utilisateur pour gérer le web
sudo useradd -m -s /bin/bash webadmin

# Donner des permissions spécifiques
sudo visudo -f /etc/sudoers.d/webadmin
```

Configuration pour webadmin

```
# Fichier: /etc/sudoers.d/webadmin
webadmin ALL=(ALL) /bin/systemctl restart apache2, \
                   /bin/systemctl restart nginx, \
                   /bin/systemctl reload apache2, \
                   /bin/systemctl reload nginx, \
                   /bin/systemctl status apache2, \
                   /bin/systemctl status nginx, \
                   /usr/sbin/a2ensite *, \
                   /usr/sbin/a2dissite *
```

Tester la configuration sudo ✅

```
# Vérifier la syntaxe de tous les fichiers sudoers
sudo visudo -c
```

Résultat du test

```
/etc/sudoers: parsed OK
/etc/sudoers.d/alice: parsed OK
/etc/sudoers.d/webadmin: parsed OK
```

Si erreur, visudo indique la ligne problématique !

Récapitulatif sudo

Ce qu'on a appris :

- ✅ Pourquoi utiliser sudo plutôt que root
- ✅ **Méthode simple** : ajouter au groupe sudo (Ubuntu) ou wheel (RHEL)
- ✅ **Méthode avancée** : éditer `/etc/sudoers` avec `visudo`
- ✅ Différences entre distributions
- ✅ Limiter les commandes autorisées
- ✅ NOPASSWD pour l'automatisation
- ✅ Traçabilité et logs
- ✅ Bonnes pratiques de sécurité

Groupes système importants 🎖️

sudo/wheel : administration système

- Peut utiliser sudo

docker : utiliser Docker sans sudo

- Accès au socket Docker

www-data : serveur web

- Propriétaire des fichiers web

systemd-journal : lire les logs systemd

adm : lire les logs dans /var/log

Quotas disque : pourquoi ? 💾

Problème : Un utilisateur remplit le disque → tout le système crashe

Solution : les quotas

- Limiter l'espace disque par utilisateur/groupe
- Limiter le nombre de fichiers (inodes)

Analogie : le forfait mobile 📱

- **Sans quota** : utilisation illimitée → un utilisateur peut tout consommer
- **Avec quota** : chacun a son forfait (10 Go par exemple)
- **Soft limit** : avertissement à 8 Go (tu vas bientôt dépasser)
- **Hard limit** : blocage à 10 Go (c'est fini, tu ne peux plus rien télécharger)

Exemple concret :

Imaginez une entreprise avec 100 employés et un serveur de 1 To :

- Sans quotas : un employé peut stocker 500 Go de vidéos personnelles
- Avec quotas : chacun a droit à 8 Go → espace équitablement réparti

Activer les quotas

1. Installer les outils :

```
sudo apt install quota quotatool    # Debian/Ubuntu
sudo dnf install quota              # RHEL/Fedora
```

Activer les quotas (suite)

2. Modifier /etc/fstab :

```
sudo nano /etc/fstab
```

Ajouter `usrquota,grpquota` :

```
/dev/sda1 / ext4 defaults,usrquota,grpquota 0 1
```

Revenir au sommaire

Activer les quotas (suite)

3. Remonter et initialiser :

```
sudo mount -o remount /
sudo quotacheck -cugm /
sudo quotaon /
```

Définir des quotas 📏

```
# Éditer les quotas d'un utilisateur
sudo edquota alice

# Interface interactive :
# Disk quotas for user alice (uid 1001):
#   Filesystem  blocks   soft   hard  inodes  soft  hard
#   /dev/sda1      100   5000  10000      50   500  1000

# blocks : kilo-octets utilisés
# soft : avertissement
# hard : limite absolue
```

Quotas : soft vs hard 🎚️

Soft limit (souple) :

- Peut être dépassé temporairement
- Période de grâce (grace period) : 7 jours par défaut
- Avertissement à l'utilisateur

Hard limit (dure) :

- Ne peut JAMAIS être dépassé
- Écriture refusée immédiatement

Analogie :

- Soft = "Tu as presque fini tes 50 Go, supprime des trucs"
- Hard = "C'est fini, tu ne peux plus rien ajouter"

Gérer les quotas en ligne de commande

```
# Définir un quota directement
sudo setquota -u alice 1000000 2000000 0 0 /

# Vérifier les quotas d'un utilisateur
sudo quota -u alice

# Voir un rapport de tous les utilisateurs
sudo repquota -a

# Désactiver les quotas
sudo quotaoff /

# Réactiver
sudo quotaon /
```

Structure détaillée du système de fichiers 📂

Rappel des principaux répertoires :

- `/bin` : binaires essentiels (ls, cat, cp, bash)
- `/sbin` : binaires système (fsck, reboot)
- `/usr/bin` : programmes utilisateur
- `/usr/sbin` : programmes admin

Note moderne : sur beaucoup de distributions, `/bin` , `/sbin` , `/lib` sont des liens symboliques vers `/usr/bin` , `/usr/sbin` , `/usr/lib`

/etc : configuration 📋

Fichiers de configuration système

```
/etc/
├── passwd              # Utilisateurs
├── shadow              # Mots de passe
├── group               # Groupes
├── sudoers             # Configuration sudo
├── fstab               # Montages automatiques
├── hostname            # Nom de la machine
├── hosts               # Résolution de noms locale
├── resolv.conf         # Serveurs DNS
├── network/            # Configuration réseau
├── ssh/                # Configuration SSH
├── systemd/            # Configuration systemd
└── apt/                # Configuration apt (Debian/Ubuntu)
```

/var : données variables 📈

Fichiers qui changent fréquemment

```
/var/
├── log/                # Logs système et applications
│   ├── syslog
│   ├── auth.log
│   └── kern.log
├── cache/              # Caches d'applications
├── tmp/                # Fichiers temporaires (persistants)
├── spool/              # Files d'attente (impression, mail)
├── lib/                # Données d'état (bases de données)
└── www/                # Données du serveur web
```

/home : répertoires utilisateurs 🏠

Chaque utilisateur a son espace :

```
/home/
├── alice/
│   ├── Documents/
│   ├── Downloads/
│   ├── .bashrc          # Configuration bash
│   ├── .ssh/            # Clés SSH
│   └── .config/         # Configs d'applications
└── bob/
    └── ...
```

Fichiers cachés (commençant par `.`) :

- Configuration personnelle
- Non affichés par `ls` (mais visibles avec `ls -a`)

/tmp et /var/tmp : temporaires ⏱️

/tmp :

- Effacé à chaque redémarrage
- Accessible à tous (sticky bit)
- Utiliser pour fichiers très temporaires

/var/tmp :

- Persistant entre redémarrages
- Nettoyé périodiquement (selon config)

```
 # Créer un fichier temporaire
mktemp
# → /tmp/tmp.xYz123abc

# Créer un répertoire temporaire
mktemp -d
```

/proc : système de fichiers virtuel

Interface virtuelle avec le noyau

```
cat /proc/cpuinfo       # Informations CPU
cat /proc/meminfo       # Informations mémoire
cat /proc/version       # Version du noyau
cat /proc/uptime        # Temps depuis démarrage
cat /proc/loadavg       # Charge système

# Informations d'un processus
cat /proc/1234/cmdline  # Commande du PID 1234
cat /proc/1234/status   # État du processus
```

Aucun fichier réel sur le disque ! Tout est généré à la volée.

/sys : autre système virtuel ⚡

Interface avec les périphériques et pilotes

```
# Luminosité de l'écran
cat /sys/class/backlight/*/brightness

# Température CPU
cat /sys/class/thermal/thermal_zone*/temp

# État de la batterie
cat /sys/class/power_supply/BAT0/capacity
```

Utile pour scripts d’automatisation et monitoring

Bonnes pratiques de gestion des utilisateurs ✅

1. **Ne jamais travailler en root directement**
   - Toujours utiliser sudo
2. **Principes du moindre privilège**
   - Donner uniquement les droits nécessaires
3. **Groupes plutôt qu'utilisateurs individuels**
   - Plus facile à gérer
4. **Désactiver les comptes inutilisés**
   - `sudo usermod -L ancien_user`

Bonnes pratiques de gestion des utilisateurs (suite) ✅

5. **Auditer régulièrement**

```
# Qui s'est connecté récemment ?
last

# Qui est connecté maintenant ?
who
w

# Historique des commandes sudo
sudo cat /var/log/auth.log | grep sudo
```

6. **Politique de mots de passe forte**
   - Expiration, complexité, longueur

Outils de monitoring des utilisateurs

```
# Connexions récentes
last -10

# Échecs de connexion
lastb

# Utilisateurs connectés
who
w

# Depuis combien de temps un user est inactif
w

# Historique des connexions d'un user
last alice

# Lister tous les utilisateurs réels
awk -F: '$3>=1000 {print $1}' /etc/passwd
```

Sécuriser l'accès root 🔒

Désactiver la connexion root directe :

```
# Dans /etc/ssh/sshd_config
sudo nano /etc/ssh/sshd_config

# Modifier cette ligne :
PermitRootLogin no

# Redémarrer SSH
sudo systemctl restart sshd
```

Forcer l'usage de sudo :

- Traçabilité (qui a fait quoi)
- Double authentification (password user + sudo)
- Limitations par commande possibles

Récapitulatif du Module 4 ✅

Ce que vous avez appris :

- ✅ Gestion des utilisateurs (useradd, usermod, userdel)
- ✅ Gestion des groupes (groupadd, groupmod, gpasswd)
- ✅ Fichiers /etc/passwd, /etc/shadow, /etc/group
- ✅ Permissions Unix (chmod, chown, umask)
- ✅ Sudo et élévation de privilèges
- ✅ Quotas disque
- ✅ Structure détaillée du système de fichiers
- ✅ Bonnes pratiques de sécurité

Prochaine étape 🎯

Module 5 : Stockage et système de fichiers

- Disques et partitions (fdisk, parted)
- Systèmes de fichiers (ext4, xfs, btrfs)
- Signaux Unix
- systemd et gestion des services
- Démarrage et arrêt du système
- Logs et journalctl

Questions ? 🤔

N’hésitez pas à poser vos questions maintenant !

Le prochain module explorera en profondeur la gestion des processus et des services système.

QCM Module 4 : Gestion utilisateurs ✅

10 questions sur les utilisateurs, groupes et permissions

Question 1

Quel fichier contient les informations des utilisateurs ?

A) `/etc/users`

B) `/etc/passwd`

C) `/etc/shadow`

D) `/etc/accounts`

Question 2

Quelle commande crée un nouvel utilisateur ?

A) `adduser` ou `useradd`

B) `newuser`

C) `createuser`

D) `mkuser`

Revenir au sommaire

Question 3

Que signifie `chmod 644 fichier.txt` ?

A) rw-r–r–

B) rwxr–r–

C) rw-rw-r–

D) r–r–r–

Revenir au sommaire

Question 4

Quelle option de `usermod` ajoute un utilisateur à un groupe ?

A) `-g`

B) `-G`

C) `-aG`

D) `-a`

Question 5

Que fait la commande `sudo` ?

A) Connecte en tant que root

B) Exécute une commande avec les privilèges root

C) Crée un nouvel utilisateur

D) Change de groupe

Revenir au sommaire

Question 6

Quel fichier configure les droits sudo ?

A) `/etc/sudo`

B) `/etc/sudoers`

C) `/etc/sudo.conf`

D) `/etc/security/sudo`

Question 7

Que fait `umask 0022` ?

A) Définit les permissions par défaut des nouveaux fichiers

B) Masque les fichiers cachés

C) Crypte les fichiers

D) Change le propriétaire

Question 8

Comment changer le groupe d'un fichier ?

A) `chmod`

B) `chown`

C) `chgrp`

D) B et C sont corrects

Revenir au sommaire

Question 9

Que fait la commande `passwd` ?

A) Affiche les mots de passe

B) Change le mot de passe

C) Crypte les fichiers

D) Liste les utilisateurs

Question 10

Quelle commande liste les groupes d’un utilisateur ?

A) `groups utilisateur`

B) `id utilisateur`

C) `whoami`

D) A et B sont corrects

Revenir au sommaire

Réponses du Module 4 📝

Réponse 1 : B) `/etc/passwd`

- `/etc/passwd` contient les informations utilisateurs (sauf les mots de passe qui sont dans `/etc/shadow`)

Réponse 2 : A) `adduser` ou `useradd`

- `useradd` est bas niveau, `adduser` est interactif (Debian/Ubuntu)

Réponse 3 : A) rw-r–r–

- 6 = rw- (propriétaire), 4 = r-- (groupe), 4 = r-- (autres)

Réponse 4 : C) `-aG`

- `-aG` (append Group) ajoute sans supprimer les groupes existants

Réponse 5 : B) Exécute une commande avec les privilèges root

- `sudo` permet d'élever les privilèges temporairement

Revenir au sommaire

Réponses du Module 4 (suite) 📝

Réponse 6 : B) `/etc/sudoers`

- À éditer UNIQUEMENT avec `visudo`

Réponse 7 : A) Définit les permissions par défaut des nouveaux fichiers

- umask soustrait des permissions : 666 - 022 = 644 pour les fichiers

Réponse 8 : D) B et C sont corrects

- `chgrp groupe fichier` ou `chown :groupe fichier`

Réponse 9 : B) Change le mot de passe

- `passwd` change votre mot de passe, `sudo passwd user` change celui d'un autre utilisateur

Réponse 10 : D) A et B sont corrects

- `groups` et `id` affichent les appartenances aux groupes

Revenir au sommaire

Score du Module 4 📊

Comptez vos bonnes réponses :

- **9-10** : Excellent ! ⭐⭐⭐
- **7-8** : Bien ! ⭐⭐
- **5-6** : À revoir ⭐
- **< 5** : Relire le module 2

Revenir au sommaire

🎯 Exercices pratiques - Module 4

Gestion utilisateurs, groupes et permissions

Exercice 1 : Création d'utilisateurs 👥

Objectif : Créer et gérer des utilisateurs

Consignes :

1. Créez trois utilisateurs : `dev1` , `dev2` , `admin1`
2. Définissez des mots de passe pour chacun
3. Vérifiez leur présence dans `/etc/passwd`
4. Affichez les informations de `dev1` avec `id`
5. Listez les dernières connexions avec `last`

Solution Exercice 1 💡

```
# 1. Créer les utilisateurs
sudo useradd -m -s /bin/bash dev1
sudo useradd -m -s /bin/bash dev2
sudo useradd -m -s /bin/bash admin1

# 2. Définir les mots de passe
sudo passwd dev1
sudo passwd dev2
sudo passwd admin1

# Ou en une ligne (automatisé)
echo "dev1:password123" | sudo chpasswd
echo "dev2:password123" | sudo chpasswd
echo "admin1:admin123" | sudo chpasswd

# 3. Vérifier dans passwd
grep -E "dev1|dev2|admin1" /etc/passwd

# 4. Informations de dev1
id dev1

# 5. Dernières connexions
sudo last | head -20
```

Exercice 2 : Gestion des groupes

Objectif : Créer et gérer des groupes

Consignes :

1. Créez deux groupes : `developers` et `admins`
2. Ajoutez `dev1` et `dev2` au groupe `developers`
3. Ajoutez `admin1` au groupe `admins`
4. Vérifiez les appartenances avec `groups`
5. Affichez le contenu de `/etc/group`

Solution Exercice 2 💡

```
# 1. Créer les groupes
sudo groupadd developers
sudo groupadd admins

# 2. Ajouter dev1 et dev2 au groupe developers
sudo usermod -aG developers dev1
sudo usermod -aG developers dev2

# 3. Ajouter admin1 au groupe admins
sudo usermod -aG admins admin1

# 4. Vérifier les appartenances
groups dev1
groups dev2
groups admin1

# Ou avec id
id dev1

# 5. Voir /etc/group
grep -E "developers|admins" /etc/group
```

Exercice 3 : Permissions de base 🔐

Objectif : Maîtriser les permissions Unix

Consignes :

1. Créez un fichier `projet.txt` appartenant à `dev1`
2. Donnez les permissions `rw-r--r--` (644)
3. Créez un fichier `secret.txt` avec permissions `rw-------` (600)
4. Créez un script `hello.sh` et rendez-le exécutable
5. Créez un répertoire `shared/` avec permissions `rwxrwxr-x` (775)

Solution Exercice 3 💡

```
# 1. Créer le fichier
sudo -u dev1 touch /home/dev1/projet.txt

# 2. Permissions 644
sudo chmod 644 /home/dev1/projet.txt
# Vérifier
ls -l /home/dev1/projet.txt

# 3. Fichier secret
sudo -u dev1 touch /home/dev1/secret.txt
sudo chmod 600 /home/dev1/secret.txt

# 4. Script exécutable
echo '#!/bin/bash' | sudo -u dev1 tee /home/dev1/hello.sh
echo 'echo "Hello World!"' | sudo -u dev1 tee -a /home/dev1/hello.sh
sudo chmod +x /home/dev1/hello.sh
# ou
sudo chmod 755 /home/dev1/hello.sh

# 5. Répertoire partagé
sudo mkdir /home/dev1/shared
sudo chmod 775 /home/dev1/shared
```

Exercice 4 : Projet collaboratif 🤝

Objectif : Mettre en place un espace de travail partagé

Scénario : Vous devez créer un espace où `dev1` et `dev2` peuvent collaborer

Consignes :

1. Créez `/shared/project`
2. Changez le propriétaire en `root` et le groupe en `developers`
3. Permissions : rwxrwx— (770)
4. Créez un fichier dedans en tant que `dev1`
5. Vérifiez que `dev2` peut le modifier
6. Vérifiez que `admin1` ne peut PAS y accéder

Solution Exercice 4 💡

```
# 1. Créer le répertoire
sudo mkdir -p /shared/project

# 2. Changer propriétaire et groupe
sudo chown root:developers /shared/project

# 3. Permissions 770
sudo chmod 770 /shared/project

# Vérifier
ls -ld /shared/project

# 4. Créer un fichier en tant que dev1
sudo -u dev1 touch /shared/project/code.txt
sudo -u dev1 bash -c 'echo "Hello from dev1" > /shared/project/code.txt'

# 5. Modifier en tant que dev2
sudo -u dev2 bash -c 'echo "Modified by dev2" >> /shared/project/code.txt'
cat /shared/project/code.txt

# 6. Tester l'accès de admin1 (devrait échouer)
sudo -u admin1 ls /shared/project/
# → Permission denied
```

Exercice 5 : Utilisation de sudo 🔑

Objectif : Configurer sudo de manière sécurisée

Consignes :

1. Donnez à `admin1` les droits sudo complets
2. Donnez à `dev1` le droit de redémarrer nginx uniquement
3. Créez un groupe `sysadmin` avec tous les droits sudo
4. Testez les commandes avec chaque utilisateur
5. Vérifiez les logs sudo dans `/var/log/auth.log`

Solution Exercice 5 💡

```
# 1. Donner tous les droits à admin1
echo "admin1 ALL=(ALL:ALL) ALL" | sudo tee /etc/sudoers.d/admin1
sudo chmod 440 /etc/sudoers.d/admin1

# 2. Droits limités pour dev1
echo "dev1 ALL=(ALL) NOPASSWD: /bin/systemctl restart nginx" | \
    sudo tee /etc/sudoers.d/dev1
sudo chmod 440 /etc/sudoers.d/dev1

# 3. Groupe sysadmin
sudo groupadd sysadmin
echo "%sysadmin ALL=(ALL:ALL) ALL" | sudo tee /etc/sudoers.d/sysadmin
sudo chmod 440 /etc/sudoers.d/sysadmin

# 4. Tester
# En tant qu'admin1
sudo -u admin1 sudo apt update

# En tant que dev1 (devrait échouer)
sudo -u dev1 sudo apt update  # → Devrait échouer

# 5. Voir les logs
sudo tail -f /var/log/auth.log | grep sudo
```

Exercice 6 : Gestion avancée des permissions 🎯

Objectif : Utiliser chown, chmod et umask

Consignes :

1. Créez `/project` appartenant à `dev1:developers`
2. Mettez umask à `0027` pour `dev1`
3. Créez un fichier en tant que `dev1` et vérifiez ses permissions
4. Changez récursivement le propriétaire de `/project` en `dev2`
5. Donnez les permissions `rwxr-x---` à `/project`

Solution Exercice 6 💡

```
# 1. Créer /project
sudo mkdir /project
sudo chown dev1:developers /project

# 2. Modifier umask pour dev1
sudo -u dev1 bash -c 'echo "umask 0027" >> /home/dev1/.bashrc'

# 3. Créer un fichier avec le nouveau umask
sudo -u dev1 bash -c 'umask 0027; touch /project/test.txt'
ls -l /project/test.txt
# → -rw-r----- (permissions 640)

# 4. Changer propriétaire récursif
sudo chown -R dev2:developers /project

# 5. Permissions sur /project
sudo chmod 750 /project
ls -ld /project
```

Exercice bonus : Script de gestion d'utilisateurs 🤖

Objectif : Automatiser la création d'utilisateurs

Créez un script `create_users.sh` **qui :**

- Lit un fichier CSV avec : nom,groupe,shell
- Crée les utilisateurs automatiquement
- Génère des mots de passe aléatoires
- Crée un rapport

Exemple CSV :

```
alice,developers,/bin/bash
bob,admins,/bin/bash
charlie,developers,/bin/zsh
```

Solution Exercice bonus 💡

```bash
#!/bin/bash
# Script de création d'utilisateurs

CSV_FILE="users.csv"
REPORT="users_report.txt"

echo "===== Rapport de création =====" > $REPORT
echo "Date: $(date)" >> $REPORT
echo >> $REPORT

while IFS=',' read -r username group shell; do
    # Ignorer l'en-tête
    [[ "$username" == "nom" ]] && continue

    echo "Création de $username..."

    # Créer l'utilisateur
    sudo useradd -m -s "$shell" "$username" 2>/dev/null

    # Créer le groupe si nécessaire
    sudo groupadd "$group" 2>/dev/null

    # Ajouter au groupe
    sudo usermod -aG "$group" "$username"

    # Générer mot de passe
    PASSWORD=$(openssl rand -base64 12)
    echo "$username:$PASSWORD" | sudo chpasswd

    # Rapport
    echo "Utilisateur: $username" >> $REPORT
    echo "Groupe: $group" >> $REPORT
    echo "Shell: $shell" >> $REPORT
    echo "Password: $PASSWORD" >> $REPORT
    echo "---" >> $REPORT

done < "$CSV_FILE"

echo "Terminé ! Voir $REPORT"
```

Points clés à retenir 📌

Sécurité :

- Ne jamais travailler en root
- Utiliser sudo avec parcimonie
- Permissions minimales nécessaires

Bonnes pratiques :

- Groupes pour organiser
- umask pour sécuriser par défaut
- Documenter les configurations sudo

Commandes essentielles :

- `useradd` , `usermod` , `userdel`
- `groupadd` , `groupmod` , `gpasswd`
- `chmod` , `chown` , `chgrp`
- `sudo` , `visudo`

💾 Module 5

Gestion du stockage et du système de fichiers

Maîtriser disques, partitions et systèmes de fichiers

Revenir au sommaire

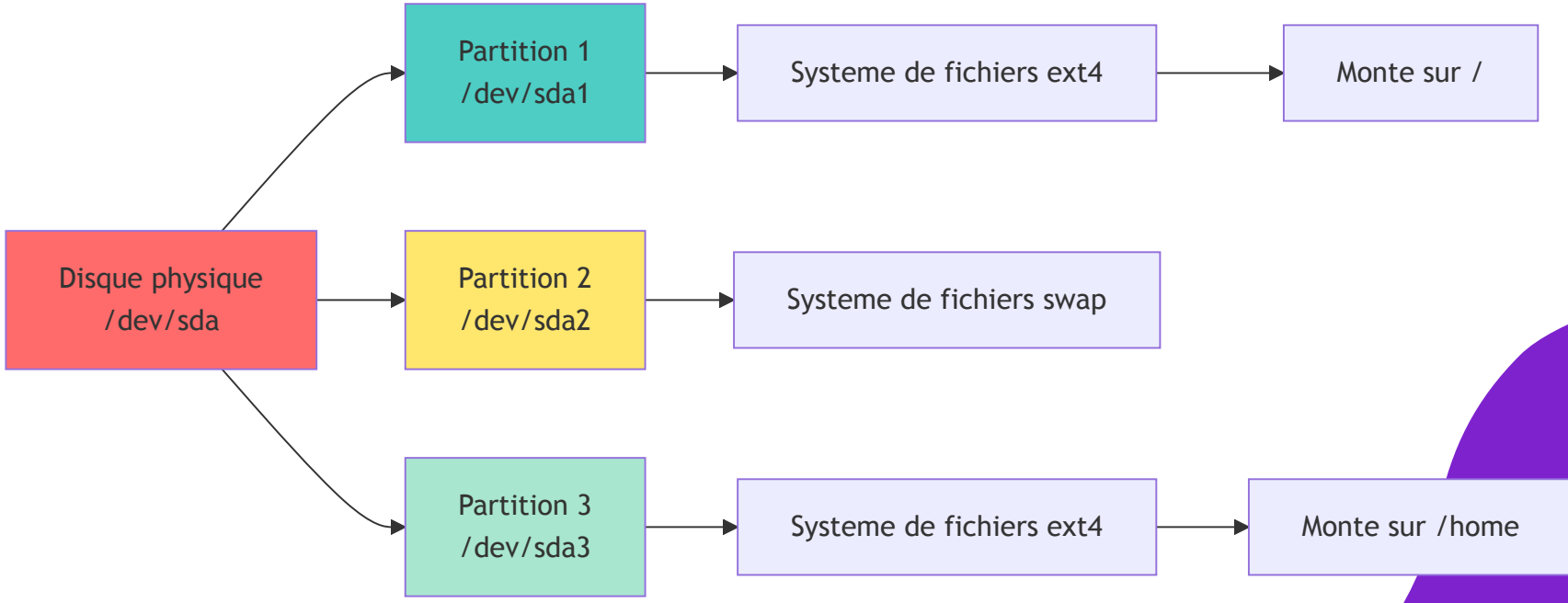
Comprendre le stockage 💾

Analogie : une bibliothèque 📚

- **Disque dur** = le bâtiment entier
- **Partition** = un étage de la bibliothèque
- **Système de fichiers** = méthode de rangement des livres
- **Montage** = ouvrir l'accès à un étage
- **Fichier** = un livre
- **Répertoire** = une étagère

Sans partition ni système de fichiers, un disque est inutilisable !

Hiérarchie du stockage



Revenir au sommaire

Processus classique de création d'une partition :

1. Identifier le disque
2. Créer des partitions
3. Créer un système de fichiers
4. Monter pour utiliser

Nomenclature des disques

Disques SATA/SAS/USB :

- `/dev/sda` : premier disque
- `/dev/sdb` : deuxième disque
- `/dev/sdc` : troisième disque

Partitions :

- `/dev/sda1` : première partition du disque sda
- `/dev/sda2` : deuxième partition du disque sda

Disques NVMe (SSD modernes) :

- `/dev/nvme0n1` : premier disque NVMe
- `/dev/nvme0n1p1` : première partition

Analogie : les placards numérotés** 🗄️

Imaginez un garde-meuble :

- **sda, sdb, sdc** : les différents placards (disques)
- **sda1, sda2** : les étagères dans le placard A (partitions)
- **Système de fichiers** : comment vous organisez chaque étagère

Exemple concret :

```
 # Lister vos disques
lsblk

# Résultat typique sur un PC :
# sda         (disque dur principal 500 Go)
# ├─sda1      (partition EFI 512 Mo)
# ├─sda2      (partition / 100 Go)
# └─sda3      (partition /home 399 Go)
# sdb         (clé USB 32 Go)
# └─sdb1      (partition unique 32 Go)
```

Lister les disques et partitions 📋

```
# Lister les disques et partitions
lsblk

# Format arborescent avec tailles
lsblk -f

# Afficher les UUID
sudo blkid

# Informations détaillées
sudo fdisk -l

# Taille des disques
df -h

# Utilisation de l'espace par répertoire
du -sh /home/*
```

Exemple de sortie lsblk 📊

```
lsblk
```

```
 NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
sda      8:0    0   500G  0 disk
├─sda1   8:1    0   512M  0 part /boot/efi
├─sda2   8:2    0   450G  0 part /
└─sda3   8:3    0    50G  0 part /home
sdb      8:16   0     1T  0 disk
└─sdb1   8:17   0     1T  0 part /mnt/data
nvme0n1  259:0  0   256G  0 disk
└─nvme0n1p1 259:1 0  256G  0 part /opt
```

Colonnes :

- `NAME` : nom du périphérique
- `SIZE` : taille
- `TYPE` : disk ou part(ition)
- `MOUNTPOINT` : où c'est monté

df : espace disque disponible 📏

```
# Format lisible
df -h

# Seulement les systèmes de fichiers locaux
df -h -x tmpfs -x devtmpfs

# Inodes disponibles
df -i

# Type de système de fichiers
df -T
```

Exemple de sortie :

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2       450G  120G  308G  28% /
/dev/sda3        50G   15G   33G  31% /home
/dev/sdb1       1.0T  500G  500G  50% /mnt/data
```

du : utilisation du disque

```
# Taille d'un répertoire
du -sh /home/alice

# Détail des sous-répertoires
du -h --max-depth=1 /var

# Les 10 plus gros répertoires
du -h /home | sort -hr | head -10

# Exclure certains répertoires
du -sh --exclude="*.cache" /home/alice

# Total uniquement
du -sh /var/log
```

ncdu : analyseur interactif d'espace disque 🎨

```
# Installer ncdu
sudo apt install ncdu          # Debian/Ubuntu
sudo dnf install ncdu          # RHEL/Fedora

# Analyser un répertoire
ncdu /home

# Analyser et sauvegarder
ncdu -o /tmp/scan.ncdu /var
# Charger plus tard
ncdu -f /tmp/scan.ncdu
```

Navigation :

- Flèches : naviguer
- Entrée : entrer dans un répertoire
- `d` : supprimer
- `q` : quitter

Partitionnement : MBR vs GPT 🔀

MBR (Master Boot Record) : ancien standard (1983)

- Max 4 partitions primaires
- Max 2 To par disque
- Compatible BIOS

GPT (GUID Partition Table) : moderne (2000s)

- 128 partitions
- Disques > 2 To
- UEFI requis
- Plus robuste (backup de la table)

Recommandation 2025 : toujours utiliser GPT !

⚠ RÈGLE FONDAMENTALE : Un disque = Un seul format !

Un disque ne peut avoir QU'UN SEUL type de table de partitions : MBR OU GPT, jamais les deux !

Conséquence : Si vous changez de MBR vers GPT (ou inversement), **toutes les données sont perdues !**

Revenir au sommaire

Vérifier le type de table : MBR ou GPT ? 🔍

Méthode 1 : `fdisk -l` (la plus simple)

```
sudo fdisk -l /dev/sda
```

```
Disque /dev/sda : 100 GiB
Type de table de partition : gpt    # <-- ICI !
```

Méthode 2 : `parted` (plus détaillé)

```
sudo parted /dev/sda print
```

```
Modèle: ATA VBOX HARDDISK
Disque /dev/sda : 107GB
Table de partitions : gpt          # <-- ICI !
```

Méthode 3 : `lsblk` avec l'option PTTYPE

```
lsblk -o NAME,SIZE,PTTYPE /dev/sda
```

```
 NAME   SIZE PTTYPE
sda    100G gpt     # <-- ICI !
 ├─sda1  50G
 └─sda2  50G
```

Méthode 4 : `blkid` (rapide)

```
sudo blkid -o value -s PTTYPE /dev/sda
gpt
```

💡 **Astuce mémo :** `fdisk -l` est la méthode la plus universelle !

Créer des partitions : fdisk 🔧

fdisk : outil classique pour partitionner

```
# Lancer fdisk sur un disque
sudo fdisk /dev/sdb

# Commandes interactives :
# m : aide (menu)
# p : afficher les partitions actuelles
# g : créer une nouvelle table GPT (efface tout !)
# o : créer une nouvelle table MBR (efface tout !)
# n : nouvelle partition
# d : supprimer une partition
# t : changer le type de partition
# w : écrire et quitter (APPLIQUE les changements)
# q : quitter sans sauvegarder
```

⚠️ **ATTENTION :** Les changements sont écrits seulement avec `w`

fdisk : Créer une partition GPT de A à Z

Scénario : Nouveau disque `/dev/sdb` vide, on veut une partition GPT

Étape 1 : Lancer fdisk

```
sudo fdisk /dev/sdb
```

Étape 2 : Créer la table GPT (si disque neuf ou changement de format)

```
Command (m for help): g    # <-- Crée une table GPT vide
Created a new GPT disklabel (GUID: xxxx-xxxx-xxxx)
```

⚠️ **Cette commande efface TOUTE la table de partitions existante !**

Revenir au sommaire

fdisk : Créer une partition GPT (suite)

Étape 3 : Créer la partition

```
Command (m for help): n     # <-- Nouvelle partition
Partition number (1-128, default 1): 1    # <-- Entrée (accepter 1)
First sector: [Entrée]      # <-- Début par défaut
Last sector: [Entrée]       # <-- Tout le disque
                            # Ou +50G pour 50 Go, +500M pour 500 Mo

Created a new partition 1 of type 'Linux filesystem'
```

Étape 4 : Vérifier avant d'écrire

```
Command (m for help): p     # <-- Affiche les partitions
Device     Start      End  Sectors Size Type
/dev/sdb1   2048 209715166 209713119 100G Linux filesystem
```

fdisk : Créer une partition GPT (fin) 📝

Étape 5 : Écrire les changements

```
Command (m for help): w     # <-- ÉCRIT sur le disque !
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

Étape 6 : Vérifier que la partition existe

```
lsblk /dev/sdb
```

```
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sdb      8:16   0  100G  0 disk
└─sdb1   8:17   0  100G  0 part    # <-- Partition créée !
```

Prochaine étape : Formater avec `mkfs.ext4 /dev/sdb1` puis monter !

fdisk : Créer une partition MBR 📝

Si vous avez besoin de MBR (compatibilité ancien BIOS) :

```
sudo fdisk /dev/sdb
```

```
Command (m for help): o      # <-- Crée une table MBR (au lieu de 'g')
Created a new DOS disklabel

Command (m for help): n      # <-- Nouvelle partition
Partition type:
   p   primary (0 primary, 0 extended, 4 free)
   e   extended
Select (default p): p        # <-- Partition primaire
Partition number (1-4): 1    # <-- Numéro 1
First sector: [Entrée]
Last sector: [Entrée]        # <-- Tout le disque

Command (m for help): w      # <-- Écrire
```

💡 **Rappel MBR :** Maximum 4 partitions primaires (ou 3 primaires + 1 étendue avec des logiques)

Revenir au sommaire

🧪 Tester sans risque : Créer un faux disque (loop device)

Problème : Vous êtes en VM et voulez tester `fdisk` , `mkfs` , `mount` ... sans ajouter de disque ?

Solution : Créer un **fichier** qui se comporte comme un disque !

C'est quoi un loop device ?

- Un fichier ordinaire que Linux traite comme un périphérique bloc
- Apparaît comme `/dev/loop0` , `/dev/loop1` , etc.
- Parfait pour apprendre et tester sans risque !

Analogie 🎭 : C'est comme un acteur qui joue le rôle d'un disque dur

Revenir au sommaire

Créer un faux disque : Étape par étape 📝

Étape 1 : Créer le fichier image (500 Mo)

```
# Crée un fichier rempli de zéros de 500 Mo
dd if=/dev/zero of=/tmp/fake_disk.img bs=1M count=500
# if = Input File (source) → /dev/zero = source infinie de zéros
# of = Output File (destination) → le fichier image à créer
# bs = Block Size → taille des blocs (1M = 1 Mo)
# count = nombre de blocs à copier → 500 × 1M = 500 Mo
```

⚠ Attention : Loop devices déjà utilisés !

Sur Ubuntu, les loop devices (`loop0` , `loop1` ...) sont souvent **déjà utilisés par Snap** !

```
# Voir les loop devices existants
losetup -l
```

```
NAME       SIZELIMIT  BACK-FILE
/dev/loop0         0  /var/lib/snapd/snaps/core22_1564.snap
/dev/loop1         0  /var/lib/snapd/snaps/firefox_4848.snap
/dev/loop2         0  /var/lib/snapd/snaps/gnome-42_178.snap
```

Si vous essayez `fdisk /dev/loop0` **→ ERREUR !**

```
This disk is currently in use...
fdisk: failed to write disklabel: Operation not permitted
```

Solution : Utilisez `losetup -f` qui trouve automatiquement un loop **libre** !

Créer un faux disque : Étape par étape (suite) 📝

Étape 2 : Associer le fichier à un loop device

```
# -f : trouve automatiquement le prochain loop libre
# -P : crée aussi les partitions (loopXp1, loopXp2...)
sudo losetup -fP /tmp/fake_disk.img
```

Étape 3 : Trouver QUEL loop device a été attribué ⚠️

```
# Filtrer pour trouver VOTRE fichier
losetup -l | grep fake_disk
```

```
 NAME         SIZELIMIT OFFSET AUTOCLEAR RO BACK-FILE
/dev/loop12          0      0         0  0 /tmp/fake_disk.img
```

→ C'est `/dev/loop12` **! (le numéro varie selon votre système)**

```
# Alternative : voir tous les loops
losetup -a | grep fake_disk
```

```
/dev/loop12: []: (/tmp/fake_disk.img)
```

💡 **Notez bien le numéro (ex: loop12) pour les commandes suivantes !**

Utiliser le faux disque comme un vrai ! 🎯

Remplacez `loopX` **par votre numéro (ex:** `loop12` **) !**

```
# Partitionner (GPT)
sudo fdisk /dev/loopX      # Ex: sudo fdisk /dev/loop12
# g → n → Entrée → Entrée → Entrée → w

# Vérifier la partition
lsblk /dev/loopX
```

```
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
loop12        7:12   0  500M  0 loop
└─loop12p1    7:13   0  499M  0 part
```

Revenir au sommaire

Utiliser le faux disque : Formater et monter 🎯

```
# Formater en ext4 (remplacez loopX par votre numéro !)
sudo mkfs.ext4 /dev/loopXp1     # Ex: /dev/loop12p1
```

```
# Créer un point de montage
sudo mkdir -p /mnt/test
```

```
# Monter
sudo mount /dev/loopXp1 /mnt/test
```

```
# Tester !
echo "Ça marche !" | sudo tee /mnt/test/test.txt
cat /mnt/test/test.txt
df -h /mnt/test
```

Nettoyer le faux disque 🧹

Quand vous avez fini vos tests :

```
# 1. Démonter la partition
sudo umount /mnt/test

# 2. Détacher le loop device (remplacez loopX !)
sudo losetup -d /dev/loopX     # Ex: /dev/loop12

# 3. Vérifier qu'il est bien détaché
losetup -l | grep fake_disk
# (ne devrait plus rien afficher)

# 4. Supprimer le fichier image (optionnel)
rm /tmp/fake_disk.img
```

💡 **Astuce :** Vous pouvez créer plusieurs faux disques simultanément !

Récapitulatif : Commandes loop device 📋

Action	Commande
Créer fichier image	`dd if=/dev/zero of=/tmp/disk.img bs=1M count=500`
Attacher au loop	`sudo losetup -fP /tmp/disk.img`
Trouver le numéro attribué	`losetup -l | grep disk.img`
Lister tous les loops	`losetup -l` ou `losetup -a`
Détacher un loop	`sudo losetup -d /dev/loopX`
Détacher tous	`sudo losetup -D`

Pourquoi utiliser les loop devices ?

- ✅ Aucun risque pour vos vrais disques
- ✅ Pas besoin d'éteindre la VM
- ✅ Créez autant de "disques" que vous voulez
- ✅ Parfait pour apprendre `fdisk` , `mkfs` , `mount` , LVM...

⚠️ **Sur Ubuntu** : Les `loop0` , `loop1` ... sont souvent pris par Snap → utilisez `losetup -f` !

Créer des partitions : gdisk/parted 🔧

gdisk : pour GPT (comme fdisk)

```
sudo gdisk /dev/sdb
```

parted : supporte MBR et GPT, scriptable

```
# Mode interactif
sudo parted /dev/sdb

# Mode commande
sudo parted /dev/sdb mklabel gpt
sudo parted /dev/sdb mkpart primary ext4 0% 100%
```

Exemple pratique : ajouter un disque

Étape 1 : Identifier le nouveau disque

```
lsblk
# → Nouveau disque : /dev/sdb
```

Exemple pratique : ajouter un disque (suite)

Étape 2 : Créer une partition

```
sudo fdisk /dev/sdb

# Dans fdisk :
# n → nouvelle partition
# p → primaire
# 1 → numéro de partition
# Entrée → début par défaut
# Entrée → fin par défaut (tout le disque)
# w → écrire et quitter
```

Exemple pratique : ajouter un disque (suite)

Étape 3 : Créer un système de fichiers

```
sudo mkfs.ext4 /dev/sdb1
```

Exemple pratique : ajouter un disque (suite)

Étape 4 : Créer un point de montage et monter

```
sudo mkdir /mnt/data
sudo mount /dev/sdb1 /mnt/data
```

Exemple pratique : ajouter un disque (suite)

Étape 5 : Rendre permanent (dans /etc/fstab)

```
sudo nano /etc/fstab

# Ajouter :
/dev/sdb1  /mnt/data  ext4  defaults  0  2
```

Revenir au sommaire

Systèmes de fichiers 📂

ext4 : standard Linux

- Stable, performant, mature
- Journaling
- Max 16 To par fichier, 1 Eo par volume
- Par défaut sur la plupart des distributions

xfs : haute performance

- Excellent pour gros fichiers
- Utilisé par Red Hat par défaut
- Pas de shrink (réduire taille) possible

Systèmes de fichiers (suite) 📂

btrfs : moderne, fonctionnalités avancées

- Snapshots, compression
- Checksums (intégrité)
- Redimensionnement en ligne
- En développement actif

tmpfs : en RAM

- Très rapide
- Volatile (effacé au redémarrage)
- Utilisé pour /tmp, /run

Systèmes de fichiers réseau

NFS : Network File System

- Standard Unix/Linux
- Partage entre serveurs Linux

CIFS/SMB : partage Windows

- Samba sur Linux
- Compatible avec Windows

sshfs : via SSH

- Monter un répertoire distant comme local
- Sécurisé

Créer un système de fichiers 🛠️

```
# ext4
sudo mkfs.ext4 /dev/sdb1

# xfs
sudo mkfs.xfs /dev/sdb1

# btrfs
sudo mkfs.btrfs /dev/sdb1

# FAT32 (pour clés USB)
sudo mkfs.vfat /dev/sdb1

# NTFS (pour compatibilité Windows)
sudo mkfs.ntfs /dev/sdb1

# Avec label
sudo mkfs.ext4 -L "MesDocuments" /dev/sdb1
```

Monter un système de fichiers 🔗

```
# Monter temporairement
sudo mount /dev/sdb1 /mnt/data

# Monter en lecture seule
sudo mount -o ro /dev/sdb1 /mnt/data

# Monter avec options
sudo mount -o rw,noexec,nosuid /dev/sdb1 /mnt/data

# Monter tous les systèmes de /etc/fstab
sudo mount -a

# Démonter
sudo umount /mnt/data

# Démonter avec force (si occupé)
sudo umount -f /mnt/data
sudo umount -l /mnt/data  # lazy unmount
```

Options de montage courantes

rw / **ro** : lecture-écriture / lecture seule

noexec : interdire l'exécution de binaires

nosuid : ignorer les bits SUID/SGID

nodev : ignorer les périphériques

noatime : ne pas mettre à jour l'access time (performance)

nodiratime : ne pas mettre à jour l'access time des répertoires

defaults : rw, suid, dev, exec, auto, nouser, async

/etc/fstab : montages automatiques

Format : `device mountpoint fstype options dump pass`

```
# UUID recommandé (stable)
UUID=abc-123  /home  ext4  defaults  0  2

# Ancien style (peut changer au reboot)
/dev/sda3  /home  ext4  defaults  0  2

# NFS
server:/export  /mnt/nfs  nfs  defaults  0  0

# tmpfs
tmpfs  /tmp  tmpfs  defaults,noatime,mode=1777  0  0

# Swap
/dev/sda2  none  swap  sw  0  0
```

Colonnes de /etc/fstab

1. **Device** : UUID ou /dev/sdX
2. **Mount point** : où monter
3. **Filesystem type** : ext4, xfs, nfs, etc.
4. **Options** : defaults, ro, noexec, etc.
5. **Dump** : sauvegarde avec dump (0 = non, 1 = oui)
6. **Pass** : ordre de vérification fsck (0 = jamais, 1 = racine, 2 = autres)

Trouver l'UUID d'un disque 🔍

```
# Méthode 1 : blkid
sudo blkid /dev/sdb1
# → UUID="abc-123-def-456"

# Méthode 2 : lsblk
lsblk -f

# Méthode 3 : depuis /dev/disk
ls -l /dev/disk/by-uuid/

# Utiliser l'UUID dans fstab
UUID=abc-123-def-456  /mnt/data  ext4  defaults  0  2
```

Pourquoi UUID ? Car `/dev/sdX` peut changer au redémarrage !

Revenir au sommaire

Tester /etc/fstab sans rebooter 🧪

```
# Éditer fstab
sudo nano /etc/fstab

# Tester le montage
sudo mount -a

# Si erreur, corriger immédiatement !
# Sinon le système ne démarrera pas

# Vérifier
df -h
mount | grep /mnt/data
```

⚠️ **Un fstab incorrect peut empêcher le boot !**

Swap : mémoire d'échange 💫

Swap : extension de la RAM sur disque

Analogie :

- **RAM** = bureau de travail (rapide)
- **Swap** = armoire (lent mais plus d'espace)

Utilisations :

- Quand la RAM est pleine
- Hibernation (suspend to disk)
- Recommandé même avec beaucoup de RAM

Créer et activer un swap 🔄

```
# Méthode 1 : partition swap
sudo mkswap /dev/sda2
sudo swapon /dev/sda2

# Méthode 2 : fichier swap
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

# Rendre permanent dans /etc/fstab
/swapfile  none  swap  sw  0  0

# Vérifier le swap
swapon --show
free -h
```

Gérer le swap

```
# Voir l'utilisation du swap
free -h
swapon --show

# Désactiver le swap
sudo swapoff /swapfile

# Réactiver
sudo swapon /swapfile

# Désactiver tout le swap
sudo swapoff -a

# Priorité du swap (multiple swaps)
sudo swapon -p 10 /swapfile
```

Swappiness : agressivité du swap ⚖️

swappiness : tendance à utiliser le swap (0-100)

```
# Voir la valeur actuelle
cat /proc/sys/vm/swappiness
# → 60 (par défaut)

# Changer temporairement
sudo sysctl vm.swappiness=10

# Rendre permanent
echo "vm.swappiness=10" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

Valeurs :

- `0` : swap uniquement si nécessaire
- `60` : défaut (équilibre)
- `100` : swap agressif

Recommandation : 10 pour desktop, 60 pour serveur

Liens symboliques et durs 🔗

Lien dur (hard link) : même inode, plusieurs noms

```
ln fichier.txt lien_dur.txt
```

- Partagent le même contenu
- Supprimer un n'affecte pas l'autre
- Impossible pour les répertoires
- Doivent être sur la même partition

Liens symboliques et durs (suite) 🔗

Lien symbolique (soft/symbolic link) : pointeur vers un fichier

```
ln -s /chemin/vers/fichier.txt lien_sym.txt
ln -s /opt/app /usr/local/bin/app
```

- Peut pointer vers des répertoires
- Peut traverser les partitions
- Si la cible est supprimée, le lien est cassé

Revenir au sommaire

Analogie des liens 🎭

Lien dur : comme deux clés différentes pour la même porte

- Les deux clés ouvrent la même pièce
- Perdre une clé ne change rien

Lien symbolique : comme une pancarte indiquant "La maison est là →"

- Si la maison est détruite, la pancarte pointe dans le vide

Comparer les liens 🔬

```
# Créer un fichier
echo "Contenu" > original.txt

# Lien dur
ln original.txt dur.txt

# Lien symbolique
ln -s original.txt sym.txt

# Comparer les inodes
ls -li
# 12345 original.txt
# 12345 dur.txt         ← même inode
# 12346 sym.txt → original.txt
```

Qu'est ce qu'un inode ?

Un inode = index node est une structure interne utilisée par les systèmes de fichiers pour stocker toutes les informations d'un fichier, sauf son nom et son contenu.

Un inode stocke par exemple :

- 📏 La taille du fichier
- 👤 Le propriétaire (UID) et le groupe (GID)
- 🔐 Les permissions (rwx)
- 🕒 Les dates (création, modification, dernier accès)
- 📍 L'emplacement des données sur le disque
- 🔗 Le nombre de liens vers ce fichier

❗ Le nom du fichier n'est PAS dans l'inode Le nom est stocké dans le dossier (qui fonctionne comme une table de correspondance : nom → inode).

pour afficher l'inode d'un fichier ou dossier :

```
#fichier
ls -i fichier.txt
#dossier
ls -di dossier/
```

Permissions étendues : ACL 🔐

ACL (Access Control Lists) : permissions plus fines que le système classique rwx

Le problème des permissions classiques :

```
-rw-r----- 1 alice devs 1024 rapport.txt
           │     │
           │     └── UN seul groupe (devs)
           └──── UN seul propriétaire (alice)
```

Que faire si :

- Bob (pas dans devs) doit pouvoir lire ce fichier ?
- Charlie (pas dans devs) doit pouvoir le modifier ?
- ❌ Impossible avec les permissions classiques !

ACL : La solution flexible 🎯

Analogie : Le badge d'accès 🏢

Système	Analogie	Limitation
Permissions classiques	Clé du bâtiment	1 clé = 1 porte = tout le monde pareil
ACL	Badge électronique	Chaque personne a ses propres accès

Avec les ACL, vous pouvez dire :

- "Alice peut lire ET écrire"
- "Bob peut seulement lire"
- "Charlie ne peut rien faire"
- "Le groupe 'devs' peut tout faire"
- **Le tout sur le MÊME fichier !**

Syntaxe ACL : Décryptage complet 📖

Format : `TYPE:NOM:PERMISSIONS`

Partie	Signification	Exemples
TYPE	`u` = utilisateur, `g` = groupe, `o` = autres	`u` , `g` , `o`
NOM	Nom de l'utilisateur ou du groupe	`alice` , `bob` , `devs`
PERMISSIONS	Combinaison de r, w, x	`r` , `rw` , `rwx` , `rx`

Exemples concrets :

Commande	Signification
`u:alice:r`	**U**tilisateur alice → lecture seule
`u:bob:rw`	**U**tilisateur bob → lecture + écriture
`g:devs:rwx`	**G**roupe devs → lecture + écriture + exécution

Revenir au sommaire

Installer et utiliser ACL 🛠️

```
# Installer les outils ACL
sudo apt install acl          # Debian/Ubuntu
sudo dnf install acl          # RHEL/Fedora

# Vérifier si le système de fichiers supporte ACL
mount | grep acl
```

Commandes ACL essentielles 📋

`getfacl` = **GET** File ACL (voir les ACL)

```
getfacl fichier.txt
```

`setfacl` = **SET** File ACL (modifier les ACL)

```
# -m = modify (ajouter/modifier une ACL)
setfacl -m u:alice:r fichier.txt      # alice peut lire
setfacl -m u:bob:rw fichier.txt       # bob peut lire+écrire
setfacl -m g:devs:rwx dossier/        # groupe devs a tous les droits
```

Exemple concret : Projet partagé 📁

Scénario : Vous avez un dossier `/projet` et voulez :

- Alice (chef de projet) : tous les droits
- Bob (développeur) : lecture + écriture
- Charlie (stagiaire) : lecture seule
- Groupe "clients" : lecture seule

```
# Créer le dossier
sudo mkdir /projet

# Donner les accès personnalisés
sudo setfacl -m u:alice:rwx /projet   # Chef → tout
sudo setfacl -m u:bob:rw /projet      # Dev → lire+écrire
sudo setfacl -m u:charlie:r /projet   # Stagiaire → lire
sudo setfacl -m g:clients:r /projet   # Clients → lire

# Vérifier
getfacl /projet
```

Résultat : 4 niveaux d'accès différents sur le MÊME dossier ! 🎉

ACL avancées 🎯

```
# Supprimer une ACL spécifique
setfacl -x u:alice fichier.txt

# Supprimer toutes les ACL
setfacl -b fichier.txt

# ACL récursive
setfacl -R -m u:alice:rx /projet/

# ACL par défaut (pour les nouveaux fichiers)
setfacl -d -m u:alice:rw /projet/

# Copier les ACL d'un fichier
getfacl fichier1.txt | setfacl --set-file=- fichier2.txt
```

Exemple d'ACL

```
# Créer un répertoire partagé
sudo mkdir /shared
sudo setfacl -m u:alice:rwx /shared
sudo setfacl -m u:bob:rx /shared
sudo setfacl -m g:devs:rwx /shared

# ACL par défaut pour les sous-fichiers
sudo setfacl -d -m u:alice:rwx /shared
sudo setfacl -d -m g:devs:rw /shared

# Vérifier
getfacl /shared
```

Sortie de getfacl 📋

```
getfacl /shared
```

```
 # file: /shared
# owner: root
# group: root
user::rwx
user:alice:rwx
user:bob:r-x
group::r-x
group:devs:rwx
mask::rwx
other::r-x
```

Décryptage de getfacl ligne par ligne 🔍

Ligne	Signification
`# file: /shared`	Chemin du fichier/dossier
`# owner: root`	Propriétaire du fichier
`# group: root`	Groupe propriétaire
`user::rwx`	Permissions du **propriétaire** (root) → rwx
`user:alice:rwx`	**ACL** : alice a rwx
`user:bob:r-x`	**ACL** : bob a r-x (lecture + exécution, pas écriture)
`group::r-x`	Permissions du **groupe propriétaire** (root)
`group:devs:rwx`	**ACL** : groupe devs a rwx
`mask::rwx`	Permissions **maximales** pour les ACL
`other::r-x`	Permissions pour **tous les autres**

Attributs de fichiers étendus

Attributs immuables avec chattr

```
# Rendre un fichier immuable (même root ne peut pas le modifier)
sudo chattr +i fichier.txt

# Append only (on peut ajouter, pas supprimer)
sudo chattr +a fichier.log

# Voir les attributs
lsattr fichier.txt

# Retirer l'attribut
sudo chattr -i fichier.txt
```

Cas d'usage :

- Protéger des fichiers critiques
- Logs append-only
- Empêcher la suppression accidentelle

Vérifier l'intégrité : fsck 🔍

fsck : File System Check (vérifier et réparer)

```
# Vérifier une partition (doit être démontée !)
sudo umount /dev/sdb1
sudo fsck /dev/sdb1

# Forcer la vérification
sudo fsck -f /dev/sdb1

# Réparer automatiquement
sudo fsck -y /dev/sdb1

# Vérifier au prochain reboot
sudo touch /forcefsck
sudo reboot
```

⚠️ **Ne jamais fsck une partition montée !**

Redimensionner un système de fichiers

ext4 : agrandir

```
# 1. Agrandir la partition (avec fdisk/parted)
# 2. Informer le noyau
sudo partprobe

# 3. Agrandir le système de fichiers
sudo resize2fs /dev/sdb1
```

ext4 : réduire

```
# 1. Démonter
sudo umount /dev/sdb1

# 2. Vérifier
sudo e2fsck -f /dev/sdb1

# 3. Réduire le système de fichiers
sudo resize2fs /dev/sdb1 50G

# 4. Réduire la partition
```

LVM : Logical Volume Manager 🎛️

LVM = **L**ogical **V**olume **M**anager

C'est quoi ? Une couche intelligente entre vos disques et votre système.

Le problème des partitions classiques :

```
Disque 500 Go
├── /boot   (500 Mo)   ← Taille FIXE !
├── /       (50 Go)    ← Plein ? Tant pis ! 😤
└── /home   (449 Go)   ← Vide mais impossible de donner l'espace à /
```

Avec LVM, tout devient flexible ! 🎉

LVM vs Partitions classiques 🆚

Aspect	Partition classique	LVM
Taille	Fixe à la création	Dynamique (agrandir/réduire)
Modifier	Difficile, risqué	Simple, à chaud
Snapshots	❌ Impossible	✅ Oui !
Fusionner disques	❌ Impossible	✅ Plusieurs disques = 1 volume
Complexité	Simple	Un peu plus complexe

Cas d'usage idéaux pour LVM :

- 🖥️ Serveurs (espace qui évolue)
- 💾 Bases de données (snapshots avant mise à jour)
- 🏢 Entreprises (ajout de disques sans downtime)

Analogie : La piscine modulable 🏊

Partitions classiques = Piscines en béton

- Taille fixée à la construction
- Pour agrandir → tout casser et reconstruire

LVM = Piscine avec parois mobiles

- Ajoutez de l'espace quand vous voulez
- Réduisez si besoin
- Connectez plusieurs bassins ensemble

Les 3 niveaux de LVM

```
┌──────────────────────────────────────┐
│ NIVEAU 3 : LV (Logical Volume)       │
│ → Ce que vous utilisez : /, /home, /var │
│ → C'est votre "faux disque" flexible │
├──────────────────────────────────────┤
│ NIVEAU 2 : VG (Volume Group)         │
│ → Le "réservoir" qui regroupe les disques │
│ → Exemple : vg_data                  │
├──────────────────────────────────────┤
│ NIVEAU 1 : PV (Physical Volume)      │
│ → Vos vrais disques/partitions       │
│ → /dev/sda, /dev/sdb, /dev/nvme0n1   │
└──────────────────────────────────────┘
```

Analogie cuisine 🍳 :

- **PV** = Les ingrédients (farine, œufs, lait)
- **VG** = Le saladier où vous mélangez tout
- **LV** = Les parts de gâteau que vous servez

Schéma LVM complet

Exemple concret : Agrandir sans rien casser 📈

Situation initiale :

- 1 disque de 500 Go avec LVM
- `/home` fait 200 Go et est **plein** !

Solution SANS LVM : 😰

1. Sauvegarder toutes les données
2. Ajouter un disque
3. Réinstaller ou tout reconfigurer
4. Restaurer les données

Solution AVEC LVM : 😎

```
# 1. Brancher le nouveau disque (1 To)
# 2. L'ajouter au LVM
sudo pvcreate /dev/sdb
sudo vgextend vg_storage /dev/sdb

# 3. Agrandir /home de 500 Go
sudo lvextend -L +500G /dev/vg_storage/home
sudo resize2fs /dev/vg_storage/home

# FINI ! Pas de reboot, pas de perte de données ! 🎉
```

Créer un LVM de A à Z 🛠️

Étape 1 : Créer les Physical Volumes (PV)

```
sudo pvcreate /dev/sdb /dev/sdc
# pv = Physical Volume, create = créer
# → Marque les disques comme utilisables par LVM
```

Étape 2 : Créer le Volume Group (VG)

```
sudo vgcreate mon_vg /dev/sdb /dev/sdc
# vg = Volume Group, create = créer
# → Fusionne les disques dans un "réservoir" nommé mon_vg
```

Créer un LVM de A à Z (suite) 🛠️

Étape 3 : Créer les Logical Volumes (LV)

```
sudo lvcreate -L 50G -n vol_data mon_vg
# lv = Logical Volume, create = créer
# -L 50G = taille de 50 Go
# -n vol_data = nom du volume
# mon_vg = depuis quel VG
```

Étape 4 : Formater et monter

```
# Formater
sudo mkfs.ext4 /dev/mon_vg/vol_data

# Monter
sudo mkdir /mnt/data
sudo mount /dev/mon_vg/vol_data /mnt/data
```

Le chemin du LV est toujours : `/dev/NOM_VG/NOM_LV`

Commandes LVM : Le mémo 📋

Action	Commande	Exemple
Créer PV	`pvcreate`	`pvcreate /dev/sdb`
Créer VG	`vgcreate`	`vgcreate mon_vg /dev/sdb`
Créer LV	`lvcreate`	`lvcreate -L 50G -n data mon_vg`
Voir PV	`pvs` ou `pvdisplay`	`pvs`
Voir VG	`vgs` ou `vgdisplay`	`vgs`
Voir LV	`lvs` ou `lvdisplay`	`lvs`
Agrandir LV	`lvextend`	`lvextend -L +10G /dev/vg/lv`
Réduire LV	`lvreduce`	`lvreduce -L -5G /dev/vg/lv`
Ajouter disque au VG	`vgextend`	`vgextend mon_vg /dev/sdc`

MINISTÈRE DES ARMÉES

Revenir au sommaire

Astuce mnémotechnique :

Voir l'état du LVM 🔍

```
# Vue rapide (recommandé)
sudo pvs    # Physical Volumes
sudo vgs    # Volume Groups
sudo lvs    # Logical Volumes

# Vue détaillée
sudo vgdisplay
sudo vgs

# Voir les LV
sudo lvdisplay
sudo lvs

# Agrandir un LV
sudo lvextend -L +10G /dev/mon_vg/vol_data
sudo resize2fs /dev/mon_vg/vol_data

# Réduire (attention, risque de perte de données)
sudo umount /dev/mon_vg/vol_data
sudo e2fsck -f /dev/mon_vg/vol_data
sudo resize2fs /dev/mon_vg/vol_data 40G
sudo lvreduce -L 40G /dev/mon_vg/vol_data
```

Snapshots LVM 📸

Snapshot : copie instantanée d'un volume

```
# Créer un snapshot
sudo lvcreate -L 5G -s -n snapshot_data /dev/mon_vg/vol_data

# Monter le snapshot
sudo mkdir /mnt/snapshot
sudo mount /dev/mon_vg/snapshot_data /mnt/snapshot

# Restaurer depuis le snapshot
sudo umount /dev/mon_vg/vol_data
sudo lvconvert --merge /dev/mon_vg/snapshot_data
# Au prochain reboot, le volume sera restauré

# Supprimer le snapshot
sudo lvremove /dev/mon_vg/snapshot_data
```

RAID : Redundant Array of Independent Disks 💾💾

RAID 0 : performance (striping)

- Données réparties sur tous les disques
- Si un disque meurt, tout est perdu

RAID 1 : miroir

- Données dupliquées sur tous les disques
- Tolérance de panne : N-1 disques

RAID 5 : parité distribuée

- Min 3 disques
- Tolérance : 1 disque

RAID 6 : double parité

- Min 4 disques
- Tolérance : 2 disques

RAID 10 : miroir + striping

- Min 4 disques
- Performance + redondance

Créer un RAID logiciel 🔧

```
# Installer mdadm
sudo apt install mdadm

# Créer un RAID 1
sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb1 /dev/sdc1

# Vérifier
cat /proc/mdstat
sudo mdadm --detail /dev/md0

# Créer le système de fichiers
sudo mkfs.ext4 /dev/md0

# Sauvegarder la config
sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf
```

Gérer un RAID 🛠️

```
# Marquer un disque défaillant
sudo mdadm --manage /dev/md0 --fail /dev/sdb1

# Retirer le disque
sudo mdadm --manage /dev/md0 --remove /dev/sdb1

# Ajouter un nouveau disque
sudo mdadm --manage /dev/md0 --add /dev/sdd1

# Surveiller l'état
watch cat /proc/mdstat
```

NFS : partage réseau

Server side (export) :

```
# Installer NFS server
sudo apt install nfs-kernel-server

# Configurer /etc/exports
sudo nano /etc/exports

# Ajouter :
/data  192.168.1.0/24(rw,sync,no_subtree_check)

# Appliquer
sudo exportfs -a
# Note: Le redémarrage du service sera vu dans le Module 6
```

NFS : client side 🖥️

```
# Installer NFS client
sudo apt install nfs-common

# Monter temporairement
sudo mount server:/data /mnt/nfs

# Monter automatiquement dans /etc/fstab
server:/data  /mnt/nfs  nfs  defaults  0  0

# Vérifier
df -h | grep nfs
```

Quota disque : rappel 💾

Configurer et surveiller les quotas

```
# Vérifier les quotas d'un utilisateur
quota -u alice

# Rapport détaillé
sudo repquota -a

# Quota d'un groupe
sudo quota -g developers

# Grace period restante
sudo repquota -a | grep alice
```

Nettoyer l'espace disque 🧹

Trouver les gros fichiers :

```
# Les 20 plus gros fichiers
find / -type f -printf '%s %p\n' 2>/dev/null | sort -nr | head -20

# Fichiers > 100M modifiés récemment
find /home -type f -size +100M -mtime -30

# Fichiers log volumineux
du -sh /var/log/*
```

Nettoyer l'espace disque (suite) 🧹

Nettoyer le cache apt :

```
sudo apt clean
sudo apt autoclean
sudo apt autoremove
```

Nettoyer les logs :

```
sudo journalctl --vacuum-time=7d
sudo find /var/log -name "*.gz" -mtime +30 -delete
```

Nettoyer l'espace disque (suite) 🧹

Trouver les doublons :

```
# Installer fdupes
sudo apt install fdupes

# Trouver les doublons
fdupes -r /home/alice

# Supprimer interactivement
fdupes -rd /home/alice
```

Analyser l'utilisation I/O

```
# Installer iotop
sudo apt install iotop

# Surveiller l'I/O par processus
sudo iotop

# iostat : statistiques I/O
sudo apt install sysstat
iostat -x 2    # Rafraîchir toutes les 2s

# I/O wait (attente disque)
top
# Regarder la colonne %wa
```

Performance : options noatime 🚀

atime : dernière fois qu'un fichier a été lu

Problème : chaque lecture génère une écriture (mise à jour atime)

Solution :

```
# Dans /etc/fstab
UUID=xxx  /home  ext4  defaults,noatime  0  2

# Ou relatime (compromis)
UUID=xxx  /home  ext4  defaults,relatime  0  2
```

Gain de performance : significatif sur serveurs avec beaucoup de lectures

Bonnes pratiques 📋

1. **Toujours utiliser UUID dans /etc/fstab**
   - Les /dev/sdX peuvent changer
2. **Tester fstab avant de rebooter**
   - `sudo mount -a`
3. **Sauvegarder avant des opérations dangereuses**
   - Partitionnement, resize, etc.
4. **Monitorer l'espace disque régulièrement**
   - Alertes à 80% d'utilisation

Bonnes pratiques (suite) 📋

5. **Utiliser LVM pour la flexibilité**
   - Agrandissement facile
   - Snapshots avant mise à jour
6. **ACL pour permissions complexes**
   - Plus maintenable que chmod
7. **noatime pour la performance**
   - Surtout sur les serveurs
8. **RAID ou backup pour les données critiques**
   - Jamais les deux en même temps !

Récapitulatif du Module 5 ✅

Ce que vous avez appris :

- ✅ Nomenclature des disques et partitions
- ✅ Partitionnement (fdisk, gdisk, parted)
- ✅ Systèmes de fichiers (ext4, xfs, btrfs, NFS)
- ✅ Montage et /etc/fstab
- ✅ Swap et swappiness
- ✅ Liens symboliques et durs
- ✅ ACL (permissions étendues)
- ✅ LVM pour la gestion flexible
- ✅ RAID pour la redondance
- ✅ Gestion de l'espace disque (df, du, ncdu)

Prochaine étape 🎯

Module 6 : Gestion des processus et services

- Comprendre les processus et leurs états
- Gestion avec systemd et systemctl
- Gestion du pare-feu (sera vu dans Module 7)
- SSH et transfert de fichiers sécurisé
- Résolution DNS
- Configuration d'interfaces

Questions ? 🤔

N’hésitez pas à poser vos questions maintenant !

Le prochain module explorera la configuration et le diagnostic réseau.

QCM Module 5 : Stockage et fichiers ✅

10 questions sur disques, partitions et systèmes de fichiers

Question 1

Quelle commande affiche l'espace disque disponible ?

A) `du`

B) `df`

C) `disk`

D) `free`

Question 2

Où sont stockées les configurations de montage automatique ?

A) `/etc/mount`

B) `/etc/fstab`

C) `/etc/filesystems`

D) `/boot/mount.conf`

Question 3

Quelle commande monte un système de fichiers ?

A) `mount`

B) `attach`

C) `connect`

D) `link`

Revenir au sommaire

Question 4

Que signifie LVM ?

A) Linux Virtual Memory

B) Logical Volume Manager

C) Linux Volume Management

D) Local Virtual Machine

Question 5

Comment créer un lien symbolique ?

A) `link`

B) `ln`

C) `ln -s`

D) `symlink`

Revenir au sommaire

Question 6

Que fait `fsck` ?

A) Monte un système de fichiers

B) Vérifie l'intégrité d'un système de fichiers

C) Formate un disque

D) Affiche l'espace disque

Revenir au sommaire

Question 7

Qu’est-ce qu’une ACL ?

A) Un type de partition

B) Un système de fichiers

C) Une permission étendue (Access Control List)

D) Un type de disque

Revenir au sommaire

Question 8

Quelle commande affiche les ACL d'un fichier ?

A) `acl`

B) `getfacl`

C) `showacl`

D) `chmod`

Revenir au sommaire

Question 9

Qu’est-ce que le swap ?

A) Une partition de sauvegarde

B) Une extension de la RAM sur disque

C) Un type de RAID

D) Un système de fichiers

Revenir au sommaire

Question 10

Quelle commande liste les disques et partitions ?

A) `disks`

B) `partitions`

C) `lsblk`

D) `listdisk`

Réponses du Module 5

Réponse 1 : B) `df`

- `df` (Disk Free) affiche l'espace par partition. `du` affiche l'usage par répertoire

Réponse 2 : B) `/etc/fstab`

- fstab = File System TABle

Réponse 3 : A) `mount`

- `mount /dev/sdb1 /mnt/data` monte la partition

Réponse 4 : B) Logical Volume Manager

- LVM permet une gestion flexible du stockage

Réponse 5 : C) `ln -s`

- `ln -s cible lien` crée un lien symbolique

Revenir au sommaire

Réponses du Module 5 (suite) 📝

Réponse 6 : B) Vérifie l'intégrité d'un système de fichiers

- ⚠️ Ne jamais fsck une partition montée !

Réponse 7 : C) Une permission étendue (Access Control List)

- Les ACL permettent des permissions plus fines que rwx

Réponse 8 : B) `getfacl`

- `setfacl` pour définir, `getfacl` pour lire

Réponse 9 : B) Une extension de la RAM sur disque

- Utilisé quand la RAM est pleine

Réponse 10 : C) `lsblk`

- lsblk = list block devices

Revenir au sommaire

Score du Module 5 📊

Comptez vos bonnes réponses :

- **9-10** : Excellent ! ⭐⭐⭐
- **7-8** : Bien ! ⭐⭐
- **5-6** : À revoir ⭐
- **< 5** : Relire le module 4

🎯 Exercices pratiques - Module 5

Gestion du stockage et du système de fichiers

Exercice 1 : Analyse de l'espace disque 💾

Objectif : Identifier l'utilisation du disque

Consignes :

1. Affichez l'espace disque de toutes les partitions
2. Trouvez le répertoire qui occupe le plus d'espace dans `/var`
3. Listez tous les fichiers de plus de 100 Mo
4. Affichez l'utilisation des inodes
5. Trouvez les 10 plus gros fichiers du système

Solution Exercice 1 💡

```
# 1. Espace disque
df -h

# 2. Plus gros répertoires dans /var
sudo du -h /var --max-depth=1 | sort -hr | head -10

# 3. Fichiers > 100 Mo
sudo find / -type f -size +100M 2>/dev/null
# Avec détails
sudo find / -type f -size +100M -exec ls -lh {} \; 2>/dev/null

# 4. Utilisation des inodes
df -i

# 5. Top 10 plus gros fichiers
sudo find / -type f -exec du -h {} + 2>/dev/null | sort -hr | head -10
```

Exercice 2 : Manipulation de fichiers image

Objectif : Créer et utiliser des fichiers image

Consignes :

1. Créez un fichier image de 100 Mo
2. Formatez-le en ext4
3. Montez-le dans `/mnt/test`
4. Créez des fichiers dedans
5. Démontez proprement
6. Remontez et vérifiez que les fichiers sont toujours là

Revenir au sommaire

Solution Exercice 2 💡

```
# 1. Créer fichier image
dd if=/dev/zero of=~/disk.img bs=1M count=100

# 2. Formater en ext4
sudo mkfs.ext4 ~/disk.img

# 3. Monter
sudo mkdir -p /mnt/test
sudo mount -o loop ~/disk.img /mnt/test

# Vérifier
df -h /mnt/test
mount | grep test

# 4. Créer des fichiers
sudo touch /mnt/test/fichier{1..5}.txt
echo "Test" | sudo tee /mnt/test/fichier1.txt
ls -l /mnt/test/

# 5. Démonter
sudo umount /mnt/test

# 6. Remonter et vérifier
sudo mount -o loop ~/disk.img /mnt/test
ls -l /mnt/test/
# Les fichiers sont toujours là !

# Nettoyer
sudo umount /mnt/test
rm ~/disk.img
```

Exercice 3 : Liens symboliques et durs 🔗

Objectif : Comprendre les liens

Consignes :

1. Créez un fichier `original.txt` avec du contenu
2. Créez un lien dur `hard_link.txt`
3. Créez un lien symbolique `soft_link.txt`
4. Comparez les inodes
5. Supprimez `original.txt` et voyez ce qui se passe
6. Créez un lien symbolique vers un répertoire

Solution Exercice 3 💡

```
# 1. Fichier original
echo "Contenu original" > /tmp/original.txt

# 2. Lien dur
ln /tmp/original.txt /tmp/hard_link.txt

# 3. Lien symbolique
ln -s /tmp/original.txt /tmp/soft_link.txt

# 4. Comparer inodes
ls -li /tmp/*.txt
# original et hard_link ont le MÊME inode
# soft_link a un inode différent

# Vérifier le contenu
cat /tmp/hard_link.txt
cat /tmp/soft_link.txt

# 5. Supprimer l'original
rm /tmp/original.txt

# hard_link fonctionne toujours !
cat /tmp/hard_link.txt  # ✅ OK

# soft_link est cassé !
cat /tmp/soft_link.txt  # ❌ No such file

# 6. Lien vers répertoire
ln -s /var/log /tmp/logs_link
ls -l /tmp/logs_link

# Nettoyer
rm /tmp/hard_link.txt /tmp/soft_link.txt /tmp/logs_link
```

Exercice 4 : ACL - Permissions étendues 🔐

Objectif : Utiliser les ACL pour des permissions fines

Consignes :

1. Créez un fichier `projet.txt`
2. Donnez à `alice` la permission de lecture (en plus des permissions Unix)
3. Donnez à `bob` la permission d'écriture
4. Affichez les ACL
5. Supprimez les ACL de `bob`
6. Supprimez toutes les ACL

Solution Exercice 4 💡

```
# 1. Créer le fichier
touch /tmp/projet.txt
echo "Projet secret" > /tmp/projet.txt

# 2. ACL pour alice (lecture)
setfacl -m u:alice:r /tmp/projet.txt

# 3. ACL pour bob (écriture)
setfacl -m u:bob:rw /tmp/projet.txt

# 4. Afficher les ACL
getfacl /tmp/projet.txt

# ls avec ACL
ls -l /tmp/projet.txt
# Un '+' apparaît : -rw-rw-r--+

# 5. Supprimer ACL de bob
setfacl -x u:bob /tmp/projet.txt
getfacl /tmp/projet.txt

# 6. Supprimer toutes les ACL
setfacl -b /tmp/projet.txt
getfacl /tmp/projet.txt
```

Exercice 5 : Montage automatique avec fstab 📋

Objectif : Configurer des montages automatiques

Consignes :

1. Créez un fichier image et formatez-le
2. Créez un point de montage `/data`
3. Ajoutez une entrée dans `/etc/fstab` pour montage automatique
4. Testez avec `mount -a`
5. Redémarrez et vérifiez que c'est monté automatiquement

⚠️ **Attention :** Erreur dans fstab = système qui ne boot pas !

Solution Exercice 5 💡

```
# 1. Créer et formater
dd if=/dev/zero of=/opt/data.img bs=1M count=200
sudo mkfs.ext4 /opt/data.img

# 2. Point de montage
sudo mkdir /data

# 3. Sauvegarder fstab avant modification
sudo cp /etc/fstab /etc/fstab.backup

# Ajouter entrée
echo "/opt/data.img /data ext4 loop,defaults 0 0" | \
    sudo tee -a /etc/fstab

# 4. Tester
sudo mount -a
df -h /data

# Si erreur, restaurer :
# sudo cp /etc/fstab.backup /etc/fstab

# 5. Redémarrer
sudo reboot

# Après reboot, vérifier
df -h /data
mount | grep data
```

Exercice bonus : Script de nettoyage disque 🧹

Objectif : Automatiser le nettoyage

Créez un script qui :

- Supprime les fichiers temp de plus de 7 jours
- Vide les corbeilles
- Nettoie les caches apt
- Nettoie les vieux logs
- Affiche un rapport

Solution Exercice bonus 💡

```bash
#!/bin/bash
# Script de nettoyage disque

echo "===== Nettoyage disque $(date) ====="

# Espace avant
BEFORE=$(df -h / | awk 'NR==2 {print $4}')
echo "Espace libre avant : $BEFORE"

# 1. Fichiers temp > 7 jours
echo -e "\n🗑️  Nettoyage /tmp..."
sudo find /tmp -type f -atime +7 -delete 2>/dev/null
FILES_DELETED=$(sudo find /tmp -type f -atime +7 2>/dev/null | wc -l)
echo "Fichiers supprimés : $FILES_DELETED"

# 2. Caches apt
echo -e "\n📦 Nettoyage apt..."
sudo apt clean
sudo apt autoclean
sudo apt autoremove -y

# 3. Logs anciens
echo -e "\n📜 Nettoyage logs..."
sudo journalctl --vacuum-time=7d
sudo find /var/log -name "*.log.*" -mtime +30 -delete

# 4. Thumbnails
echo -e "\n🖼️  Nettoyage thumbnails..."
rm -rf ~/.cache/thumbnails/*

# Espace après
AFTER=$(df -h / | awk 'NR==2 {print $4}')
echo -e "\n✅ Espace libre après : $AFTER"
echo "============================="
```

Revenir au sommaire

Points clés à retenir 📌

Gestion disque :

- `df` pour partitions, `du` pour répertoires
- `lsblk` pour voir l'arborescence
- `mount` / `umount` pour monter/démonter

Liens :

- Lien dur : même inode, même contenu
- Lien symbolique : pointeur, peut casser

ACL :

- Permissions plus fines que rwx
- `setfacl` pour définir, `getfacl` pour lire

Module 6

Gestion des processus et des services

Maîtriser l'exécution et les services système

Revenir au sommaire

Qu'est-ce qu'un processus ? 🏃

Un processus = un programme en cours d'exécution

Différence programme vs processus :

Programme	Processus
Fichier sur le disque	Instance en mémoire
Statique (ne bouge pas)	Dynamique (s'exécute)
Un seul exemplaire	Plusieurs copies possibles
`/usr/bin/firefox`	Firefox qui tourne sur votre écran

Analogie : Le restaurant 🍽️

Imaginez un restaurant :

Concept Linux	Analogie Restaurant
Programme	La recette de cuisine (dans un livre)
Processus	Le cuisinier qui prépare le plat
PID	Le numéro de commande (ticket)
CPU	Le fourneau (un seul à la fois peut l'utiliser)
Mémoire	Le plan de travail du cuisinier
Fichiers ouverts	Les ingrédients sortis du frigo

Un même programme peut créer plusieurs processus :

- La même recette (Firefox) peut être préparée par 3 cuisiniers (3 onglets = 3 processus)

Le PID : Process ID 🆔

PID : identifiant unique d'un processus (comme un numéro de ticket)

```
# Voir les processus en cours
ps

# Afficher avec les PID
ps aux

# PID 1 : le processus init (systemd) - LE CHEF !
ps -p 1
```

Caractéristiques :

- Numéro unique attribué à la création
- **PID 1 = systemd** (le "patron" qui démarre tous les autres)
- Réutilisé après la terminaison du processus
- Maximum : 32768 par défaut

💡 **Astuce :** `$$` donne le PID du shell actuel : `echo $$`

PPID : Parent Process ID

Chaque processus a un parent (comme un arbre généalogique)

```
# Voir PID et PPID
ps -ef

# Arbre des processus (très visuel !)
pstree

# Détails d'un processus
ps -fp 1234
```

L'arbre généalogique des processus 🌳

```
systemd (PID 1) ← L'ancêtre de TOUS les processus
├── sshd (PID 500) ← Service SSH
│   └── sshd (PID 1200) ← Votre session SSH
│       └── bash (PID 1201) ← Votre shell
│           └── vim (PID 1500) ← L'éditeur que vous avez lancé
├── nginx (PID 600)
└── cron (PID 700)
```

Règles de la famille :

- **Tout le monde descend de PID 1** (systemd est l'ancêtre commun)
- **Si un parent meurt** → les enfants sont "adoptés" par PID 1
- **Processus zombie** → enfant mort mais parent n'a pas encore lu l'acte de décès

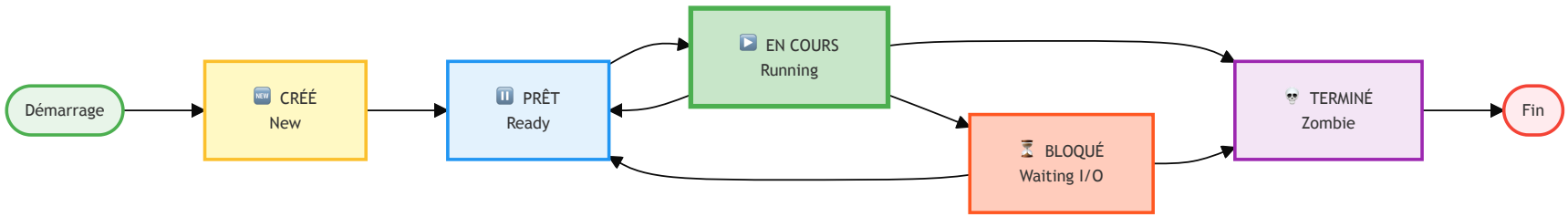
```
# Voir l'arbre complet
pstree -p    # Avec les PIDs
pstree -u    # Avec les utilisateurs
```

Anatomie d'un processus 🧬

Composants d'un processus :

1. **Code** : instructions du programme
2. **Données** : variables globales
3. **Pile (stack)** : variables locales, appels de fonctions
4. **Tas (heap)** : allocation dynamique
5. **Descripteurs de fichiers** : fichiers ouverts, sockets
6. **Registres CPU** : état du processeur
7. **PID, PPID, UID, GID** : identifiants

États d'un processus 🔄



Revenir au sommaire

Les états : Analogie de la salle d'attente 🏥

Imaginez une salle d'attente chez le médecin (le CPU) :

État	Lettre	Analogie	Exemple concret
Running	`R`	Vous êtes avec le médecin !	`firefox` affiche une page
Sleeping	`S`	Vous attendez qu'on vous appelle	`bash` attend votre commande
Disk Sleep	`D`	Vous attendez vos résultats d'analyse	Lecture d'un gros fichier
Stopped	`T`	Vous êtes sorti fumer (pause)	Processus après Ctrl+Z
Zombie	`Z`	Consultation terminée, mais pas encore parti	Processus fini, parent pas informé

Les états en détail 📊

Running ® : en cours d'exécution sur un CPU

- 💡 Le processus travaille activement

Sleeping (S) : en attente, peut être réveillé

- 💡 Attend une entrée utilisateur, un timer, etc.

Disk Sleep (D) : en attente d'I/O disque

- ⚠️ **Ne peut PAS être interrompu** (même pas par kill -9 !)
- 💡 Si bloqué longtemps → problème matériel probable

Stopped (T) : mis en pause (Ctrl+Z)

- 💡 Peut être repris avec `fg` ou `bg`

Zombie (Z) : mort mais pas enterré

- 💡 Le parent doit "récupérer" le code de sortie

Revenir au sommaire

Voir les processus : ps 🔍

```
# Processus de votre session
ps

# Tous les processus (format BSD)
ps aux

# Tous les processus (format System V)
ps -ef

# Processus d'un utilisateur
ps -u alice

# Processus avec un nom spécifique
ps aux | grep nginx
```

Format de sortie de ps aux 📋

```
ps aux
```

```
 USER        PID  %CPU %MEM    VSZ   RSS TTY    STAT START   TIME COMMAND
root          1   0.0  0.1 169316 11704 ?      Ss   10:00   0:02 /sbin/init
alice      1234   0.5  2.3 423680 94532 ?      Sl   10:15   1:23 firefox
bob        5678   0.0  0.1  12456  4532 pts/0  R+   14:23   0:00 ps aux
```

Décryptage de ps aux colonne par colonne 🔍

Colonne	Signification	Exemple	Explication
`USER`	Propriétaire	`alice`	Qui a lancé le processus
`PID`	Process ID	`1234`	Numéro unique du processus
`%CPU`	Utilisation CPU	`0.5`	0.5% du CPU utilisé
`%MEM`	Utilisation RAM	`2.3`	2.3% de la RAM utilisée
`VSZ`	Mémoire virtuelle	`423680`	En Ko (demandée, pas forcément utilisée)
`RSS`	Mémoire réelle	`94532`	En Ko (réellement en RAM)
`TTY`	Terminal	`pts/0` ou `?`	`?` = pas de terminal (daemon)
`STAT`	État	`Sl`	S=sleeping, l=multi-thread
`START`	Heure de début	`10:15`	Quand le processus a démarré
`TIME`	Temps CPU	`1:23`	1 min 23 sec de CPU utilisé

Astuce : VSZ vs RSS - Quelle différence ? 🤔

VSZ (Virtual Size) = Ce que le processus a **demandé**

RSS (Resident Set Size) = Ce qu'il **utilise vraiment** en RAM

Analogie du restaurant 🍽️ :

- **VSZ** = Vous réservez une table de 10 personnes (au cas où)
- **RSS** = Seulement 4 personnes sont venues (vraiment occupé)

En pratique :

```
firefox : VSZ=423680 RSS=94532
          (413 Mo demandés, 92 Mo vraiment utilisés)
```

💡 **Pour surveiller la mémoire, regardez RSS, pas VSZ !**

Les codes STAT déchiffrés 🔐

Codes principaux :

- `R` : Running (en cours)
- `S` : Sleeping (sommeil interruptible)
- `D` : Disk sleep (sommeil non interruptible)
- `T` : Stopped (arrêté)
- `Z` : Zombie

Modificateurs :

- `<` : haute priorité
- `N` : basse priorité
- `+` : au premier plan (foreground)
- `s` : leader de session
- `l` : multi-threadé

Voir les processus en temps réel : top ⏱️

```
top
```

Interface interactive :

- `q` : quitter
- `h` : aide
- `k` : tuer un processus
- `r` : changer la priorité (renice)
- `M` : trier par mémoire
- `P` : trier par CPU
- `u` : filtrer par utilisateur
- `1` : afficher tous les CPU

Alternative moderne : htop 🎨

```
# Installer htop
sudo apt install htop          # Debian/Ubuntu
sudo dnf install htop          # RHEL/Fedora

# Lancer
htop
```

Avantages :

- Interface colorée
- Navigation intuitive (flèches, F-keys)
- Arbre des processus
- Recherche facile
- Kill/renice intuitif

Trouver un processus : pgrep 🔎

```
# Trouver le PID d'un processus par nom
pgrep nginx
# → 1234

# Avec détails
pgrep -l nginx
# → 1234 nginx

# Processus d'un utilisateur
pgrep -u alice

# Premier match uniquement
pgrep -n firefox

# Dernier match uniquement
pgrep -o firefox
```

Signaux Unix : communiquer avec les processus 📡

Un signal = un message/ordre envoyé à un processus

Analogie : Le bureau et le téléphone 📞

Imaginez que chaque processus est un employé dans un bureau. Les signaux sont des appels téléphoniques :

Signal	Analogie téléphone	Ce que fait le processus
SIGTERM	"Merci de ranger et partir"	Sauvegarde et quitte proprement
SIGKILL	Agent de sécurité qui vous sort	Arrêt immédiat, pas de sauvegarde !
SIGSTOP	"Pause café !"	Se fige, attend SIGCONT
SIGHUP	"Relis tes instructions"	Recharge sa configuration

```
# Envoyer un signal
kill -SIGNAL PID
```

SIGTERM vs SIGKILL : La différence cruciale ⚠️

SIGTERM (15) = "S'il te plaît, arrête-toi"

```
kill 1234        # Envoie SIGTERM par défaut
kill -15 1234    # Même chose, explicite
kill -TERM 1234  # Même chose, avec le nom
```

- ✅ Le processus **peut** l'ignorer ou le gérer
- ✅ Il a le temps de **sauvegarder** ses données
- ✅ Il **ferme proprement** les fichiers et connexions

SIGKILL (9) = "STOP ! Tu n'as pas le choix !"

```
kill -9 1234
kill -KILL 1234
```

- ❌ Le processus **ne peut PAS** l'ignorer
- ❌ **Aucune sauvegarde** possible
- ❌ Fichiers temporaires non nettoyés

MINISTÈRE DES ARMÉES

À utiliser en DERNIER recours !

Revenir au sommaire

Signaux courants 🚦

SIGTERM (15) : terminaison propre (par défaut)

- Le processus peut nettoyer avant de quitter
- Enregistrer des données, fermer des fichiers
- **Toujours essayer en premier !**

SIGKILL (9) : terminaison immédiate

- Le processus est tué de force
- Pas de nettoyage possible
- **Utilisez en dernier recours !**

SIGHUP (1) : rechargement de configuration

- "Hang up" - historiquement : déconnexion du terminal
- Souvent utilisé pour recharger la config

SIGINT (2) : interruption (Ctrl+C)

SIGSTOP (19) : pause (Ctrl+Z)

SIGCONT (18) : reprise après pause

Utiliser kill 🔫

```
# Terminaison propre (SIGTERM)
kill 1234
kill -15 1234
kill -TERM 1234

# Terminaison forcée (SIGKILL)
kill -9 1234
kill -KILL 1234

# Rechargement de config
kill -HUP 1234

# Lister tous les signaux disponibles
kill -l
```

killall et pkill 🎯

```
# Tuer tous les processus avec un nom
killall firefox

# Tuer par motif
pkill -f "python.*script.py"

# Tuer les processus d'un utilisateur
pkill -u alice

# Avec un signal spécifique
killall -9 nginx
pkill -TERM firefox
```

⚠️ **Attention :** `killall` sur certains Unix (Solaris) tue TOUS les processus !

Revenir au sommaire

Priorité des processus : nice

Nice value : priorité d'un processus (-20 à +19)

- **-20** : priorité maximale (réservé root)
- **0** : priorité par défaut
- **+19** : priorité minimale

Analogie : la file d'attente au restaurant

- VIP (-20) : servi en premier
- Client normal (0) : dans l'ordre
- Pas pressé (+19) : attendra longtemps

Plus le nice est élevé, plus le processus est "gentil" en laissant les autres passer

Exemple pratique :

Imaginez que vous lancez un traitement vidéo qui prend 2 heures :

```
# Sans nice : utilise tout le CPU → votre navigateur rame
ffmpeg -i video.mp4 output.mp4

# Avec nice : laisse les autres processus passer en priorité
nice -n 15 ffmpeg -i video.mp4 output.mp4
# → Le traitement prendra peut-être 2h15, mais vous pouvez
#   travailler normalement pendant ce temps !
```

Cas d'usage réels :

- Sauvegardes nocturnes : nice +19 (non urgentes)
- Serveur web : nice 0 (priorité normale)
- Calculs critiques : nice -10 (urgent, root only)

Lancer un processus avec nice 🚀

```
# Lancer avec nice +10 (basse priorité)
nice -n 10 ./mon_script.sh

# Lancer avec nice -10 (haute priorité, root requis)
sudo nice -n -10 ./processus_important.sh

# Lancer en arrière-plan avec nice
nice -n 15 ./backup.sh &
```

Changer la priorité : renice 🔧

```
# Changer le nice d'un processus existant
renice 10 -p 1234

# Pour tous les processus d'un utilisateur
renice 15 -u alice

# Diminuer le nice (augmenter priorité) - root requis
sudo renice -5 -p 1234

# Dans htop : appuyez sur F7/F8 pour changer
```

Premier plan / Arrière-plan 🎭

```
# Lancer en arrière-plan
./long_script.sh &

# Voir les jobs en cours
jobs
# [1]  Running    ./long_script.sh &
# [2]- Running    ./autre.sh &
# [3]+ Stopped    nano fichier.txt

# Mettre en avant-plan
fg %1               # Job numéro 1
fg                  # Dernier job (+)

# Mettre en arrière-plan
bg %2
```

Suspendre et reprendre ⏸️▶️

```
# Lancer un programme
nano fichier.txt

# Suspendre (Ctrl+Z)
^Z
[1]+  Stopped    nano fichier.txt

# Reprendre en arrière-plan
bg

# Reprendre en avant-plan
fg

# Tuer un job
kill %1
```

nohup : processus persistant

nohup : "no hang up" - continue après déconnexion

```
# Lancer un processus qui survit à la déconnexion
nohup ./long_script.sh &

# Sortie redirigée vers nohup.out
nohup ./script.sh > output.log 2>&1 &

# Vérifier
cat nohup.out
```

Cas d'usage :

- Scripts qui tournent longtemps
- Connexion SSH instable
- Lancement de services manuellement

screen et tmux : multiplexeurs de terminal 📺

Outils pour gérer plusieurs sessions terminales

screen : classique et simple

```
# Installer
sudo apt install screen

# Nouvelle session
screen

# Détacher (Ctrl+A puis D)
# Lister les sessions
screen -ls

# Rattacher
screen -r

# Nouvelle fenêtre : Ctrl+A puis C
# Naviguer : Ctrl+A puis N (next) ou P (previous)
```

tmux : alternative moderne 🚀

```
# Installer
sudo apt install tmux

# Nouvelle session
tmux

# Détacher (Ctrl+B puis D)
# Lister les sessions
tmux ls

# Rattacher
tmux attach

# Nouvelle fenêtre : Ctrl+B puis C
# Split vertical : Ctrl+B puis %
# Split horizontal : Ctrl+B puis "
# Naviguer : Ctrl+B puis flèches
```

Qu'est-ce qu'un service ?

Service (ou daemon) : programme qui tourne **en permanence** en arrière-plan

Différence processus vs service :

Processus normal	Service (daemon)
Vous le lancez manuellement	Démarre automatiquement au boot
Se termine quand vous fermez	Tourne 24h/24
Interactif (vous tapez dedans)	Pas d'interface directe
Ex: `vim` , `firefox`	Ex: `nginx` , `sshd` , `mysql`

Analogie : Le personnel invisible d'un hôtel 🏨

Les services sont comme le staff d'un hôtel qui travaille 24h/24 :

Service Linux	Rôle dans l'hôtel	Ce qu'il fait
nginx/apache	Réceptionniste	Accueille les visiteurs web
sshd	Portier sécurité	Vérifie les badges et ouvre les portes
cron	Femme de ménage	Fait le ménage à heures fixes
rsyslog	Comptable	Note tout ce qui se passe
NetworkManager	Standardiste	Gère les communications
cups	Service courrier	Gère les impressions

Caractéristiques communes :

- 🌙 Travaillent même quand vous dormez
- 👻 Invisibles (pas de fenêtre)
- 🔄 Redémarrent automatiquement si crash

systemd : C'est quoi exactement ? 🎛️

systemd = le "chef d'orchestre" de votre système Linux

Analogie : Le directeur d'hôtel 🏨

Quand l'hôtel ouvre le matin (= quand Linux démarre) :

1. Le directeur (systemd) arrive en premier (PID 1)
2. Il réveille le personnel dans le bon ordre :
   - D'abord l'électricien (réseau)
   - Puis le réceptionniste (services web)
   - Enfin les femmes de chambre (tâches planifiées)
3. Si un employé tombe malade → le directeur le remplace automatiquement

Ce que systemd gère :

- 🚀 Démarrage des services (dans le bon ordre)
- 👀 Surveillance (redémarre si crash)
- 📝 Logs (journald)
- ⏰ Tâches planifiées (timers)

Pourquoi systemd a remplacé les anciens systèmes ?

Avant (SysVinit) :

```
# Scripts shell dans /etc/init.d/
/etc/init.d/nginx start
/etc/init.d/nginx stop
```

- ❌ Lent (scripts lancés un par un)

- ❌ Pas de gestion des dépendances
- ❌ Pas de redémarrage automatique

Maintenant (systemd) :

```
sudo systemctl start nginx
sudo systemctl stop nginx
```

- ✅ Rapide (services lancés en parallèle)

- ✅ Gère les dépendances automatiquement
- ✅ Redémarrage automatique si crash
- ✅ Logs centralisés (journalctl)

systemctl : contrôler les services 🎮

```
# Démarrer un service
sudo systemctl start nginx

# Arrêter un service
sudo systemctl stop nginx

# Redémarrer un service
sudo systemctl restart nginx

# Recharger la config (sans redémarrer)
sudo systemctl reload nginx

# Redémarrer ou recharger (intelligent)
sudo systemctl reload-or-restart nginx
```

systemctl : état des services

```
# Voir l'état d'un service
systemctl status nginx

# Vérifier si actif
systemctl is-active nginx

# Vérifier si activé au démarrage
systemctl is-enabled nginx

# Lister tous les services
systemctl list-units --type=service

# Lister les services qui ont échoué
systemctl --failed
```

Exemple de sortie systemctl status 📋

```
systemctl status nginx
```

```
● nginx.service - A high performance web server
     Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset: enabled)
     Active: active (running) since Mon 2025-11-06 10:00:00 CET; 2h 30min ago
       Docs: man:nginx(8)
   Main PID: 1234 (nginx)
      Tasks: 5 (limit: 4915)
     Memory: 12.5M
        CPU: 1.234s
     CGroup: /system.slice/nginx.service
             ├─1234 nginx: master process /usr/sbin/nginx
             └─1235 nginx: worker process

Nov 06 10:00:00 hostname systemd[1]: Starting A high performance web server...
Nov 06 10:00:00 hostname systemd[1]: Started A high performance web server.
```

Décryptage de systemctl status 🔍

Ligne	Signification
`● nginx.service`	● = en cours (🟢 vert = OK, 🔴 rouge = erreur)
`Loaded: loaded (...; enabled)`	Fichier trouvé + **activé au boot**
`Active: active (running)`	Service **en cours d'exécution**
`since Mon 2025-11-06 10:00:00`	Démarré depuis cette date
`Main PID: 1234`	PID du processus principal
`Tasks: 5`	Nombre de tâches/threads
`Memory: 12.5M`	RAM utilisée
`CGroup: ...`	Arbre des processus du service

États possibles de `Active:` :

- 🟢 `active (running)` → Tout va bien !
- 🟡 `inactive (dead)` → Arrêté normalement
- 🔴 `failed` → Erreur ! Regardez les logs avec `journalctl -u nginx`
- 🔵 `activating` → En cours de démarrage

Revenir au sommaire

start vs enable : La différence ! ⚠️

Confusion très courante chez les débutants !

Commande	Ce que ça fait	Analogie
`systemctl start nginx`	Démarre **maintenant**	"Réveille-toi !"
`systemctl enable nginx`	Démarre **au prochain boot**	"Programme ton réveil pour demain"

Exemple concret :

```
# Vous installez nginx
sudo apt install nginx

# Vous le démarrez MAINTENANT
sudo systemctl start nginx    # ✅ nginx tourne

# MAIS... au prochain reboot, nginx ne redémarrera pas !
# Il faut aussi l'activer au boot :
sudo systemctl enable nginx   # ✅ démarre automatiquement au boot
```

💡 **Astuce :** Faites les deux en une commande !

```
sudo systemctl enable --now nginx   # Active ET démarre
```

Activer/désactiver au démarrage

```
# Activer un service au démarrage (ne le démarre pas !)
sudo systemctl enable nginx

# Désactiver (ne sera plus lancé au boot)
sudo systemctl disable nginx

# Activer ET démarrer en une commande
sudo systemctl enable --now nginx

# Désactiver ET arrêter
sudo systemctl disable --now nginx

# Masquer un service (empêcher TOUT démarrage, même manuel)
sudo systemctl mask nginx

# Démasquer
sudo systemctl unmask nginx
```

Dépendances de services 🔗

Les services dépendent les uns des autres

```
# Voir les dépendances d'un service
systemctl list-dependencies nginx

# Voir qui dépend de ce service
systemctl list-dependencies nginx --reverse

# Voir uniquement les services (pas targets, etc.)
systemctl list-dependencies nginx --plain
```

Exemple : nginx dépend de network.target

Units systemd 📦

systemd gère plusieurs types d'unités :

- `.service` : services
- `.target` : groupes de services (comme les runlevels)
- `.socket` : sockets réseau/IPC
- `.timer` : tâches planifiées (comme cron)
- `.mount` : points de montage
- `.device` : périphériques
- `.path` : surveillance de fichiers/répertoires

```
# Lister tous les types d'unités
systemctl list-units --all
```

Créer un service systemd custom 🛠️

Fichier : `/etc/systemd/system/mon-service.service`

PS : nous prenons le cas d'une app python qui tourne en tant que service systemd.

```
[Unit]
Description=Mon service personnalisé
After=network.target

[Service]
Type=simple
User=alice
WorkingDirectory=/opt/mon-app
ExecStart=/usr/bin/python3 /opt/mon-app/server.py
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

Créer un service systemd (suite) 🛠️

```
# Recharger systemd après création/modification
sudo systemctl daemon-reload

# Activer le service
sudo systemctl enable mon-service

# Démarrer le service
sudo systemctl start mon-service

# Vérifier l'état
systemctl status mon-service
```

Types de services systemd 🎭

Type=simple : processus principal (par défaut)

Type=forking : processus qui fork (comme les daemons traditionnels)

Type=oneshot : s'exécute une fois et se termine

Type=notify : le service notifie systemd quand il est prêt

Type=idle : attend que tous les autres services soient démarrés

Redémarrage automatique 🔄

```
[Service]
Restart=always           # Toujours redémarrer
Restart=on-failure       # Seulement en cas d'échec
Restart=on-abnormal      # Seulement si terminaison anormale
Restart=on-abort         # Seulement si signal d'arrêt
Restart=on-watchdog      # Seulement si watchdog timeout

RestartSec=5s            # Attendre 5s avant de redémarrer
StartLimitBurst=3        # Max 3 redémarrages
StartLimitIntervalSec=60 # Dans une fenêtre de 60s
```

Targets systemd : les "runlevels" 🎯

Targets : groupes de services (équivalent des runlevels)

Targets principaux :

- `poweroff.target` : arrêt
- `rescue.target` : mode rescue (single-user)
- `multi-user.target` : multi-utilisateurs, pas de GUI
- `graphical.target` : multi-utilisateurs avec GUI
- `reboot.target` : redémarrage

```
# Target actuel
systemctl get-default

# Changer le target par défaut
sudo systemctl set-default multi-user.target
```

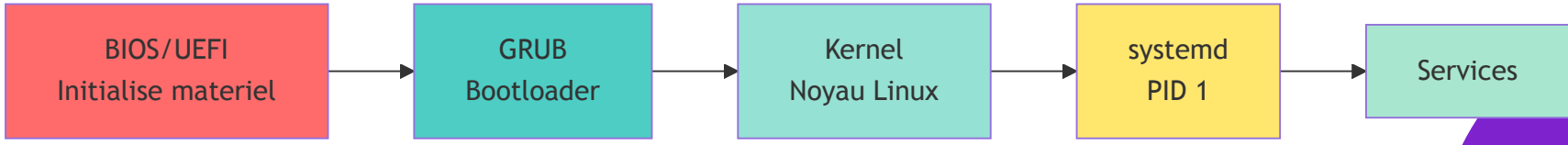
Changer de target (runlevel) 🔀

```
# Passer en mode rescue
sudo systemctl isolate rescue.target

# Passer en mode multi-user
sudo systemctl isolate multi-user.target

# Passer en mode graphique
sudo systemctl isolate graphical.target

# Équivalents traditionnels :
# init 1 → systemctl isolate rescue.target
# init 3 → systemctl isolate multi-user.target
# init 5 → systemctl isolate graphical.target
```

Le démarrage du système 🚀

Séquence de boot :

1. **BIOS/UEFI** : initialise le matériel
2. **Bootloader (GRUB)** : charge le noyau
3. **Noyau Linux** : initialise les périphériques
4. **systemd (PID 1)** : lance les services



Analyser le temps de démarrage ⏱️

```
# Temps total de démarrage
systemd-analyze

# Temps par service
systemd-analyze blame

# Chaîne critique
systemd-analyze critical-chain

# Graphique (génère un SVG)
systemd-analyze plot > boot.svg
```

Logs système : journald 📜

journalctl : voir les logs systemd (le "journal de bord" du système)

Analogie : La boîte noire d'un avion ✈️

Tout ce qui se passe sur votre système est enregistré :

- Services qui démarrent/s'arrêtent
- Erreurs et avertissements
- Connexions utilisateurs
- Problèmes matériels

journalctl : Les commandes essentielles 📋

```
# Tous les logs (attention, BEAUCOUP de données !)
journalctl

# Logs depuis le dernier boot (plus utile)
journalctl -b

# Logs d'UN service spécifique
journalctl -u nginx

# Suivre en temps réel (comme tail -f)
journalctl -f

# Suivre un service en temps réel
journalctl -u nginx -f

# Les 50 dernières lignes
journalctl -n 50
```

💡 **Astuce :** Quand un service ne marche pas → `journalctl -u le-service`

journalctl : filtres avancés 🔍

```
# Depuis une heure
journalctl --since "1 hour ago"

# Depuis une date
journalctl --since "2025-11-01"

# Période
journalctl --since "2025-11-01" --until "2025-11-02"

# Par utilisateur
journalctl _UID=1000

# Par priorité (erreur et plus grave)
journalctl -p err

# Inverser l'ordre (plus récent en premier)
journalctl -r
```

journalctl : format et export 📤

```
# Format JSON
journalctl -u nginx -o json

# Format court
journalctl -o short

# Format verbeux
journalctl -o verbose

# Export vers fichier
journalctl -u nginx > nginx.log

# Taille du journal
journalctl --disk-usage

# Nettoyer les vieux logs
sudo journalctl --vacuum-time=7d    # Garder 7 jours
sudo journalctl --vacuum-size=500M  # Garder 500 Mo
```

Arrêt et redémarrage du système 🔌

```
# Arrêt immédiat
sudo shutdown now
sudo poweroff
sudo systemctl poweroff

# Arrêt dans 10 minutes
sudo shutdown +10

# Arrêt à une heure précise
sudo shutdown 22:30

# Redémarrage
sudo reboot
sudo systemctl reboot
sudo shutdown -r now

# Annuler un shutdown planifié
sudo shutdown -c
```

Niveaux d'arrêt 🛑

halt : arrête le CPU (système reste sous tension)

```
sudo halt
```

poweroff : coupe l'alimentation

```
sudo poweroff
```

reboot : redémarre

```
sudo reboot
```

Différence : subtile et dépend du matériel

- Modern practice : utiliser `systemctl`

Processus zombie

Zombie : processus terminé mais pas "récupéré" par le parent

```
# Identifier les zombies
ps aux | grep Z

# Ou
ps aux | awk '$8 == "Z"'
```

Comment les éliminer ?

- Impossible de tuer un zombie (déjà mort !)
- Il faut tuer le parent ou attendre qu'il récupère le code de sortie
- Les zombies ne consomment pas de ressources (juste une entrée dans la table)

Les processus orphelins 👻

Orphelin : processus dont le parent est mort

Ce qui se passe :

- Le processus est adopté par init (PID 1)
- systemd nettoie automatiquement

Pas un problème généralement, contrairement aux zombies

Surveiller l'utilisation des ressources

```
# Utilisation globale
top
htop

# Utilisation par processus
ps aux --sort=-%mem | head -10    # Top 10 mémoire
ps aux --sort=-%cpu | head -10    # Top 10 CPU

# Statistiques I/O
sudo iotop

# Statistiques réseau par processus
sudo nethogs
```

uptime et load average ⚖️

```
uptime
# 14:23:45 up 5 days, 3:15, 2 users, load average: 0.52, 0.38, 0.31
```

Load average : nombre moyen de processus en attente du CPU

- Trois valeurs : 1 min, 5 min, 15 min
- `< nombre de CPUs` : OK
- `= nombre de CPUs` : pleine charge
- `> nombre de CPUs` : surchargé

Exemple : 4 CPUs

- Load 2.0 : OK (50% d'utilisation)
- Load 4.0 : pleine charge
- Load 8.0 : 2x trop de travail

Limites de ressources : ulimit 🚧

ulimit : limiter les ressources d'un processus

```
# Voir les limites actuelles
ulimit -a

# Limite de fichiers ouverts
ulimit -n 4096

# Limite de taille de fichier
ulimit -f 10000   # En blocs de 512 octets

# Limite de processus
ulimit -u 200

# Limite de mémoire
ulimit -m 1000000  # En Ko
```

Limites permanentes : /etc/security/limits.conf 📝

```
sudo nano /etc/security/limits.conf
```

Format : `domain type item value`

```
 alice    soft    nproc    100
alice    hard    nproc    150
@devs    soft    nofile   4096
@devs    hard    nofile   8192
*        soft    core     0
```

- **soft** : limite par défaut (peut être augmentée par l'utilisateur)
- **hard** : limite maximale (root uniquement)

cgroups : C'est quoi ? 🎛️

cgroups (control groups) = système pour **limiter et isoler** les ressources des processus

Analogie : Le budget familial 💰

Imaginez une famille avec un budget limité :

Ressource système	Analogie budget	Exemple
CPU	Temps de travail	"Tu n'as droit qu'à 2h de TV par jour"
Mémoire (RAM)	Argent de poche	"Tu as 50€ max par mois"
I/O disque	Vitesse de dépense	"Pas plus de 10€ par jour"
Réseau	Forfait téléphone	"2 Go de data max"

Sans cgroups : Un processus gourmand peut tout prendre (comme un enfant qui mange tout le gâteau 🍰)

Avec cgroups : Chaque processus a sa "part" définie (équité garantie)

Revenir au sommaire

cgroups : Pourquoi c'est important ? 🤔

Problème sans cgroups :

```
Serveur avec 8 Go de RAM
├── nginx (serveur web) : utilise 500 Mo ✅
├── mysql (base de données) : utilise 2 Go ✅
└── script_bugué.py : FUITE MÉMOIRE → utilise 10 Go ! 💥
    → Tout le serveur plante !
```

Solution avec cgroups :

```
Serveur avec 8 Go de RAM
├── nginx : limite 1 Go (utilise 500 Mo) ✅
├── mysql : limite 4 Go (utilise 2 Go) ✅
└── script_bugué.py : limite 1 Go → TUÉ quand il dépasse ! ✅
    → Le serveur continue de fonctionner !
```

Qui utilise les cgroups ?

- 🐳 **Docker** : isole chaque conteneur
- ⚙️ **systemd** : limite les services
- ☸️ **Kubernetes** : gère les ressources des pods

Limiter les ressources d'un service 🔒

Dans un fichier service systemd :

```
[Service]
# Limite CPU (50% max)
CPUQuota=50%

# Limite mémoire (500 Mo max, tué si dépassé)
MemoryMax=500M

# Limite I/O disque
IOReadBandwidthMax=/dev/sda 10M
IOWriteBandwidthMax=/dev/sda 5M

# Limite nombre de processus/threads
TasksMax=100
```

Exemple concret : Protéger votre serveur

Scénario : Vous avez un script Python qui parfois consomme trop de RAM

```
# Créer un service avec limites
sudo nano /etc/systemd/system/mon-script.service
```

```
[Unit]
Description=Mon script Python limité

[Service]
ExecStart=/usr/bin/python3 /opt/script.py
MemoryMax=256M       # Max 256 Mo de RAM
CPUQuota=25%         # Max 25% du CPU
Restart=on-failure   # Redémarre si tué pour dépassement

[Install]
WantedBy=multi-user.target
```

Résultat : Si le script dépasse 256 Mo → il est tué automatiquement et redémarré proprement !

Outils de diagnostic 🔧

```
# Informations système
uname -a                    # Noyau
hostname                    # Nom de la machine
uptime                      # Durée de fonctionnement

# Processus
ps aux
top
htop

# Services
systemctl status
systemctl --failed

# Logs
journalctl -xe
dmesg                       # Logs du noyau
```

Scénarios pratiques 💼

Service qui ne démarre pas :

```
# 1. Vérifier l'état
systemctl status nom-service

# 2. Voir les logs
journalctl -u nom-service -n 50

# 3. Vérifier la config
systemctl cat nom-service

# 4. Tester manuellement
/usr/bin/mon-programme --config /etc/config
```

Scénarios pratiques (suite) 💼

Système lent :

```
# 1. Vérifier la charge
uptime
top

# 2. Identifier les processus gourmands
ps aux --sort=-%cpu | head
ps aux --sort=-%mem | head

# 3. Vérifier l'I/O
sudo iotop

# 4. Vérifier les logs
journalctl -p err -n 50
```

Bonnes pratiques ✅

1. **Utiliser systemd pour les services**
   - Ne pas lancer les services manuellement en production
   - Configurer le restart automatique
2. **Surveiller les logs régulièrement**
   - `journalctl -f` pendant les déploiements
3. **Nommer explicitement les processus en arrière-plan**
   - Facilite le debugging
4. **Limiter les ressources des services**
   - Éviter qu'un service monopolise le système

Bonnes pratiques (suite) ✅

5. **Ne jamais faire `kill -9` en premier**
   - Toujours essayer `kill` (SIGTERM) d'abord
   - Laisser le processus nettoyer
6. **Utiliser les cgroups en production**
   - Isoler les applications critiques
7. **Monitorer les zombies**
   - Pas critique, mais symptôme d'un bug
8. **Automatiser avec systemd timers**
   - Plutôt que cron pour les tâches système

Récapitulatif du Module 6 ✅

Ce que vous avez appris :

- ✅ Anatomie et états des processus (PID, PPID, états)
- ✅ Gestion des processus (ps, top, htop, kill)
- ✅ Signaux Unix (SIGTERM, SIGKILL, SIGHUP, etc.)
- ✅ Priorités (nice, renice)
- ✅ Jobs et arrière-plan (fg, bg, nohup)
- ✅ systemd et gestion des services
- ✅ Création de services personnalisés
- ✅ Logs avec journalctl
- ✅ Démarrage et arrêt du système
- ✅ Limites de ressources (ulimit, cgroups)

Prochaine étape 🎯

Module 7 : Configuration réseau

- Configuration IP (ip, nmcli)
- SSH et connexions sécurisées
- Montage et /etc/fstab
- Permissions étendues (ACL)
- Liens symboliques et durs
- Gestion de l'espace disque

Questions ? 🤔

N’hésitez pas à poser vos questions maintenant !

Le prochain module explorera la gestion du stockage et des systèmes de fichiers.

QCM Module 6 : Processus et services ✅

10 questions sur la gestion des processus et systemd

Question 1

Quel est le PID du premier processus système ?

A) 0

B) 1

C) 10

D) 100

Question 2

Quelle commande affiche les processus en temps réel ?

A) `ps`

B) `top`

C) `pgrep`

D) `htop`

Question 3

Quel signal envoie `kill` par défaut ?

A) SIGKILL (9)

B) SIGTERM (15)

C) SIGHUP (1)

D) SIGINT (2)

Question 4

Comment démarrer un service avec systemd ?

A) `systemctl start service`

B) `service start`

C) `start service`

D) `systemd start service`

Question 5

Quelle est la différence entre `start` et `enable` ?

A) Aucune

B) `start` démarre maintenant, `enable` au boot

C) `enable` démarre maintenant, `start` au boot

D) Les deux font la même chose

Revenir au sommaire

Question 6

Où voir les logs d'un service systemd ?

A) `/var/log/service.log`

B) `journalctl -u service`

C) `systemctl logs service`

D) `dmesg`

Revenir au sommaire

Question 7

Comment changer la priorité d'un processus ?

A) `priority`

B) `nice` ou `renice`

C) `top`

D) `ps`

Revenir au sommaire

Question 8

Que fait `Ctrl+Z` ?

A) Tue le processus

B) Suspend le processus

C) Quitte le programme

D) Redémarre le processus

Revenir au sommaire

Question 9

Comment lister les services qui ont échoué ?

A) `systemctl failed`

B) `systemctl --failed`

C) `systemctl list-failed`

D) `journalctl --failed`

Question 10

Qu’est-ce qu’un processus zombie ?

A) Un processus qui consomme beaucoup de CPU

B) Un processus terminé mais non récupéré par le parent

C) Un processus malveillant

D) Un processus sans PID

Réponses du Module 6 📝

Réponse 1 : B) 1

- Le PID 1 est `systemd` (ou `init` sur les anciens systèmes)

Réponse 2 : B) `top` (ou D `htop`)

- Les deux affichent les processus en temps réel, `htop` est plus moderne

Réponse 3 : B) SIGTERM (15)

- SIGTERM permet au processus de se terminer proprement

Réponse 4 : A) `systemctl start service`

- systemd utilise la commande `systemctl`

Réponse 5 : B) `start` démarre maintenant, `enable` au boot

- `enable` configure le démarrage automatique

Revenir au sommaire

Réponses du Module 6 (suite) 📝

Réponse 6 : B) `journalctl -u service`

- journalctl est l'outil de logs de systemd

Réponse 7 : B) `nice` ou `renice`

- `nice` au lancement, `renice` pour un processus existant

Réponse 8 : B) Suspend le processus

- Le processus passe en arrière-plan suspendu. `fg` pour reprendre

Réponse 9 : B) `systemctl --failed`

- Affiche tous les services en état "failed"

Réponse 10 : B) Un processus terminé mais non récupéré par le parent

- Il reste dans la table des processus jusqu'à ce que le parent récupère son code de sortie

Revenir au sommaire

Score du Module 6 📊

Comptez vos bonnes réponses :

- **9-10** : Excellent ! ⭐⭐⭐
- **7-8** : Bien ! ⭐⭐
- **5-6** : À revoir ⭐
- **< 5** : Relire le module 3

Revenir au sommaire

🎯 Exercices pratiques - Module 6

Gestion des processus et des services

Exercice 1 : Gestion de processus 🏃

Objectif : Manipuler les processus

Consignes :

1. Lancez la commande `sleep 300` en arrière-plan
2. Listez les jobs en cours
3. Trouvez le PID du processus `sleep`
4. Changez sa priorité (nice +10)
5. Tuez le processus proprement
6. Vérifiez qu'il est bien terminé

Solution Exercice 1 💡

```
# 1. Lancer en arrière-plan
sleep 300 &

# 2. Lister les jobs
jobs
# [1]+  Running    sleep 300 &

# 3. Trouver le PID
pgrep sleep
# ou
ps aux | grep sleep
# ou avec jobs
jobs -l

# 4. Changer la priorité
PID=$(pgrep sleep)
sudo renice 10 -p $PID

# Vérifier
ps -o pid,ni,comm -p $PID

# 5. Tuer proprement (SIGTERM)
kill $PID

# 6. Vérifier
ps -p $PID
# → No such process
jobs
# → Terminated
```

Exercice 2 : Monitoring des processus

Objectif : Surveiller l'activité système

Consignes :

1. Affichez les 10 processus qui consomment le plus de CPU
2. Affichez les 10 processus qui consomment le plus de mémoire
3. Trouvez tous les processus appartenant à votre utilisateur
4. Comptez le nombre total de processus
5. Utilisez `top` pour surveiller en temps réel (puis quittez avec `q`)

Solution Exercice 2 💡

```
# 1. Top 10 CPU
ps aux --sort=-%cpu | head -11

# 2. Top 10 Mémoire
ps aux --sort=-%mem | head -11

# 3. Processus de l'utilisateur
ps -u $USER
# ou
ps aux | grep ^$USER

# 4. Nombre total
ps aux | wc -l
# ou
ps -e | wc -l

# 5. Monitoring temps réel
top
# Dans top :
# - M : trier par mémoire
# - P : trier par CPU
# - u : filtrer par utilisateur
# - k : tuer un processus
# - q : quitter
```

Exercice 3 : Services systemd 🎛️

Objectif : Gérer les services avec systemd

Consignes :

1. Listez tous les services actifs
2. Vérifiez l'état du service SSH
3. Redémarrez le service SSH (si installé)
4. Activez le démarrage automatique au boot
5. Listez les services qui ont échoué
6. Affichez les 50 dernières lignes de logs du service SSH

Solution Exercice 3 💡

```
# 1. Services actifs
systemctl list-units --type=service --state=active

# 2. État de SSH
systemctl status sshd
# ou
systemctl status ssh

# 3. Redémarrer
sudo systemctl restart sshd

# 4. Activer au boot
sudo systemctl enable sshd
# Vérifier
systemctl is-enabled sshd

# 5. Services échoués
systemctl --failed

# 6. Logs SSH
journalctl -u sshd -n 50
# ou en temps réel
journalctl -u sshd -f
```

Exercice 4 : Créer un service personnalisé 🛠️

Objectif : Créer et gérer son propre service

Consignes :

Créez un service qui :

1. Lance un script Python simple toutes les 10 secondes
2. Se relance automatiquement en cas de crash
3. Démarre automatiquement au boot
4. Enregistre des logs

Script Python à créer : `/opt/hello_service.py`

Solution Exercice 4 - Script 💡

```
# 1. Créer le script
sudo mkdir -p /opt
sudo tee /opt/hello_service.py << 'EOF'
#!/usr/bin/env python3
import time
import datetime

while True:
    now = datetime.datetime.now()
    print(f"[{now}] Service en cours d'exécution...")
    time.sleep(10)
EOF

# Rendre exécutable
sudo chmod +x /opt/hello_service.py
```

Solution Exercice 4 - Service 💡

```
# 2. Créer le service systemd
sudo tee /etc/systemd/system/hello.service << 'EOF'
[Unit]
Description=Hello Service
After=network.target

[Service]
Type=simple
User=nobody
ExecStart=/usr/bin/python3 /opt/hello_service.py
Restart=always
RestartSec=5s
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
EOF

# 3. Recharger systemd
sudo systemctl daemon-reload

# 4. Démarrer le service
sudo systemctl start hello.service

# 5. Activer au boot
sudo systemctl enable hello.service

# 6. Vérifier
systemctl status hello.service
journalctl -u hello.service -f
```

Exercice 5 : Analyse du démarrage 🚀

Objectif : Analyser les performances de démarrage

Consignes :

1. Affichez le temps total de démarrage
2. Listez les 10 services les plus lents à démarrer
3. Affichez la chaîne critique de démarrage
4. Générez un graphique du démarrage (SVG)
5. Désactivez un service inutile pour accélérer le boot

Solution Exercice 5 💡

```
# 1. Temps total
systemd-analyze

# 2. Top 10 services lents
systemd-analyze blame | head -10

# 3. Chaîne critique
systemd-analyze critical-chain

# 4. Graphique
systemd-analyze plot > /tmp/boot.svg
# Ouvrir avec un navigateur
firefox /tmp/boot.svg &

# 5. Désactiver un service inutile
# Exemple : bluetooth si non utilisé
sudo systemctl disable bluetooth.service
# Voir la différence
systemd-analyze
```

Exercice 6 : Gestion des logs 📜

Objectif : Maîtriser journalctl

Consignes :

1. Affichez les logs depuis le dernier boot
2. Affichez seulement les erreurs
3. Suivez les logs en temps réel
4. Affichez les logs d'hier
5. Exportez les logs dans un fichier
6. Nettoyez les logs de plus de 7 jours

Solution Exercice 6 💡

```
# 1. Logs du dernier boot
journalctl -b

# 2. Seulement les erreurs
journalctl -p err
# ou toutes les priorités <= error
journalctl -p err -b

# 3. Temps réel
journalctl -f
# Ctrl+C pour arrêter

# 4. Logs d'hier
journalctl --since yesterday --until today

# 5. Exporter
journalctl > /tmp/all_logs.txt
# ou pour un service spécifique
journalctl -u nginx > /tmp/nginx_logs.txt

# 6. Nettoyer (> 7 jours)
sudo journalctl --vacuum-time=7d

# Voir l'espace utilisé
journalctl --disk-usage
```

Exercice bonus : Script de monitoring

Objectif : Créer un script de surveillance système

Créez `monitor.sh` **qui :**

- Affiche les 5 processus utilisant le plus de CPU
- Affiche les 5 processus utilisant le plus de mémoire
- Affiche la charge système (load average)
- Alerte si un service critique est arrêté
- Sauvegarde les infos dans un fichier log

Solution Exercice bonus 💡

```bash
#!/bin/bash
# Script de monitoring système

LOG_FILE="/var/log/monitor.log"
CRITICAL_SERVICES=("sshd" "nginx" "mysql")

echo "===== Monitoring $(date) =====" | tee -a $LOG_FILE

# Top 5 CPU
echo -e "\n🔥 Top 5 CPU:" | tee -a $LOG_FILE
ps aux --sort=-%cpu | head -6 | tail -5 | tee -a $LOG_FILE

# Top 5 Mémoire
echo -e "\n📊 Top 5 Mémoire:" | tee -a $LOG_FILE
ps aux --sort=-%mem | head -6 | tail -5 | tee -a $LOG_FILE

# Load average
echo -e "\n⚖️  Charge système:" | tee -a $LOG_FILE
uptime | tee -a $LOG_FILE

# Vérifier services critiques
echo -e "\n🔍 Services critiques:" | tee -a $LOG_FILE
for service in "${CRITICAL_SERVICES[@]}"; do
    if systemctl is-active --quiet $service; then
        echo "✅ $service: OK" | tee -a $LOG_FILE
    else
        echo "❌ $service: ARRÊTÉ !" | tee -a $LOG_FILE
        # Envoyer une alerte
        logger -t MONITOR "ALERTE: $service est arrêté"
    fi
done

echo "============================" | tee -a $LOG_FILE
```

Revenir au sommaire

Automatiser avec cron :

```
# Exécuter toutes les 5 minutes
*/5 * * * * /usr/local/bin/monitor.sh
```

Points clés à retenir 📌

Processus :

- PID, PPID, états (R, S, D, Z, T)
- nice/renice pour la priorité
- kill avec signaux (TERM, KILL, HUP)

systemd :

- `start` , `stop` , `restart` , `reload`
- `enable` , `disable` pour le boot
- `journalctl` pour les logs

Monitoring :

- `ps` , `top` , `htop`
- `systemd-analyze` pour le boot
- `journalctl` pour l'historique

Module 7

Réseau

Configuration, diagnostic et sécurité réseau

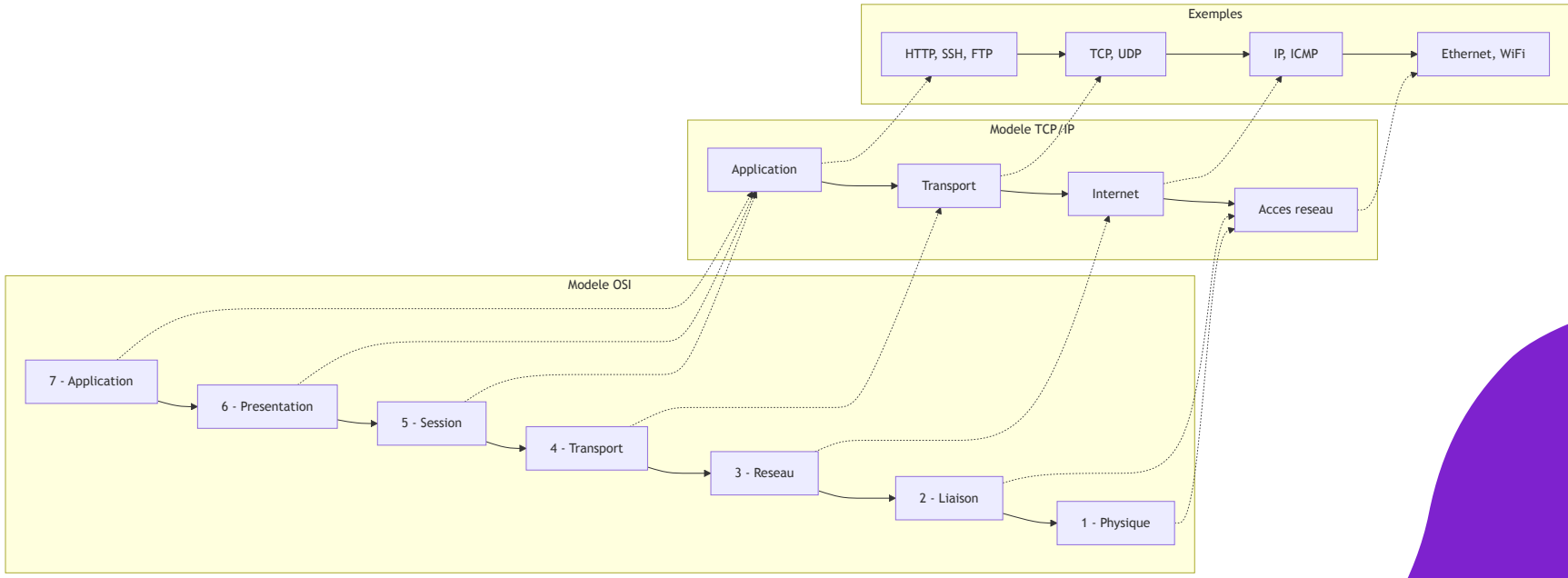
Revenir au sommaire

Le réseau : une analogie postale 📮

Imaginez le réseau comme le système postal :

- **Adresse IP** = adresse postale de votre maison
- **Masque de sous-réseau** = code postal (définit le quartier)
- **Passerelle (Gateway)** = bureau de poste local
- **DNS** = annuaire téléphonique (nom → adresse)
- **Port** = numéro d'appartement
- **Routeur** = centre de tri postal
- **Pare-feu** = gardien qui filtre le courrier

Modèle OSI vs TCP/IP 🏗️



En pratique, on utilise le modèle TCP/IP

Comprendre les modèles réseau : L'analogie de la lettre 📪

Étape	Modèle OSI	Ce que ça fait	Analogie postale
7	**Application**	Vous écrivez le message	✍️ Rédiger la lettre
6	**Présentation**	Mise en forme, chiffrement	🔐 Mettre dans une enveloppe scellée
5	**Session**	Établir la conversation	📞 Décider à qui écrire
4	**Transport**	Découper en morceaux numérotés	📦 Numéroter les pages si lettre longue
3	**Réseau**	Ajouter les adresses	🏠 Écrire l'adresse expéditeur/destinataire
2	**Liaison**	Préparer pour le transport local	📮 Mettre dans la boîte aux lettres
1	**Physique**	Transmettre physiquement	🚚 Le camion qui transporte

Revenir au sommaire

Modèle OSI : Détail des 7 couches 📚

Couche 7 - Application : Ce que VOUS voyez

- HTTP (web), FTP (fichiers), SMTP (mail), SSH (terminal distant)
- 💡 *C'est votre navigateur, votre client mail...*

Couche 6 - Présentation : Traduction et formatage

- Chiffrement SSL/TLS, compression, encodage (UTF-8, ASCII)
- 💡 *Convertit les données pour qu'elles soient compréhensibles*

Couche 5 - Session : Gestion de la conversation

- Ouverture, maintien, fermeture de session
- 💡 *"Bonjour, je voudrais parler à..."*

Couche 4 - Transport : Fiabilité de bout en bout

- TCP (fiable, ordonné) ou UDP (rapide, sans garantie)
- 💡 *"Je vérifie que tout est bien arrivé"*

Modèle OSI : Couches basses (1-3) 🔌

Couche 3 - Réseau : Routage et adressage IP

- Détermine le CHEMIN que prennent les données
- Protocole IP : adressage logique
- 💡 *"Comment aller de Paris à Tokyo ?"*

Couche 2 - Liaison de données : Communication locale

- Adresse MAC (physique, gravée dans la carte réseau)
- Détection d'erreurs, contrôle de flux local
- 💡 *"Communication avec le voisin immédiat"*

Couche 1 - Physique : Les bits sur le fil

- Câbles Ethernet, fibres optiques, ondes WiFi
- Tensions électriques, fréquences radio
- 💡 *"Le support physique qui transporte les 0 et 1"*

Astuce mémo : "**A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing" (Application → Physique)

TCP/IP : Le modèle pratique simplifié 🌐

Pourquoi TCP/IP plutôt qu'OSI ?

Le modèle OSI est **théorique** (créé pour enseigner). Le modèle TCP/IP est **pratique** (créé pour Internet).

TCP/IP	Équivalent OSI	Protocoles	Analogie
Application	5, 6, 7	HTTP, SSH, DNS, FTP	L'application que vous utilisez
Transport	4	TCP, UDP	Le facteur qui garantit la livraison
Internet	3	IP, ICMP	Le système d'adressage mondial
Accès réseau	1, 2	Ethernet, WiFi	Le camion et la route physique

En pratique, vous manipulez :

- 🖥️ **Application** : `curl` , navigateur, `ssh`
- 🔢 **Transport** : Ports (22, 80, 443)
- 🌍 **Internet** : Adresses IP (192.168.1.100)
- 🔌 **Accès réseau** : `eth0` , `wlan0` , câble RJ45

Flux de communication réseau 🔄

Exemple : Envoi d'un email

1. **Application** → L'utilisateur écrit et envoie l'email (SMTP)
2. **Transport** → Les données sont découpées en segments (TCP)
3. **Internet** → Chaque segment reçoit une adresse IP source/destination
4. **Accès réseau** → Les paquets sont transmis physiquement sur le réseau

Le processus inverse se produit à la réception ⬅️

- Chaque couche retire son en-tête
- Les données remontent jusqu'à l'application destinataire
- L'email est reconstitué et affiché

💡 **Astuce** : Chaque couche communique uniquement avec la couche adjacente, comme des étages dans un immeuble !

Adresses IP : Qu'est-ce que c'est ? 🏠

Une adresse IP, c'est l'adresse postale de votre machine sur le réseau.

Sans adresse IP, impossible de :

- ❌ Recevoir des données
- ❌ Envoyer des données
- ❌ Être identifié sur le réseau

Deux versions coexistent :

	IPv4	IPv6
Créé en	1983	1998
Bits	32 bits	128 bits
Format	`192.168.1.100`	`2001:db8::1`
Adresses	~4,3 milliards	~340 sextillions
Situation	Épuisé !	Adoption progressive

Revenir au sommaire

IPv4 en détail : Anatomie d'une adresse 🔍

Format : 4 nombres de 0 à 255, séparés par des points

Revenir au sommaire

Chaque octet = 8 bits = valeur de 0 à 255

```
# En binaire, 192.168.1.100 s'écrit :
11000000.10101000.00000001.01100100
```

Analogie téléphonique 📞

IPv6 : L'avenir du réseau 🚀

Pourquoi IPv6 ?

Avec ~4,3 milliards d'adresses IPv4 et **20+ milliards** d'appareils connectés → pénurie !

Format IPv6 : 8 groupes de 4 caractères hexadécimaux

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
  1    2    3    4    5    6    7    8
```

Simplification autorisée :

```
Complet :   2001:0db8:0000:0000:0000:0000:0000:0001
Simplifié : 2001:db8::1
```

- **Zéros initiaux** : peuvent être supprimés (`0db8` → `db8`)
- **Groupes de zéros** : remplacés par `::` (une seule fois)

IPv4 vs IPv6 : Comparaison pratique 📊

Combien d'adresses ?

Version	Nombre	Analogie
IPv4	4 294 967 296	1 adresse pour 2 humains
IPv6	340 282 366 920 938 463 463 374 607 431 768 211 456	50 000 milliards d'adresses par **millimètre carré** de la Terre !

En pratique aujourd'hui :

```
# Voir vos adresses IPv4 et IPv6
ip -4 addr    # Seulement IPv4
ip -6 addr    # Seulement IPv6
```

Pourquoi on utilise encore IPv4 ?

- 🏠 Réseaux locaux privés (192.168.x.x)
- 🔄 NAT (plusieurs machines partagent 1 IP publique)
- 📚 Habitude et compatibilité

IPv6 est utilisé pour :

- 🌍 Connexions Internet directes
- 📱 Opérateurs mobiles (Free, Orange…)
- ☁️ Services cloud modernes

Exemple concret :

```
# Voir vos adresses IP
ip addr show

# Résultat typique :
# inet 192.168.1.100/24    (IPv4 privée)
# inet6 fe80::a00:27ff:fe4e:66a1/64  (IPv6 locale)
```

Classes d'adresses IPv4 privées 🏠

Plages privées (non routables sur Internet) :

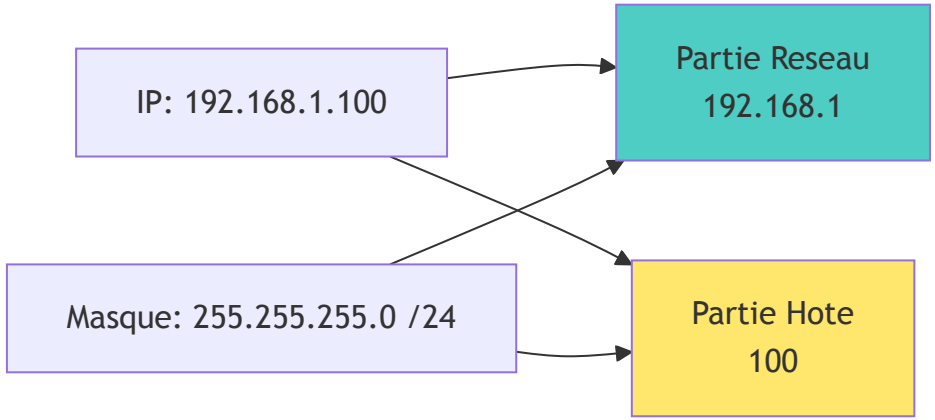
- **Classe A :** `10.0.0.0` à `10.255.255.255` (/8)
  - 16 millions d'adresses
  - 💼 **Exemple** : Grandes entreprises, data centers → `10.50.120.45`
- **Classe B :** `172.16.0.0` à `172.31.255.255` (/12)
  - 1 million d'adresses
  - 🏢 **Exemple** : Entreprises moyennes, campus universitaires → `172.20.15.100`
- **Classe C :** `192.168.0.0` à `192.168.255.255` (/16)
  - 65 536 adresses
  - 🏠 **Exemple** : Réseaux domestiques, petits bureaux → `192.168.1.25`

Loopback : `127.0.0.1` (localhost)

- 🔄 **Exemple** : Tester un serveur web local → `http://127.0.0.1:8080`

Masque de sous-réseau 🎭

Masque : définit quelle partie est réseau vs hôte



Notation CIDR : /24 = 255.255.255.0

Revenir au sommaire

Comprendre le masque de sous-réseau 🎓

C'est quoi concrètement ?

Le masque détermine **quelle partie de l'IP identifie le réseau** et **quelle partie identifie la machine**.

Exemple avec `192.168.1.100` **et masque** `255.255.255.0` **(/24)**

```
IP :      192  .  168  .   1   .  100
Masque :  255  .  255  .  255  .   0

          [    Réseau    ] [Hôte]
          192.168.1         .100
```

- **Partie réseau** (`192.168.1`) → Toutes les machines du même réseau local
- **Partie hôte** (`100`) → Identifie UNE machine spécifique sur ce réseau

En pratique ça veut dire quoi ? 🤔

Avec ce masque, votre machine `192.168.1.100` peut communiquer directement avec :

- ✅ `192.168.1.1` (même réseau)
- ✅ `192.168.1.50` (même réseau)
- ❌ `192.168.2.100` (autre réseau → besoin d'un routeur)

Notation CIDR - Les masques courants 📊

Format `/XX` **= nombre de bits pour le réseau**

CIDR	Masque	Adresses dispo	Cas d'usage	Exemple réseau
`/8`	`255.0.0.0`	16 777 216	Très grands réseaux	`10.0.0.0/8`
`/16`	`255.255.0.0`	65 536	Grands réseaux	`172.16.0.0/16`
`/24`	`255.255.255.0`	256 (254 utiles)	Réseau standard	`192.168.1.0/24`
`/30`	`255.255.255.252`	4 (2 utiles)	Liaison point-à-point	`10.0.0.0/30`

Pourquoi 254 et pas 256 en /24 ? 🤔

- **Adresse réseau :** `192.168.1.0` → réservée (identifie le réseau)
- **Adresse broadcast :** `192.168.1.255` → réservée (message à tous)
- **Adresses utilisables :** `192.168.1.1` à `192.168.1.254` (254 machines)

Analogie du quartier : Comprendre les masques 🏘️

Imaginez que les adresses IP sont des adresses postales dans des quartiers :

CIDR	Analogie	Nombre de "maisons"
/24	**Un petit quartier** 🏠	254 maisons numérotées de 1 à 254
/16	**Une ville entière** 🏙️	65 534 maisons (254 quartiers × 254 maisons)
/8	**Une mégalopole** 🌆	16 millions de maisons !
/30	**Un duplex** 🏡	Seulement 2 maisons (liaison directe)

Exemple concret avec /24 (255.255.255.0) :

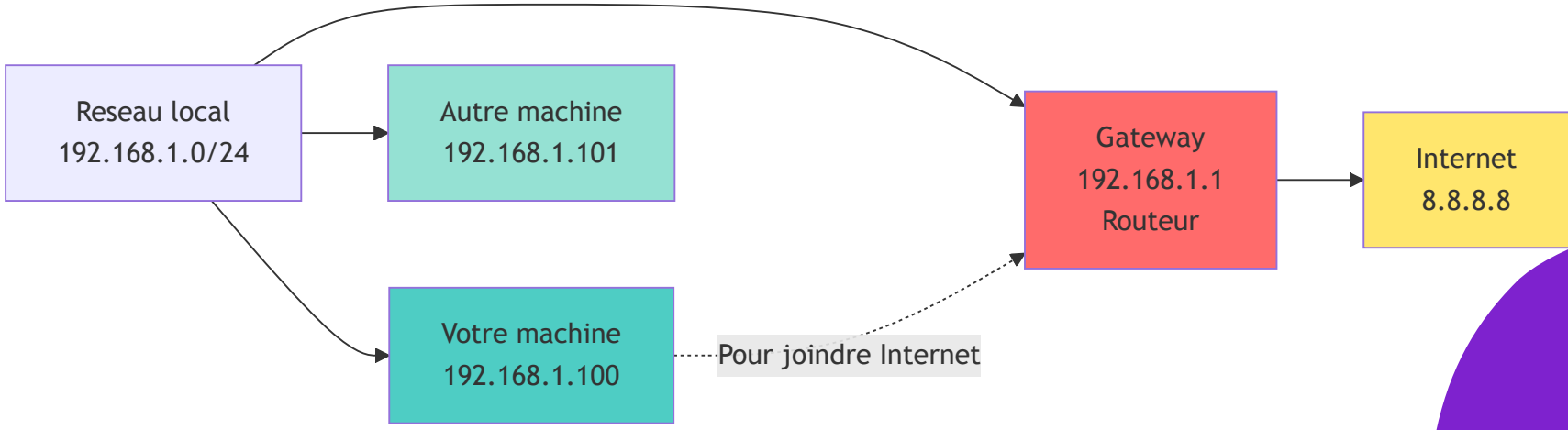
```
Quartier : 192.168.1.x  (le "x" = numéro de maison)

🏠 Maison 0    → Réservée (panneau "Bienvenue dans le quartier")
🏠 Maisons 1-254 → Habitables (vos machines)
🏠 Maison 255  → Réservée (haut-parleur pour parler à tout le quartier)
```

Plus le /XX est petit, plus le "quartier" est grand !

- /24 = petit quartier où tout le monde se connaît
- /8 = immense ville où il faut des "routeurs" pour trouver son chemin

Passerelle (Gateway) 🚪

Gateway : le routeur qui permet de sortir du réseau local



Analogie : la porte d'entrée de votre quartier

Comprendre la Passerelle (Gateway) 🚪

C'est quoi une Gateway ?

C'est **le routeur** qui fait le pont entre votre réseau local et le monde extérieur (Internet).

Fonctionnement concret :

Imaginons que vous voulez accéder à `google.com` depuis votre ordinateur `192.168.1.100` :

1. **Votre machine** vérifie : "Est-ce que Google est dans mon réseau local (192.168.1.x) ?"
   - ❌ Non → Google est sur Internet
2. **Votre machine** envoie alors le paquet à la **Gateway** (`192.168.1.1`)
   - 💬 "Tiens, routeur ! Peux-tu transmettre ça à Google pour moi ?"
3. **La Gateway** (routeur) :
   - Reçoit le paquet
   - Change l'adresse IP source (de privée à publique via NAT)
   - Envoie le paquet vers Internet
   - Reçoit la réponse et la renvoie à votre machine

En résumé : Sans Gateway, vous êtes prisonnier de votre réseau local ! 🔒

Revenir au sommaire

Cas concret : Configuration réseau complète

Scénario : Vous travaillez depuis chez vous sur un réseau domestique

Configuration de votre ordinateur :

```
Adresse IP :     192.168.1.50
Masque :         255.255.255.0  (/24)
Gateway :        192.168.1.1
DNS :            8.8.8.8
```

Décryptage :

Élément	Signification	Analogie
192.168.1.50	Votre adresse sur le réseau local	Votre numéro de maison dans le quartier
255.255.255.0	Vous pouvez parler directement aux IP de 192.168.1.1 à 192.168.1.254	Votre quartier contient 254 maisons
192.168.1.1	L'adresse du routeur pour sortir du réseau	La porte d'entrée du quartier
8.8.8.8	Serveur Google pour traduire les noms de domaine	L'annuaire téléphonique

Exemples de communication :

- 💬 Accéder à `192.168.1.10` (imprimante réseau) → ✅ Communication directe (même réseau)
- 💬 Accéder à `192.168.1.100` (NAS) → ✅ Communication directe (même réseau)
- 💬 Accéder à `google.com` → ❌ Pas dans le réseau local
  - → Demander au DNS : "C'est quoi l'IP de google.com ?" → `142.250.185.46`
  - → Envoyer le paquet à la Gateway (`192.168.1.1`)
  - → La Gateway transmet vers Internet

DNS : Domain Name System 🌐

Pourquoi le DNS existe ?

Les ordinateurs communiquent avec des **adresses IP** (142.250.185.46) Les humains préfèrent des **noms** (google.com)

Le DNS fait la traduction ! 📖

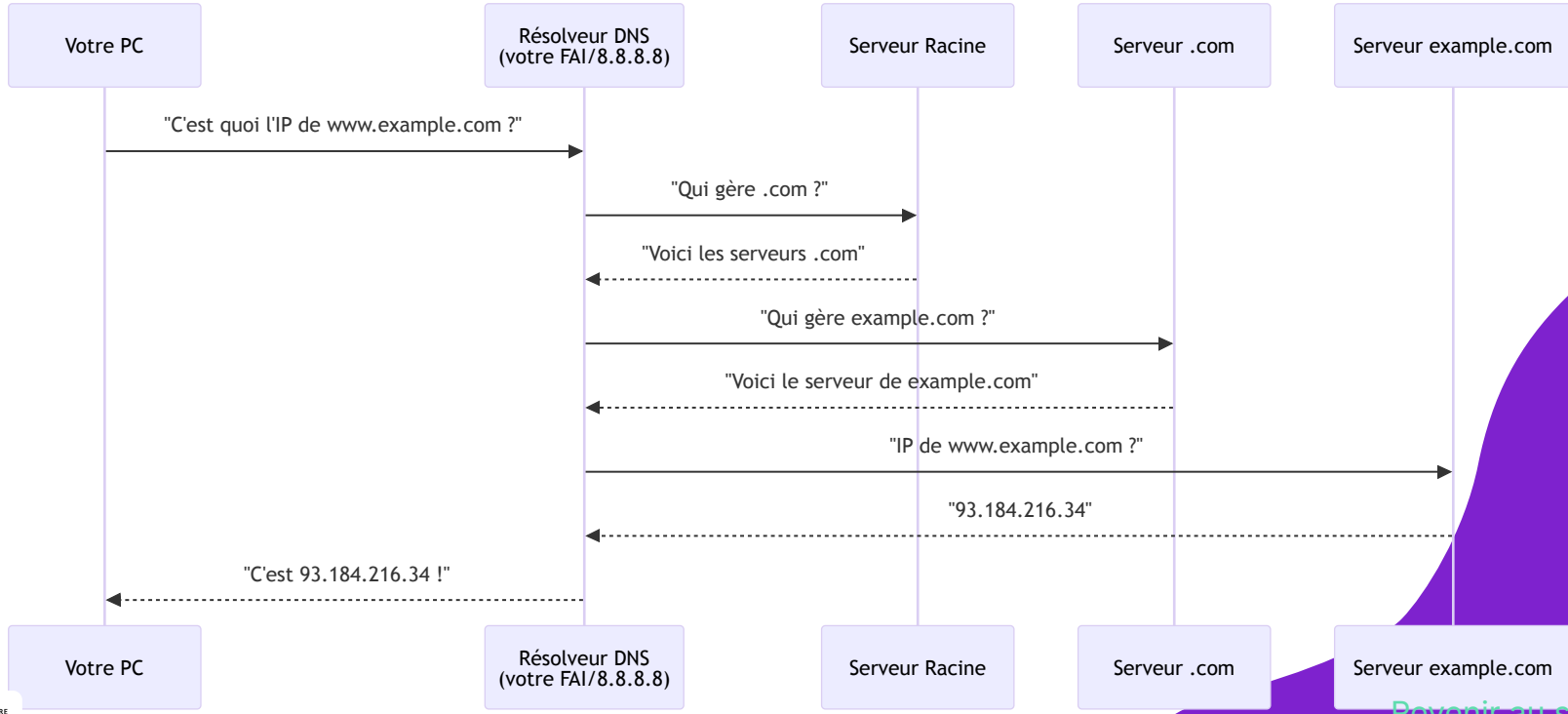
```
 Vous tapez :     google.com
DNS traduit :    142.250.185.46
Votre PC :       "Ah OK, je contacte 142.250.185.46"
```

Analogie : L'annuaire téléphonique 📞

- Vous cherchez "Pizza du coin" → L'annuaire vous donne 01 23 45 67 89
- Vous cherchez "google.com" → Le DNS vous donne 142.250.185.46

Comment fonctionne le DNS ? 🔄

Quand vous tapez `www.example.com` **dans votre navigateur :**



Revenir au Sommaire

La hiérarchie DNS 🏛️

Le DNS est organisé en arbre inversé :

Les niveaux :

- **Racine (.)** : 13 serveurs mondiaux (a.root-servers.net → m.root-servers.net)
- **TLD** : .com, .org, .fr, .net... (géré par des registres)
- **Domaine** : google.com, example.fr (géré par le propriétaire)
- **Sous-domaine** : www.google.com, mail.google.com

Types d'enregistrements DNS 📋

Type	Signification	Exemple	Usage
A	Adresse IPv4	`93.184.216.34`	IP du serveur
AAAA	Adresse IPv6	`2606:2800:220:1::`	IP v6 du serveur
CNAME	Alias	`www` → `example.com`	Redirection de nom
MX	Mail eXchanger	`mail.example.com`	Serveur email
NS	Name Server	`ns1.example.com`	Serveur DNS autoritaire
TXT	Texte libre	`v=spf1 include:...`	Vérifications (SPF, DKIM)
PTR	Reverse	`34.216.184.93.in-addr.arpa`	IP → Nom (inverse)

Exemple concret :

```
# Voir tous les enregistrements
dig example.com ANY

# Seulement les MX (serveurs mail)
dig example.com MX
```

Serveurs DNS publics recommandés 🌍

Fournisseur	DNS Primaire	DNS Secondaire	Avantages
Google	`8.8.8.8`	`8.8.4.4`	Rapide, fiable
Cloudflare	`1.1.1.1`	`1.0.0.1`	Très rapide, vie privée
Quad9	`9.9.9.9`	`149.112.112.112`	Sécurité (bloque malwares)
OpenDNS	`208.67.222.222`	`208.67.220.220`	Filtrage parental dispo

Lequel choisir ?

- 🚀 **Performance** : Cloudflare (1.1.1.1)
- 🔒 **Sécurité** : Quad9 (bloque les sites malveillants)
- 🏢 **Entreprise** : Souvent DNS interne + Google/Cloudflare en backup

```
# Tester la vitesse d'un DNS
time dig @1.1.1.1 google.com
time dig @8.8.8.8 google.com
```

Voir la configuration réseau 🔍

```
# Méthode moderne (depuis 2010)
ip addr
ip a

# Méthode traditionnelle (obsolète mais encore utilisée)
ifconfig

# Voir seulement les adresses IPv4
ip -4 addr

# Voir seulement les adresses IPv6
ip -6 addr

# Afficher une interface spécifique
ip addr show eth0
```

Exemple de sortie ip addr 📋

```
ip addr
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
    inet6 fe80::a00:27ff:fe4e:66a1/64 scope link
```

Décryptage de `ip addr` ligne par ligne (1/2) 🔍

Ligne 1 : `1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536`

Élément	Signification
`1:`	Numéro de l'interface (ordre d'apparition)
`lo`	Nom de l'interface (loopback = boucle locale)
`<LOOPBACK>`	Type d'interface (boucle interne)
`UP`	Interface **activée** au niveau système
`LOWER_UP`	Câble branché / signal présent
`mtu 65536`	Taille max d'un paquet (65 Ko pour loopback)

Les flags entre `< >` sont cruciaux :

- `UP` = activée → Sans ça, l'interface ne fonctionne pas !
- `LOWER_UP` = connexion physique OK → Si absent = câble débranché
- `BROADCAST` = peut envoyer à tous les appareils du réseau
- `MULTICAST` = peut envoyer à un groupe d'appareils

Décryptage de `ip addr` ligne par ligne (2/2) 🔍

Ligne : `inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0`

Élément	Signification
`inet`	Adresse **IPv4** (inet6 = IPv6)
`192.168.1.100`	Votre adresse IP
`/24`	Masque de sous-réseau (255.255.255.0)
`brd 192.168.1.255`	Adresse de **broadcast** (message à tous)
`scope global`	Utilisable pour communiquer **partout**
`eth0`	Interface à laquelle cette IP est liée

Les différents "scopes" :

Scope	Signification	Exemple
`global`	Routable partout	192.168.1.100 (réseau local)
`link`	Seulement sur le lien local	fe80::... (IPv6 link-local)
`host`	Seulement cette machine	127.0.0.1 (localhost)

💡 **Si vous n'avez pas d'adresse** `inet` **→ pas de réseau IPv4 !**

Interfaces réseau : Les noms 🔌

Pourquoi les noms sont-ils bizarres ?

Avant, c'était simple : `eth0` , `eth1` , `wlan0` ... Problème : l'ordre pouvait **changer au redémarrage** ! 😱

Solution moderne : noms prévisibles (systemd)

Les noms sont basés sur l'**emplacement physique** du matériel.

Nomenclature des interfaces : Décodage 🔍

Structure : `<type><emplacement>`

Préfixe	Signification	Exemple
`en`	**E**ther**n**et (filaire)	`enp0s3`
`wl`	**W**ire**l**ess (WiFi)	`wlp3s0`
`ww`	**W**ireless **W**AN (4G/5G)	`wwan0`
`lo`	**Lo**opback (boucle locale)	`lo`

Revenir au sommaire

Décodage de l'emplacement :

Suffixe	Signification	Exemple complet
`o<N>`	**O**n-board (intégré carte mère)	`eno1` = Ethernet intégré #1
`p<bus>s<slot>`	**P**CI bus X, **S**lot Y	`enp0s3` = Ethernet PCI bus 0, slot 3
`s<slot>`	Hot-plug **S**lot	`ens33` = Slot 33 (VMware)

Exemples concrets :

- `enp0s3` → Ethernet, PCI bus 0, slot 3 (VirtualBox)
- `ens33` → Ethernet, slot 33 (VMware)
- `wlp3s0` → WiFi, PCI bus 3, slot 0
- `eno1` → Premier Ethernet intégré à la carte mère

Trouver le nom de votre interface 🎯

```
# Lister toutes les interfaces
ip link show

# Résultat typique :
# 1: lo: <LOOPBACK,UP>
# 2: enp0s3: <BROADCAST,MULTICAST,UP>
# 3: wlp2s0: <BROADCAST,MULTICAST>
```

Astuce : identifier rapidement

```
# Interface avec une adresse IP (probablement la principale)
ip -br addr

# Résultat :
# lo        UNKNOWN  127.0.0.1/8
# enp0s3    UP       192.168.1.100/24   <-- C'est celle-ci !
```

💡 En VM, les noms courants sont :

- **VirtualBox :** `enp0s3` , `enp0s8`
- **VMware :** `ens33` , `ens34`
- **QEMU/KVM :** `enp1s0` , `ens3`

Revenir au sommaire

Activer/désactiver une interface

```
# Désactiver
sudo ip link set eth0 down

# Activer
sudo ip link set eth0 up

# Voir l'état
ip link show eth0

# Méthode alternative (ifconfig)
sudo ifconfig eth0 down
sudo ifconfig eth0 up
```

Configuration IP manuelle 🔧

```
# Attribuer une IP
sudo ip addr add 192.168.1.100/24 dev eth0

# Supprimer une IP
sudo ip addr del 192.168.1.100/24 dev eth0

# Ajouter une passerelle par défaut
sudo ip route add default via 192.168.1.1

# Supprimer la passerelle
sudo ip route del default

# Méthode ifconfig (obsolète)
sudo ifconfig eth0 192.168.1.100 netmask 255.255.255.0
```

⚠️ **Ces changements sont temporaires !**

Configuration permanente : méthode traditionnelle

Debian/Ubuntu (/etc/network/interfaces) :

```
sudo nano /etc/network/interfaces
```

```
auto eth0
iface eth0 inet static
    address 192.168.1.100
    netmask 255.255.255.0
    gateway 192.168.1.1
    dns-nameservers 8.8.8.8 8.8.4.4
```

```
# Redémarrer le réseau
sudo systemctl restart networking
```

Configuration permanente : Netplan (Ubuntu moderne) NEW

Netplan est le système de configuration réseau par défaut sur Ubuntu (depuis 17.10)

Fichier : `/etc/netplan/01-netcfg.yaml`

```yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: no
      addresses:
        - 192.168.1.100/24
      gateway4: 192.168.1.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

Netplan : Explication ligne par ligne (1/3) 📖

Structure de base :

```
network:
  version: 2
```

- `network:` → Racine du fichier de configuration réseau
- `version: 2` → Utilise le format Netplan v2 (standard actuel)

Revenir au sommaire

Netplan : Explication ligne par ligne (2/3) 📖

Choix du renderer et interface :

```
  renderer: networkd
ethernets:
  eth0:
```

- `renderer: networkd` → Utilise **systemd-networkd** pour gérer le réseau
  - Alternative : `NetworkManager` (plus courant sur desktop)
- `ethernets:` → Section pour les interfaces Ethernet filaires
- `eth0:` → Nom de l'interface à configurer

⚠️ **Important** : Le nom peut varier (`ens33` , `enp0s3` ...) → Vérifier avec `ip a`

Revenir au sommaire

Netplan : Explication ligne par ligne (3/3) 📖

Configuration IP statique :

```
    dhcp4: no
    addresses:
      - 192.168.1.100/24
    gateway4: 192.168.1.1
    nameservers:
      addresses:
        - 8.8.8.8
        - 8.8.4.4
```

Ligne	Signification
`dhcp4: no`	Désactive l'attribution automatique d'IP (DHCP)
`addresses: - 192.168.1.100/24`	IP fixe avec masque /24 (255.255.255.0)
`gateway4: 192.168.1.1`	Passerelle par défaut (routeur)
`nameservers:`	Serveurs DNS à utiliser
`8.8.8.8 / 8.8.4.4`	DNS publics de Google

Netplan : Commandes d'application 🚀

```
# Appliquer la configuration
sudo netplan apply
```

```
# Tester avant d'appliquer (rollback automatique après 120s)
sudo netplan try
```

```
# Générer la config et voir les erreurs
sudo netplan generate
```

```
# Afficher la configuration actuelle
sudo netplan get
```

⚠️ **Attention** : Une erreur de syntaxe YAML peut couper le réseau !

Configuration permanente : Red Hat/CentOS/AlmaLinux 🎩

Fichier : `/etc/sysconfig/network-scripts/ifcfg-eth0`

```
TYPE=Ethernet
BOOTPROTO=static
NAME=eth0
DEVICE=eth0
ONBOOT=yes
IPADDR=192.168.1.100
NETMASK=255.255.255.0
GATEWAY=192.168.1.1
DNS1=8.8.8.8
DNS2=8.8.4.4
```

Red Hat/CentOS : Appliquer les changements 🚀

```
# Redémarrer l'interface (méthode classique)
sudo ifdown eth0 && sudo ifup eth0
```

```
# Ou via NetworkManager (recommandé)
sudo nmcli connection reload
sudo nmcli connection up eth0
```

```
# Ou redémarrer le service complet
sudo systemctl restart NetworkManager
```

Vérifier que ça fonctionne :

```
# Voir la nouvelle configuration
ip addr show eth0

# Tester la connectivité
ping -c 3 8.8.8.8
```

DHCP : configuration automatique 🤖

DHCP : Dynamic Host Configuration Protocol

```
# Netplan
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: yes
```

Le serveur DHCP fournit automatiquement :

- Adresse IP
- Masque de sous-réseau
- Passerelle
- Serveurs DNS

NetworkManager : gestion graphique et CLI 🎛️

nmcli : outil en ligne de commande pour NetworkManager

```
# Voir les connexions
nmcli connection show

# Voir les périphériques
nmcli device status

# Activer/désactiver une connexion
nmcli connection up eth0
nmcli connection down eth0

# Voir les détails d'une connexion
nmcli connection show eth0
```

Créer une connexion avec nmcli 🔧

```
# DHCP
sudo nmcli connection add type ethernet con-name "Mon Ethernet" \
  ifname eth0

# IP statique
sudo nmcli connection add type ethernet con-name "Mon Ethernet" \
  ifname eth0 \
  ipv4.addresses 192.168.1.100/24 \
  ipv4.gateway 192.168.1.1 \
  ipv4.dns "8.8.8.8 8.8.4.4" \
  ipv4.method manual
```

Modifier une connexion avec nmcli 🔧

```
# Changer l'IP
sudo nmcli connection modify eth0 ipv4.addresses 192.168.1.200/24

# Changer la passerelle
sudo nmcli connection modify eth0 ipv4.gateway 192.168.1.1

# Changer les DNS
sudo nmcli connection modify eth0 ipv4.dns "1.1.1.1 8.8.8.8"

# Passer en DHCP
sudo nmcli connection modify eth0 ipv4.method auto

# Appliquer les changements
sudo nmcli connection up eth0
```

Configuration DNS

Deux fichiers importants pour la résolution de noms :

1. `/etc/resolv.conf` **- Configuration des serveurs DNS**

```
cat /etc/resolv.conf
```

```
nameserver 8.8.8.8
nameserver 8.8.4.4
search example.com
```

2. `/etc/hosts` **- Résolution locale (prioritaire)**

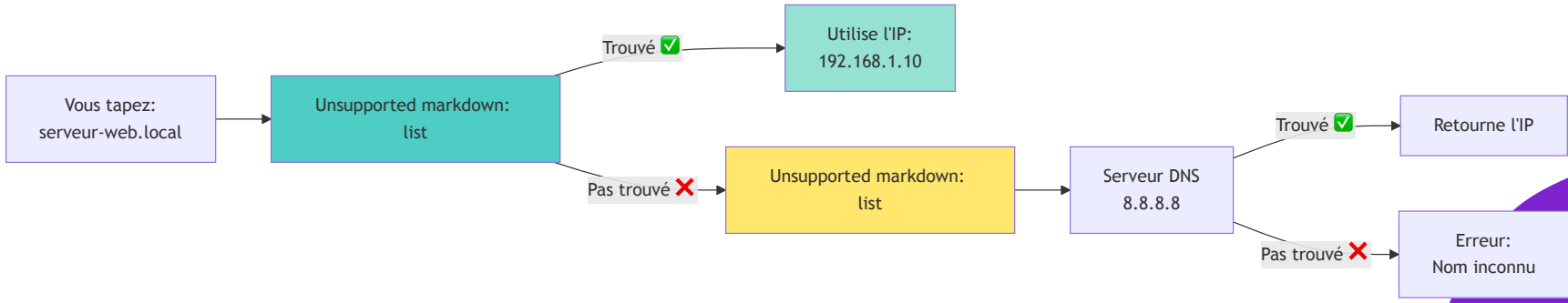
```
cat /etc/hosts
```

```
127.0.0.1       localhost
127.0.1.1       ma-machine
192.168.1.10    serveur-web.local serveur-web
192.168.1.20    base-donnees.local db
```

⚠ `/etc/resolv.conf` **est souvent géré automatiquement par systemd-resolved ou NetworkManager**

/etc/hosts vs /etc/resolv.conf - Quelle différence ? 🤔

Ordre de résolution des noms :



Revenir au sommaire

En résumé :

Fichier	Rôle	Priorité	Cas d'usage
`/etc/hosts`	Association directe nom → IP	✅ **1er** (prioritaire)	Résolution locale rapide, alias, tests
`/etc/resolv.conf`	Liste des serveurs DNS à interroger	2ème (si hosts ne trouve pas)	Résolution des noms Internet

Cas d'usage pratiques 💡

Exemple 1 : Forcer une résolution locale avec `/etc/hosts`

```
# Bloquer un site (redirige vers localhost)
echo "127.0.0.1    facebook.com" | sudo tee -a /etc/hosts

# Créer des alias pour vos serveurs internes
echo "192.168.1.50    db" | sudo tee -a /etc/hosts
echo "192.168.1.51    web" | sudo tee -a /etc/hosts

# Ensuite vous pouvez faire :
ssh db        # Au lieu de ssh 192.168.1.50
ping web      # Au lieu de ping 192.168.1.51
```

Exemple 2 : Changer de serveur DNS dans `/etc/resolv.conf`

```
# Voir les DNS actuels
cat /etc/resolv.conf

# Utiliser les DNS de Cloudflare (plus rapides/privés)
echo "nameserver 1.1.1.1" | sudo tee /etc/resolv.conf
echo "nameserver 1.0.0.1" | sudo tee -a /etc/resolv.conf
```

Analogie complète 📞

- `/etc/hosts` = Votre carnet de contacts personnel (consulté en premier)
- `/etc/resolv.conf` = L'annuaire téléphonique public (consulté si pas dans vos contacts)

systemd-resolved NEW

Service moderne de résolution DNS

```
# Voir l'état
systemctl status systemd-resolved

# Voir la configuration
resolvectl status

# Vider le cache DNS
sudo resolvectl flush-caches

# Statistiques
resolvectl statistics
```

Tester la connectivité : ping 🏓

```
# Ping vers une IP
ping 8.8.8.8

# Ping vers un nom de domaine
ping google.com

# Limiter le nombre de paquets
ping -c 4 google.com

# Intervalle entre les pings (secondes)
ping -i 0.5 google.com

# Flood ping (root requis, ATTENTION !)
sudo ping -f 192.168.1.1
```

Ce que ping teste :

- Connectivité réseau
- Temps de réponse (latence)
- Perte de paquets

Interpréter les résultats de ping

```
ping -c 4 google.com
```

```
PING google.com (142.250.185.46) 56(84) bytes of data.
64 bytes from par21s04-in-f14.1e100.net (142.250.185.46): icmp_seq=1 ttl=115 time=15.2 ms
64 bytes from par21s04-in-f14.1e100.net (142.250.185.46): icmp_seq=2 ttl=115 time=14.8 ms
64 bytes from par21s04-in-f14.1e100.net (142.250.185.46): icmp_seq=3 ttl=115 time=15.1 ms
64 bytes from par21s04-in-f14.1e100.net (142.250.185.46): icmp_seq=4 ttl=115 time=15.0 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 14.825/15.025/15.194/0.141 ms
```

Informations clés :

- `time` : latence (plus bas = mieux)
- `0% packet loss` : aucune perte (parfait)
- `ttl` : Time To Live (nombre de sauts max)

Revenir au sommaire

Tracer le chemin : traceroute 🗺️

```
# Installer
sudo apt install traceroute

# Tracer la route vers un hôte
traceroute google.com

# Utiliser ICMP au lieu d'UDP
sudo traceroute -I google.com

# Limiter le nombre de sauts
traceroute -m 15 google.com
```

Montre tous les routeurs intermédiaires

Exemple de traceroute

```
traceroute google.com
```

```
traceroute to google.com (142.250.185.46), 30 hops max, 60 byte packets
 1  192.168.1.1 (192.168.1.1)  1.234 ms  1.123 ms  1.056 ms
 2  10.0.0.1 (10.0.0.1)  5.432 ms  5.123 ms  5.234 ms
 3  * * *
 4  142.250.185.46 (142.250.185.46)  15.123 ms  14.987 ms  15.234 ms
```

Signification :

- Chaque ligne = un routeur
- `* * *` = routeur qui ne répond pas (normal, firewall)
- Temps en ms = latence

Revenir au sommaire

Résolution DNS : dig et nslookup 🔍

dig : outil moderne et complet

```
# Installer
sudo apt install dnsutils

# Résolution simple
dig google.com

# Seulement la réponse courte
dig +short google.com

# Serveur DNS spécifique
dig @8.8.8.8 google.com

# Type d'enregistrement spécifique
dig google.com MX     # Mail servers
dig google.com NS     # Name servers
dig google.com AAAA   # IPv6
```

Résolution DNS : nslookup 🔍

nslookup : outil traditionnel

```
# Résolution simple
nslookup google.com

# Serveur DNS spécifique
nslookup google.com 8.8.8.8

# Mode interactif
nslookup
> server 8.8.8.8
> google.com
> exit
```

Résolution inverse : IP → nom 🔄

```
# dig
dig -x 8.8.8.8

# nslookup
nslookup 8.8.8.8

# host
host 8.8.8.8
```

/etc/hosts : résolution locale

Fichier `/etc/hosts` **: résolution avant DNS**

```
sudo nano /etc/hosts
```

```
127.0.0.1       localhost
127.0.1.1       ma-machine
192.168.1.10    serveur-web.local serveur-web
192.168.1.20    base-donnees.local db
```

Cas d'usage :

- Tests en développement
- Bloquer des domaines (ad blocker)
- Noms personnalisés pour le réseau local

Table de routage 🛣️

```
# Voir la table de routage
ip route
ip route show

# Route par défaut
ip route show default

# Méthode ancienne
route -n
netstat -rn
```

Exemple de table de routage 📊

```
ip route
```

```
default via 192.168.1.1 dev eth0 proto dhcp metric 100
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.100 metric 100
```

Signification :

- `default via 192.168.1.1` : passerelle par défaut
- `192.168.1.0/24 dev eth0` : réseau local directement accessible

Ajouter/supprimer des routes 🛤️

```
# Ajouter une route vers un réseau spécifique
sudo ip route add 10.0.0.0/24 via 192.168.1.254

# Supprimer une route
sudo ip route del 10.0.0.0/24

# Changer la route par défaut
sudo ip route change default via 192.168.1.2

# Route permanente (dans /etc/network/interfaces)
up ip route add 10.0.0.0/24 via 192.168.1.254
```

Ports : C'est quoi exactement ? 🚪

Problème : Une machine a UNE adresse IP, mais fait tourner PLUSIEURS services (web, mail, SSH...)

Comment distinguer les services ? → Les **ports** !

Analogie : L'immeuble 🏢

```
Adresse IP = Adresse de l'immeuble (10 rue de Paris)
Port       = Numéro d'appartement

192.168.1.100:22   → Immeuble 192.168.1.100, appartement 22 (SSH)
192.168.1.100:80   → Immeuble 192.168.1.100, appartement 80 (Web)
192.168.1.100:443  → Immeuble 192.168.1.100, appartement 443 (Web sécurisé)
```

Format complet d'une connexion :

```
IP_source:port_source  →  IP_destination:port_destination
192.168.1.50:54321     →  142.250.185.46:443
  (Votre PC)                 (Google HTTPS)
```

Les 3 catégories de ports

Catégorie	Plage	Qui les utilise	Exemples
Bien connus	0 - 1023	Services système (root)	SSH (22), HTTP (80), DNS (53)
Enregistrés	1024 - 49151	Applications	MySQL (3306), Minecraft (25565)
Dynamiques	49152 - 65535	Connexions temporaires	Votre navigateur utilise un port ici

Pourquoi cette séparation ?

- Ports 0-1023 : Réservés aux services critiques (besoin d'être root pour les ouvrir)
- Ports éphémères : Votre PC en prend un au hasard pour chaque connexion sortante

Ports essentiels à connaître par cœur 🧠

Port	Service	Protocole	À quoi ça sert
22	SSH	TCP	Connexion à distance sécurisée
80	HTTP	TCP	Sites web non sécurisés
443	HTTPS	TCP	Sites web sécurisés (TLS)
21	FTP	TCP	Transfert de fichiers (ancien)
25	SMTP	TCP	Envoi d'emails
53	DNS	UDP/TCP	Résolution de noms
67/68	DHCP	UDP	Attribution d'IP automatique
3306	MySQL	TCP	Base de données MySQL
5432	PostgreSQL	TCP	Base de données PostgreSQL

Astuce mémo :

- 🔐 **22** = "2 personnes discutent en **S**ecret" (SSH)
- 🌐 **80** = "**H**TTP" (le H ressemble à 8+0)
- 🔒 **443** = 80 × 5.5 ≈ "HTTP × Sécurité"

Voir les ports ouverts : ss 🔍

ss : Socket Statistics (remplace netstat)

```
# Tous les ports en écoute
sudo ss -tuln

# Avec les noms de processus
sudo ss -tulnp

# Seulement TCP
sudo ss -tln

# Seulement UDP
sudo ss -uln

# Connexions établies
sudo ss -tn
```

Options :

- `-t` : TCP
- `-u` : UDP
- `-l` : en écoute (listening)
- `-n` : numérique (pas de résolution DNS)
- `-p` : processus

Exemple de sortie ss 📋

```
sudo ss -tulnp
```

```
Netid  State   Recv-Q  Send-Q  Local Address:Port  Peer Address:Port  Process
tcp    LISTEN  0       128     0.0.0.0:22          0.0.0.0:*          users:(("sshd",pid=1234,fd=3))
tcp    LISTEN  0       128     0.0.0.0:80          0.0.0.0:*          users:(("nginx",pid=5678,fd=6))
tcp    LISTEN  0       128     [::]:22             [::]:*             users:(("sshd",pid=1234,fd=4))
udp    UNCONN  0       0       0.0.0.0:68          0.0.0.0:*          users:(("dhclient",pid=9012,fd=6))
```

Décryptage de `ss -tulnp` colonne par colonne 🔍

Prenons cette ligne :

```
tcp    LISTEN  0       128     0.0.0.0:22          0.0.0.0:*          users:(("sshd",pid=1234,fd=3))
```

Colonne	Valeur	Signification
Netid	`tcp`	Protocole (tcp ou udp)
State	`LISTEN`	État : en attente de connexions
Recv-Q	`0`	File d'attente réception (0 = OK)
Send-Q	`128`	Taille max de la file d'attente
Local Address:Port	`0.0.0.0:22`	Écoute sur **toutes les interfaces**, port 22
Peer Address:Port	`0.0.0.0:*`	Accepte des connexions de **n'importe où**
Process	`sshd,pid=1234`	Programme qui écoute (SSH daemon, PID 1234)

Comprendre les adresses dans ss 🎯

Local Address : Sur quelle(s) interface(s) le service écoute ?

Adresse	Signification	Sécurité
`0.0.0.0:80`	Écoute sur **TOUTES** les interfaces IPv4	⚠️ Accessible de partout
`127.0.0.1:3306`	Écoute **SEULEMENT** sur localhost	✅ Seulement local
`192.168.1.100:22`	Écoute sur cette IP spécifique	⚠️ Accessible du réseau local
`[::]:80`	Écoute sur **TOUTES** les interfaces IPv6	⚠️ Accessible de partout
`[::1]:5432`	IPv6 localhost uniquement	✅ Seulement local

Peer Address : Qui peut se connecter ?

Adresse	Signification
`0.0.0.0:*`	N'importe quelle IP, n'importe quel port
`192.168.1.50:54321`	Connexion établie avec cette IP:port

État (State) pour TCP :

- `LISTEN` : En attente de connexions (serveur)
- `ESTABLISHED` : Connexion active
- `TIME_WAIT` : Connexion en cours de fermeture
- `CLOSE_WAIT` : L'autre côté a fermé, on attend la fermeture locale

Alternative : netstat 🔍

netstat : outil traditionnel (toujours utile)

```
# Tous les ports en écoute
sudo netstat -tuln

# Avec les processus
sudo netstat -tulnp

# Connexions actives
netstat -tan

# Statistiques réseau
netstat -s

# Table de routage
netstat -rn
```

Tester un port : telnet et nc 🔌

telnet : tester la connectivité TCP

```
telnet google.com 80
# Tapez : GET / HTTP/1.1
# Puis : Host: google.com
# Puis : Entrée deux fois

# Ctrl+] puis quit pour sortir
```

nc (netcat) : couteau suisse réseau

```
# Tester un port
nc -zv google.com 80

# Scanner des ports
nc -zv 192.168.1.1 20-80

# Écouter sur un port
nc -l 12345

# Envoyer vers un port
echo "Message" | nc 192.168.1.100 12345
```

Capturer le trafic réseau : tcpdump 📦

```
# Installer
sudo apt install tcpdump

# Capturer sur eth0
sudo tcpdump -i eth0

# Limiter le nombre de paquets
sudo tcpdump -i eth0 -c 10

# Sauvegarder dans un fichier
sudo tcpdump -i eth0 -w capture.pcap

# Filtrer par port
sudo tcpdump -i eth0 port 80

# Filtrer par hôte
sudo tcpdump -i eth0 host 192.168.1.100
```

Wireshark : analyseur graphique

```
# Installer Wireshark
sudo apt install wireshark

# Lancer (graphique)
sudo wireshark

# Ou en CLI
sudo tshark -i eth0
```

Wireshark : analyse détaillée du trafic

- Interface graphique
- Filtres puissants
- Décodage de protocoles
- Statistiques

Pare-feu : introduction 🔥

Pare-feu (firewall) : filtre le trafic réseau

Analogie : le videur d'une boîte de nuit

- Contrôle qui entre et qui sort
- Bloque les indésirables
- Autorise les invités

3 outils principaux sur Linux :

1. **iptables** : bas niveau, complexe
2. **ufw** : simple, Debian/Ubuntu
3. **firewalld** : moderne, RHEL/Fedora

iptables : le pare-feu traditionnel

iptables : très puissant mais complexe

Chaînes principales :

- **INPUT** : trafic entrant
- **OUTPUT** : trafic sortant
- **FORWARD** : trafic transité

Actions :

- **ACCEPT** : autoriser
- **DROP** : bloquer (silencieux)
- **REJECT** : bloquer (avec message)

Exemples iptables 🔧

```
# Voir les règles
sudo iptables -L -n -v

# Autoriser SSH (port 22)
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Autoriser HTTP et HTTPS
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Autoriser les connexions établies
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Bloquer tout le reste
sudo iptables -P INPUT DROP
```

Sauvegarder les règles iptables 💾

```
# Debian/Ubuntu
sudo iptables-save > /etc/iptables/rules.v4

# Restaurer
sudo iptables-restore < /etc/iptables/rules.v4

# RHEL/Fedora
sudo iptables-save > /etc/sysconfig/iptables

# Ou utiliser iptables-persistent
sudo apt install iptables-persistent
```

ufw : Uncomplicated Firewall

ufw : interface simple pour iptables (Ubuntu/Debian)

```
# Installer
sudo apt install ufw

# Activer
sudo ufw enable

# Désactiver
sudo ufw disable

# Voir l'état
sudo ufw status
sudo ufw status verbose
```

Règles ufw 🔧

```
# Autoriser un port
sudo ufw allow 22
sudo ufw allow ssh
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Refuser un port
sudo ufw deny 23

# Autoriser depuis une IP spécifique
sudo ufw allow from 192.168.1.100

# Autoriser une plage de ports
sudo ufw allow 6000:6010/tcp

# Supprimer une règle
sudo ufw delete allow 80
```

ufw : règles avancées 🎯

```
# Autoriser seulement depuis un sous-réseau
sudo ufw allow from 192.168.1.0/24 to any port 22

# Limiter les tentatives (protection brute-force)
sudo ufw limit ssh

# Bloquer une IP
sudo ufw deny from 203.0.113.51

# Réinitialiser toutes les règles
sudo ufw reset

# Numéroter les règles
sudo ufw status numbered

# Supprimer par numéro
sudo ufw delete 3
```

ufw : profils d'application 📱

```
# Lister les applications disponibles
sudo ufw app list

# Voir les détails d'une application
sudo ufw app info OpenSSH

# Autoriser une application
sudo ufw allow 'Nginx Full'
sudo ufw allow 'OpenSSH'
sudo ufw allow 'Apache'
```

firewalld : pare-feu moderne 🔥

firewalld : par défaut sur RHEL/Fedora/CentOS

Concepts :

- **Zones** : niveaux de confiance
- **Services** : ensembles de ports prédéfinis
- **Permanent** : règles qui survivent au redémarrage

```
# Installer
sudo dnf install firewalld

# Démarrer
sudo systemctl start firewalld
sudo systemctl enable firewalld
```

Zones firewalld 🎯

Zones prédéfinies (du plus sécurisé au moins) :

- **drop** : tout est bloqué
- **block** : bloqué avec rejet
- **public** : par défaut, accès limité
- **external** : masquerading activé (NAT)
- **dmz** : accès limité aux services publics
- **work** : réseau de travail
- **home** : réseau domestique
- **internal** : réseau interne de confiance
- **trusted** : tout est autorisé

Utiliser firewalld 🔧

```
# Voir la zone par défaut
sudo firewall-cmd --get-default-zone

# Changer la zone par défaut
sudo firewall-cmd --set-default-zone=home

# Lister les zones actives
sudo firewall-cmd --get-active-zones

# Voir les règles de la zone
sudo firewall-cmd --list-all

# Services autorisés
sudo firewall-cmd --list-services
```

Ajouter des règles firewalld 🔧

```
# Autoriser un service
sudo firewall-cmd --add-service=http
sudo firewall-cmd --add-service=https

# Autoriser un port
sudo firewall-cmd --add-port=8080/tcp

# Rendre permanent
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-port=8080/tcp

# Recharger la config
sudo firewall-cmd --reload

# Supprimer une règle
sudo firewall-cmd --remove-service=http
sudo firewall-cmd --permanent --remove-service=http
```

Règles riches firewalld 🎨

```
# Autoriser SSH seulement depuis un sous-réseau
sudo firewall-cmd --add-rich-rule='rule family="ipv4" source address="192.168.1.0/24" service name="ssh" accept'

# Bloquer une IP
sudo firewall-cmd --add-rich-rule='rule family="ipv4" source address="203.0.113.51" reject'

# Limitation de débit (protection DDoS)
sudo firewall-cmd --add-rich-rule='rule service name="http" accept limit value="100/m"'
```

SSH : Secure Shell 🔒

SSH : Se connecter à une machine distante de manière **sécurisée**

Avant SSH (années 90) : Telnet, rsh → Tout passait en **clair** sur le réseau ! 😱

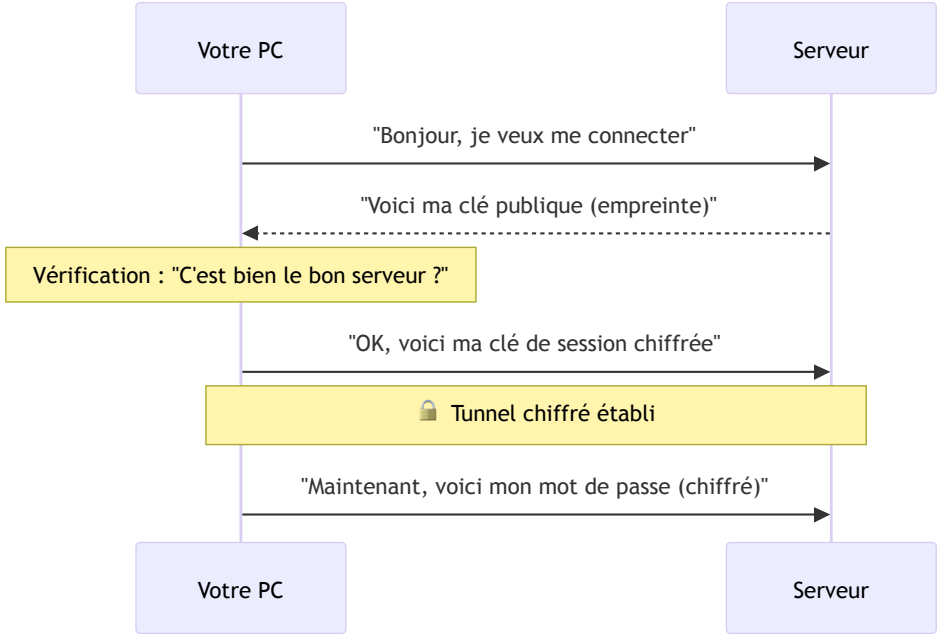
- Mot de passe visible par n'importe qui sur le réseau
- Commandes lisibles
- Aucune garantie que c'est le bon serveur

Avec SSH : Tout est **chiffré** de bout en bout 🔐

- Mot de passe illisible même si intercepté
- Authentification du serveur (pas d'usurpation)
- Intégrité des données garantie

Comment SSH sécurise la connexion ? 🔐

Étape 1 : Établissement du tunnel chiffré



Étape 2 : Authentification

- Par **mot de passe** : Vous tapez votre mot de passe (chiffré dans le tunnel)
- Par **clé SSH** : Preuve cryptographique sans mot de passe

SSH : Connexion basique

```
# Connexion simple
ssh utilisateur@serveur

# Avec un port différent
ssh -p 2222 utilisateur@serveur

# Avec une clé privée spécifique
ssh -i ~/.ssh/ma_cle utilisateur@serveur

# Exécuter une commande à distance (sans session interactive)
ssh utilisateur@serveur 'ls -la /var/log'

# Activer le forwarding X11 (applications graphiques)
ssh -X utilisateur@serveur
```

Première connexion ? Normal de voir ça :

```
The authenticity of host 'serveur (192.168.1.100)' can't be established.
ED25519 key fingerprint is SHA256:xxxxxxxxxxxxxxxxxxx.
Are you sure you want to continue connecting (yes/no)?
```

→ Tapez `yes` pour accepter et mémoriser l'empreinte du serveur

Authentification par clés SSH : Pourquoi ? 🔑

Problèmes avec les mots de passe :

- ❌ Peuvent être devinés (attaque brute-force)
- ❌ Doivent être tapés à chaque fois
- ❌ Peuvent être volés (keylogger, phishing)

Avantages des clés SSH :

- ✅ Impossibles à deviner (4096 bits = astronomiquement complexe)
- ✅ Pas besoin de taper de mot de passe
- ✅ Peuvent être protégées par une passphrase locale

Clés SSH : L'analogie du cadenas 🔓

Principe de la cryptographie asymétrique :

```
🔐 Clé PRIVÉE (id_ed25519)     →  Votre clé de maison (JAMAIS partagée)
🔓 Clé PUBLIQUE (id_ed25519.pub)  →  Un cadenas que vous distribuez

Vous donnez votre cadenas (clé publique) au serveur.
Le serveur vous envoie un défi verrouillé avec votre cadenas.
Seule votre clé privée peut l'ouvrir → Vous êtes authentifié !
```

Concrètement :

1. Vous générez une **paire** de clés (privée + publique)
2. Vous copiez la clé **publique** sur le serveur
3. À chaque connexion, le serveur vérifie que vous avez la clé **privée**

Générer des clés SSH 🔑

```
# Méthode recommandée en 2025 : Ed25519 (plus sûr, plus rapide)
ssh-keygen -t ed25519 -C "mon-email@example.com"

# Alternative : RSA 4096 bits (compatibilité maximale)
ssh-keygen -t rsa -b 4096 -C "mon-email@example.com"
```

Ce qui se passe :

```
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/user/.ssh/id_ed25519): [Entrée]
Enter passphrase (empty for no passphrase): [optionnel mais recommandé]
Enter same passphrase again:

Your identification has been saved in /home/user/.ssh/id_ed25519       ← PRIVÉE
Your public key has been saved in /home/user/.ssh/id_ed25519.pub      ← PUBLIQUE
```

⚠️ RÈGLE D'OR :

- Clé **privée** → NE JAMAIS la partager, la copier, l'envoyer par mail !
- Clé **publique** → Peut être partagée librement

Copier la clé publique sur un serveur 📤

```
# Méthode automatique
ssh-copy-id utilisateur@serveur

# Avec port spécifique
ssh-copy-id -p 2222 utilisateur@serveur

# Méthode manuelle
cat ~/.ssh/id_rsa.pub | ssh utilisateur@serveur 'mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys'

# Ou copier-coller dans ~/.ssh/authorized_keys sur le serveur
```

Configuration SSH client

Fichier : `~/.ssh/config`

```
# Serveur de production
Host prod
    HostName 203.0.113.10
    User admin
    Port 2222
    IdentityFile ~/.ssh/cle_prod

# Serveur de dev
Host dev
    HostName 192.168.1.100
    User developer
    ForwardAgent yes

# Utilisation :
ssh prod       # au lieu de ssh -p 2222 -i ~/.ssh/cle_prod admin@203.0.113.10
ssh dev
```

Sécuriser le serveur SSH 🔐

Fichier : `/etc/ssh/sshd_config`

```
# Désactiver root login
PermitRootLogin no

# Désactiver mot de passe (clés uniquement)
PasswordAuthentication no

# Changer le port (obscurité, pas de sécurité réelle)
Port 2222

# Autoriser seulement certains utilisateurs
AllowUsers alice bob

# Timeout
ClientAliveInterval 300
ClientAliveCountMax 2
```

```
# Redémarrer SSH
sudo systemctl restart sshd
```

Transfert de fichiers : scp 📂

scp : Secure Copy (basé sur SSH)

```
# Copier vers un serveur
scp fichier.txt utilisateur@serveur:/chemin/distant/

# Copier depuis un serveur
scp utilisateur@serveur:/chemin/fichier.txt ./

# Copier un répertoire
scp -r dossier/ utilisateur@serveur:/chemin/distant/

# Avec port spécifique
scp -P 2222 fichier.txt utilisateur@serveur:/chemin/

# Conserver permissions et dates
scp -p fichier.txt utilisateur@serveur:/chemin/
```

Transfert de fichiers : rsync 🔄

rsync : synchronisation efficace

```
# Synchroniser un répertoire
rsync -avz /source/ utilisateur@serveur:/destination/

# Options :
# -a : mode archive (préserve tout)
# -v : verbose
# -z : compression
# -P : progression et reprise
# --delete : supprimer les fichiers supprimés de la source

# Synchronisation avec suppression
rsync -avz --delete /source/ utilisateur@serveur:/destination/

# Dry-run (simulation)
rsync -avz --dry-run /source/ utilisateur@serveur:/destination/
```

Transfert de fichiers : sftp 📦

sftp : FTP sécurisé via SSH

```
# Se connecter
sftp utilisateur@serveur

# Commandes dans sftp :
# ls : lister distant
# lls : lister local
# cd : changer répertoire distant
# lcd : changer répertoire local
# put : envoyer fichier
# get : télécharger fichier
# exit : quitter
```

```
sftp> put fichier.txt
sftp> get document.pdf
sftp> exit
```

Tunnels SSH : C'est quoi ? 🚇

Problème courant :

- Vous êtes chez vous
- Vous voulez accéder à une base de données qui n'est accessible QUE depuis le serveur
- La base n'est pas exposée sur Internet (sécurité !)

Solution : Créer un "tunnel" à travers SSH

Analogie : Le tunnel sous la montagne ⛰️ Au lieu de contourner la montagne (Internet, firewalls), vous passez **à travers** via SSH.

Local Forwarding : Ramener un service distant 📥

Cas d'usage : Accéder à un service qui n'est accessible que depuis le serveur

```
ssh -L port_local:destination:port_distant utilisateur@serveur
```

Exemple concret : Accéder à MySQL distant

```
# MySQL n'écoute que sur localhost du serveur (sécurité)
# On crée un tunnel pour y accéder depuis notre PC

ssh -L 3307:localhost:3306 admin@serveur-prod

# Maintenant sur votre PC :
mysql -h 127.0.0.1 -P 3307 -u root -p
# → Connecté à MySQL du serveur distant !
```

Schéma :

```
[Votre PC :3307] ──SSH──> [serveur-prod] ──> [localhost:3306 MySQL]
```

Remote Forwarding : Exposer un service local 📤

Cas d'usage : Montrer votre site en développement à un collègue

```
ssh -R port_distant:localhost:port_local utilisateur@serveur
```

Exemple : Partager votre app locale sur un serveur public

```
# Votre app tourne sur localhost:3000
# Vous voulez qu'elle soit accessible sur serveur.com:8080

ssh -R 8080:localhost:3000 admin@serveur.com

# Maintenant n'importe qui peut accéder à :
# http://serveur.com:8080 → voit votre localhost:3000 !
```

Schéma :

```
[Internet] → [serveur.com:8080] ──SSH──> [Votre PC localhost:3000]
```

⚠️ Le serveur doit avoir `GatewayPorts yes` dans `/etc/ssh/sshd_config`

Récapitulatif des tunnels SSH 📊

Type	Option	Direction	Cas d'usage
Local	`-L`	Distant → Local	Accéder à un service distant depuis votre PC
Remote	`-R`	Local → Distant	Exposer votre service local sur un serveur
Dynamic	`-D`	Proxy complet	Tout le trafic passe par le serveur

Syntaxe mnémotechnique :

```
# L = Local : je RAMÈNE le port distant chez moi
ssh -L local:cible:distant user@serveur

# R = Remote : j'ENVOIE mon port local vers le serveur
ssh -R distant:localhost:local user@serveur

# D = Dynamic : TOUT passe par là
ssh -D port_proxy user@serveur
```

Astuce : garder le tunnel en arrière-plan

```
ssh -fN -L 3307:localhost:3306 admin@serveur
# -f : passe en arrière-plan
# -N : pas de commande à exécuter (juste le tunnel)
```

Diagnostiquer les problèmes réseau 🔧

Checklist de diagnostic :

1. **Interface activée ?**

```
ip link show eth0
```

2. **IP correcte ?**

```
ip addr show eth0
```

Diagnostiquer les problèmes réseau (suite) 🔧

3. **Passerelle joignable ?**

```
ping 192.168.1.1
```

4. **DNS fonctionne ?**

```
dig google.com
```

Diagnostiquer les problèmes réseau (suite) 🔧

5. **Connectivité Internet ?**

```
ping 8.8.8.8
```

6. **Pare-feu ?**

```
sudo ufw status
sudo firewall-cmd --list-all
```

DHCP : attribution automatique d'IP

DHCP (Dynamic Host Configuration Protocol)

Fonctionnement :

1. Client démarre : "J'ai besoin d'une IP !"
2. Serveur DHCP : "Voici 192.168.1.50"
3. Client : "Merci, je l'utilise"

Le serveur DHCP fournit :

- Adresse IP
- Masque de sous-réseau
- Passerelle par défaut
- Serveurs DNS
- Durée de location (lease time)

Installer un serveur DHCP 📥

```
# Installer isc-dhcp-server
sudo apt update
sudo apt install isc-dhcp-server

# Sur RHEL/Fedora
sudo dnf install dhcp-server
```

Fichier de configuration :

- Debian/Ubuntu : `/etc/dhcp/dhcpd.conf`
- RHEL/Fedora : `/etc/dhcp/dhcpd.conf`

Configuration DHCP de base

Fichier : `/etc/dhcp/dhcpd.conf`

```
sudo nano /etc/dhcp/dhcpd.conf
```

```
 # Paramètres globaux
default-lease-time 600;
max-lease-time 7200;
authoritative;

# Subnet 192.168.1.0/24
subnet 192.168.1.0 netmask 255.255.255.0 {
    range 192.168.1.100 192.168.1.200;
    option routers 192.168.1.1;
    option subnet-mask 255.255.255.0;
    option domain-name-servers 8.8.8.8, 1.1.1.1;
    option domain-name "example.local";
}
```

DHCP : Explication des paramètres globaux 📖

Paramètre	Valeur	Signification
`default-lease-time`	`600`	Durée par défaut du "bail" : **10 minutes** (600 secondes)
`max-lease-time`	`7200`	Durée maximale si le client demande plus : **2 heures**
`authoritative`	-	Ce serveur est **le seul DHCP** légitime sur ce réseau

C'est quoi un "bail" (lease) ? 🤔

Le serveur DHCP **prête** une IP au client pour une durée limitée :

```
Client : "Je veux une IP !"
Serveur : "Tiens, voici 192.168.1.105, mais c'est un PRÊT de 10 min"
Client : (après 5 min) "Je suis toujours là, je peux garder ?"
Serveur : "OK, prolongé de 10 min"
Client : (déconnecté) ...
Serveur : (après expiration) "Il est parti, je récupère l'IP"
```

Pourquoi `authoritative` **?** → Évite les conflits si un autre serveur DHCP "pirate" apparaît

DHCP : Explication du bloc subnet 📖

```
subnet 192.168.1.0 netmask 255.255.255.0 {
    range 192.168.1.100 192.168.1.200;
    option routers 192.168.1.1;
    option subnet-mask 255.255.255.0;
    option domain-name-servers 8.8.8.8, 1.1.1.1;
    option domain-name "example.local";
}
```

Ligne	Signification
`subnet ... netmask ...`	Définit le réseau géré (192.168.1.0/24)
`range 192.168.1.100 192.168.1.200`	Plage d'IP à distribuer (101 adresses disponibles)
`option routers`	Passerelle par défaut (gateway) envoyée aux clients
`option subnet-mask`	Masque de sous-réseau envoyé aux clients
`option domain-name-servers`	Serveurs DNS envoyés aux clients
`option domain-name`	Nom de domaine local (pour la recherche DNS)

Analogie 🏨 **:** Le serveur DHCP est comme un hôtel qui attribue des chambres (IP) aux clients, avec les infos utiles (WiFi, restaurant = DNS, passerelle)

Configuration DHCP : réservations 📌

Réserver une IP pour une machine spécifique (par MAC address) :

```
 # Réservation pour l'imprimante
host imprimante {
    hardware ethernet 00:11:22:33:44:55;
    fixed-address 192.168.1.10;
}

# Réservation pour le serveur
host serveur-web {
    hardware ethernet AA:BB:CC:DD:EE:FF;
    fixed-address 192.168.1.20;
}
```

Avantage : toujours la même IP, mais géré par DHCP

Configurer l'interface d'écoute

Debian/Ubuntu : `/etc/default/isc-dhcp-server`

```
sudo nano /etc/default/isc-dhcp-server
```

```
# Interface sur laquelle le serveur DHCP écoute
INTERFACESv4="eth0"
INTERFACESv6=""
```

RHEL/Fedora : `/etc/sysconfig/dhcpd`

```
DHCPDARGS="eth0"
```

Démarrer le serveur DHCP ▶️

```
# Tester la configuration
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf

# Démarrer le service
sudo systemctl start isc-dhcp-server      # Debian/Ubuntu
sudo systemctl start dhcpd                # RHEL/Fedora

# Activer au démarrage
sudo systemctl enable isc-dhcp-server
sudo systemctl enable dhcpd

# Vérifier l'état
sudo systemctl status isc-dhcp-server
```

Logs et dépannage DHCP 🔍

Voir les logs DHCP :

```
# Debian/Ubuntu
sudo tail -f /var/log/syslog | grep dhcpd

# RHEL/Fedora
sudo journalctl -u dhcpd -f
```

Voir les baux (leases) actifs :

```
# Fichier des baux
cat /var/lib/dhcp/dhcpd.leases

# Ou utiliser dhcp-lease-list (à installer)
sudo apt install dhcp-helper
dhcp-lease-list
```

Test du serveur DHCP 🧪

Sur une machine cliente :

```
# Libérer l'adresse actuelle
sudo dhclient -r

# Demander une nouvelle adresse
sudo dhclient eth0

# Vérifier l'IP reçue
ip addr show eth0
```

Vérifier que le client a bien reçu :

- Une IP dans la plage définie
- La passerelle correcte : `ip route`
- Les DNS corrects : `cat /etc/resolv.conf`

Récapitulatif du Module 7 ✅

Ce que vous avez appris :

- ✅ Concepts réseau (IP, masque, gateway, DNS)
- ✅ Configuration réseau (ip, nmcli, Netplan)
- ✅ Diagnostic (ping, traceroute, dig, ss)
- ✅ Pare-feu (iptables, ufw, firewalld)
- ✅ SSH et clés SSH
- ✅ Transfert de fichiers (scp, rsync, sftp)
- ✅ Tunnels SSH
- ✅ Dépannage réseau
- ✅ Serveur DHCP (attribution automatique IP)

Prochaine étape 🎯

Module 8 : Automatisation et scripts

- Scripts Bash avancés
- Tâches planifiées avec cron
- Gestion des dépôts
- Dépendances
- Sécurité et mises à jour

Revenir au sommaire

Questions ? 🤔

N’hésitez pas à poser vos questions maintenant !

Le prochain module explorera la gestion des paquets logiciels et la maintenance du système.

QCM Module 7 : Réseau ✅

20 questions sur la configuration et le diagnostic réseau

Revenir au sommaire

Question 1

Quelle adresse IP est localhost ?

A) `192.168.1.1`

B) `127.0.0.1`

C) `10.0.0.1`

D) `0.0.0.0`

Question 2

Quel port utilise SSH par défaut ?

A) 21

B) 22

C) 23

D) 80

Revenir au sommaire

Question 3

Quelle commande teste la connectivité réseau ?

A) `test`

B) `ping`

C) `connect`

D) `check`

Question 4

Quelle commande affiche la configuration IP ?

A) `ipconfig`

B) `ifconfig`

C) `ip addr`

D) B et C sont corrects

Revenir au sommaire

Question 5

Quelle commande affiche les connexions actives ?

A) `connections`

B) `netstat` ou `ss`

C) `active`

D) `ports`

Question 6

Quel outil génère une paire de clés SSH ?

A) `ssh-key`

B) `ssh-keygen`

C) `keygen`

D) `ssh-create`

Question 7

Où sont stockées les clés SSH de l'utilisateur ?

A) `/etc/ssh/`

B) `~/.ssh/`

C) `/var/ssh/`

D) `/root/ssh/`

Question 8

Quel pare-feu est simple à utiliser sur Ubuntu ?

A) `iptables`

B) `ufw`

C) `firewalld`

D) `firewall`

Question 9

Comment copier un fichier via SSH ?

A) `copy`

B) `scp`

C) `ssh-copy`

D) `transfer`

Question 10

Quelle commande résout un nom de domaine en IP ?

A) `resolve`

B) `nslookup` ou `dig`

C) `dns`

D) `lookup`

Revenir au sommaire

Question 11

Que signifie la notation CIDR /24 ?

A) 24 machines dans le réseau

B) Masque 255.255.255.0 (256 adresses)

C) 24 ports ouverts

D) 24 sous-réseaux

Question 12

Quelle option SSH crée un tunnel local (Local Forwarding) ?

A) `-R`

B) `-L`

C) `-D`

D) `-T`

Question 13

Quel fichier contient les serveurs DNS sur Linux ?

A) `/etc/dns.conf`

B) `/etc/resolv.conf`

C) `/etc/network/dns`

D) `/etc/dns/servers`

Question 14

Quelle commande synchronise des fichiers via SSH avec reprise ?

A) `scp`

B) `sftp`

C) `rsync`

D) `ssh-sync`

Question 15

Quel algorithme de clé SSH est recommandé en 2025 ?

A) RSA 1024

B) DSA

C) Ed25519

D) ECDSA 256

Question 16

Que fait la commande `traceroute` ?

A) Teste la vitesse de connexion

B) Affiche le chemin réseau vers un hôte

C) Liste les interfaces réseau

D) Scanne les ports ouverts

Revenir au sommaire

Question 17

Quel outil configure le réseau sur Ubuntu moderne (22.04+) ?

A) ifconfig

B) network-scripts

C) Netplan

D) wicked

Question 18

Quel fichier permet de définir des alias DNS locaux ?

A) `/etc/hosts`

B) `/etc/aliases`

C) `/etc/dns/local`

D) `/etc/hostname`

Question 19

Quelle est la plage de ports "bien connus" (well-known) ?

A) 1-100

B) 0-1023

C) 1024-49151

D) 49152-65535

Question 20

Quel service attribue automatiquement des IP aux clients ?

A) DNS

B) NAT

C) DHCP

D) ARP

Réponses du Module 7 📝 (1/4)

Réponse 1 : B) `127.0.0.1`

- Toute la plage 127.0.0.0/8 est réservée au loopback

Réponse 2 : B) 22

- Port 22 = SSH, Port 21 = FTP, Port 23 = Telnet, Port 80 = HTTP

Réponse 3 : B) `ping`

- `ping google.com` teste si le serveur est joignable

Réponse 4 : D) B et C sont corrects

- `ip addr` est moderne, `ifconfig` est ancien mais toujours utilisé

Réponse 5 : B) `netstat` ou `ss`

- `ss` est la version moderne de `netstat`

Réponses du Module 7 📝 (2/4)

Réponse 6 : B) `ssh-keygen`

- `ssh-keygen -t ed25519` est recommandé en 2025

Réponse 7 : B) `~/.ssh/`

- Répertoire personnel de chaque utilisateur

Réponse 8 : B) `ufw`

- ufw = Uncomplicated Firewall (interface simple pour iptables)

Réponse 9 : B) `scp`

- `scp fichier user@serveur:/chemin/`

Réponse 10 : B) `nslookup` ou `dig`

- `dig` est plus moderne et détaillé

Revenir au sommaire

Réponses du Module 7 📝 (3/4)

Réponse 11 : B) Masque 255.255.255.0 (256 adresses)

- /24 signifie 24 bits pour le réseau, 8 bits pour les hôtes (2^8 = 256)

Réponse 12 : B) `-L`

- `-L` = Local forwarding (ramène un port distant vers local)
- `-R` = Remote forwarding, `-D` = Dynamic (proxy SOCKS)

Réponse 13 : B) `/etc/resolv.conf`

- Contient les serveurs DNS avec `nameserver 8.8.8.8`

Réponse 14 : C) `rsync`

- `rsync -avz` synchronise avec compression et reprise en cas d'interruption

Réponse 15 : C) Ed25519

- Ed25519 est le plus moderne, sûr et performant. RSA 4096 reste acceptable.

Revenir au sommaire

Réponses du Module 7 📝 (4/4)

Réponse 16 : B) Affiche le chemin réseau vers un hôte

- Montre chaque routeur traversé entre vous et la destination

Réponse 17 : C) Netplan

- Netplan est l'outil standard sur Ubuntu 18.04+ pour configurer le réseau

Réponse 18 : A) `/etc/hosts`

- Permet de définir des correspondances nom → IP localement

Réponse 19 : B) 0-1023

- Well-known ports (réservés aux services système), nécessitent root

Réponse 20 : C) DHCP

- Dynamic Host Configuration Protocol : attribution automatique d'IP, masque, gateway, DNS

Score du Module 7 📊

Comptez vos bonnes réponses :

- **18-20** : Expert réseau ! ⭐⭐⭐
- **14-17** : Très bien ! ⭐⭐
- **10-13** : Bien, à consolider ⭐
- **< 10** : Relire le module 7

Revenir au sommaire

🎯 Exercices pratiques - Module 7

Configuration et diagnostic réseau

Exercice 1 : Diagnostic réseau de base 🔍

Objectif : Diagnostiquer la connectivité

Consignes :

1. Affichez votre adresse IP
2. Affichez la passerelle par défaut
3. Testez la connectivité vers `8.8.8.8`
4. Testez la résolution DNS vers `google.com`
5. Tracez la route vers `cloudflare.com`
6. Affichez les connexions actives

Solution Exercice 1 💡

```
# 1. Adresse IP
ip addr show
# ou
hostname -I
# ou
ifconfig  # si installé

# 2. Passerelle
ip route show default
# ou
route -n | grep ^0.0.0.0

# 3. Test connectivité
ping -c 4 8.8.8.8

# 4. Test DNS
ping -c 4 google.com
# ou
nslookup google.com
# ou
dig google.com +short

# 5. Trace route
traceroute cloudflare.com
# ou avec ICMP
sudo traceroute -I cloudflare.com

# 6. Connexions actives
ss -tunap
# ou
sudo netstat -tunap
```

Exercice 2 : Configuration SSH 🔐

Objectif : Configurer et sécuriser SSH

Consignes :

1. Générez une paire de clés SSH (ed25519)
2. Affichez votre clé publique
3. Copiez votre clé vers `localhost` (test)
4. Connectez-vous sans mot de passe
5. Changez le port SSH en 2222 (config)
6. Désactivez l'authentification par mot de passe

Revenir au sommaire

Solution Exercice 2 💡

```
# 1. Générer clés
ssh-keygen -t ed25519 -C "votre@email.com"
# Appuyez sur Entrée pour accepter les valeurs par défaut

# 2. Afficher clé publique
cat ~/.ssh/id_ed25519.pub

# 3. Copier vers localhost
ssh-copy-id localhost
# ou manuellement :
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# 4. Connexion sans mot de passe
ssh localhost
# Devrait fonctionner directement !

# 5 & 6. Configuration SSH (sur serveur distant)
sudo nano /etc/ssh/sshd_config

# Modifier :
# Port 2222
# PasswordAuthentication no
# PubkeyAuthentication yes
# PermitRootLogin no

# Tester la config
sudo sshd -t

# Redémarrer
sudo systemctl restart sshd

# Se connecter avec nouveau port
ssh -p 2222 user@server
```

Exercice 3 : Pare-feu avec UFW 🔥

Objectif : Configurer un pare-feu simple

Consignes :

1. Installez et activez UFW
2. Autorisez SSH (port 22)
3. Autorisez HTTP et HTTPS
4. Bloquez le ping (ICMP)
5. Affichez les règles
6. Testez les règles

Solution Exercice 3 💡

```
# 1. Installer et activer
sudo apt install ufw
sudo ufw enable

# 2. Autoriser SSH (IMPORTANT : avant d'activer !)
sudo ufw allow 22/tcp
# ou
sudo ufw allow ssh

# 3. HTTP et HTTPS
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
# ou
sudo ufw allow 'Nginx Full'

# 4. Bloquer ping
sudo ufw deny proto icmp

# 5. Afficher règles
sudo ufw status verbose
sudo ufw status numbered

# 6. Tester
# Depuis une autre machine
ping votre_ip  # Devrait être bloqué
curl http://votre_ip  # Devrait fonctionner

# Supprimer une règle (par numéro)
sudo ufw delete 3

# Désactiver temporairement
sudo ufw disable
```

Exercice 4 : Transfert de fichiers 📤

Objectif : Transférer des fichiers de manière sécurisée

Consignes :

1. Créez un fichier de test de 10 Mo
2. Transférez-le vers `/tmp` sur localhost avec `scp`
3. Transférez un répertoire complet avec `scp -r`
4. Utilisez `rsync` pour synchroniser un répertoire
5. Comparez les temps de transfert

Revenir au sommaire

Solution Exercice 4 💡

```
# 1. Créer fichier de test
dd if=/dev/zero of=/tmp/test_10mo.bin bs=1M count=10

# 2. Transférer avec scp
time scp /tmp/test_10mo.bin localhost:/tmp/test_copy.bin

# 3. Répertoire complet
mkdir -p /tmp/source/{dir1,dir2,dir3}
touch /tmp/source/dir{1,2,3}/file{1..5}.txt
time scp -r /tmp/source localhost:/tmp/dest

# 4. Rsync (plus efficace)
time rsync -avz /tmp/source/ localhost:/tmp/dest_rsync/

# Modifier un fichier
echo "modifié" >> /tmp/source/dir1/file1.txt

# Re-synchroniser (seulement les changements)
time rsync -avz /tmp/source/ localhost:/tmp/dest_rsync/

# 5. Comparer
# rsync est plus rapide pour les mises à jour !

# Rsync avec progression
rsync -avz --progress /tmp/source/ localhost:/tmp/dest2/
```

Exercice 5 : Serveur web simple 🌐

Objectif : Mettre en place un serveur web basique

Consignes :

1. Installez nginx
2. Créez une page HTML simple
3. Configurez un virtual host
4. Testez avec curl et navigateur
5. Ajoutez une entrée `/etc/hosts`
6. Vérifiez les logs d'accès

Solution Exercice 5 💡

```
# 1. Installer nginx
sudo apt update
sudo apt install nginx

# 2. Page HTML
sudo mkdir -p /var/www/monsite
sudo tee /var/www/monsite/index.html << 'EOF'
<!DOCTYPE html>
<html>
<head><title>Mon Site</title></head>
<body>
  <h1>Bienvenue sur mon site !</h1>
  <p>Ceci est une page de test.</p>
</body>
</html>
EOF

# 3. Virtual host
sudo tee /etc/nginx/sites-available/monsite << 'EOF'
server {
    listen 80;
    server_name monsite.local;
    root /var/www/monsite;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
EOF

# Activer
sudo ln -s /etc/nginx/sites-available/monsite /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx

# 5. /etc/hosts
echo "127.0.0.1 monsite.local" | sudo tee -a /etc/hosts

# 4. Tester
curl http://monsite.local
firefox http://monsite.local &

# 
sudo tail -f /var/log/nginx/access.log
```

Revenir au sommaire

Exercice bonus : Script de monitoring réseau

Objectif : Surveiller la connectivité

Créez un script qui :

- Teste la connectivité vers plusieurs serveurs
- Mesure la latence
- Enregistre les résultats
- Alerte en cas de problème

Solution Exercice bonus 💡

```bash
#!/bin/bash
# Script de monitoring réseau

SERVERS=("8.8.8.8" "1.1.1.1" "google.com" "github.com")
LOG_FILE="/var/log/network_monitor.log"
ALERT_THRESHOLD=200  # ms

echo "===== Monitoring $(date) =====" | tee -a $LOG_FILE

for server in "${SERVERS[@]}"; do
    echo -n "Test de $server... " | tee -a $LOG_FILE

    # Ping 3 fois
    result=$(ping -c 3 -W 2 $server 2>/dev/null | grep "avg" | \
             awk -F'/' '{print $5}')

    if [ -z "$result" ]; then
        echo "❌ ÉCHEC" | tee -a $LOG_FILE
        logger -t NET_MONITOR "ALERTE: $server injoignable"
    else
        latency=${result%.*}  # Arrondir
        echo "✅ OK (${latency}ms)" | tee -a $LOG_FILE

        # Alerte si latence élevée
        if [ $latency -gt $ALERT_THRESHOLD ]; then
            echo "  ⚠️  Latence élevée!" | tee -a $LOG_FILE
            logger -t NET_MONITOR "WARNING: $server latence ${latency}ms"
        fi
    fi

    sleep 1
done

echo "============================" | tee -a $LOG_FILE
```

Automatiser avec cron :

```
# Toutes les 5 minutes
*/5 * * * * /usr/local/bin/network_monitor.sh
```

Points clés à retenir 📌

Diagnostic :

- `ping` : connectivité
- `traceroute` : chemin
- `dig` / `nslookup` : DNS
- `ss` / `netstat` : connexions

SSH :

- Clés publiques/privées
- `ssh-keygen` , `ssh-copy-id`
- Configuration dans `/etc/ssh/sshd_config`

Pare-feu :

- UFW pour simplicité
- `allow` / `deny` pour les règles
- Toujours autoriser SSH avant d'activer !

🤖 Module 8

Automatisation, scripts et tâches planifiées

Automatiser pour gagner du temps et éviter les erreurs

Revenir au sommaire

Pourquoi automatiser ? 🤖

Analogie : la machine à café programmable ☕

Sans automatisation :

- Se lever à 6h pour faire le café
- Risque d'oublier
- Toujours la même tâche répétitive

Avec automatisation :

- Programmée pour 6h30
- Café prêt au réveil
- Zéro effort

Avantages de l'automatisation ✨

1. Gain de temps

- Tâches répétitives automatisées
- Vous dormez, le script travaille

2. Fiabilité

- Pas d'oubli
- Pas d'erreur humaine
- Traçabilité

3. Cohérence

- Toujours la même procédure
- Reproductible

4. Scalabilité

- Un script pour 1 serveur = un script pour 100 serveurs

Shell scripting : bases 📝

Script : fichier texte contenant des commandes

Premier script :

```
#!/bin/bash
# Mon premier script

echo "Hello, World!"
echo "Date actuelle : $(date)"
echo "Utilisateur : $USER"
```

```
# Rendre exécutable
chmod +x script.sh

# Exécuter
./script.sh
```

Le shebang #! 🔧

Shebang : première ligne qui indique l'interpréteur

```
#!/bin/bash      # Bash
#!/bin/sh        # Shell POSIX
#!/usr/bin/env python3    # Python
#!/usr/bin/env node       # Node.js
```

Pourquoi `/usr/bin/env` **?**

- Cherche l'interpréteur dans le PATH
- Plus portable

Variables 📦

```bash
#!/bin/bash

# Définir une variable
nom="Alice"
age=30

# Utiliser une variable
echo "Bonjour $nom"
echo "Vous avez $age ans"

# Ou avec accolades (recommandé)
echo "Bonjour ${nom}"
echo "Fichier : ${nom}.txt"
```

⚠️ Pas d'espace autour du `=` !

```bash
nom = "Alice"    # ERREUR
nom="Alice"      # CORRECT
```

Variables d'environnement 🌍

```bash
#!/bin/bash

# Variables système
echo "Home : $HOME"
echo "User : $USER"
echo "Path : $PATH"
echo "Shell : $SHELL"
echo "PWD : $PWD"

# Définir une variable d'environnement
export MA_VARIABLE="valeur"

# Dans un autre script/programme, $MA_VARIABLE sera disponible
```

Variables spéciales 🎁

```bash
#!/bin/bash

echo "Script : $0"           # Nom du script
echo "Premier arg : $1"      # Premier argument
echo "Deuxième arg : $2"     # Deuxième argument
echo "Tous les args : $@"    # Tous les arguments
echo "Nombre d'args : $#"    # Nombre d'arguments
echo "PID : $$"              # PID du script
echo "Dernier code retour : $?"  # Code retour dernière commande
```

Analogie : une fonction en cuisine 👨‍🍳

Imaginez une recette (script) avec des ingrédients (arguments) :

- **$0** : le nom de la recette ("Tarte aux pommes")
- **$1, $2** : les ingrédients ("pommes", "sucre")
- **$#** : nombre d'ingrédients (2)
- **$$** : le numéro de la cuisine (PID unique)
- **$?** : "est-ce que ça a réussi ?" (0=succès, autre=échec)

Exemple concret :

```bash
#!/bin/bash
# Script : backup.sh

if [ $# -lt 2 ]; then
    echo "Usage: $0 <source> <destination>"
    echo "Exemple: $0 /home/user /backup"
    exit 1
fi

echo "Sauvegarde de $1 vers $2..."
tar -czf "$2/backup-$(date +%Y%m%d).tar.gz" "$1"

if [ $? -eq 0 ]; then
    echo "✅ Sauvegarde réussie !"
else
    echo "❌ Erreur lors de la sauvegarde"
    exit 1
fi
```

Exemple avec arguments 📋

```
#!/bin/bash

echo "Nombre d'arguments : $#"
echo "Premier argument : $1"
echo "Deuxième argument : $2"
echo "Tous les arguments : $@"

# Usage :
# ./script.sh alice bob
```

Sortie :

```
Nombre d'arguments : 2
Premier argument : alice
Deuxième argument : bob
Tous les arguments : alice bob
```

Lire l'entrée utilisateur 🎤

```bash
#!/bin/bash

echo "Quel est votre nom ?"
read nom

echo "Bonjour $nom!"

# Lecture avec prompt
read -p "Votre âge : " age
echo "Vous avez $age ans"

# Lecture silencieuse (mot de passe)
read -sp "Mot de passe : " password
echo
echo "Mot de passe saisi (chut!)"
```

Conditions : if/else 🔀

```bash
#!/bin/bash

age=25

if [ $age -ge 18 ]; then
    echo "Vous êtes majeur"
else
    echo "Vous êtes mineur"
fi
```

⚠️ Espaces importants !

```bash
if [ $age -ge 18 ]; then    # CORRECT
if [$age -ge 18]; then      # ERREUR
```

Opérateurs de comparaison 🔢

Nombres :

```
-eq    # égal (equal)
-ne    # différent (not equal)
-lt    # inférieur (less than)
-le    # inférieur ou égal (less or equal)
-gt    # supérieur (greater than)
-ge    # supérieur ou égal (greater or equal)
```

Exemple :

```
if [ $age -ge 18 ]; then
    echo "Majeur"
fi
```

Comparaison de chaînes 🔤

```
=       # égal
!=      # différent
-z      # chaîne vide
-n      # chaîne non vide
```

Exemple :

```
#!/bin/bash

nom="Alice"

if [ "$nom" = "Alice" ]; then
    echo "Bonjour Alice"
fi

if [ -z "$nom" ]; then
    echo "Nom vide"
fi
```

⚠️ **Toujours mettre les variables entre guillemets !**

Tests de fichiers 📁

```
-e fichier    # existe
-f fichier    # est un fichier régulier
-d fichier    # est un répertoire
-r fichier    # est lisible
-w fichier    # est inscriptible
-x fichier    # est exécutable
-s fichier    # taille > 0
```

Exemple :

```
if [ -f "/etc/passwd" ]; then
    echo "Le fichier existe"
fi

if [ -d "/home/alice" ]; then
    echo "Le répertoire existe"
fi
```

if/elif/else 🔀

```bash
#!/bin/bash

read -p "Votre note : " note

if [ $note -ge 16 ]; then
    echo "Très bien"
elif [ $note -ge 14 ]; then
    echo "Bien"
elif [ $note -ge 12 ]; then
    echo "Assez bien"
elif [ $note -ge 10 ]; then
    echo "Passable"
else
    echo "Insuffisant"
fi
```

Opérateurs logiques 🧠

```
&&      # ET (AND)
||      # OU (OR)
!       # NON (NOT)
```

Exemples :

```
# ET
if [ $age -ge 18 ] && [ $permis = "oui" ]; then
    echo "Vous pouvez conduire"
fi

# OU
if [ $jour = "samedi" ] || [ $jour = "dimanche" ]; then
    echo "Week-end!"
fi

# NON
if [ ! -f "/tmp/lock" ]; then
    echo "Pas de fichier verrou"
fi
```

Case : switch/case 🎯

```bash
#!/bin/bash

read -p "Entrez une commande (start/stop/restart) : " cmd

case $cmd in
    start)
        echo "Démarrage..."
        ;;
    stop)
        echo "Arrêt..."
        ;;
    restart)
        echo "Redémarrage..."
        ;;
    *)
        echo "Commande inconnue"
        ;;
esac
```

Boucle for 🔁

```bash
#!/bin/bash

# Itérer sur une liste
for fruit in pomme banane orange; do
    echo "J'aime les ${fruit}s"
done

# Itérer sur des fichiers
for fichier in *.txt; do
    echo "Fichier : $fichier"
done

# Itérer sur une plage de nombres
for i in {1..5}; do
    echo "Numéro $i"
done
```

Boucle for style C 🔁

```bash
#!/bin/bash

# Syntaxe C-style
for ((i=1; i<=10; i++)); do
    echo "Compteur : $i"
done

# Avec un pas de 2
for ((i=0; i<=20; i+=2)); do
    echo "Nombre pair : $i"
done
```

Boucle while ∞

```bash
#!/bin/bash

# Boucle tant que condition vraie
compteur=1

while [ $compteur -le 5 ]; do
    echo "Compteur : $compteur"
    ((compteur++))
done

# Lecture d'un fichier ligne par ligne
while read ligne; do
    echo "Ligne : $ligne"
done < fichier.txt
```

Boucle until 🔄

```bash
#!/bin/bash

# Boucle jusqu'à ce que condition vraie
compteur=1

until [ $compteur -gt 5 ]; do
    echo "Compteur : $compteur"
    ((compteur++))
done
```

Différence with/until :

- `while` : continue tant que VRAI
- `until` : continue tant que FAUX

Break et continue ⏭️

```bash
#!/bin/bash

# Break : sortir de la boucle
for i in {1..10}; do
    if [ $i -eq 5 ]; then
        break
    fi
    echo $i
done
# Affiche : 1 2 3 4

# Continue : passer à l'itération suivante
for i in {1..5}; do
    if [ $i -eq 3 ]; then
        continue
    fi
    echo $i
done
# Affiche : 1 2 4 5
```

Fonctions 📦

```bash
#!/bin/bash

# Définir une fonction
saluer() {
    echo "Bonjour!"
}

# Appeler la fonction
saluer

# Fonction avec arguments
saluer_nom() {
    echo "Bonjour $1!"
}

saluer_nom "Alice"
saluer_nom "Bob"
```

Fonctions avec retour

```bash
#!/bin/bash

# Fonction avec return (code de sortie 0-255)
est_pair() {
    if [ $(($1 % 2)) -eq 0 ]; then
        return 0    # Vrai
    else
        return 1    # Faux
    fi
}

if est_pair 4; then
    echo "4 est pair"
fi

# Fonction avec echo (retourner une valeur)
additionner() {
    echo $(($1 + $2))
}

resultat=$(additionner 5 3)
echo "Résultat : $resultat"
```

Revenir au sommaire

Variables locales et globales

```bash
#!/bin/bash

global_var="Je suis globale"

ma_fonction() {
    local local_var="Je suis locale"
    echo $global_var    # Accessible
    echo $local_var     # Accessible
}

ma_fonction

echo $global_var        # Accessible
echo $local_var         # Vide (hors de portée)
```

Tableaux

```bash
#!/bin/bash

# Définir un tableau
fruits=("pomme" "banane" "orange")

# Accéder à un élément
echo ${fruits[0]}       # pomme
echo ${fruits[1]}       # banane

# Tous les éléments
echo ${fruits[@]}

# Nombre d'éléments
echo ${#fruits[@]}

# Ajouter un élément
fruits+=("kiwi")

# Parcourir le tableau
for fruit in "${fruits[@]}"; do
    echo $fruit
done
```

Calculs arithmétiques

```bash
#!/bin/bash

# Méthode 1 : $(( ))
a=5
b=3
echo $((a + b))         # 8
echo $((a - b))         # 2
echo $((a * b))         # 15
echo $((a / b))         # 1 (division entière)
echo $((a % b))         # 2 (modulo)

# Incrémenter
((a++))
echo $a                 # 6

# Méthode 2 : expr (ancien)
resultat=$(expr $a + $b)

# Méthode 3 : bc pour décimaux
echo "scale=2; 10 / 3" | bc    # 3.33
```

Substitution de commandes

```bash
#!/bin/bash

# Méthode moderne : $( )
date_actuelle=$(date)
echo "Date : $date_actuelle"

fichiers=$(ls -1 | wc -l)
echo "Nombre de fichiers : $fichiers"

# Ancien style : ` ` (backticks)
date_actuelle=`date`
```

Chaînes de caractères 🧵

```bash
#!/bin/bash

nom="Alice"

# Longueur
echo ${#nom}            # 5

# Sous-chaîne (à partir de position, longueur)
echo ${nom:0:2}         # Al

# Remplacement
phrase="J'aime les chats"
echo ${phrase/chats/chiens}    # J'aime les chiens

# Tout en majuscules
echo ${nom^^}           # ALICE

# Tout en minuscules
echo ${nom,,}           # alice
```

Redirection dans les scripts 📤

```bash
#!/bin/bash

# Rediriger stdout vers un fichier
echo "Log entry" > log.txt

# Ajouter à un fichier
echo "Another entry" >> log.txt

# Rediriger stderr
commande_inexistante 2> erreurs.txt

# Rediriger tout
commande > tout.txt 2>&1

# Ignorer les erreurs
commande 2> /dev/null
```

Gestion des erreurs 🚨

```bash
#!/bin/bash

# Exit si erreur
set -e

# Vérifier le code de retour
if ! commande; then
    echo "Erreur lors de l'exécution"
    exit 1
fi

# Ou avec $?
commande
if [ $? -ne 0 ]; then
    echo "Erreur"
    exit 1
fi

# Exit avec message
error_exit() {
    echo "Erreur : $1" >&2
    exit 1
}

[ -f "fichier.txt" ] || error_exit "Fichier manquant"
```

set : options de script 🔧

```bash
#!/bin/bash

set -e    # Arrêter si une commande échoue
set -u    # Erreur si variable non définie
set -x    # Afficher les commandes exécutées (debug)
set -o pipefail    # Échec si une commande du pipe échoue

# Combiné (recommandé)
set -euo pipefail

# Ou sur la ligne shebang
#!/bin/bash -euo pipefail
```

Exemple de script pratique : sauvegarde 💾

```bash
#!/bin/bash
set -euo pipefail

# Variables
SOURCE="/home/alice"
DESTINATION="/backup"
DATE=$(date +%Y%m%d_%H%M%S)
ARCHIVE="backup_${DATE}.tar.gz"

# Fonction de log
log() {
    echo "[$(date +%Y-%m-%d\ %H:%M:%S)] $1"
}

# Vérifier que la source existe
if [ ! -d "$SOURCE" ]; then
    log "ERREUR : $SOURCE n'existe pas"
    exit 1
fi
```

Exemple de script pratique : sauvegarde (suite) 💾

```
# Créer le répertoire de destination si nécessaire
mkdir -p "$DESTINATION"

# Sauvegarde
log "Début de la sauvegarde de $SOURCE"

if tar -czf "${DESTINATION}/${ARCHIVE}" "$SOURCE"; then
    log "Sauvegarde réussie : ${ARCHIVE}"
else
    log "ERREUR lors de la sauvegarde"
    exit 1
fi

# Supprimer les sauvegardes de plus de 7 jours
find "$DESTINATION" -name "backup_*.tar.gz" -mtime +7 -delete
log "Anciennes sauvegardes supprimées"

log "Fin de la sauvegarde"
```

cron : planificateur de tâches ⏰

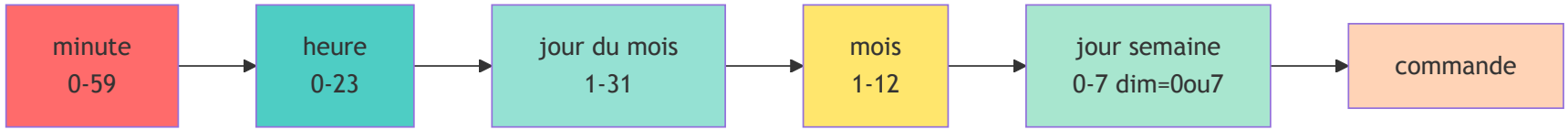
cron : exécute des commandes à intervalles réguliers

Analogie : l'alarme de votre téléphone

- Programmée pour sonner à des moments précis
- Peut se répéter (tous les jours, toutes les semaines)
- Vous rappelle des tâches

```
# Éditer le crontab
crontab -e

# Voir le crontab actuel
crontab -l

# Supprimer le crontab
crontab -r
```

Format du crontab 📋



Format : `* * * * * commande`

Exemples :

```
# Tous les jours à 2h30
30 2 * * * /chemin/script.sh

# Toutes les heures
0 * * * * /chemin/script.sh

# Tous les lundis à 9h
0 9 * * 1 /chemin/script.sh
```

Exemples de crontab ⏰

```
# Toutes les 5 minutes
*/5 * * * * /chemin/script.sh

# Tous les jours à minuit
0 0 * * * /chemin/backup.sh

# Le premier jour de chaque mois à 6h
0 6 1 * * /chemin/rapport.sh

# Du lundi au vendredi à 8h
0 8 * * 1-5 /chemin/script.sh

# Samedis et dimanches à 10h
0 10 * * 6,7 /chemin/weekend.sh

# Au démarrage du système
@reboot /chemin/startup.sh
```

Raccourcis cron

```
@reboot         # Au démarrage
@yearly         # Une fois par an : 0 0 1 1 *
@annually       # Idem
@monthly        # Une fois par mois : 0 0 1 * *
@weekly         # Une fois par semaine : 0 0 * * 0
@daily          # Une fois par jour : 0 0 * * *
@midnight       # Idem
@hourly         # Une fois par heure : 0 * * * *
```

Exemple :

```
@daily /usr/local/bin/cleanup.sh
@hourly /usr/local/bin/check_health.sh
```

Variables d'environnement dans cron 🌍

```
# Définir des variables
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
MAILTO=admin@example.com

# Ensuite les tâches cron
0 2 * * * /chemin/backup.sh
```

⚠️ **Problème courant :** PATH limité dans cron

- Toujours utiliser des chemins absolus
- Ou définir PATH au début du crontab

Redirection dans cron 📤

```
# Envoyer stdout vers un fichier
0 2 * * * /chemin/script.sh > /var/log/script.log

# Ajouter à un fichier
0 2 * * * /chemin/script.sh >> /var/log/script.log

# Rediriger stdout et stderr
0 2 * * * /chemin/script.sh >> /var/log/script.log 2>&1

# Ignorer toute sortie (silencieux)
0 2 * * * /chemin/script.sh > /dev/null 2>&1

# Envoyer par email (si MAILTO défini)
0 2 * * * /chemin/script.sh
```

Cron système : /etc/cron.* 🗂️

Répertoires système :

```
/etc/cron.hourly/     # Scripts exécutés toutes les heures
/etc/cron.daily/      # Scripts quotidiens
/etc/cron.weekly/     # Scripts hebdomadaires
/etc/cron.monthly/    # Scripts mensuels
```

Utilisation :

```
# Copier un script dans le répertoire
sudo cp mon_script.sh /etc/cron.daily/

# Rendre exécutable
sudo chmod +x /etc/cron.daily/mon_script.sh
```

Plus simple que crontab pour tâches standards !

anacron : pour les machines non 24/7 🏠

anacron : exécute les tâches manquées

Différence avec cron :

- **cron** : exécute à un moment précis
- **anacron** : exécute quand la machine est allumée

Utile pour :

- Ordinateurs portables
- Machines de bureau
- Serveurs redémarrés souvent

```
# Configuration : /etc/anacrontab
# Période  Délai  Identifiant  Commande
1          5      cron.daily   run-parts /etc/cron.daily
7          10     cron.weekly  run-parts /etc/cron.weekly
@monthly   15     cron.monthly run-parts /etc/cron.monthly
```

at : tâche ponctuelle unique ⏱️

at : exécuter une commande une seule fois

```
# Installer
sudo apt install at

# Activer
sudo systemctl enable --now atd

# Planifier une tâche
at 10:30
> /chemin/script.sh
> Ctrl+D

# Autres formats
at now + 1 hour
at tomorrow
at 14:00 tomorrow
at 2:30 PM Friday
```

Gérer les tâches at

```
# Lister les tâches en attente
atq
# ou
at -l

# Voir les détails d'une tâche
at -c 1

# Supprimer une tâche
atrm 1

# Planifier depuis un fichier
at -f script.sh now + 1 hour
```

systemd timers : alternative moderne ⏱️

systemd timer : remplaçant moderne de cron

Avantages :

- Intégration avec systemd
- Logs dans journalctl
- Dépendances
- Répétition flexible

Inconvénient :

- Plus complexe à configurer

Créer un systemd timer 🛠️

1. Créer le service : `/etc/systemd/system/mon-script.service`

```
[Unit]
Description=Mon script de maintenance

[Service]
Type=oneshot
ExecStart=/usr/local/bin/mon-script.sh
```

Créer un systemd timer (suite) 🛠️

2. Créer le timer : `/etc/systemd/system/mon-script.timer`

```
[Unit]
Description=Timer pour mon script

[Timer]
OnCalendar=daily
OnCalendar=*-*-* 02:30:00
Persistent=true

[Install]
WantedBy=timers.target
```

Créer un systemd timer (suite) 🛠️

3. Activer et démarrer

```
# Recharger systemd
sudo systemctl daemon-reload

# Activer le timer
sudo systemctl enable mon-script.timer

# Démarrer le timer
sudo systemctl start mon-script.timer

# Vérifier l'état
systemctl status mon-script.timer

# Lister tous les timers
systemctl list-timers
```

Syntaxe OnCalendar

```
# Tous les jours à minuit
OnCalendar=daily

# Toutes les heures
OnCalendar=hourly

# Tous les lundis à 9h
OnCalendar=Mon *-*-* 09:00:00

# Tous les 15 minutes
OnCalendar=*:0/15

# Premier jour du mois
OnCalendar=*-*-01 00:00:00

# Vérifier la syntaxe
systemd-analyze calendar "Mon *-*-* 09:00:00"
```

Logs : où sont-ils ? 📜

Logs système :

```
/var/log/syslog         # Logs système général (Debian/Ubuntu)
/var/log/messages       # Logs système (RHEL/Fedora)
/var/log/auth.log       # Authentification (Debian/Ubuntu)
/var/log/secure         # Authentification (RHEL/Fedora)
/var/log/kern.log       # Noyau
/var/log/dmesg          # Messages de boot
/var/log/cron           # Logs cron
```

Voir les logs : tail et grep 🔍

```
# Dernières lignes d'un log
tail /var/log/syslog

# Suivre en temps réel
tail -f /var/log/syslog

# 50 dernières lignes
tail -n 50 /var/log/syslog

# Rechercher dans les logs
grep "error" /var/log/syslog

# Case insensitive
grep -i "error" /var/log/syslog

# Avec contexte (3 lignes avant/après)
grep -C 3 "error" /var/log/syslog
```

journalctl : logs systemd 📚

```
# Tous les logs
journalctl

# Depuis le dernier boot
journalctl -b

# Logs d'un service
journalctl -u nginx

# Suivre en temps réel
journalctl -f

# Suivre un service
journalctl -u nginx -f

# Depuis une date
journalctl --since "2025-11-06"
journalctl --since "1 hour ago"
```

journalctl : filtres avancés 🔎

```
# Par priorité
journalctl -p err          # Erreurs
journalctl -p warning      # Warnings et plus

# Période
journalctl --since "2025-11-06 10:00" --until "2025-11-06 12:00"

# Combinaisons
journalctl -u nginx -p err --since "1 day ago"

# Par utilisateur
journalctl _UID=1000

# Kernel messages
journalctl -k

# Export JSON
journalctl -u nginx -o json
```

Gestion des logs : logrotate 🔄

logrotate : rotation automatique des logs

Configuration : `/etc/logrotate.conf` **et** `/etc/logrotate.d/`

Exemple : `/etc/logrotate.d/nginx`

```
/var/log/nginx/*.log {
    daily
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 www-data adm
    sharedscripts
    postrotate
        [ -f /var/run/nginx.pid ] && kill -USR1 $(cat /var/run/nginx.pid)
    endscript
}
```

Configuration logrotate

Options courantes :

```
 daily          # Rotation quotidienne
weekly         # Rotation hebdomadaire
monthly        # Rotation mensuelle
rotate 7       # Garder 7 fichiers
compress       # Compresser les anciens logs
delaycompress # Compresser au prochain cycle
notifempty     # Ne pas tourner si vide
create 0644 user group  # Créer nouveau fichier avec permissions
size 100M      # Tourner si > 100M
```

Tester logrotate 🧪

```
# Tester la configuration
sudo logrotate -d /etc/logrotate.conf

# Forcer la rotation
sudo logrotate -f /etc/logrotate.conf

# Voir l'état
cat /var/lib/logrotate/status
```

Alertes et notifications 🔔

Envoyer un email depuis un script :

```
#!/bin/bash

# Installer mailutils
sudo apt install mailutils

# Envoyer un email simple
echo "Le script a terminé" | mail -s "Notification" admin@example.com

# Avec un fichier
mail -s "Rapport" admin@example.com < rapport.txt

# Dans un script
if ! /chemin/backup.sh; then
    echo "La sauvegarde a échoué" | mail -s "ERREUR Backup" admin@example.com
fi
```

Notifications système : notify-send 💬

```
# Installer
sudo apt install libnotify-bin

# Notification simple
notify-send "Titre" "Message"

# Avec icône
notify-send -i info "Info" "Message informatif"

# Avec urgence
notify-send -u critical "Erreur" "Problème critique!"

# Dans un script
if [ $espace_libre -lt 10 ]; then
    notify-send -u critical "Disque plein" "Moins de 10% d'espace libre"
fi
```

Exemple : script de monitoring 📊

```bash
#!/bin/bash

# Vérifier l'espace disque
espace=$(df -h / | awk 'NR==2 {print $5}' | tr -d '%')

if [ $espace -gt 80 ]; then
    echo "ALERTE : Disque à ${espace}%" | \
        mail -s "Alerte disque" admin@example.com
fi

# Vérifier si un service tourne
if ! systemctl is-active --quiet nginx; then
    echo "ALERTE : Nginx est arrêté" | \
        mail -s "Service down" admin@example.com
    systemctl start nginx
fi
```

Bonnes pratiques de scripting 📋

1. **Toujours commencer par un shebang**

```
#!/bin/bash
```

2. **Utiliser set -euo pipefail**

```
set -euo pipefail
```

3. **Commenter votre code**

```
# Ce script fait une sauvegarde
```

4. **Utiliser des variables pour les valeurs répétées**

```
LOG_DIR=/var/log/mon-app
```

Bonnes pratiques de scripting (suite) 📋

5. **Vérifier les dépendances au début**

```
command -v rsync >/dev/null || { echo "rsync requis"; exit 1; }
```

6. **Toujours mettre les variables entre guillemets**

```
echo "$variable"    # CORRECT
echo $variable      # Risqué
```

7. **Utiliser des fonctions pour le code répété**

8. **Logger les actions importantes**

Bonnes pratiques de scripting (suite) 📋

9. **Tester avant de déployer**
    - Toujours dans un environnement de test
10. **Gérer les erreurs**

```
commande || { echo "Erreur"; exit 1; }
```

11. **Utiliser des chemins absolus dans cron**

```
/usr/bin/python3 /home/user/script.py
```

12. **Ne jamais faire confiance aux entrées utilisateur**

Débugger un script 🐛

```bash
# Mode debug (affiche les commandes)
bash -x script.sh

# Ou dans le script
set -x

# Activer/désactiver ponctuellement
set -x
commandes_à_debugger
set +x

# Vérifier la syntaxe sans exécuter
bash -n script.sh

# ShellCheck : outil de linting
sudo apt install shellcheck
shellcheck script.sh
```

Récapitulatif du Module 8 ✅

Ce que vous avez appris :

- ✅ Bases du shell scripting (variables, conditions, boucles)
- ✅ Fonctions et tableaux
- ✅ Gestion des erreurs
- ✅ cron pour tâches récurrentes
- ✅ at pour tâches ponctuelles
- ✅ systemd timers (alternative moderne)
- ✅ Gestion des logs (journalctl, logrotate)
- ✅ Alertes et notifications
- ✅ Bonnes pratiques de scripting

Prochaine étape 🎯

Module 9 : Sécurité système

- Durcissement SSH et fail2ban
- SELinux et AppArmor
- Pare-feu et durcissement système
- Audit et logs de sécurité
- Fail2ban
- Gestion des mots de passe

Questions ? 🤔

N'hésitez pas à poser vos questions maintenant !

Le prochain module se concentrera sur la sécurité du système Linux.

QCM Module 8 : Automatisation et scripts ✅

10 questions sur le scripting et les tâches planifiées

Revenir au sommaire

Question 1

Quelle est la première ligne d'un script bash ?

A) `#bash`

B) `#!/bin/bash`

C) `<bash>`

D) `//bin/bash`

Question 2

Comment rendre un script exécutable ?

A) `chmod +x script.sh`

B) `execute script.sh`

C) `make executable script.sh`

D) `run script.sh`

Question 3

Comment définir une variable en bash ?

A) `var = value`

B) `var=value`

C) `set var value`

D) `$var=value`

Revenir au sommaire

Question 4

Comment accéder à la valeur d'une variable ?

A) `var`

B) `$var`

C) `&var`

D) `@var`

Question 5

Quelle commande planifie des tâches récurrentes ?

A) `at`

B) `cron`

C) `timer`

D) `schedule`

Question 6

Que signifie `0 2 * * *` dans un crontab ?

A) Toutes les 2 minutes

B) À 2h du matin tous les jours

C) Tous les 2 jours

D) Le 2 de chaque mois

Revenir au sommaire

Question 7

Comment voir les logs systemd en temps réel ?

A) `tail -f /var/log/syslog`

B) `journalctl -f`

C) `logwatch`

D) `logs -f`

Question 8

Quelle structure crée une boucle en bash ?

A) `loop ... end`

B) `for ... done`

C) `while ... endwhile`

D) `repeat ... until`

Question 9

Comment rediriger stdout ET stderr vers un fichier ?

A) `commande > fichier`

B) `commande 2> fichier`

C) `commande &> fichier`

D) `commande >> fichier`

Question 10

Quelle commande planifie une tâche ponctuelle ?

A) `cron`

B) `at`

C) `once`

D) `schedule`

Revenir au sommaire

Réponses du Module 8

Réponse 1 : B) `#!/bin/bash`

- Le shebang indique l'interpréteur à utiliser

Réponse 2 : A) `chmod +x script.sh`

- Puis l'exécuter avec `./script.sh`

Réponse 3 : B) `var=value`

- ⚠️ Pas d'espace autour du `=` !

Réponse 4 : B) `$var`

- Ou `${var}` (forme recommandée)

Réponse 5 : B) `cron`

- `crontab -e` pour éditer les tâches cron

Revenir au sommaire

Réponses du Module 8 (suite)

Réponse 6 : B) À 2h du matin tous les jours

- Format : minute heure jour mois jour_semaine

Réponse 7 : B) `journalctl -f`

- `-f` = follow (comme `tail -f`)

Réponse 8 : B) `for ... done`

- Exemple : `for i in {1..5}; do echo $i; done`

Réponse 9 : C) `commande &> fichier`

- Ou `commande > fichier 2>&1`

Réponse 10 : B) `at`

- `at now + 1 hour` pour dans 1 heure

Revenir au sommaire

Score du Module 8 📊

Comptez vos bonnes réponses :

- **9-10** : Excellent ! ⭐⭐⭐
- **7-8** : Bien ! ⭐⭐
- **5-6** : À revoir ⭐
- **< 5** : Relire le module 7

Revenir au sommaire

🎯 Exercices pratiques - Module 8

Automatisation, scripts et tâches planifiées

Exercice 1 : Premier script Bash

Objectif : Créer un script fonctionnel

Consignes :

Créez un script `sysinfo.sh` qui affiche :

1. Le nom d'utilisateur
2. Le hostname
3. La date et l'heure
4. Le répertoire actuel
5. L'espace disque disponible
6. Les 5 processus utilisant le plus de mémoire

Solution Exercice 1 💡

```
#!/bin/bash
# Script d'informations système

echo "======================================"
echo "  Informations Système"
echo "======================================"
echo

echo "Utilisateur : $(whoami)"
echo "Hostname : $(hostname)"
echo "Date : $(date '+%Y-%m-%d %H:%M:%S')"
echo "Répertoire : $(pwd)"
echo

echo "======================================"
echo "  Espace Disque"
echo "======================================"
df -h / | tail -1
echo

echo "======================================"
echo "  Top 5 Processus (Mémoire)"
echo "======================================"
ps aux --sort=-%mem | head -6 | tail -5
echo

echo "======================================"
```

Utilisation :

```
chmod +x sysinfo.sh
./sysinfo.sh
```

Exercice 2 : Script avec arguments 🎯

Objectif : Gérer les arguments

Créez `backup.sh` **qui :**

- Prend 2 arguments : source et destination
- Vérifie que les arguments sont fournis
- Vérifie que la source existe
- Crée une archive tar.gz avec timestamp
- Affiche un message de succès/échec

Solution Exercice 2 💡

```bash
#!/bin/bash
# Script de sauvegarde

# Vérifier les arguments
if [ $# -ne 2 ]; then
    echo "Usage: $0 <source> <destination>"
    echo "Exemple: $0 /home/user/documents /backup"
    exit 1
fi

SOURCE="$1"
DEST="$2"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
ARCHIVE="backup_${TIMESTAMP}.tar.gz"

# Vérifier que la source existe
if [ ! -d "$SOURCE" ]; then
    echo "❌ Erreur: $SOURCE n'existe pas"
    exit 1
fi

# Créer le répertoire destination si nécessaire
mkdir -p "$DEST"

# Créer l'archive
echo "Sauvegarde de $SOURCE vers $DEST/$ARCHIVE..."
tar -czf "$DEST/$ARCHIVE" "$SOURCE" 2>/dev/null

if [ $? -eq 0 ]; then
    SIZE=$(du -h "$DEST/$ARCHIVE" | cut -f1)
    echo "✅ Sauvegarde réussie ! Taille: $SIZE"
    ls -lh "$DEST/$ARCHIVE"
else
    echo "❌ Erreur lors de la sauvegarde"
    exit 1
fi
```

Exercice 3 : Boucles et conditions 🔄

Objectif : Utiliser les structures de contrôle

Créez un script qui :

1. Boucle sur tous les fichiers .log dans /var/log
2. Affiche le nom et la taille de chaque fichier
3. Archive les fichiers > 10 Mo
4. Supprime les archives > 30 jours

Solution Exercice 3 💡

```bash
#!/bin/bash
# Gestion des logs

LOG_DIR="/var/log"
ARCHIVE_DIR="/backup/logs"
MAX_SIZE=10485760  # 10 Mo en octets

mkdir -p "$ARCHIVE_DIR"

echo "Analyse des fichiers logs..."

# Boucle sur les fichiers .log
for logfile in $LOG_DIR/*.log; do
    # Vérifier que le fichier existe
    [ -f "$logfile" ] || continue

    filename=$(basename "$logfile")
    size=$(stat -f%z "$logfile" 2>/dev/null || stat -c%s "$logfile")
    size_mb=$(echo "scale=2; $size / 1048576" | bc)

    echo "📄 $filename : ${size_mb}Mo"

    # Si > 10 Mo, archiver
    if [ $size -gt $MAX_SIZE ]; then
        echo "  ⚠️  Fichier volumineux, archivage..."
        timestamp=$(date +%Y%m%d)
        gzip -c "$logfile" > "$ARCHIVE_DIR/${filename}_${timestamp}.gz"

        if [ $? -eq 0 ]; then
            echo "  ✅ Archivé"
            # Vider le fichier (ne pas supprimer)
            > "$logfile"
        fi
    fi
done

# Supprimer archives > 30 jours
echo
echo "Nettoyage des anciennes archives..."
find "$ARCHIVE_DIR" -name "*.gz" -mtime +30 -delete
echo "✅ Terminé"
```

Exercice 4 : Fonctions Bash 🎭

Objectif : Créer des fonctions réutilisables

Créez un script avec ces fonctions :

- `log_info()` : affiche un message info
- `log_error()` : affiche un message d'erreur
- `check_root()` : vérifie si on est root
- `check_command()` : vérifie si une commande existe

Solution Exercice 4 💡

```bash
#!/bin/bash
# Script avec fonctions

# Fonction de log info
log_info() {
    echo "[$(date '+%H:%M:%S')] ℹ️  INFO: $1"
}

# Fonction de log erreur
log_error() {
    echo "[$(date '+%H:%M:%S')] ❌ ERROR: $1" >&2
}

# Vérifier si root
check_root() {
    if [ "$EUID" -ne 0 ]; then
        log_error "Ce script doit être exécuté en root"
        exit 1
    fi
}

# Vérifier si une commande existe
check_command() {
    if ! command -v $1 &> /dev/null; then
        log_error "Commande '$1' non trouvée"
        return 1
    fi
    log_info "Commande '$1' trouvée"
    return 0
}

# Exemple d'utilisation
log_info "Démarrage du script"

check_command "git"
check_command "docker"
check_command "kubectl"  # Va échouer si non installé

log_info "Script terminé"
```

Exercice 5 : Tâches planifiées avec cron ⏰

Objectif : Automatiser avec cron

Consignes :

1. Créez un script qui nettoie `/tmp`
2. Planifiez-le tous les jours à 2h du matin
3. Créez une tâche qui sauvegarde `/etc` tous les lundis à 3h
4. Listez vos tâches cron
5. Redirigez les sorties vers un fichier log

Solution Exercice 5 💡

```
# 1. Script de nettoyage
cat > /usr/local/bin/clean_tmp.sh << 'EOF'
#!/bin/bash
find /tmp -type f -mtime +7 -delete
find /tmp -type d -empty -delete
echo "[$(date)] Nettoyage /tmp effectué" >> /var/log/clean_tmp.log
EOF

chmod +x /usr/local/bin/clean_tmp.sh

# 2. Éditer crontab
crontab -e

# Ajouter:
# Nettoyage /tmp tous les jours à 2h
0 2 * * * /usr/local/bin/clean_tmp.sh

# 3. Sauvegarde /etc tous les lundis à 3h
0 3 * * 1 tar -czf /backup/etc_$(date +\%Y\%m\%d).tar.gz /etc

# 4. Lister les tâches
crontab -l

# 5. Avec redirection
0 2 * * * /usr/local/bin/clean_tmp.sh >> /var/log/cron_clean.log 2>&1
0 3 * * 1 tar -czf /backup/etc_$(date +\%Y\%m\%d).tar.gz /etc >> /var/log/backup.log 2>&1

# Exemples de syntaxe cron:
# */5 * * * *     # Toutes les 5 minutes
# 0 */2 * * *     # Toutes les 2 heures
# 0 9-17 * * 1-5  # 9h-17h, lundi-vendredi
# @reboot         # Au démarrage
# @daily          # Quotidien (minuit)
# @weekly         # Hebdomadaire
```

Exercice 6 : Gestion des logs avec journalctl

Objectif : Filtrer et analyser les logs

Consignes :

1. Créez un script qui analyse les logs systemd
2. Extrait les erreurs des 24 dernières heures
3. Compte les erreurs par service
4. Envoie un rapport
5. Exécutez-le quotidiennement

Solution Exercice 6 💡

```bash
#!/bin/bash
# Analyse des logs système

REPORT="/tmp/log_report_$(date +%Y%m%d).txt"

echo "===== Rapport d'analyse des logs =====" > $REPORT
echo "Date: $(date)" >> $REPORT
echo >> $REPORT

# 1 & 2. Erreurs des 24h
echo "📅 Erreurs des dernières 24h:" >> $REPORT
journalctl --since "24 hours ago" -p err --no-pager >> $REPORT
echo >> $REPORT

# 3. Compter par service
echo "📊 Erreurs par service:" >> $REPORT
journalctl --since "24 hours ago" -p err --no-pager | \
    grep -oP '(?<=\w{3} \d+ \d+:\d+:\d+ \w+ )\w+' | \
    sort | uniq -c | sort -rn | head -10 >> $REPORT
echo >> $REPORT

# Warnings aussi
echo "⚠️  Warnings des dernières 24h:" >> $REPORT
journalctl --since "24 hours ago" -p warning --no-pager | wc -l >> $REPORT
echo >> $REPORT

# Services failed
echo "❌ Services en échec:" >> $REPORT
systemctl --failed >> $REPORT 2>&1
echo >> $REPORT

echo "===== Fin du rapport =====" >> $REPORT

# Afficher le rapport
cat $REPORT

# Envoyer par email (optionnel)
# mail -s "Rapport Logs $(hostname)" admin@example.com < $REPORT
```

Revenir au sommaire

Planifier quotidiennement à 7h :

```
0 7 * * * /usr/local/bin/log_analysis.sh
```

Exercice bonus : Dashboard système interactif

Objectif : Script de monitoring avancé

Créez un script qui affiche en continu :

- Charge système
- Top processus
- Connexions réseau
- Espace disque
- Se rafraîchit automatiquement

Solution Exercice bonus 💡

```bash
#!/bin/bash
# Dashboard système interactif

while true; do
    clear

    echo "╔═══════════════════════════════════════╗"
    echo "║      DASHBOARD SYSTÈME - $(date +%H:%M:%S)     ║"
    echo "╚═══════════════════════════════════════╝"
    echo

    # Charge système
    echo "📊 CHARGE SYSTÈME"
    uptime
    echo

    # CPU et Mémoire
    echo "💻 UTILISATION"
    printf "CPU: "
    top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1
    printf "MEM: "
    free | grep Mem | awk '{printf "%.1f%%\n", $3/$2 * 100.0}'
    echo

    # Top 5 processus
    echo "🔥 TOP 5 PROCESSUS (CPU)"
    ps aux --sort=-%cpu | head -6 | tail -5 | \
        awk '{printf "%-10s %5s%%  %s\n", $1, $3, $11}'
    echo

    # Espace disque
    echo "💾 ESPACE DISQUE"
    df -h / | tail -1 | \
        awk '{printf "Utilisé: %s / %s (%s)\n", $3, $2, $5}'
    echo

    # Connexions réseau
    echo "🌐 CONNEXIONS"
    echo "Actives: $(ss -tan | grep ESTAB | wc -l)"
    echo "En écoute: $(ss -tln | grep LISTEN | wc -l)"
    echo

    # Services
    echo "⚙️  SERVICES CRITIQUES"
    for svc in sshd nginx mysql; do
        if systemctl is-active --quiet $svc 2>/dev/null; then
```

Revenir au sommaire

Points clés à retenir 📌

Scripts :

- Shebang `#!/bin/bash`
- Arguments : `$1` , `$2` , `$@` , `$#`
- Conditions : `if` , `test` , `[ ]`
- Boucles : `for` , `while` , `until`

Cron :

- Format : minute heure jour mois jour_semaine
- `crontab -e` pour éditer
- Redirection importante pour les logs

Logs :

- `journalctl` pour systemd
- Filtres : `-p` , `-u` , `--since`
- `logger` pour écrire dans syslog

Module 9

Sécurité système

Protéger et durcir votre système Linux

Revenir au sommaire

La sécurité : un principe de couches 🧅

Analogie : la sécurité d'une maison 🏠

1. **Portail** (pare-feu)
2. **Porte blindée** (authentification forte)
3. **Alarme** (monitoring)
4. **Coffre-fort** (chiffrement)
5. **Caméras** (logs d'audit)

Principe de défense en profondeur :

- Une couche échoue → les autres protègent
- Jamais se reposer sur une seule protection

Les principes de sécurité

1. Moindre privilège

- Donner uniquement les droits nécessaires
- Ne jamais travailler en root

2. Défense en profondeur

- Plusieurs couches de sécurité

3. Sécurité par la simplicité

- Moins de services = moins de vulnérabilités

4. Mise à jour régulière

- Correctifs de sécurité critiques

5. Audit et surveillance

- Savoir ce qui se passe sur le système

Analogie : la sécurité de votre banque** 🏦

Moindre privilège : L'employé au guichet ne peut pas ouvrir le coffre-fort

Défense en profondeur :

1. Vigiles à l'entrée (pare-feu)
2. Porte blindée (authentification)
3. Caméras (logs)
4. Alarme (fail2ban)
5. Coffre-fort (chiffrement)

Sécurité par simplicité : Moins de portes = moins de points d'entrée

Mise à jour : Changer les codes d'accès régulièrement

Audit : Vérifier les enregistrements vidéo

Sécurité des comptes utilisateurs 👤

Rappels des bonnes pratiques :

- ✅ Ne jamais utiliser root directement
- ✅ Désactiver le login root en SSH
- ✅ Utiliser sudo avec parcimonie
- ✅ Politique de mots de passe forte
- ✅ Supprimer les comptes inutilisés
- ✅ Auditer régulièrement les comptes

Politique de mot de passe : PAM 🔑

PAM : Pluggable Authentication Modules

Configuration : `/etc/pam.d/common-password`

```
# Installer
sudo apt install libpam-pwquality

# Configuration
sudo nano /etc/security/pwquality.conf
```

Configuration pwquality

```
# Longueur minimum
minlen = 12

# Au moins 1 minuscule
lcredit = -1

# Au moins 1 majuscule
ucredit = -1

# Au moins 1 chiffre
dcredit = -1

# Au moins 1 caractère spécial
ocredit = -1

# Nombre de caractères différents minimum
difok = 3

# Interdire les mots du dictionnaire
dictcheck = 1
```

Expiration des mots de passe ⏳

```
# Configurer l'expiration (90 jours)
sudo chage -M 90 alice

# Prévenir 7 jours avant
sudo chage -W 7 alice

# Délai minimum entre changements (1 jour)
sudo chage -m 1 alice

# Voir la config
sudo chage -l alice

# Forcer le changement au prochain login
sudo chage -d 0 alice
```

Configuration par défaut dans /etc/login.defs 📋

```
sudo nano /etc/login.defs
```

```
PASS_MAX_DAYS   90      # Expiration
PASS_MIN_DAYS   1       # Délai minimum
PASS_MIN_LEN    12      # Longueur minimum
PASS_WARN_AGE   7       # Avertissement
```

⚠️ **S'applique uniquement aux nouveaux utilisateurs**

Verrouillage automatique après échecs 🔒

Fichier : `/etc/pam.d/common-auth`

```
auth required pam_tally2.so deny=5 unlock_time=900
```

Paramètres :

- `deny=5` : bloquer après 5 échecs
- `unlock_time=900` : débloquer après 15 minutes (900 secondes)

Gérer les verrouillages 🗝️

```
# Voir les tentatives d'un utilisateur
sudo pam_tally2 -u alice

# Réinitialiser le compteur
sudo pam_tally2 -u alice --reset

# Voir tous les utilisateurs
sudo pam_tally2
```

Sudo : configuration avancée 🎛️

Rappel : toujours utiliser `visudo` !

```
sudo visudo
```

Log des commandes sudo :

```
Defaults logfile="/var/log/sudo.log"
Defaults log_year, log_host, loglinelen=0
```

Timeout du mot de passe (en minutes) :

```
Defaults timestamp_timeout=5
```

Toujours demander le mot de passe :

```
Defaults timestamp_timeout=0
```

Sudo : restreindre les commandes 🔧

```
# Alice peut seulement redémarrer nginx
alice ALL=(ALL) /usr/bin/systemctl restart nginx

# Bob peut gérer les paquets
bob ALL=(ALL) /usr/bin/apt, /usr/bin/dnf

# Groupe ops peut tout
%ops ALL=(ALL) ALL

# Interdire certaines commandes
charlie ALL=(ALL) ALL, !/bin/su, !/bin/bash
```

Sudo : environnement sécurisé 🌍

Nettoyer les variables d'environnement :

```
Defaults env_reset
Defaults env_keep += "LANG LC_* HOME"
Defaults secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
```

Empêcher l'utilisation de LD_PRELOAD :

```
Defaults !env_keep += "LD_PRELOAD"
```

Fail2ban : protection contre brute-force 🚫

Fail2ban : bannit les IP après plusieurs échecs

```
# Installer
sudo apt install fail2ban

# Démarrer
sudo systemctl enable --now fail2ban

# État
sudo systemctl status fail2ban
```

Configuration Fail2ban 📝

Fichier : `/etc/fail2ban/jail.local`

```
[DEFAULT]
bantime = 3600          # Bannir 1 heure
findtime = 600          # Fenêtre de 10 minutes
maxretry = 5            # 5 échecs max
destemail = admin@example.com
sendername = Fail2ban
action = %(action_mwl)s   # Ban + email avec logs

[sshd]
enabled = true
port = ssh
logpath = /var/log/auth.log
maxretry = 3
```

Gérer Fail2ban 🎮

```
# Voir l'état des jails
sudo fail2ban-client status

# État d'une jail spécifique
sudo fail2ban-client status sshd

# Bannir manuellement une IP
sudo fail2ban-client set sshd banip 203.0.113.51

# Débannir une IP
sudo fail2ban-client set sshd unbanip 203.0.113.51

# Recharger la config
sudo fail2ban-client reload
```

Créer une jail personnalisée 🔧

Protection du serveur web :

```
[nginx-403]
enabled = true
port = http,https
filter = nginx-403
logpath = /var/log/nginx/error.log
maxretry = 10
findtime = 600
bantime = 7200
```

Fichier de filtre : `/etc/fail2ban/filter.d/nginx-403.conf`

```
[Definition]
failregex = ^<HOST>.*"(GET|POST|HEAD).*" 403
ignoreregex =
```

SELinux : Security-Enhanced Linux

SELinux : système de contrôle d'accès obligatoire (MAC)

Analogie : badges d'accès dans une entreprise

- **Permissions Unix** : "vous êtes autorisé"
- **SELinux** : "vous êtes autorisé ET vous avez le bon badge"

Par défaut sur : RHEL, CentOS, Fedora, AlmaLinux

3 modes :

- **Enforcing** : applique les règles
- **Permissive** : log mais n'applique pas
- **Disabled** : désactivé

Vérifier l'état de SELinux

```
# État actuel
getenforce

# Informations détaillées
sestatus

# Changer temporairement (jusqu'au reboot)
sudo setenforce 0    # Permissive
sudo setenforce 1    # Enforcing
```

Configurer SELinux

Fichier : `/etc/selinux/config`

```
SELINUX=enforcing
SELINUXTYPE=targeted
```

Modes :

- `enforcing` : actif
- `permissive` : logs uniquement
- `disabled` : désactivé

⚠️ **Changer de disabled à enforcing nécessite un relabel complet !**

```
# Forcer le relabel au prochain boot
sudo touch /.autorelabel
sudo reboot
```

SELinux : contextes

Chaque fichier/processus a un contexte SELinux

```
# Voir le contexte des fichiers
ls -Z /var/www/html

# Contexte d'un processus
ps auxZ | grep nginx

# Contexte de l'utilisateur
id -Z
```

Format : `user:role:type:level`

Exemple : `system_u:object_r:httpd_sys_content_t:s0`

Changer les contextes SELinux 🔧

```
# Changer temporairement
sudo chcon -t httpd_sys_content_t /var/www/html/index.html

# Changer récursivement
sudo chcon -R -t httpd_sys_content_t /var/www/html

# Restaurer le contexte par défaut
sudo restorecon /var/www/html/index.html
sudo restorecon -R /var/www/html

# Définir le contexte par défaut (persistant)
sudo semanage fcontext -a -t httpd_sys_content_t "/srv/web(/.*)?"
sudo restorecon -R /srv/web
```

Booléens SELinux

Booléens : options on/off pour des fonctionnalités

```
# Lister tous les booléens
getsebool -a

# Voir un booléen spécifique
getsebool httpd_can_network_connect

# Activer temporairement
sudo setsebool httpd_can_network_connect on

# Activer de façon permanente
sudo setsebool -P httpd_can_network_connect on
```

Exemples courants :

- `httpd_can_network_connect` : Apache peut se connecter au réseau
- `httpd_read_user_content` : Apache peut lire les fichiers utilisateurs
- `ftpd_full_access` : FTP a accès complet

Déboguer SELinux

```
# Installer les outils
sudo dnf install setroubleshoot-server

# Voir les alertes
sudo sealert -a /var/log/audit/audit.log

# Analyser un problème spécifique
sudo sealert -l <ID-de-alerte>

# Les logs SELinux
sudo tail -f /var/log/audit/audit.log | grep denied
```

⚠️ **En cas de problème, passer temporairement en permissive pour tester**

AppArmor : alternative à SELinux 🍎

AppArmor : plus simple que SELinux

Par défaut sur : Ubuntu, Debian, SUSE

```
# Vérifier si AppArmor est actif
sudo aa-status

# Profils actifs
sudo aa-enabled
```

Modes de profils AppArmor 🎭

Enforce : applique les restrictions **Complain** : log mais n'applique pas

```
# Mettre un profil en mode complain
sudo aa-complain /etc/apparmor.d/usr.sbin.nginx

# Remettre en mode enforce
sudo aa-enforce /etc/apparmor.d/usr.sbin.nginx

# Désactiver un profil
sudo ln -s /etc/apparmor.d/usr.sbin.nginx /etc/apparmor.d/disable/
sudo apparmor_parser -R /etc/apparmor.d/usr.sbin.nginx

# Réactiver
sudo rm /etc/apparmor.d/disable/usr.sbin.nginx
sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.nginx
```

Créer un profil AppArmor 🔧

```
# Générer un profil de base
sudo aa-genprof /usr/bin/mon-programme

# Améliorer un profil existant
sudo aa-logprof

# Recharger tous les profils
sudo systemctl reload apparmor
```

Exemple de profil AppArmor

Fichier : `/etc/apparmor.d/usr.bin.mon-app`

```
#include <tunables/global>

/usr/bin/mon-app {
  #include <abstractions/base>

  # Lecture de la config
  /etc/mon-app/** r,

  # Écriture des logs
  /var/log/mon-app/** rw,

  # Accès réseau
  network inet stream,

  # Exécuter des binaires
  /usr/bin/curl rix,
}
```

Revenir au sommaire

Durcissement SSH 🔐

Fichier : `/etc/ssh/sshd_config`

```
# Désactiver root login
PermitRootLogin no

# Désactiver l'authentification par mot de passe
PasswordAuthentication no

# Seulement clés publiques
PubkeyAuthentication yes

# Désactiver les mots de passe vides
PermitEmptyPasswords no

# Changer le port (obscurité, pas sécurité)
Port 2222
```

Durcissement SSH (suite) 🔐

```
# Limiter les utilisateurs autorisés
AllowUsers alice bob

# Ou par groupe
AllowGroups sshusers

# Timeout de connexion
ClientAliveInterval 300
ClientAliveCountMax 2

# Nombre de tentatives de connexion
MaxAuthTries 3

# Désactiver X11 forwarding (si non utilisé)
X11Forwarding no

# Désactiver le forwarding de port (si non utilisé)
AllowTcpForwarding no
```

Durcissement SSH : authentification à deux facteurs 🔑🔑

Google Authenticator :

```
# Installer
sudo apt install libpam-google-authenticator

# Configurer pour l'utilisateur
google-authenticator

# Modifier /etc/pam.d/sshd
auth required pam_google_authenticator.so
```

Fichier `/etc/ssh/sshd_config` :

```
ChallengeResponseAuthentication yes
AuthenticationMethods publickey,keyboard-interactive
```

```
# Redémarrer SSH
sudo systemctl restart sshd
```

Audit du système : auditd

auditd : système d'audit du noyau

```
# Installer
sudo apt install auditd

# Démarrer
sudo systemctl enable --now auditd

# État
sudo auditctl -s
```

Règles d'audit 🔍

```
# Auditer l'accès à un fichier
sudo auditctl -w /etc/passwd -p wa -k passwd_changes

# Auditer les modifications de permissions
sudo auditctl -a always,exit -F arch=b64 -S chmod,fchmod,fchmodat -k perm_mod

# Auditer les suppressions de fichiers
sudo auditctl -a always,exit -F arch=b64 -S unlink,unlinkat,rename,renameat -k delete

# Lister les règles
sudo auditctl -l
```

Options :

- `-w` : watch (surveiller un fichier)
- `-p` : permissions (r=read, w=write, x=execute, a=attribute)
- `-k` : key (identifiant pour recherche)

Rechercher dans les logs d'audit 🔎

```
# Tous les événements
sudo ausearch -k passwd_changes

# Événements d'un utilisateur
sudo ausearch -ua alice

# Événements récents
sudo ausearch -ts today

# Événements entre deux dates
sudo ausearch -ts 10:00 -te 12:00

# Générer un rapport
sudo aureport

# Rapport des modifications de fichiers
sudo aureport -f
```

Rendre les règles d'audit permanentes 💾

Fichier : `/etc/audit/rules.d/audit.rules`

```
# Surveiller /etc
-w /etc/ -p wa -k etc_changes

# Surveiller sudo
-w /var/log/sudo.log -p wa -k sudo_log

# Surveiller les connexions
-w /var/log/lastlog -p wa -k logins

# Recharger
sudo augenrules --load
```

Détection d'intrusion : AIDE 🚨

AIDE : Advanced Intrusion Detection Environment

```
# Installer
sudo apt install aide

# Initialiser la base de données
sudo aideinit

# Copier la base de données
sudo mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db

# Vérifier l'intégrité
sudo aide --check

# Mettre à jour la base après changements légitimes
sudo aide --update
```

Configuration AIDE

Fichier : `/etc/aide/aide.conf`

```
# Surveiller /bin
/bin    p+i+n+u+g+s+b+m+c+md5+sha256

# Surveiller /etc
/etc    p+i+n+u+g+s+b+m+c+md5+sha256

# Ignorer certains fichiers
!/var/log
!/tmp
```

Revenir au sommaire

Chiffrement des disques 💿🔒

LUKS : Linux Unified Key Setup

```
# Chiffrer une partition
sudo cryptsetup luksFormat /dev/sdb1

# Ouvrir une partition chiffrée
sudo cryptsetup luksOpen /dev/sdb1 encrypted_disk

# Le périphérique apparaît dans /dev/mapper/
ls -l /dev/mapper/encrypted_disk

# Créer un système de fichiers
sudo mkfs.ext4 /dev/mapper/encrypted_disk

# Monter
sudo mount /dev/mapper/encrypted_disk /mnt/secure

# Démonter et fermer
sudo umount /mnt/secure
sudo cryptsetup luksClose encrypted_disk
```

Montage automatique de partition chiffrée 🔐

Fichier : `/etc/crypttab`

```
encrypted_disk /dev/sdb1 none luks
```

Fichier : `/etc/fstab`

```
/dev/mapper/encrypted_disk /mnt/secure ext4 defaults 0 2
```

Au boot, le système demandera la passphrase

Chiffrer le home directory 🏠🔒

ecryptfs : chiffrement du home

```
# Installer
sudo apt install ecryptfs-utils

# Migrer un utilisateur existant
sudo ecryptfs-migrate-home -u alice

# L'utilisateur doit se reconnecter pour chiffrer
```

Lors de la première connexion après migration :

```
ecryptfs-unwrap-passphrase
```

⚠️ Noter la passphrase de récupération !

Revenir au sommaire

Limites de ressources : ulimit et cgroups 🚧

Prévenir les attaques par déni de service

```
# Limite de processus pour un utilisateur
echo "alice hard nproc 100" | sudo tee -a /etc/security/limits.conf

# Limite de fichiers ouverts
echo "alice hard nofile 1024" | sudo tee -a /etc/security/limits.conf

# Limite de mémoire
echo "alice hard as 1000000" | sudo tee -a /etc/security/limits.conf
```

Désactiver les services inutiles 🔌

Principe : moins de services = moins de surface d'attaque

```
# Lister les services actifs
systemctl list-units --type=service --state=running

# Désactiver un service inutile
sudo systemctl disable --now cups    # Impression
sudo systemctl disable --now bluetooth
sudo systemctl disable --now avahi-daemon    # mDNS

# Vérifier les ports en écoute
sudo ss -tulnp
```

Modules du noyau : désactiver les inutiles 🧠

```
# Lister les modules chargés
lsmod

# Désactiver des modules (USB, FireWire si non utilisés)
echo "install usb-storage /bin/true" | sudo tee -a /etc/modprobe.d/disable-usb-storage.conf

# Désactiver IPv6 si non utilisé
echo "net.ipv6.conf.all.disable_ipv6 = 1" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

Kernel hardening : sysctl

Fichier : `/etc/sysctl.conf`

```
# Protection contre IP spoofing
net.ipv4.conf.all.rp_filter = 1

# Ignorer les pings (optionnel)
net.ipv4.icmp_echo_ignore_all = 1

# Pas de redirection ICMP
net.ipv4.conf.all.accept_redirects = 0

# Pas de source routing
net.ipv4.conf.all.accept_source_route = 0

# SYN cookies (protection SYN flood)
net.ipv4.tcp_syncookies = 1

# Logs des paquets suspects
net.ipv4.conf.all.log_martians = 1
```

Kernel hardening (suite)

```
# Randomisation de l'espace d'adressage (ASLR)
kernel.randomize_va_space = 2

# Empêcher l'accès aux dmesg
kernel.dmesg_restrict = 1

# Interdire les core dumps
kernel.core_uses_pid = 1
fs.suid_dumpable = 0

# Appliquer les changements
sudo sysctl -p
```

Vérifier les ports ouverts 🔍

```
# Ports en écoute
sudo ss -tulnp

# Avec netstat
sudo netstat -tulnp

# Scanner les ports externes (depuis une autre machine)
nmap -sS target-ip

# Vérifier les connexions établies
sudo ss -tn
```

Détection de rootkits : rkhunter 🕵️

```
# Installer
sudo apt install rkhunter

# Mettre à jour la base de données
sudo rkhunter --update

# Scanner le système
sudo rkhunter --check

# Scanner silencieusement
sudo rkhunter --check --sk --nocolors
```

Détection de rootkits : chkrootkit 🕵️

```
# Installer
sudo apt install chkrootkit

# Scanner
sudo chkrootkit

# Vérifier seulement certains tests
sudo chkrootkit -l
sudo chkrootkit sniffer
```

Antivirus : ClamAV

```
# Installer
sudo apt install clamav clamav-daemon

# Mettre à jour les signatures
sudo freshclam

# Scanner un répertoire
sudo clamscan -r /home

# Scanner avec suppression des infections
sudo clamscan -r --remove /home

# Scanner en arrière-plan
sudo systemctl enable --now clamav-daemon
```

Sauvegarde et récupération 💾

3-2-1 Rule :

- **3** copies de vos données
- **2** supports différents
- **1** copie hors site

```
# Sauvegarde avec rsync
rsync -avz --delete /home /backup/

# Sauvegarde distante
rsync -avz --delete /home user@backup-server:/backup/

# Sauvegarde chiffrée avec tar
tar -czf - /home | openssl enc -aes-256-cbc -e > backup.tar.gz.enc
```

Bonnes pratiques de sécurité 📋

1. **Principe du moindre privilège**
   - Uniquement les droits nécessaires
2. **Mises à jour régulières**
   - Automatiser les correctifs de sécurité
3. **Authentification forte**
   - Clés SSH, 2FA
4. **Surveillance et logs**
   - Monitoring actif, alertes
5. **Sauvegardes régulières**
   - Testées et hors site

Bonnes pratiques de sécurité (suite) 📋

6. **Chiffrement**
    - Disques, communications (SSH, HTTPS)
7. **Pare-feu**
    - Bloquer tout sauf nécessaire
8. **Désactiver l'inutile**
    - Services, modules kernel
9. **Audit régulier**
    - Comptes, permissions, ports ouverts
10. **Documentation**
    - Procédures de sécurité, incidents

Checklist de durcissement système ✅

Utilisateurs et authentification :

- ☐Root login désactivé
- ☐Sudo configuré correctement
- ☐Politique de mot de passe forte
- ☐Comptes inutilisés supprimés
- ☐SSH par clés uniquement

Réseau :

- ☐Pare-feu activé et configuré
- ☐Services inutiles désactivés
- ☐Fail2ban installé
- ☐SSH durci

Checklist de durcissement système (suite) ✅

Système :

- ☐Mises à jour automatiques de sécurité
- ☐SELinux/AppArmor activé
- ☐Audit activé (auditd)
- ☐Modules kernel inutiles désactivés
- ☐sysctl durci

Surveillance :

- ☐Logs centralisés
- ☐Monitoring en place
- ☐Alertes configurées
- ☐Détection d’intrusion (AIDE, rkhunter)

Ressources et certifications 📚

Certifications :

- **CompTIA Security+** : bases de sécurité
- **CEH** : Certified Ethical Hacker
- **OSCP** : Offensive Security Certified Professional
- **GIAC** : diverses spécialisations

Ressources :

- CIS Benchmarks (guides de durcissement)
- NIST Cybersecurity Framework
- OWASP (sécurité web)
- CVE/NVD (vulnérabilités)

Récapitulatif du Module 9 ✅

Ce que vous avez appris :

- ✅ Sécurisation des comptes (PAM, sudo, fail2ban)
- ✅ SELinux et AppArmor (MAC)
- ✅ Durcissement SSH et 2FA
- ✅ Audit système (auditd, AIDE)
- ✅ Chiffrement (LUKS, ecryptfs)
- ✅ Pare-feu et durcissement réseau
- ✅ Détection d'intrusion (rkhunter, chkrootkit)
- ✅ Kernel hardening (sysctl)
- ✅ Bonnes pratiques et checklist

Prochaine étape 🎯

Module 10 : Services serveurs

- Serveur web (Nginx)
- Base de données (MySQL)
- Serveur DNS
- Serveur de base de données
- Monitoring et supervision
- Haute disponibilité

Questions ? 🤔

N’hésitez pas à poser vos questions maintenant !

Le prochain module explorera la mise en place et la gestion de services serveurs courants.

QCM Module 9 : Sécurité système ✅

10 questions sur la sécurisation du système

Revenir au sommaire

Question 1

Qu’est-ce que SELinux ?

A) Un gestionnaire de paquets

B) Un système de contrôle d’accès obligatoire (MAC)

C) Un pare-feu

D) Un éditeur de texte

Question 2

Quel outil bloque les IP après plusieurs échecs de connexion ?

A) `iptables`

B) `fail2ban`

C) `ufw`

D) `firewalld`

Question 3

Où sont stockés les mots de passe chiffrés ?

A) `/etc/passwd`

B) `/etc/shadow`

C) `/etc/security`

D) `/etc/passwords`

Question 4

Quelle commande désactive temporairement SELinux ?

A) `selinux disable`

B) `setenforce 0`

C) `systemctl stop selinux`

D) `disable selinux`

Question 5

Quel fichier configure sudo ?

A) `/etc/sudo`

B) `/etc/sudoers`

C) `/etc/sudo.conf`

D) `/etc/security/sudo`

Question 6

Qu’est-ce que fail2ban surveille ?

A) Les performances système

B) Les logs pour détecter des tentatives d’intrusion

C) L’espace disque

D) Les processus

Revenir au sommaire

Question 7

Quelle commande affiche les tentatives de connexion échouées ?

A) `lastlog`

B) `last`

C) `lastb`

D) `faillog`

Revenir au sommaire

Question 8

Quelle est la meilleure pratique pour SSH ?

A) Utiliser des mots de passe forts

B) Utiliser des clés SSH

C) Changer le port

D) Tout ce qui précède

Revenir au sommaire

Question 9

Que fait `auditd` ?

A) Optimise les performances

B) Audite les actions système

C) Nettoie le système

D) Sauvegarde les fichiers

Revenir au sommaire

Question 10

Comment chiffrer une partition avec Linux ?

A) `encrypt`

B) `LUKS`

C) `cryptfs`

D) `secure`

Revenir au sommaire

Réponses du Module 9

Réponse 1 : B) Un système de contrôle d'accès obligatoire (MAC)

- SELinux ajoute une couche de sécurité au-delà des permissions Unix

Réponse 2 : B) `fail2ban`

- Protection contre le brute-force

Réponse 3 : B) `/etc/shadow`

- Accessible uniquement par root

Réponse 4 : B) `setenforce 0`

- 0 = permissive, 1 = enforcing

Réponse 5 : B) `/etc/sudoers`

- À éditer UNIQUEMENT avec `visudo`

Revenir au sommaire

Réponses du Module 9 (suite)

Réponse 6 : B) Les logs pour détecter des tentatives d'intrusion

- Bannit les IPs suspectes temporairement

Réponse 7 : C) `lastb`

- `last` affiche les connexions réussies

Réponse 8 : D) Tout ce qui précède

- Defense en profondeur : plusieurs couches de sécurité

Réponse 9 : B) Audite les actions système

- Enregistre qui fait quoi et quand

Réponse 10 : B) `LUKS`

- LUKS = Linux Unified Key Setup

Score du Module 9 📊

Comptez vos bonnes réponses :

- **9-10** : Excellent ! ⭐⭐⭐
- **7-8** : Bien ! ⭐⭐
- **5-6** : À revoir ⭐
- **< 5** : Relire le module 8

Revenir au sommaire

🎯 Exercices pratiques - Module 9

Sécurité système

Revenir au sommaire

Exercice 1 : Durcissement SSH 🔒

Objectif : Sécuriser SSH

Consignes :

1. Désactivez l'authentification root
2. Changez le port SSH (2222)
3. Désactivez l'authentification par mot de passe
4. Limitez les utilisateurs autorisés
5. Activez fail2ban pour SSH
6. Testez la configuration

Solution Exercice 1 💡

```
# 1-4. Configuration SSH
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup

sudo tee -a /etc/ssh/sshd_config << 'EOF'

# Configuration sécurisée
Port 2222
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers alice bob
MaxAuthTries 3
ClientAliveInterval 300
ClientAliveCountMax 2
EOF

# Tester la config
sudo sshd -t

# Redémarrer
sudo systemctl restart sshd

# 5. Installer fail2ban
sudo apt install fail2ban -y

# Configurer
sudo tee /etc/fail2ban/jail.local << 'EOF'
[sshd]
enabled = true
port = 2222
logpath = /var/log/auth.log
maxretry = 3
bantime = 3600
findtime = 600
EOF

sudo systemctl restart fail2ban

# 6. Tester
# Depuis une autre machine
ssh -p 2222 alice@serveur

# ier fail2ban
s il2ban-client status sshd
```

Revenir au sommaire

Exercice 2 : Audit des permissions 🔍

Objectif : Trouver les fichiers à risque

Consignes :

1. Trouvez tous les fichiers world-writable
2. Trouvez tous les fichiers SUID
3. Trouvez tous les fichiers SGID
4. Trouvez les fichiers sans propriétaire
5. Générez un rapport
6. Corrigez les problèmes critiques

Solution Exercice 2 💡

```bash
#!/bin/bash
# Audit de sécurité des permissions

REPORT="/tmp/security_audit_$(date +%Y%m%d).txt"

echo "===== AUDIT SÉCURITÉ =====" > $REPORT
echo "Date: $(date)" >> $REPORT
echo >> $REPORT

# 1. Fichiers world-writable
echo "🔓 FICHIERS WORLD-WRITABLE:" >> $REPORT
find / -type f -perm -002 ! -path "/proc/*" ! -path "/sys/*" \
    2>/dev/null | tee -a $REPORT
echo >> $REPORT

# 2. Fichiers SUID
echo "⚠️  FICHIERS SUID:" >> $REPORT
find / -type f -perm -4000 2>/dev/null | tee -a $REPORT
echo >> $REPORT

# 3. Fichiers SGID
echo "⚠️  FICHIERS SGID:" >> $REPORT
find / -type f -perm -2000 2>/dev/null | tee -a $REPORT
echo >> $REPORT

# 4. Fichiers sans propriétaire
echo "❓ FICHIERS ORPHELINS:" >> $REPORT
find / -nouser -o -nogroup 2>/dev/null | tee -a $REPORT
echo >> $REPORT

# Vérifier /etc/passwd
echo "👤 COMPTES SYSTÈME:" >> $REPORT
awk -F: '$3 == 0 {print $1}' /etc/passwd | tee -a $REPORT
echo >> $REPORT

cat $REPORT

# 6. Corrections
echo "Corrections recommandées..."

# Retirer world-writable (exemple)
# find / -type f -perm -002 -exec chmod o-w {} \; 2>/dev/null
```

Revenir au sommaire

Exercice 3 : Configuration de fail2ban

Objectif : Protéger contre le brute-force

Consignes :

1. Configurez fail2ban pour SSH, HTTP et FTP
2. Définissez des règles strictes
3. Testez en générant des échecs
4. Vérifiez les IPs bannies
5. Débannissez une IP
6. Consultez les logs

Solution Exercice 3 💡

```
# 1 & 2. Configuration
sudo tee /etc/fail2ban/jail.local << 'EOF'
[DEFAULT]
bantime = 3600
findtime = 600
maxretry = 3
destemail = admin@example.com
action = %(action_mwl)s

[sshd]
enabled = true
port = ssh
logpath = /var/log/auth.log
maxretry = 3

[nginx-http-auth]
enabled = true
port = http,https
logpath = /var/log/nginx/error.log

[nginx-noscript]
enabled = true
port = http,https
logpath = /var/log/nginx/access.log
maxretry = 6

[nginx-badbots]
enabled = true
port = http,https
logpath = /var/log/nginx/access.log
maxretry = 2
EOF

sudo systemctl restart fail2ban

# 4. Voir les IPs bannies
sudo fail2ban-client status
sudo fail2ban-client status sshd

# 5. Débannir une IP
sudo fail2ban-client set sshd unbanip 192.168.1.100

# 6. Logs
sudo tail -f /var/log/fail2ban.log

# Tester (depuis une autre machine)
```

Revenir au sommaire

Exercice 4 : Chiffrement avec LUKS 🔐

Objectif : Chiffrer un volume

Consignes :

1. Créez une image disque de 100 Mo
2. Chiffrez-la avec LUKS
3. Ouvrez le volume chiffré
4. Formatez et montez
5. Créez des fichiers sensibles
6. Fermez et testez la sécurité

Solution Exercice 4 💡

```
# 1. Créer image
dd if=/dev/zero of=/tmp/encrypted.img bs=1M count=100

# 2. Chiffrer avec LUKS
sudo cryptsetup luksFormat /tmp/encrypted.img
# Entrez un mot de passe fort

# 3. Ouvrir le volume
sudo cryptsetup luksOpen /tmp/encrypted.img encrypted_vol

# 4. Formater
sudo mkfs.ext4 /dev/mapper/encrypted_vol

# Monter
sudo mkdir /mnt/encrypted
sudo mount /dev/mapper/encrypted_vol /mnt/encrypted

# 5. Créer des fichiers
echo "Données confidentielles" | sudo tee /mnt/encrypted/secret.txt
sudo ls -la /mnt/encrypted/

# 6. Fermer
sudo umount /mnt/encrypted
sudo cryptsetup luksClose encrypted_vol

# Tester : impossible de lire sans le mot de passe !
# sudo mount /tmp/encrypted.img /mnt/test  # Échec

# Réouvrir
sudo cryptsetup luksOpen /tmp/encrypted.img encrypted_vol
# Entrez le mot de passe
sudo mount /dev/mapper/encrypted_vol /mnt/encrypted
cat /mnt/encrypted/secret.txt

# Nettoyage
sudo umount /mnt/encrypted
sudo cryptsetup luksClose encrypted_vol
rm /tmp/encrypted.img
```

Exercice 5 : Audit avec auditd

Objectif : Surveiller les actions système

Consignes :

1. Installez auditd
2. Surveillez les modifications de `/etc/passwd`
3. Surveillez les commandes sudo
4. Générez des événements
5. Consultez les rapports
6. Recherchez des événements spécifiques

Solution Exercice 5 💡

```
# 1. Installer
sudo apt install auditd audispd-plugins -y

# 2. Règle pour /etc/passwd
sudo auditctl -w /etc/passwd -p wa -k passwd_changes

# 3. Règle pour sudo
sudo auditctl -w /var/log/sudo.log -p wa -k sudo_log

# Règles pour exécutions
sudo auditctl -a always,exit -F arch=b64 -S execve -k exec_commands

# Voir les règles
sudo auditctl -l

# Rendre permanent
sudo tee -a /etc/audit/rules.d/custom.rules << 'EOF'
-w /etc/passwd -p wa -k passwd_changes
-w /etc/shadow -p wa -k shadow_changes
-w /etc/sudoers -p wa -k sudoers_changes
-w /var/log/sudo.log -p wa -k sudo_log
EOF

# 4. Générer des événements
sudo useradd testuser
sudo userdel testuser
sudo cat /etc/shadow > /dev/null

# 5. Consulter les rapports
sudo ausearch -k passwd_changes
sudo ausearch -k sudo_log

# Par date
sudo ausearch -ts today

# 6. Recherche spécifique
sudo ausearch -ua alice
sudo ausearch -x /usr/bin/sudo

# Rapport détaillé
sudo aureport --summary
sudo aureport --auth
sudo aureport --file
```

Revenir au sommaire

Exercice bonus : Script de hardening automatique

Objectif : Script de sécurisation

Créez un script qui :

- Désactive les services inutiles
- Configure le pare-feu
- Sécurise SSH
- Met en place fail2ban
- Configure les mises à jour auto
- Génère un rapport

Solution Exercice bonus 💡

```
#!/bin/bash
# Script de hardening système

if [ "$EUID" -ne 0 ]; then
    echo "Doit être exécuté en root"
    exit 1
fi

echo "===== HARDENING SYSTÈME ====="

# 1. Désactiver services inutiles
echo "Désactivation services inutiles..."
for svc in bluetooth cups avahi-daemon; do
    systemctl disable $svc 2>/dev/null
    systemctl stop $svc 2>/dev/null
done

# 2. Pare-feu
echo "Configuration pare-feu..."
ufw --force enable
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp

# 3. SSH sécurisé
echo "Sécurisation SSH..."
cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup
sed -i 's/#PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config
sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
systemctl restart sshd

# 4. Fail2ban
echo "Installation fail2ban..."
apt install fail2ban -y
systemctl enable fail2ban
systemctl start fail2ban

# 5. Mises à jour auto
echo "Configuration mises à jour automatiques..."
apt install unattended-upgrades -y
d  configure -plow unattended-upgrades

#  rmissions /tmp
echo "Sécurisation /tmp..."
```

Revenir au sommaire

Points clés à retenir 📌

SSH :

- Désactiver root login
- Clés au lieu de mots de passe
- Changer le port par défaut
- fail2ban pour protection

Audits :

- Fichiers SUID/SGID à surveiller
- World-writable dangereux
- auditd pour traçabilité

Chiffrement :

- LUKS pour volumes
- GPG pour fichiers
- SSH pour communications

Revenir au sommaire

Module 10

Services serveurs

Installer et gérer des services courants

Revenir au sommaire

Qu'est-ce qu'un serveur ? 🖥️

Serveur : ordinateur qui fournit des services à d'autres (clients)

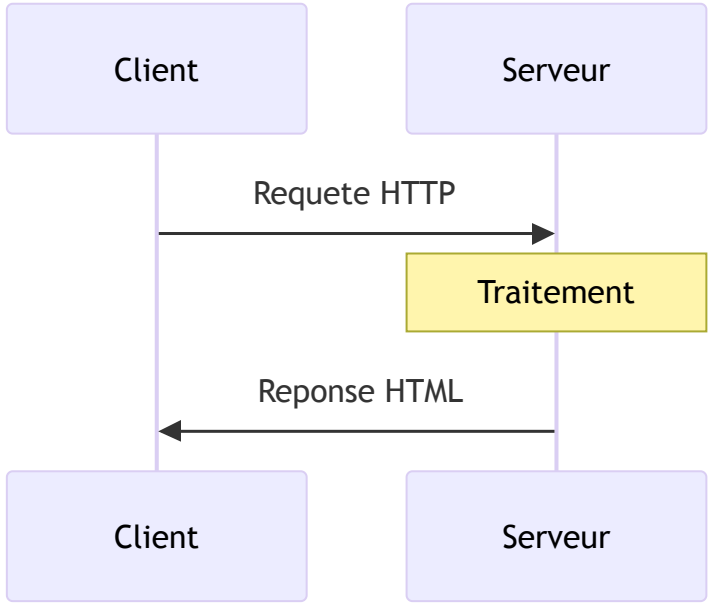
Analogie : un restaurant 🍽️

- **Serveur web** : serveur qui apporte les plats (pages web)
- **Clients** : les convives (navigateurs)
- **Cuisine** : l'arrière-plan (traitement)
- **Menu** : l'API
- **Réservations** : gestion des sessions

Linux : système de prédilection pour les serveurs

- Stable, performant, gratuit
- Grande communauté
- Outils puissants

Architecture client-serveur 🔄



Exemples :

- **Web** : navigateur ↔ Nginx/Apache
- **Email** : client mail ↔ serveur SMTP
- **Base de données** : app ↔ MySQL/PostgreSQL
- **DNS** : système ↔ serveur DNS

Serveur web : Nginx 🚀

Nginx : serveur web moderne et performant

Avantages :

- Très performant (async)
- Faible consommation mémoire
- Reverse proxy efficace
- Load balancing
- Configuration simple

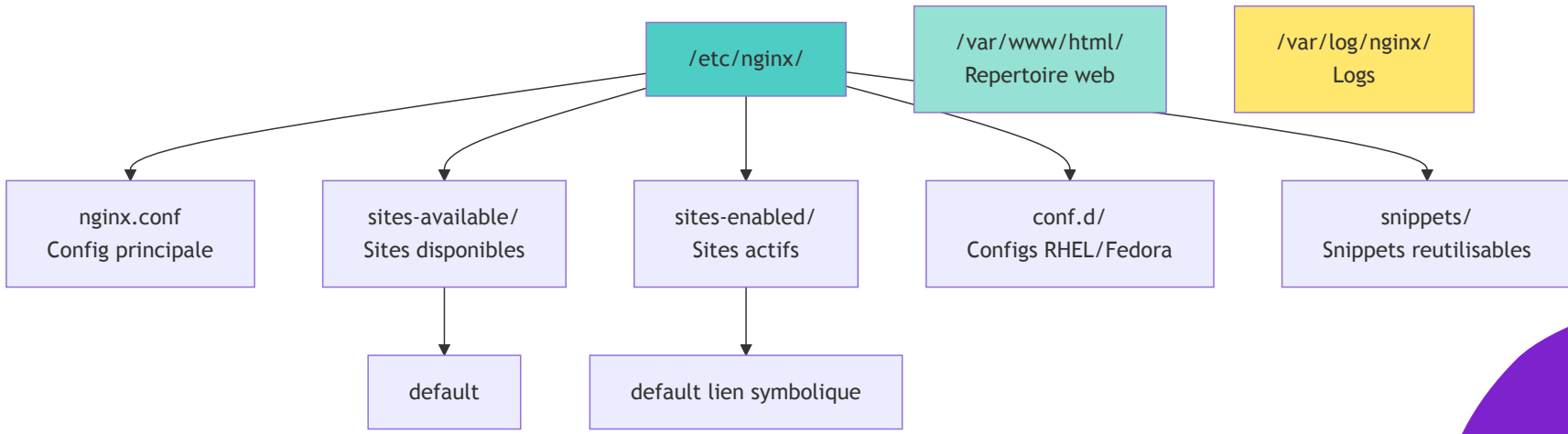
Cas d'usage :

- Servir des sites statiques
- Reverse proxy pour apps (Node, Python, etc.)
- Load balancer
- Cache

Installer Nginx 📥

```
# Debian/Ubuntu
sudo apt update
sudo apt install nginx

# RHEL/Fedora
sudo dnf install nginx

# Démarrer
sudo systemctl start nginx
sudo systemctl enable nginx

# Vérifier
sudo systemctl status nginx
curl http://localhost
```

Structure de Nginx 📁



Configuration Nginx de base

Fichier : `/etc/nginx/sites-available/default`

```
server {
    listen 80;
    listen [::]:80;

    server_name example.com www.example.com;

    root /var/www/html;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ =404;
    }

    access_log /var/log/nginx/access.log;
    error_log /var/log/nginx/error.log;
}
```

Créer un virtual host

1. Créer la configuration

```
sudo nano /etc/nginx/sites-available/mon-site
```

```
server {
    listen 80;
    server_name mon-site.local;

    root /var/www/mon-site;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Créer un virtual host (suite)

2. Créer le répertoire et le contenu

```
sudo mkdir -p /var/www/mon-site
echo "<h1>Mon Site</h1>" | sudo tee /var/www/mon-site/index.html
sudo chown -R www-data:www-data /var/www/mon-site
```

Créer un virtual host (suite)

3. Activer le site

```
# Créer le lien symbolique
sudo ln -s /etc/nginx/sites-available/mon-site /etc/nginx/sites-enabled/

# Tester la configuration
sudo nginx -t

# Recharger Nginx
sudo systemctl reload nginx
```

Créer un virtual host (suite)

4. Ajouter au /etc/hosts (pour tests locaux)

```
sudo nano /etc/hosts
```

```
127.0.0.1   mon-site.local
```

Tester :

```
curl http://mon-site.local
```

Nginx comme reverse proxy 🔄

Reverse proxy : Nginx devant une application

```
server {
    listen 80;
    server_name api.example.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}
```

HTTPS avec Let's Encrypt 🔒

Certbot : obtenir des certificats SSL gratuits

```
# Installer Certbot
sudo apt install certbot python3-certbot-nginx

# Obtenir et installer le certificat
sudo certbot --nginx -d example.com -d www.example.com

# Renouvellement automatique
sudo systemctl status certbot.timer

# Tester le renouvellement
sudo certbot renew --dry-run
```

Configuration HTTPS générée 🔐

```
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name example.com;

    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    # ... reste de la config
}

server {
    listen 80;
    server_name example.com;
    return 301 https://$server_name$request_uri;
}
```

Nginx : commandes utiles 🔧

```
# Tester la configuration
sudo nginx -t

# Recharger (sans downtime)
sudo systemctl reload nginx

# Redémarrer
sudo systemctl restart nginx

# Arrêter
sudo systemctl stop nginx

# Voir les logs en temps réel
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
```

Apache : alternative à Nginx

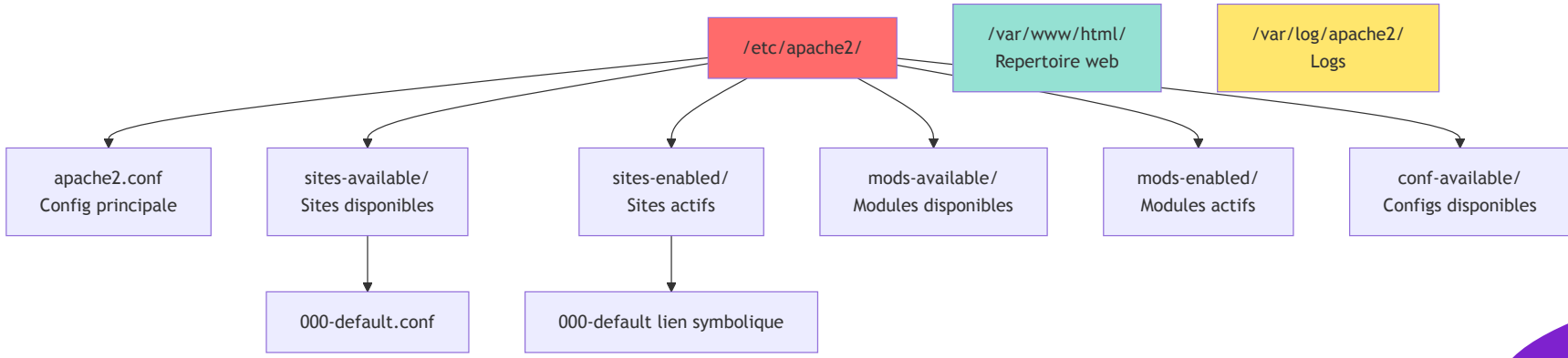
Apache : serveur web historique

Avantages :

- Très mature et stable
- Modules nombreux
- .htaccess (config par répertoire)
- Documentation extensive

```
# Installer
sudo apt install apache2

# Démarrer
sudo systemctl start apache2
sudo systemctl enable apache2
```

Structure Apache 📁



Virtual host Apache

Fichier : `/etc/apache2/sites-available/mon-site.conf`

```
<VirtualHost *:80>
    ServerName mon-site.local
    ServerAdmin admin@mon-site.local
    DocumentRoot /var/www/mon-site

    <Directory /var/www/mon-site>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/mon-site_error.log
    CustomLog ${APACHE_LOG_DIR}/mon-site_access.log combined
</VirtualHost>
```

Activer un site Apache

```
# Activer le site
sudo a2ensite mon-site.conf

# Désactiver le site par défaut
sudo a2dissite 000-default.conf

# Recharger
sudo systemctl reload apache2

# Commandes utiles
sudo a2enmod rewrite    # Activer module rewrite
sudo a2dismod status    # Désactiver module
```

Revenir au sommaire

PHP avec Apache ou Nginx 🐘

Installer PHP :

```
# Debian/Ubuntu
sudo apt install php-fpm php-mysql php-cli php-curl php-gd

# RHEL/Fedora
sudo dnf install php-fpm php-mysqlnd php-cli php-curl php-gd
```

PHP avec Nginx

```
server {
    listen 80;
    server_name example.com;
    root /var/www/html;
    index index.php index.html;

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.1-fpm.sock;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

```
# Redémarrer PHP-FPM
sudo systemctl restart php8.1-fpm
```

PHP avec Apache

```
# Installer le module PHP pour Apache
sudo apt install libapache2-mod-php

# Redémarrer Apache
sudo systemctl restart apache2

# Tester
echo "<?php phpinfo(); ?>" | sudo tee /var/www/html/info.php
curl http://localhost/info.php
```

Base de données : MySQL/MariaDB 🗄️

MariaDB : fork open source de MySQL

```
# Installer MariaDB
sudo apt install mariadb-server

# Démarrer
sudo systemctl start mariadb
sudo systemctl enable mariadb

# Sécuriser l'installation
sudo mysql_secure_installation
```

mysql_secure_installation 🔒

Questions posées :

1. Définir un mot de passe root
2. Supprimer les utilisateurs anonymes
3. Interdire le login root à distance
4. Supprimer la base de test
5. Recharger les tables de privilèges

Répondre "Y" à tout pour une installation sécurisée

Se connecter à MySQL 🔗

```
# En tant que root
sudo mysql

# Avec mot de passe
mysql -u root -p

# Commandes MySQL
mysql> SHOW DATABASES;
mysql> USE mysql;
mysql> SHOW TABLES;
mysql> SELECT user, host FROM user;
mysql> EXIT;
```

Créer une base et un utilisateur

```sql
-- Créer une base de données
CREATE DATABASE ma_base;

-- Créer un utilisateur
CREATE USER 'alice'@'localhost' IDENTIFIED BY 'mot_de_passe_fort';

-- Donner tous les privilèges sur la base
GRANT ALL PRIVILEGES ON ma_base.* TO 'alice'@'localhost';

-- Appliquer les changements
FLUSH PRIVILEGES;

-- Vérifier
SHOW GRANTS FOR 'alice'@'localhost';
```

Sauvegarder et restaurer MySQL 💾

```
# Sauvegarder une base
mysqldump -u root -p ma_base > ma_base_backup.sql

# Sauvegarder toutes les bases
mysqldump -u root -p --all-databases > all_databases.sql

# Restaurer une base
mysql -u root -p ma_base < ma_base_backup.sql

# Restaurer tout
mysql -u root -p < all_databases.sql
```

PostgreSQL : alternative à MySQL 🐘

```
# Installer PostgreSQL
sudo apt install postgresql postgresql-contrib

# Démarrer
sudo systemctl start postgresql
sudo systemctl enable postgresql

# Se connecter
sudo -u postgres psql

# Commandes PostgreSQL
postgres=# \l                  -- Lister les bases
postgres=# \c ma_base          -- Se connecter à une base
postgres=# \dt                 -- Lister les tables
postgres=# \du                 -- Lister les utilisateurs
postgres=# \q                  -- Quitter
```

Serveur DNS : BIND

BIND : Berkeley Internet Name Domain

```
# Installer
sudo apt install bind9 bind9utils bind9-doc

# Démarrer
sudo systemctl start named
sudo systemctl enable named

# Configuration
/etc/bind/
├── named.conf              # Configuration principale
├── named.conf.options      # Options
├── named.conf.local        # Zones locales
└── named.conf.default-zones
```

Configuration DNS simple

Fichier : `/etc/bind/named.conf.local`

```
zone "exemple.local" {
    type master;
    file "/etc/bind/zones/db.exemple.local";
};

zone "1.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/zones/db.192.168.1";
};
```

Fichier de zone DNS 🗺️

Fichier : `/etc/bind/zones/db.exemple.local`

```
$TTL    604800
@       IN      SOA     ns1.exemple.local. admin.exemple.local. (
                              3         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.exemple.local.
@       IN      A       192.168.1.10
ns1     IN      A       192.168.1.10
www     IN      A       192.168.1.20
mail    IN      A       192.168.1.30
```

Tester le DNS 🧪

```
# Vérifier la configuration
sudo named-checkconf
sudo named-checkzone exemple.local /etc/bind/zones/db.exemple.local

# Redémarrer BIND
sudo systemctl restart named

# Tester avec dig
dig @localhost exemple.local
dig @localhost www.exemple.local

# Tester avec nslookup
nslookup www.exemple.local localhost
```

Serveur de cache DNS : dnsmasq 🚀

dnsmasq : DNS + DHCP léger

```
# Installer
sudo apt install dnsmasq

# Configuration simple
sudo nano /etc/dnsmasq.conf
```

```
# Écouter sur localhost
listen-address=127.0.0.1

# Upstream DNS
server=8.8.8.8
server=1.1.1.1

# Taille du cache
cache-size=1000

# Résolution locale
address=/mon-serveur.local/192.168.1.10
```

Serveur FTP : vsftpd 📂

vsftpd : Very Secure FTP Daemon

```
# Installer
sudo apt install vsftpd

# Sauvegarder la config originale
sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.backup

# Éditer
sudo nano /etc/vsftpd.conf
```

Configuration vsftpd

```
# Désactiver les connexions anonymes
anonymous_enable=NO

# Autoriser les utilisateurs locaux
local_enable=YES

# Autoriser l'écriture
write_enable=YES

# Chrooter les utilisateurs dans leur home
chroot_local_user=YES

# Ports passifs
pasv_min_port=40000
pasv_max_port=40100

# Logs
xferlog_enable=YES
xferlog_file=/var/log/vsftpd.log
```

```
# Redémarrer
sudo systemctl restart vsftpd
```

Serveur de mail : Postfix

Postfix : MTA (Mail Transfer Agent) moderne

```
# Installer
sudo apt install postfix

# Pendant l'installation, choisir "Internet Site"
# Entrer le nom de domaine

# Configuration
sudo nano /etc/postfix/main.cf
```

Configuration Postfix de base

```
myhostname = mail.exemple.com
mydomain = exemple.com
myorigin = $mydomain
inet_interfaces = all
mydestination = $myhostname, localhost.$mydomain, $mydomain
mynetworks = 127.0.0.0/8, 192.168.1.0/24
home_mailbox = Maildir/
```

```
# Redémarrer
sudo systemctl restart postfix

# Tester
echo "Test" | mail -s "Sujet" user@exemple.com
```

Revenir au sommaire

Monitoring : Netdata

Netdata : monitoring en temps réel

```
# Installation automatique
bash <(curl -Ss https://my-netdata.io/kickstart.sh)

# Accéder à l'interface
# http://localhost:19999
```

Fonctionnalités :

- CPU, RAM, disque, réseau
- Temps réel (1s d'actualisation)
- Pas de base de données
- Alertes

Monitoring : Prometheus + Grafana 📈

Prometheus : collecte de métriques

```
# Installer Prometheus
sudo apt install prometheus

# Démarrer
sudo systemctl start prometheus
sudo systemctl enable prometheus

# Interface : http://localhost:9090
```

Grafana : visualisation

```
# Ajouter le dépôt
sudo apt install -y software-properties-common
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
echo "deb https://packages.grafana.com/oss/deb stable main" | \
  sudo tee /etc/apt/sources.list.d/grafana.list

# Installer
sudo apt update
sudo apt install grafana

# Démarrer
sudo systemctl start grafana-server
sudo systemctl enable grafana-server

# Interface : http://localhost:3000
# Login : admin / admin
```

Logs centralisés : ELK Stack

ELK : Elasticsearch + Logstash + Kibana

Elasticsearch : stockage et recherche **Logstash** : collecte et transformation **Kibana** : visualisation

Alternative moderne : Loki + Promtail + Grafana

Haute disponibilité : concepts 🔄

HA : High Availability (disponibilité élevée)

Objectif : minimiser le downtime

Techniques :

- **Load balancing** : répartir la charge
- **Failover** : basculement automatique
- **Réplication** : copies multiples
- **Monitoring** : détection de panne

SLA : Service Level Agreement

- 99.9% = 8h46min de downtime/an
- 99.99% = 52min/an
- 99.999% = 5min/an

Load balancing avec Nginx ⚖️

```
upstream backend {
    server backend1.example.com:8080;
    server backend2.example.com:8080;
    server backend3.example.com:8080;
}

server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Méthodes de load balancing 🎯

```
# Round-robin (par défaut)
upstream backend {
    server backend1.example.com;
    server backend2.example.com;
}

# Least connections
upstream backend {
    least_conn;
    server backend1.example.com;
    server backend2.example.com;
}

# IP hash (sticky sessions)
upstream backend {
    ip_hash;
    server backend1.example.com;
    server backend2.example.com;
}
```

Health checks 🏥

```
upstream backend {
    server backend1.example.com max_fails=3 fail_timeout=30s;
    server backend2.example.com max_fails=3 fail_timeout=30s;
    server backend3.example.com backup;  # Serveur de secours
}
```

Options :

- `max_fails` : nombre d'échecs avant marquage "down"
- `fail_timeout` : durée avant nouvelle tentative
- `backup` : utilisé seulement si les autres sont down

Keepalived : IP virtuelle 🔄

Keepalived : basculement d'IP entre serveurs

```
# Installer
sudo apt install keepalived

# Configuration
sudo nano /etc/keepalived/keepalived.conf
```

```
vrrp_instance VI_1 {
    state MASTER
    interface eth0
    virtual_router_id 51
    priority 100
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass secret
    }
    virtual_ipaddress {
        192.168.1.100
    }
}
```

Réplication MySQL 🔄

Master-Slave Replication

Sur le Master :

```
CREATE USER 'repl'@'%' IDENTIFIED BY 'password';
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%';

SHOW MASTER STATUS;
-- Noter File et Position
```

Configuration : `/etc/mysql/my.cnf`

```
[mysqld]
server-id = 1
log_bin = /var/log/mysql/mysql-bin.log
```

Réplication MySQL (suite) 🔄

Sur le Slave :

Configuration : `/etc/mysql/my.cnf`

```
[mysqld]
server-id = 2
```

```
CHANGE MASTER TO
    MASTER_HOST='master_ip',
    MASTER_USER='repl',
    MASTER_PASSWORD='password',
    MASTER_LOG_FILE='mysql-bin.000001',
    MASTER_LOG_POS=154;

START SLAVE;
SHOW SLAVE STATUS\G
```

Containers : Docker 🐳

Docker : conteneurisation d'applications

```
# Installer Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Ajouter l'utilisateur au groupe docker
sudo usermod -aG docker $USER

# Tester
docker run hello-world
```

Docker : commandes de base 🐳

```
# Lancer un conteneur
docker run -d -p 80:80 nginx

# Lister les conteneurs
docker ps

# Arrêter un conteneur
docker stop <container_id>

# Voir les logs
docker logs <container_id>

# Entrer dans un conteneur
docker exec -it <container_id> bash

# Supprimer un conteneur
docker rm <container_id>
```

Docker Compose : orchestration 🎼

```
# Installer Docker Compose
sudo apt install docker-compose
```

Fichier : `docker-compose.yml`

```
version: '3'
services:
  web:
    image: nginx
    ports:
      - "80:80"
    volumes:
      - ./html:/usr/share/nginx/html
  db:
    image: mariadb
    environment:
      MYSQL_ROOT_PASSWORD: example
    volumes:
      - db_data:/var/lib/mysql
volumes:
  db_data:
```

Docker Compose : utilisation 🎮

```
# Démarrer les services
docker-compose up -d

# Voir l'état
docker-compose ps

# Logs
docker-compose logs -f

# Arrêter
docker-compose down

# Reconstruire
docker-compose up -d --build
```

Bonnes pratiques serveurs 📋

1. **Sécurité**
   - Pare-feu activé
   - Services exposés minimaux
   - Mises à jour automatiques
2. **Monitoring**
   - Surveiller CPU, RAM, disque
   - Alertes configurées
   - Logs centralisés
3. **Sauvegardes**
   - Automatiques et régulières
   - Testées
   - Hors site

Bonnes pratiques serveurs (suite) 📋

4. **Documentation**
   - Procédures déploiement
   - Configurations
   - Contacts
5. **Haute disponibilité**
   - Load balancing
   - Réplication
   - Failover
6. **Performance**
   - Cache (Redis, Memcached)
   - CDN pour le contenu statique
   - Optimisation des requêtes DB

Récapitulatif du Module 10 ✅

Ce que vous avez appris :

- ✅ Serveurs web (Nginx, Apache)
- ✅ Virtual hosts et HTTPS
- ✅ Bases de données (MySQL, PostgreSQL)
- ✅ Serveur DNS (BIND, dnsmasq)
- ✅ Serveurs mail, FTP
- ✅ Monitoring (Netdata, Prometheus, Grafana)
- ✅ Haute disponibilité et load balancing
- ✅ Docker et containerisation
- ✅ Bonnes pratiques de gestion de serveurs

Félicitations ! 🎉

Vous avez complété les 9 modules de la formation Unix/Linux - Initiation !

Compétences acquises :

- ✅ Maîtrise des fondamentaux Unix/Linux
- ✅ Gestion des utilisateurs et permissions
- ✅ Administration système (processus, services, stockage)
- ✅ Configuration réseau
- ✅ Automatisation et scripting
- ✅ Sécurité système
- ✅ Gestion de services serveurs

Prochaine étape 🎯

Module 11 : Supervision & Analyse Système

- Monitoring avec top, htop, iotop
- Analyse des logs système
- Métriques et tableaux de bord
- Alertes et notifications

Pour aller plus loin 🚀

Certifications :

- **LPIC-1** : Linux Professional Institute Certification
- **RHCSA** : Red Hat Certified System Administrator
- **CompTIA Linux+**

Ressources :

- Linux Documentation Project
- ArchWiki (excellente doc)
- man pages et info
- Reddit : r/linux, r/linuxadmin

Pratique :

- Installer une VM ou un Raspberry Pi
- Contribuer à des projets open source
- Monter son propre serveur

Questions ? 🤔

N’hésitez pas à poser vos questions maintenant !

Prochaines étapes : exercices pratiques et QCM de validation.

QCM Module 10 : Services serveurs ✅

10 questions sur les services et serveurs

Revenir au sommaire

Question 1

Quel serveur web est connu pour ses performances ?

A) Apache

B) Nginx

C) IIS

D) Tomcat

Revenir au sommaire

Question 2

Où sont les fichiers de config de Nginx sur Ubuntu ?

A) `/etc/apache2/`

B) `/etc/nginx/`

C) `/var/www/nginx/`

D) `/usr/local/nginx/`

Revenir au sommaire

MINISTÈRE DES ARMÉES
Liberté
Égalité
Fraternité

Question 3

Comment tester la config de Nginx ?

A) `nginx test`

B) `nginx -t`

C) `systemctl test nginx`

D) `test nginx`

Question 4

Quelle commande sauvegarde une base MySQL ?

A) `mysql-backup`

B) `mysqldump`

C) `mysql-export`

D) `backup-mysql`

Question 5

Quel port utilise MySQL par défaut ?

A) 3306

B) 5432

C) 27017

D) 1433

Question 6

Qu’est-ce qu’un reverse proxy ?

A) Un proxy qui inverse les données

B) Un serveur intermédiaire devant les applications

C) Un type de pare-feu

D) Un outil de sauvegarde

Revenir au sommaire

Question 7

Comment obtenir un certificat SSL gratuit ?

A) Let’s Encrypt (Certbot)

B) Acheter chez un CA

C) Générer avec OpenSSL

D) A et C sont corrects

Question 8

Quel est le protocole pour transférer des fichiers de façon sécurisée ?

A) FTP

B) HTTP

C) SCP/SFTP

D) Telnet

Question 9

Qu’est-ce que Docker ?

A) Un gestionnaire de paquets

B) Une plateforme de conteneurisation

C) Un éditeur de texte

D) Un serveur web

Revenir au sommaire

Question 10

Comment démarrer tous les services d'un docker-compose ?

A) `docker start`

B) `docker-compose up`

C) `compose start`

D) `docker run`

Réponses du Module 10

Réponse 1 : B) Nginx

- Architecture asynchrone, faible consommation mémoire

Réponse 2 : B) `/etc/nginx/`

- `/etc/nginx/nginx.conf` est le fichier principal

Réponse 3 : B) `nginx -t`

- Toujours tester avant de recharger !

Réponse 4 : B) `mysqldump`

- `mysqldump -u root -p database > backup.sql`

Réponse 5 : A) 3306

- 5432 = PostgreSQL, 27017 = MongoDB

Revenir au sommaire

Réponses du Module 10 (suite)

Réponse 6 : B) Un serveur intermédiaire devant les applications

- Nginx est souvent utilisé comme reverse proxy

Réponse 7 : A) Let’s Encrypt (Certbot)

- Gratuit, automatisé, renouvelé automatiquement

Réponse 8 : C) SCP/SFTP

- Basés sur SSH, donc chiffrés

Réponse 9 : B) Une plateforme de conteneurisation

- Permet d’isoler les applications dans des conteneurs

Réponse 10 : B) `docker-compose up`

- `-d` pour lancer en arrière-plan

Score du Module 10 📊

Comptez vos bonnes réponses :

- **9-10** : Excellent ! ⭐⭐⭐
- **7-8** : Bien ! ⭐⭐
- **5-6** : À revoir ⭐
- **< 5** : Relire le module 9

Revenir au sommaire

🎯 Exercices pratiques - Module 10

Services serveurs

Exercice 1 : Serveur web Nginx

Objectif : Installer et configurer Nginx

Consignes :

1. Installez Nginx
2. Créez 2 sites : `site1.local` et `site2.local`
3. Configurez 2 virtual hosts
4. Ajoutez du contenu HTML
5. Testez les deux sites
6. Consultez les logs d'accès

Solution Exercice 1 💡

```
# 1. Installer
sudo apt update
sudo apt install nginx -y

# 2 & 3. Site 1
sudo mkdir -p /var/www/site1
sudo tee /var/www/site1/index.html << 'EOF'
<!DOCTYPE html>
<html>
<head><title>Site 1</title></head>
<body style="background: #4CAF50; color: white;">
  <h1>Bienvenue sur Site 1 🎯</h1>
</body>
</html>
EOF

sudo tee /etc/nginx/sites-available/site1 << 'EOF'
server {
    listen 80;
    server_name site1.local;
    root /var/www/site1;
    index index.html;
}
EOF

# Site 2
sudo mkdir -p /var/www/site2
sudo tee /var/www/site2/index.html << 'EOF'
<!DOCTYPE html>
<html>
<head><title>Site 2</title></head>
<body style="background: #2196F3; color: white;">
  <h1>Bienvenue sur Site 2 🚀</h1>
</body>
</html>
EOF

sudo tee /etc/nginx/sites-available/site2 << 'EOF'
server {
    listen 80;
    server_name site2.local;
    root /var/www/site2;
    index index.html;
}
EOF
```

Revenir au sommaire

Exercice 2 : Reverse Proxy 🔄

Objectif : Configurer Nginx comme reverse proxy

Consignes :

1. Lancez 2 serveurs Python sur ports 8001 et 8002
2. Configurez Nginx pour proxy les requêtes
3. Testez le load balancing
4. Ajoutez des headers personnalisés
5. Configurez le cache

Solution Exercice 2 💡

```
# 1. Serveurs Python
cat > /tmp/server1.py << 'EOF'
from http.server import HTTPServer, BaseHTTPRequestHandler

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'Response from Server 1')

HTTPServer(('', 8001), Handler).serve_forever()
EOF

cat > /tmp/server2.py << 'EOF'
from http.server import HTTPServer, BaseHTTPRequestHandler

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'Response from Server 2')

HTTPServer(('', 8002), Handler).serve_forever()
EOF

# Lancer en arrière-plan
python3 /tmp/server1.py &
python3 /tmp/server2.py &

# 2 & 3. Configuration Nginx
sudo tee /etc/nginx/sites-available/proxy << 'EOF'
upstream backend {
    server localhost:8001;
    server localhost:8002;
}

server {
    listen 80;
    server_name proxy.local;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

Revenir au sommaire

Exercice 3 : Base de données MySQL 🗄️

Objectif : Installer et sécuriser MySQL

Consignes :

1. Installez MySQL
2. Exécutez la sécurisation initiale
3. Créez une base de données `webapp`
4. Créez un utilisateur avec droits limités
5. Testez la connexion
6. Créez une sauvegarde

Solution Exercice 3 💡

```
# 1. Installer
sudo apt install mysql-server -y

# 2. Sécurisation
sudo mysql_secure_installation
# - Mot de passe root
# - Supprimer utilisateurs anonymes : Oui
# - Interdire connexion root à distance : Oui
# - Supprimer base test : Oui
# - Recharger privilèges : Oui

# 3 & 4. Créer base et utilisateur
sudo mysql << 'EOF'
CREATE DATABASE webapp;
CREATE USER 'webapp_user'@'localhost' IDENTIFIED BY 'SecurePass123!';
GRANT SELECT, INSERT, UPDATE, DELETE ON webapp.* TO 'webapp_user'@'localhost';
FLUSH PRIVILEGES;

-- Créer une table test
USE webapp;
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50),
    email VARCHAR(100)
);

INSERT INTO users (username, email) VALUES
('alice', 'alice@example.com'),
('bob', 'bob@example.com');
EOF

# 5. Tester
mysql -u webapp_user -p'SecurePass123!' -e "SELECT * FROM webapp.users;"

# 6. Sauvegarde
sudo mysqldump -u root webapp > /tmp/webapp_backup.sql

# Restaurer (si besoin)
# mysql -u root webapp < /tmp/webapp_backup.sql

# Script de sauvegarde automatique
cat > /usr/local/bin/mysql_backup.sh << 'EOF'
#  bash
B  DIR="/backup/mysql"
D  date +%Y%m%d_%H%M%S)
```

Revenir au sommaire

Exercice 4 : SSL avec Let's Encrypt 🔒

Objectif : Sécuriser un site avec HTTPS

Consignes :

1. Installez Certbot
2. Obtenez un certificat (domaine requis ou test local)
3. Configurez Nginx pour HTTPS
4. Forcez la redirection HTTP → HTTPS
5. Testez le certificat
6. Configurez le renouvellement automatique

Solution Exercice 4 💡

```
# 1. Installer Certbot
sudo apt install certbot python3-certbot-nginx -y

# 2. Obtenir certificat (avec domaine réel)
sudo certbot --nginx -d example.com -d www.example.com

# Pour test local (certificat auto-signé)
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout /etc/ssl/private/nginx-selfsigned.key \
    -out /etc/ssl/certs/nginx-selfsigned.crt \
    -subj "/C=FR/ST=France/L=Paris/O=Test/CN=site1.local"

# 3 & 4. Config HTTPS
sudo tee /etc/nginx/sites-available/site1-ssl << 'EOF'
server {
    listen 80;
    server_name site1.local;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name site1.local;

    ssl_certificate /etc/ssl/certs/nginx-selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/nginx-selfsigned.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    root /var/www/site1;
    index index.html;
}
EOF

sudo ln -sf /etc/nginx/sites-available/site1-ssl /etc/nginx/sites-enabled/site1
sudo nginx -t
sudo systemctl reload nginx

# 5. Tester
curl -k https://site1.local

# [illegible]nouvellement automatique (Let's Encrypt)
s[illegible]rtbot renew --dry-run

# Ajouter au cron
```

Revenir au sommaire

Exercice 5 : Monitoring avec Prometheus & Node Exporter 📊

Objectif : Mettre en place du monitoring

Consignes :

1. Installez Node Exporter
2. Configurez-le comme service
3. Testez les métriques
4. Créez un script de monitoring custom
5. Exportez des métriques personnalisées

Solution Exercice 5 💡

```
# 1. Télécharger Node Exporter
cd /tmp
wget https://github.com/prometheus/node_exporter/releases/download/v1.7.0/node_exporter-1.7.0.linux-amd64.tar.gz
tar xvfz node_exporter-1.7.0.linux-amd64.tar.gz
sudo mv node_exporter-1.7.0.linux-amd64/node_exporter /usr/local/bin/
sudo useradd -rs /bin/false node_exporter

# 2. Service systemd
sudo tee /etc/systemd/system/node_exporter.service << 'EOF'
[Unit]
Description=Node Exporter
After=network.target

[Service]
User=node_exporter
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl start node_exporter
sudo systemctl enable node_exporter

# 3. Tester
curl http://localhost:9100/metrics

# Métriques intéressantes
curl http://localhost:9100/metrics | grep node_cpu
curl http://localhost:9100/metrics | grep node_memory

# 4 & 5. Métriques custom
cat > /var/lib/node_exporter/textfile_collector/custom.prom << 'EOF'
# HELP custom_disk_usage Custom disk usage metric
# TYPE custom_disk_usage gauge
custom_disk_usage{path="/"} 75.5

# HELP custom_service_status Custom service status
# TYPE custom_service_status gauge
custom_service_status{service="nginx"} 1
custom_service_status{service="mysql"} 1
E
# t de mise à jour des métriques
cat > /usr/local/bin/update_custom_metrics.sh << 'EOF'
```

Revenir au sommaire

Exercice bonus : Stack complète Docker 🐳

Objectif : Déployer une stack avec Docker Compose

Créez un `docker-compose.yml` **avec :**

- Nginx
- PHP-FPM
- MySQL
- Redis
- PHPMyAdmin

Solution Exercice bonus 💡

```yaml
# docker-compose.yml
version: '3.8'

services:
  nginx:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./html:/var/www/html
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - php

  php:
    image: php:8.2-fpm
    volumes:
      - ./html:/var/www/html
    environment:
      - MYSQL_HOST=mysql
      - REDIS_HOST=redis

  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: webapp
      MYSQL_USER: user
      MYSQL_PASSWORD: userpass
    volumes:
      - mysql_data:/var/lib/mysql

  redis:
    image: redis:latest

  phpmyadmin:
    image: phpmyadmin:latest
    ports:
      - "8080:80"
    environment:
      PMA_HOST: mysql
      PMA_USER: root
      MA_PASSWORD: rootpass

v
  mysql_data:
```

Revenir au sommaire

Points clés à retenir 📌

Nginx :

- Virtual hosts pour multi-sites
- Reverse proxy pour applications
- SSL/TLS obligatoire en production

Bases de données :

- Sauvegardes régulières
- Utilisateurs avec droits minimaux
- Sécurisation post-installation

Monitoring :

- Métriques essentielles
- Alertes automatiques
- Logs centralisés

Module 4

Supervision & Analyse Système

Surveiller et analyser votre serveur Linux

Revenir au sommaire

Pourquoi superviser un serveur ? 🤔

Supervision : surveiller l'état et les performances d'un système

Analogie : le tableau de bord d'une voiture 🚗

- **Vitesse** = charge CPU
- **Température moteur** = température système
- **Jauge essence** = espace disque
- **Voyants d'alerte** = logs d'erreurs

Sans supervision :

- Pannes inattendues
- Pas d'anticipation des problèmes
- Difficile à diagnostiquer
- Downtime prolongé

Les 4 ressources clés à surveiller 🎯

1. CPU : charge processeur

- Trop élevé = système lent
- Objectif : < 70% en moyenne

2. RAM : mémoire vive

- Saturation = swap = lenteur
- Objectif : < 80%

3. Disque : espace de stockage

- Plein = plantages
- Objectif : < 85%

4. Réseau : bande passante

- Saturation = timeouts
- Objectif : surveiller les pics

top : le basique incontournable

```
top
```

Première ligne : load average

```
load average: 1.23, 0.87, 0.65
```

- 3 valeurs : 1 min, 5 min, 15 min - Si > nombre de CPUs : surchargé

Colonnes importantes :

- `PID` : identifiant processus
- `USER` : propriétaire
- `%CPU` : utilisation CPU
- `%MEM` : utilisation mémoire
- `COMMAND` : nom du programme

top : commandes utiles ⌨️

Raccourcis interactifs :

```
M    : trier par mémoire
P    : trier par CPU (défaut)
k    : tuer un processus (demande PID)
u    : filtrer par utilisateur
1    : afficher tous les CPU séparément
q    : quitter
```

Exemple pratique : trouver qui consomme le plus de RAM

```
top
# Appuyez sur M
# Les processus les plus gourmands en RAM sont en haut
```

Revenir au sommaire

htop : top en mieux 🎨

```
# Installer
sudo apt install htop       # Debian/Ubuntu
sudo dnf install htop       # RHEL/Fedora

# Lancer
htop
```

Avantages sur top :

- Interface colorée et intuitive
- Barres graphiques CPU/RAM
- Navigation avec les flèches
- Arbre des processus (F5)
- Recherche (F3)
- Kill facile (F9)

htop : navigation rapide 🎮

Touches de fonction :

```
F1  : Aide
F2  : Configuration
F3  : Rechercher un processus
F4  : Filtrer
F5  : Vue arbre
F6  : Trier par colonne
F9  : Kill processus
F10 : Quitter
```

Touches directes :

- `Space` : marquer un processus
- `u` : filtrer par utilisateur
- `t` : vue arbre
- `/` : recherche

iotop : surveiller les I/O disque

```
# Installer
sudo apt install iotop

# Lancer (root requis)
sudo iotop
```

Colonnes clés :

- `DISK READ` : lecture disque par seconde
- `DISK WRITE` : écriture disque par seconde
- `COMMAND` : processus responsable

Cas d'usage :

- Système lent : est-ce le disque ?
- Qui écrit énormément sur le disque ?
- Identifier un processus qui fait du I/O intensif

iotop : options utiles 🔧

```
# Mode batch (pour script)
sudo iotop -b -n 3

# Seulement les processus actifs
sudo iotop -o

# Afficher les threads
sudo iotop -P

# Trier par écriture disque
# Appuyez sur 'a' dans iotop
```

Exemple : détecter une base de données qui écrit trop

vmstat : statistiques mémoire 📈

```
# Vue instantanée
vmstat

# Mise à jour toutes les 2 secondes
vmstat 2

# 5 itérations toutes les 2 secondes
vmstat 2 5
```

Colonnes importantes :

- `r` : processus en attente CPU
- `b` : processus bloqués I/O
- `si` : swap in (Ko/s)
- `so` : swap out (Ko/s)
- `bi` : blocs lus disque
- `bo` : blocs écrits disque
- `us` : temps CPU user
- `sy` : temps CPU system
- `wa` : temps CPU attente I/O

Interpréter vmstat 🔍

```
vmstat 2
```

```
r  b   swpd   free   buff  cache   si   so    bi    bo
2  0      0  524288  89600 1048576   0    0    10    50
```

Indicateurs de problème :

- `r > nombre CPUs` : CPU surchargé
- `b > 0` : beaucoup d'attente I/O disque
- `si/so > 0` : système utilise le swap (mauvais !)
- `wa > 20` : le CPU attend le disque

Bon état :

- `r ≈ nombre CPUs`
- `si/so = 0` (pas de swap)
- `wa < 10`

Revenir au sommaire

iostat : I/O par périphérique 💾

```
# Installer
sudo apt install sysstat

# Statistiques I/O
iostat

# Mise à jour toutes les 2 secondes
iostat -x 2
```

Colonnes clés (-x pour détails) :

- `r/s` : lectures par seconde
- `w/s` : écritures par seconde
- `rkB/s` : Ko lus par seconde
- `wkB/s` : Ko écrits par seconde
- `%util` : taux d'utilisation disque

Attention : `%util > 80%` = goulot d'étranglement disque

journalctl : les logs système 📜

Logs systemd : journalctl remplace /var/log/*

```
# Tous les logs
journalctl

# Logs depuis le dernier boot
journalctl -b

# Logs d'un service
journalctl -u nginx

# Suivre en temps réel
journalctl -f

# Suivre un service
journalctl -u nginx -f
```

journalctl : filtres utiles 🔎

```
# Depuis 1 heure
journalctl --since "1 hour ago"

# Depuis aujourd'hui
journalctl --since today

# Période précise
journalctl --since "2025-11-01" --until "2025-11-02"

# Seulement les erreurs
journalctl -p err

# Seulement les erreurs et critiques
journalctl -p crit

# Les 50 dernières lignes
journalctl -n 50
```

journalctl : export et nettoyage 🧹

```
# Export vers fichier
journalctl -u nginx > nginx.log

# Taille du journal
journalctl --disk-usage

# Nettoyer : garder 7 jours
sudo journalctl --vacuum-time=7d

# Nettoyer : garder 500 Mo
sudo journalctl --vacuum-size=500M

# Vérifier l'intégrité
journalctl --verify
```

sar : System Activity Reporter 📊

sar : collecte et rapporte l'activité système

```
# Installer sysstat
sudo apt install sysstat

# Activer la collecte (éditer /etc/default/sysstat)
sudo nano /etc/default/sysstat
# ENABLED="true"

sudo systemctl enable sysstat
sudo systemctl start sysstat
```

Collecte automatique : toutes les 10 minutes par défaut

Revenir au sommaire

sar : consulter les stats 📈

```
# Stats CPU du jour
sar

# Stats mémoire
sar -r

# Stats swap
sar -S

# Stats I/O
sar -b

# Stats réseau
sar -n DEV

# Stats d'hier
sar -f /var/log/sysstat/sa$(date -d yesterday +%d)
```

Exemple : voir l'utilisation CPU à 14h

```
sar -s 14:00:00 -e 15:00:00
```

Logs traditionnels : /var/log/ 📂

Fichiers logs importants :

```
# Logs système généraux
tail -f /var/log/syslog          # Debian/Ubuntu
tail -f /var/log/messages        # RHEL/Fedora

# Authentification
tail -f /var/log/auth.log        # Debian/Ubuntu
tail -f /var/log/secure          # RHEL/Fedora

# Noyau
dmesg
tail -f /var/log/kern.log

# Services spécifiques
tail -f /var/log/nginx/access.log
tail -f /var/log/nginx/error.log
tail -f /var/log/mysql/error.log
```

logrotate : rotation automatique 🔄

logrotate : compresse et archive les vieux logs

Configuration : `/etc/logrotate.conf` et `/etc/logrotate.d/*`

Exemple : /etc/logrotate.d/nginx

```
/var/log/nginx/*.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 www-data adm
    sharedscripts
    postrotate
        systemctl reload nginx > /dev/null
    endscript
}
```

Surveiller l'espace disque 💾

```
# Espace disque global
df -h

# Utilisation par répertoire
du -sh /var/*
du -sh /home/*

# Top 10 des plus gros fichiers dans /var
sudo find /var -type f -exec du -h {} + | sort -rh | head -10

# Surveiller en temps réel
watch -n 5 df -h
```

Alerte : disque > 85% = risque de plantage

Revenir au sommaire

Exemple de script monitoring simple 🤖

Script : /usr/local/bin/monitor.sh

```
#!/bin/bash

# Seuils
CPU_THRESHOLD=80
MEM_THRESHOLD=80

# Récupérer les valeurs
CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)
MEM=$(free | grep Mem | awk '{print ($3/$2) * 100}')

# Alertes
if (( $(echo "$CPU > $CPU_THRESHOLD" | bc -l) )); then
    echo "ALERTE: CPU à ${CPU}%" | mail -s "Alerte CPU" admin@example.com
fi

if (( $(echo "$MEM > $MEM_THRESHOLD" | bc -l) )); then
    echo "ALERTE: RAM à ${MEM}%" | mail -s "Alerte RAM" admin@example.com
fi
```

Automatiser le monitoring avec cron ⏰

```
# Rendre le script exécutable
sudo chmod +x /usr/local/bin/monitor.sh

# Ajouter dans crontab
sudo crontab -e
```

```
# Vérifier toutes les 5 minutes
*/5 * * * * /usr/local/bin/monitor.sh
```

Alternative moderne : systemd timer

Checklist diagnostic serveur lent 🐌

1. CPU

```
top
# Appuyez sur '1' pour voir tous les CPU
# Processus qui utilisent > 50% CPU ?
```

2. RAM

```
free -h
# Swap utilisé ? Si oui, problème mémoire
```

3. Disque

```
df -h          # Espace disque
sudo iotop     # I/O en cours
```

Checklist diagnostic (suite) 🔍

4. Réseau

```
# Connexions actives
ss -tuln | wc -l

# Bande passante (si iftop installé)
sudo iftop
```

5. Logs

```
# Erreurs récentes
journalctl -p err --since "10 minutes ago"
dmesg | tail -20
```

Outils complémentaires utiles 🛠️

nmon : monitoring complet interactif

```
sudo apt install nmon
nmon
# Appuyez sur 'c' pour CPU, 'm' pour mémoire, etc.
```

glances : monitoring moderne

```
sudo apt install glances
glances
```

multitail : suivre plusieurs logs

```
sudo apt install multitail
multitail /var/log/nginx/access.log /var/log/nginx/error.log
```

Bonnes pratiques 📋

1. Surveiller régulièrement

- Consulter htop tous les jours
- Vérifier les logs d'erreurs

2. Définir des seuils

- CPU < 70%
- RAM < 80%
- Disque < 85%
- Load average < nb_cpus

3. Automatiser

- Scripts de monitoring
- Alertes email/SMS
- Collecte historique (sar)

Bonnes pratiques (suite) ✅

4. Garder l'historique

- sar collecte automatique
- Logs rotatés mais conservés 14-30 jours
- Graphiques de tendance

5. Documenter les incidents

- Quand ?
- Symptômes ?
- Cause ?
- Solution ?

6. Anticiper

- Tendance d'utilisation disque
- Planifier l'upgrade avant saturation

Récapitulatif du Module 4 ✅

Ce que vous avez appris :

- ✅ Outils de surveillance temps réel (top, htop, iotop)
- ✅ Statistiques système (vmstat, iostat)
- ✅ Logs système (journalctl, /var/log/*)
- ✅ Collecte historique (sar)
- ✅ Rotation logs (logrotate)
- ✅ Scripts monitoring custom
- ✅ Diagnostic de performance
- ✅ Bonnes pratiques supervision

Prochaine étape 🎯

Module 12 : Supervision avancée avec Zabbix

- Installation et configuration Zabbix
- Surveillance avancée et alertes
- Création hosts et alertes
- Notifications automatiques

Questions ? 🤔

N’hésitez pas à poser vos questions maintenant !

Le prochain module explorera la supervision avancée avec Zabbix.

QCM - Supervision & Analyse Système ✅

Module 4 : Validation des connaissances

Question 1

Que signifie un load average de 4.0 sur une machine avec 2 CPUs ?

A) Le système est sous-utilisé
B) Le système est à 50% de charge
C) Le système est surchargé (2x plus de travail que de CPUs)
D) Le système fonctionne normalement

Réponse : C

Load average > nombre de CPUs = surchargé. Ici 4.0 pour 2 CPUs = 2x trop de travail.

Question 2 🎨

Quel est l'avantage principal de htop par rapport à top ?

A) htop est plus rapide
B) htop utilise moins de ressources
C) htop a une interface colorée et plus intuitive
D) htop fonctionne sur plus de systèmes

Réponse : C

htop offre une interface colorée avec des barres graphiques, navigation au clavier et actions plus faciles.

Question 3

Quelle commande permet de voir quel processus utilise le plus d'I/O disque ?

A) top
B) htop
C) iostat
D) iotop

Réponse : D

iotop affiche l'utilisation I/O disque par processus en temps réel.

Question 4

Comment voir les logs d'erreur du service nginx avec journalctl ?

A) `journalctl nginx`
B) `journalctl -u nginx -p err`
C) `journalctl --error nginx`
D) `journalctl -e nginx`

Réponse : B

`journalctl -u nginx -p err` filtre les logs du service nginx avec priorité erreur.

Question 5 📈

Quel outil collecte automatiquement les statistiques système toutes les 10 minutes ?

A) top
B) sar (sysstat)
C) htop
D) vmstat

Réponse : B

sar (System Activity Reporter) collecte automatiquement les stats système pour analyse historique.

Question 6 🔄

Quel fichier configure la rotation automatique des logs nginx ?

A) /etc/nginx/logrotate.conf
B) /etc/logrotate.d/nginx
C) /var/log/nginx/rotate.conf
D) /etc/log/rotate/nginx

Réponse : B

Les configurations logrotate spécifiques sont dans /etc/logrotate.d/

Question 7 💾

Quel seuil d'utilisation disque est généralement considéré comme critique ?

A) > 50%
B) > 70%
C) > 85%
D) > 95%

Réponse : C

Au-delà de 85% d'utilisation disque, il y a risque de plantage ou ralentissement.

Question 8 🔍

Dans vmstat, que signifie si/so > 0 ?

A) Le CPU est surchargé
B) Le système utilise le swap (problème mémoire)
C) Le disque est lent
D) Le réseau est saturé

Réponse : B

si (swap in) et so (swap out) > 0 indiquent que le système manque de RAM et utilise le swap.

Question 9 🧹

Comment nettoyer les logs journald pour garder seulement 7 jours ?

A) `journalctl --clean 7d`
B) `journalctl --vacuum-time=7d`
C) `journalctl --delete-old 7`
D) `journalctl -r 7d`

Réponse : B

`journalctl --vacuum-time=7d` supprime les logs plus vieux que 7 jours.

Question 10 ⏰

Quelle syntaxe cron pour exécuter un script toutes les 5 minutes ?

A) `5 * * * * /script.sh`
B) `*/5 * * * * /script.sh`
C) `* 5 * * * /script.sh`
D) `0/5 * * * * /script.sh`

Réponse : B

`*/5` signifie "toutes les 5 unités" pour le champ minutes.

Score 🎯

Barème :

- 9-10 : Excellent ! ⭐⭐⭐
- 7-8 : Très bien ! ⭐⭐
- 5-6 : Bien, à consolider ⭐
- < 5 : Réviser le module

Prochaine étape : Exercices pratiques

Fin du QCM ✅

Passez maintenant aux exercices pratiques pour mettre en application ces connaissances !

Exercices - Supervision & Analyse Système 🎯

Module 4 : Mise en pratique

Revenir au sommaire

Exercice 1 : Surveillance en temps réel 📊

Objectif : Utiliser top et htop

Consignes :

1. Lancez `top` et identifiez :
   - Le load average actuel
   - Les 3 processus qui consomment le plus de CPU
   - Les 3 processus qui consomment le plus de RAM
2. Installez et lancez `htop`
   - Triez par utilisation mémoire
   - Affichez l'arbre des processus (F5)
   - Recherchez le processus "sshd" (F3)

Durée : 10 minutes

Exercice 1 : Solution 💡

```
# 1. Top
top
# Observer la ligne "load average: ..."
# Appuyer sur 'P' pour trier par CPU
# Appuyer sur 'M' pour trier par RAM
# Noter les 3 premiers processus

# 2. Htop
sudo apt install htop -y    # si pas installé
htop
# F6 puis sélectionner PERCENT_MEM
# F5 pour vue arbre
# F3 puis taper "sshd"
# F10 pour quitter
```

Exercice 2 : Analyse I/O disque

Objectif : Identifier l'activité disque

Consignes :

1. Installez `iotop`
2. Lancez-le et observez pendant 30 secondes
3. Créez un fichier de 100 Mo pour générer de l'I/O :

```
dd if=/dev/zero of=/tmp/test.img bs=1M count=100
```

4. Observez iotop pendant l'opération
5. Supprimez le fichier test

Question : Quel processus a généré le plus d'écriture disque ?

Durée : 10 minutes

Revenir au sommaire

Exercice 2 : Solution 💡

```
# 1. Installation
sudo apt install iotop -y

# 2. Lancement
sudo iotop
# Observer pendant 30s

# 3. Terminal 2 : générer I/O
dd if=/dev/zero of=/tmp/test.img bs=1M count=100

# 4. Observer iotop : le processus 'dd' apparaît
# avec DISK WRITE élevé

# 5. Nettoyage
rm /tmp/test.img
```

Réponse : Le processus `dd` génère le plus d'écriture

Exercice 3 : Analyse logs avec journalctl

Objectif : Filtrer et analyser les logs

Consignes :

1. Affichez les logs depuis le dernier boot
2. Affichez seulement les erreurs des dernières 24 heures
3. Suivez en temps réel les logs du service SSH
4. Affichez les 20 dernières lignes des logs système
5. Vérifiez la taille totale des logs journald

Durée : 15 minutes

Exercice 3 : Solution 💡

```
# 1. Logs depuis dernier boot
journalctl -b

# 2. Erreurs des 24 dernières heures
journalctl -p err --since "24 hours ago"

# 3. Suivre SSH en temps réel
journalctl -u ssh -f
# ou journalctl -u sshd -f (selon distrib)
# Ctrl+C pour arrêter

# 4. 20 dernières lignes
journalctl -n 20

# 5. Taille des logs
journalctl --disk-usage
```

Exercice 4 : Surveillance avec vmstat et iostat 📈

Objectif : Comprendre les statistiques système

Consignes :

1. Installez sysstat : `sudo apt install sysstat -y`
2. Lancez `vmstat 2 5` (5 itérations de 2 secondes)
3. Notez les valeurs moyennes de :
   - `r` (processus en attente)
   - `si` et `so` (swap)
   - `wa` (attente I/O)
4. Lancez `iostat -x 2 3`
5. Notez le `%util` de votre disque principal

Durée : 10 minutes

Exercice 4 : Solution 💡

```
# 1. Installation
sudo apt install sysstat -y

# 2. vmstat
vmstat 2 5

# Exemple de sortie :
# r  b  swpd   free   si  so  wa
# 1  0     0  524288   0   0   2

# Interprétation :
# r=1 : OK (< nb CPUs)
# si=so=0 : pas de swap (bon)
# wa=2 : peu d'attente I/O (bon)

# 4. iostat
iostat -x 2 3

# Observer %util :
# < 50% = OK
# 50-80% = chargé
# > 80% = goulot d'étranglement
```

Exercice 5 : Script de monitoring ⚙️

Objectif : Créer un script d'alerte simple

Consignes :

Créez un script `/usr/local/bin/check_disk.sh` qui :

1. Vérifie l'espace disque de `/`
2. Affiche une alerte si > 80%
3. Rend le script exécutable
4. Testez-le manuellement

Bonus : Ajoutez-le dans crontab pour exécution toutes les heures

Durée : 20 minutes

Exercice 5 : Solution 💡

```
# 1. Créer le script
sudo nano /usr/local/bin/check_disk.sh
```

```
#!/bin/bash
# Script de surveillance disque

THRESHOLD=80
USAGE=$(df -h / | tail -1 | awk '{print $5}' | sed 's/%//')

if [ $USAGE -gt $THRESHOLD ]; then
    echo "ALERTE: Disque / à ${USAGE}%"
    # echo "ALERTE: Disque à ${USAGE}%" | mail -s "Alerte Disque" admin@example.com
else
    echo "OK: Disque / à ${USAGE}%"
fi
```

Exercice 5 : Solution (suite) 💡

```
# 2. Rendre exécutable
sudo chmod +x /usr/local/bin/check_disk.sh

# 3. Tester
/usr/local/bin/check_disk.sh

# Bonus: Ajouter dans crontab
sudo crontab -e
# Ajouter la ligne :
0 * * * * /usr/local/bin/check_disk.sh >> /var/log/disk_check.log
```

Exercice 6 : Analyse avec sar 📊

Objectif : Activer et utiliser sar

Consignes :

1. Vérifiez que sysstat est installé
2. Activez la collecte :
   - Éditez `/etc/default/sysstat`
   - Mettez `ENABLED="true"`
3. Redémarrez sysstat
4. Affichez les stats CPU actuelles : `sar`
5. Affichez les stats mémoire : `sar -r`

Note : sar n'aura des données historiques qu'après quelques heures

Durée : 10 minutes

Exercice 6 : Solution 💡

```
# 1. Installation (si besoin)
sudo apt install sysstat -y

# 2. Activer collecte
sudo nano /etc/default/sysstat
# Changer ENABLED="false" en ENABLED="true"

# 3. Redémarrer
sudo systemctl enable sysstat
sudo systemctl start sysstat

# 4. Stats CPU
sar
# Si "Cannot open /var/log/sysstat/..." = normal,
# attendre 10 minutes que la collecte commence

# 5. Stats mémoire
sar -r

# 6. Stats réseau
sar -n DEV
```

Exercice 7 : Diagnostic complet 🔍

Objectif : Diagnostiquer un système "lent" (simulation)

Consignes :

Imaginez qu'un utilisateur dit "le serveur est lent".
Effectuez ce diagnostic dans l'ordre :

1. Vérifiez le load average (top ou uptime)
2. Vérifiez l'utilisation CPU (top)
3. Vérifiez la RAM et le swap (free -h)
4. Vérifiez l'espace disque (df -h)
5. Vérifiez l'activité I/O (sudo iotop pendant 10s)
6. Vérifiez les erreurs récentes (journalctl -p err --since "10 min ago")

Documentez vos observations.

Durée : 15 minutes

Exercice 7 : Solution 💡

```
# 1. Load average
uptime
top
# load < nb_cpus = OK

# 2. CPU
top
# Processus > 80% CPU ?

# 3. RAM et swap
free -h
# Swap used > 0 = problème mémoire

# 4. Disque
df -h
# / > 85% = problème espace

# 5. I/O
sudo iotop -o
# Processus avec DISK READ/WRITE élevé ?

# 6. Erreurs
journalctl -p err --since "10 min ago"
# Erreurs système ?
```

Exercice 7 : Grille de diagnostic 📋

Créez votre checklist personnelle :

```
DIAGNOSTIC SYSTÈME LENT
=======================
Date: __________  Heure: __________

☐ Load average: _____ (< nb_cpus ?)
☐ CPU max utilisé par: __________ (___%)
☐ RAM utilisée: _____% (< 80% ?)
☐ Swap utilisé: _____MB (= 0 ?)
☐ Disque / utilisé: _____% (< 85% ?)
☐ I/O disque: Processus __________ (élevé ?)
☐ Erreurs logs: Oui / Non
  └─> Si oui: ____________________

Conclusion: ______________________
```

Projet fil rouge : Monitoring automatisé 🤖

Objectif final : Script de monitoring complet

Fonctionnalités :

- Vérifier CPU, RAM, disque
- Logger dans /var/log/monitoring.log
- Envoyer alerte si seuil dépassé
- Exécuter toutes les 5 minutes

Étapes :

1. Créer le script
2. Tester
3. Ajouter dans crontab
4. Surveiller les logs pendant 1 heure

Voir exercice 5 comme base

Récapitulatif des exercices ✅

Compétences acquises :

- ✅ Utiliser top/htop pour surveillance temps réel
- ✅ Analyser l'I/O avec iotop
- ✅ Filtrer et analyser les logs avec journalctl
- ✅ Interpréter vmstat et iostat
- ✅ Créer des scripts de monitoring
- ✅ Activer et utiliser sar
- ✅ Diagnostiquer un système lent méthodiquement

Bravo ! 🎉

Vous maîtrisez maintenant les bases de la supervision système.

Prochaine étape : Module 5 - Supervision avancée avec Zabbix

Module 5

Supervision Avancée & Alertes

Monitoring professionnel avec Zabbix

Revenir au sommaire

Pourquoi un outil de supervision avancé ? 🤔

Limites des outils en ligne de commande :

- top/htop : données instantanées uniquement
- Pas d'historique graphique
- Pas d'alertes automatiques
- Surveillance manuelle
- Pas de vue d'ensemble multi-serveurs

Solution : outil de monitoring centralisé

- **Zabbix** ← On va utiliser celui-ci !
- Nagios
- Prometheus + Grafana
- Munin

Zabbix : présentation

Zabbix : solution de monitoring open source entreprise

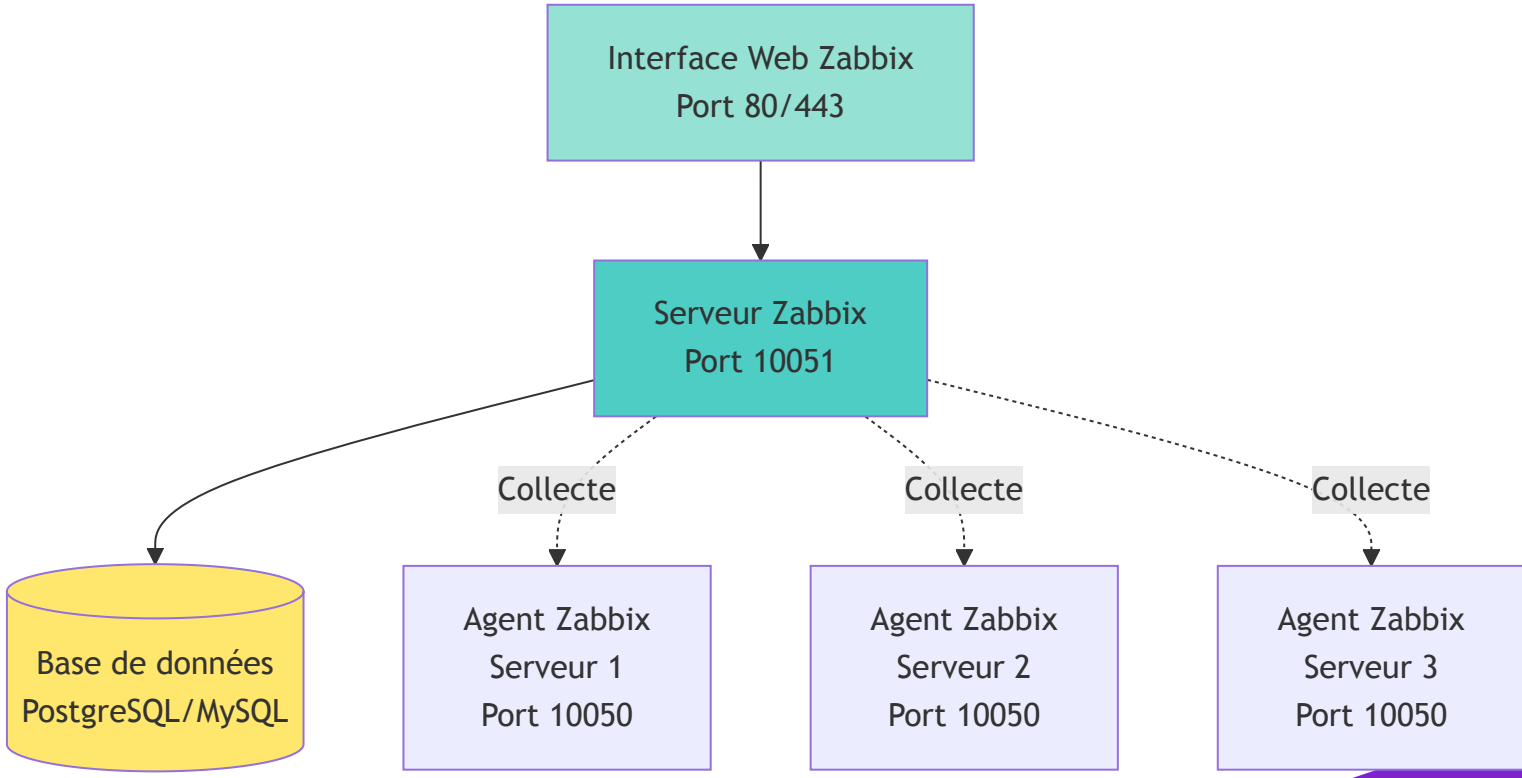
Fonctionnalités :

- Surveillance serveurs, réseaux, applications
- Historique et graphiques
- Alertes automatiques (email, SMS, Slack...)
- Templates prêts à l'emploi
- Interface web intuitive
- API REST pour automatisation
- Découverte automatique

Architecture : Client/Serveur

- **Serveur Zabbix** : collecte et stocke les données
- **Agent Zabbix** : installé sur chaque machine à surveiller

Architecture Zabbix



Revenir au sommaire

Prérequis installation 📋

Machine serveur Zabbix :

- 2 CPU minimum
- 2 Go RAM minimum (4 Go recommandé)
- 10 Go espace disque minimum
- Debian 12 / Ubuntu 22.04+ ou RHEL 9+

Composants à installer :

- Serveur Zabbix
- Interface web (Apache/Nginx + PHP)
- Base de données (PostgreSQL ou MySQL/MariaDB)

On va installer sur Ubuntu 24.04

Installation Zabbix Server 📥

```
# 1. Télécharger le dépôt Zabbix 7.0 LTS
wget https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/zabbix-release/zabbix-release_latest_7.0+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_latest_7.0+ubuntu24.04_all.deb
sudo apt update

# 2. Installer serveur, frontend, agent
sudo apt install zabbix-server-mysql zabbix-frontend-php zabbix-apache-conf zabbix-sql-scripts zabbix-agent -y

# 3. Installer MariaDB
sudo apt install mariadb-server -y
```

Configuration base de données 🗄️

```
# Se connecter à MySQL
sudo mysql

# Créer la base et l'utilisateur
CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE utf8mb4_bin;
CREATE USER 'zabbix'@'localhost' IDENTIFIED BY 'MotDePasseSecurise';
GRANT ALL PRIVILEGES ON zabbix.* TO 'zabbix'@'localhost';
SET GLOBAL log_bin_trust_function_creators = 1;
FLUSH PRIVILEGES;
EXIT;

# Importer le schéma initial
sudo zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz | mysql --default-character-set=utf8mb4 -uzabbix -p zabbix
# Entrer le mot de passe
```

Configuration Zabbix Server

```
# Éditer la config
sudo nano /etc/zabbix/zabbix_server.conf
```

Modifier ces lignes :

```
DBName=zabbix
DBUser=zabbix
DBPassword=MotDePasseSecurise
```

```
# Redémarrer MySQL pour désactiver log_bin_trust
sudo mysql -e "SET GLOBAL log_bin_trust_function_creators = 0;"

# Démarrer Zabbix
sudo systemctl restart zabbix-server zabbix-agent apache2
sudo systemctl enable zabbix-server zabbix-agent apache2
```

Configuration interface web 🌐

Accéder à l'interface :

```
http://ip-serveur/zabbix
```

Assistant d'installation :

1. **Welcome** : Next
2. **Pre-requisites** : vérifier que tout est OK → Next
3. **Database connection** :
   - Type : MySQL
   - Host : localhost
   - Database : zabbix
   - User : zabbix
   - Password : MotDePasseSecurise
4. **Settings** : donner un nom au serveur
5. **Summary** : Next
6. **Install** : Finish

Première connexion 🔑

Identifiants par défaut :

- **User :** `Admin`
- **Password :** `zabbix`

⚠️ IMPORTANT : Changer le mot de passe immédiatement !

```
Administration → Users → Admin → Change password
```

Interface : en anglais par défaut

Pour passer en français :

```
User settings (icône utilisateur) → Language : French
```

Interface Zabbix : tour rapide 🖥️

Menu principal :

- **Monitoring → Hosts** : liste des serveurs surveillés
- **Monitoring → Latest data** : dernières données collectées
- **Monitoring → Graphs** : graphiques
- **Monitoring → Dashboards** : tableaux de bord
- **Configuration → Hosts** : gérer les hôtes
- **Configuration → Templates** : templates pré-configurés
- **Reports → Availability** : disponibilité

Le serveur Zabbix se surveille lui-même 👀

Host "Zabbix server" déjà configuré

Voir les données :

1. Allez dans **Monitoring → Hosts**
2. Cliquez sur "Zabbix server"
3. Onglet **Latest data** : voir CPU, RAM, disque en temps réel
4. Onglet **Graphs** : graphiques des dernières heures

C'est votre premier host surveillé !

Installer l'agent Zabbix sur un serveur

Sur le serveur à surveiller (ex: serveur web) :

```
# Debian/Ubuntu
wget https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/zabbix-release/zabbix-release_latest_7.0+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_latest_7.0+ubuntu24.04_all.deb
sudo apt update
sudo apt install zabbix-agent -y

# Configuration
sudo nano /etc/zabbix/zabbix_agentd.conf
```

Configuration agent Zabbix

```
sudo nano /etc/zabbix/zabbix_agentd.conf
```

Modifier ces lignes :

```
Server=<IP_SERVEUR_ZABBIX>
ServerActive=<IP_SERVEUR_ZABBIX>
Hostname=serveur-web-01
```

Exemple :

```
Server=192.168.1.10
ServerActive=192.168.1.10
Hostname=serveur-web-01
```

```
# Redémarrer l'agent
sudo systemctl restart zabbix-agent
sudo systemctl enable zabbix-agent
```

Ajouter un host dans Zabbix 🖥️

Dans l'interface web Zabbix :

1. **Configuration → Hosts → Create host**
2. Remplir :
   - **Host name :** `serveur-web-01`
   - **Groups :** `Linux servers` (créer si besoin)
   - **Interfaces :** Add → Agent
     - IP address : `192.168.1.20` (IP du serveur)
     - Port : `10050`
3. **Templates :** Select → chercher `Linux by Zabbix agent`
4. **Add** (pour ajouter le template)
5. **Add** (en bas pour créer l'hôte)

Vérifier la connexion 🔍

Dans Configuration → Hosts :

- Colonne **Availability** :
  - **Vert (ZBX)** : agent accessible ✅
  - **Rouge** : problème de connexion ❌

Si rouge :

```
# Sur le serveur agent, vérifier :
sudo systemctl status zabbix-agent

# Vérifier le firewall
sudo ufw allow 10050/tcp

# Tester depuis le serveur Zabbix
telnet <IP_AGENT> 10050
```

Voir les données collectées 📊

Monitoring → Latest data

1. Filtrer par host : sélectionner `serveur-web-01`
2. Voir toutes les métriques collectées :
   - CPU utilization
   - Memory utilization
   - Disk space usage
   - Network traffic
   - Processes
   - ...

Cliquer sur "Graph" pour voir l'évolution dans le temps

Créer une alerte simple 🚨

Objectif : Recevoir un email si CPU > 80%

Étape 1 : Créer un trigger

1. **Configuration → Hosts →** `serveur-web-01`
2. **Triggers → Create trigger**
3. Remplir :
   - **Name :** `CPU élevé sur {HOST.NAME}`
   - **Severity :** High
   - **Expression :** Add → Sélectionner :
     - Item : `CPU utilization`
     - Function : `avg`
     - Last of (T) : `5m`
     - Result : `> 80`
4. **Add**

Créer une alerte : configuration email

Étape 2 : Configurer l'envoi d'emails

1. **Administration → Media types → Email**
2. **SMTP server :** `smtp.gmail.com`
3. **SMTP server port :** `587`
4. **SMTP helo :** `gmail.com`
5. **SMTP email :** `votre-email@gmail.com`
6. **Connection security :** STARTTLS
7. **Authentication :** Username and password
8. **Username :** `votre-email@gmail.com`
9. **Password :** (mot de passe d'application Gmail)
10. **Update**

Note : Pour Gmail, créer un mot de passe d'application

Créer une alerte : assigner l'email 📬

Étape 3 : Assigner l'email à l'utilisateur

1. **Administration → Users → Admin**
2. **Media → Add**
3. **Type :** Email
4. **Send to :** `votre-email@example.com`
5. **When active :** 1-7, 00:00-24:00 (toujours actif)
6. **Use if severity :** cocher toutes les cases
7. **Add → Update**

Créer une alerte : action 🎬

Étape 4 : Créer une action

1. **Configuration → Actions** (onglet Trigger actions) → **Create action**
2. **Action** (onglet) :
   - **Name :** `Alerte CPU élevé`
   - **Conditions :** Trigger severity >= High
3. **Operations** (onglet) → **Add** :
   - **Send to users :** Admin
   - **Send only to :** Email
   - **Add**
4. **Add** (en bas)

C'est prêt ! Dès que CPU > 80% pendant 5 min → email

Revenir au sommaire

Tester l'alerte 🧪

Simuler une charge CPU :

```
# Sur le serveur surveillé
# Lancer un stress test (installer si besoin)
sudo apt install stress -y

# Stresser le CPU pendant 5 minutes
stress --cpu 4 --timeout 300s
```

Observer dans Zabbix :

- **Monitoring → Problems** : voir le trigger activé
- **Boîte mail** : recevoir l'email d'alerte

Dashboards personnalisés 📈

Créer un dashboard :

1. **Monitoring → Dashboards → Create dashboard**
2. **Name :** `Mon serveur web`
3. **Add widget** :
   - **Type :** Graph
   - **Name :** `CPU usage`
   - **Host :** `serveur-web-01`
   - **Item :** `CPU utilization`
4. **Add → Apply**
5. Ajouter d'autres widgets (RAM, disque, etc.)

Résultat : Vue d'ensemble graphique de votre serveur

Templates : gagner du temps 📦

Template : ensemble pré-configuré d'items, triggers et graphiques

Templates fournis :

- `Linux by Zabbix agent` : métriques Linux de base
- `Apache by HTTP` : monitoring Apache
- `Nginx by HTTP` : monitoring Nginx
- `MySQL by Zabbix agent` : monitoring MySQL
- ...

Pour appliquer un template :

1. **Configuration → Hosts** → cliquer sur le host
2. **Templates → Select**
3. Chercher et sélectionner le template
4. **Add → Update**

Bonnes pratiques Zabbix ✅

1. Organisation

- Grouper les hosts par fonction (Web servers, DB servers, etc.)
- Nommer clairement (serveur-web-prod-01, bdd-mysql-01)

2. Alertes

- Ne pas alerter sur tout !
- Définir des seuils réalistes
- Tester les alertes

3. Maintenance

- Nettoyer l'historique ancien (> 90 jours)
- Surveiller l'espace disque de la BDD
- Mettre à jour Zabbix régulièrement

Récapitulatif du Module 5 ✅

Ce que vous avez appris :

- ✅ Architecture et concepts Zabbix
- ✅ Installation serveur et agent
- ✅ Configuration base de données
- ✅ Ajout de hosts à surveiller
- ✅ Création de triggers (alertes)
- ✅ Configuration notifications email
- ✅ Dashboards personnalisés
- ✅ Utilisation des templates

Prochaine étape 🎯

Module 13 (BONUS) : Optimisation & Performances

- Tuning kernel et système
- Optimisation des services
- Optimisation MySQL
- Cache système

Revenir au sommaire

Questions ? 🤔

N’hésitez pas à poser vos questions maintenant !

Vous disposez maintenant d’un outil professionnel de supervision.

QCM - Supervision Avancée avec Zabbix ✅

Module 5 : Validation des connaissances

Question 1

Quelle est l'architecture de Zabbix ?

A) Un agent autonome sur chaque serveur
B) Serveur centralisé + agents sur les machines surveillées
C) Uniquement une interface web
D) Un script cron qui collecte les données

Réponse : B

Zabbix utilise une architecture client/serveur : un serveur central collecte les données depuis des agents installés sur chaque machine.

Question 2 🔌

Sur quel port l'agent Zabbix écoute-t-il par défaut ?

A) 10050
B) 10051
C) 3306
D) 8080

Réponse : A

L'agent Zabbix écoute sur le port 10050. Le serveur Zabbix écoute sur 10051.

Question 3 🗄️

Quelle base de données peut être utilisée avec Zabbix ?

A) Seulement MySQL
B) Seulement PostgreSQL
C) MySQL/MariaDB ou PostgreSQL
D) SQLite uniquement

Réponse : C

Zabbix supporte MySQL/MariaDB et PostgreSQL. PostgreSQL est souvent recommandé pour les grandes installations.

Question 4 🔑

Quels sont les identifiants par défaut de Zabbix ?

A) admin / admin
B) Admin / zabbix
C) root / password
D) zabbix / zabbix

Réponse : B

User : `Admin` (avec majuscule) / Password : `zabbix`
⚠️ À changer immédiatement après installation !

Revenir au sommaire

Question 5 🚨

Qu’est-ce qu’un "trigger" dans Zabbix ?

A) Un bouton pour démarrer la surveillance
B) Une règle qui déclenche une alerte selon des conditions
C) Un type de graphique
D) Un service système

Réponse : B

Un trigger est une condition logique (ex: CPU > 80%) qui, lorsqu’elle est vraie, génère une alerte.

Question 6

Qu'est-ce qu'un template dans Zabbix ?

A) Un modèle d'email
B) Un thème graphique
C) Un ensemble pré-configuré d'items, triggers et graphiques
D) Un fichier de configuration

Réponse : C

Un template est un ensemble réutilisable de configuration (items, triggers, graphs) qu'on peut appliquer à plusieurs hosts.

Question 7

Dans le fichier zabbix_agentd.conf, à quoi sert le paramètre "Server" ?

A) Le nom du serveur à surveiller
B) L'IP du serveur Zabbix autorisé à se connecter
C) Le type de serveur (web, db, etc.)
D) Le port du serveur

Réponse : B

`Server=` indique l'adresse IP du serveur Zabbix autorisé à interroger cet agent (sécurité).

Question 8 🟢

Dans Zabbix, que signifie "ZBX" vert dans la colonne Availability ?

A) L'agent est en erreur
B) L'agent est accessible et répond
C) L'agent est en maintenance
D) L'agent est désactivé

Réponse : B

ZBX vert = l'agent Zabbix est accessible et répond correctement aux requêtes du serveur.

Question 9

Où voir les dernières données collectées pour un host ?

A) Configuration → Hosts
B) Monitoring → Latest data
C) Reports → Data
D) Administration → Queue

Réponse : B

Monitoring → Latest data affiche les dernières valeurs collectées pour tous les items des hosts.

Revenir au sommaire

Question 10

Pour recevoir des alertes par email, que faut-il configurer ?

A) Seulement le serveur SMTP
B) Seulement le trigger
C) Le media type Email + l'assigner à l'utilisateur + créer une action
D) Rien, c'est automatique

Réponse : C

Il faut : 1) Configurer le media type (Email avec SMTP), 2) L'assigner à l'utilisateur, 3) Créer une action pour envoyer l'alerte.

Score 🎯

Barème :

- 9-10 : Excellent ! ⭐⭐⭐
- 7-8 : Très bien ! ⭐⭐
- 5-6 : Bien, à consolider ⭐
- < 5 : Réviser le module

Prochaine étape : Exercices pratiques Zabbix

Fin du QCM ✅

Passez maintenant aux exercices pratiques !

Revenir au sommaire

Exercices - Supervision Avancée Zabbix 🎯

Module 5 : Mise en pratique

Revenir au sommaire

Exercice 1 : Installation Zabbix Server 🖥️

Objectif : Installer Zabbix serveur complet

Prérequis : Machine Ubuntu 24.04 (VM ou cloud)

Consignes :

1. Installer le dépôt Zabbix 7.0
2. Installer serveur + frontend + agent + Apache
3. Installer MariaDB
4. Créer la base de données zabbix
5. Importer le schéma initial
6. Configurer zabbix_server.conf
7. Démarrer tous les services
8. Accéder à l'interface web

Durée : 30 minutes

Exercice 1 : Solution 💡

```
# 1-2. Dépôt et installation
wget https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/zabbix-release/zabbix-release_latest_7.0+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_latest_7.0+ubuntu24.04_all.deb
sudo apt update
sudo apt install zabbix-server-mysql zabbix-frontend-php zabbix-apache-conf zabbix-sql-scripts zabbix-agent mariadb-server -y

# 3-5. Base de données
sudo mysql -e "CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE utf8mb4_bin;"
sudo mysql -e "CREATE USER 'zabbix'@'localhost' IDENTIFIED BY 'MonMotDePasse';"
sudo mysql -e "GRANT ALL PRIVILEGES ON zabbix.* TO 'zabbix'@'localhost';"
sudo mysql -e "SET GLOBAL log_bin_trust_function_creators = 1;"
sudo zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz | mysql --default-character-set=utf8mb4 -uzabbix -p zabbix

# 6. Configuration
sudo nano /etc/zabbix/zabbix_server.conf
# DBPassword=MonMotDePasse

# 7. Démarrage
sudo mysql -e "SET GLOBAL log_bin_trust_function_creators = 0;"
sudo systemctl restart zabbix-server zabbix-agent apache2
sudo systemctl enable zabbix-server zabbix-agent apache2

# 8. Interface
firefox http://localhost/zabbix
```

Exercice 2 : Configuration initiale 🔧

Objectif : Configurer l'interface web

Consignes :

1. Accédez à http://ip-serveur/zabbix
2. Suivez l'assistant d'installation
3. Connectez-vous avec Admin / zabbix
4. Changez le mot de passe Admin
5. Changez la langue en Français
6. Vérifiez que le host "Zabbix server" est actif (vert)

Durée : 10 minutes

Exercice 2 : Solution 💡

```
# 1. Navigateur
http://<IP_SERVEUR>/zabbix

# 2. Assistant
# - Welcome → Next
# - Prerequisites → Next
# - Database: MySQL, localhost, zabbix, zabbix, MotDePasse → Next
# - Server details → Next
# - Finish

# 3. Connexion
User: Admin
Password: zabbix

# 4. Changer mot de passe
Administration → Users → Admin → Change password
Nouveau mot de passe (2x) → Update

# 5. Langue
Icône utilisateur (coin haut droite) → Language: French → Update

# 6. Vérifier
Configuration → Hôtes
# Ligne "Zabbix server" → colonne Disponibilité = ZBX vert ✅
```

Exercice 3 : Ajouter un agent

Objectif : Installer et configurer un agent sur un second serveur

Prérequis : Un second serveur (VM ou cloud)

Consignes :

1. Sur le serveur à surveiller, installer zabbix-agent
2. Configurer zabbix_agentd.conf :
   - Server = IP du serveur Zabbix
   - Hostname = serveur-test
3. Redémarrer l'agent
4. Ouvrir le port 10050 dans le firewall
5. Vérifier que l'agent écoute

Durée : 15 minutes

Exercice 3 : Solution 💡

```
# Sur le serveur à surveiller

# 1. Installation
wget https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/zabbix-release/zabbix-release_latest_7.0+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_latest_7.0+ubuntu24.04_all.deb
sudo apt update
sudo apt install zabbix-agent -y

# 2. Configuration
sudo nano /etc/zabbix/zabbix_agentd.conf

# Modifier:
Server=192.168.1.10        # IP serveur Zabbix
ServerActive=192.168.1.10
Hostname=serveur-test

# 3. Redémarrage
sudo systemctl restart zabbix-agent
sudo systemctl enable zabbix-agent

# 4. Firewall
sudo ufw allow 10050/tcp

# 5. Vérification
sudo systemctl status zabbix-agent
sudo ss -tlnp | grep 10050
```

Exercice 4 : Ajouter le host dans Zabbix

Objectif : Déclarer le nouveau serveur dans Zabbix

Consignes :

1. Dans l'interface Zabbix : Configuration → Hôtes → Créer un hôte
2. Nom : `serveur-test`
3. Groupes : `Linux servers`
4. Interface : Agent, IP du serveur, port 10050
5. Templates : ajouter `Linux by Zabbix agent`
6. Ajouter l'hôte
7. Vérifier que ZBX devient vert
8. Consulter les données collectées

Durée : 10 minutes

Exercice 4 : Solution 💡

```
1. Configuration → Hôtes → Créer un hôte

2-3. Onglet Hôte:
   - Nom de l'hôte: serveur-test
   - Groupes: Linux servers

4. Interfaces → Ajouter:
   - Type: Agent
   - Adresse IP: 192.168.1.20 (IP du serveur test)
   - Port: 10050

5. Templates → Sélectionner:
   - Chercher "Linux by Zabbix agent"
   - Cliquer sur le nom
   - Ajouter

6. Ajouter (bouton en bas)

7. Attendre 30-60 secondes
   Configuration → Hôtes
   Colonne Disponibilité → ZBX doit être vert ✅

8. Supervision → Dernières données
   Filtrer par hôte: serveur-test
   Voir CPU, RAM, disque, etc.
```

Exercice 5 : Créer une alerte CPU 🚨

Objectif : Alerte si CPU > 80% pendant 5 minutes

Consignes :

1. Créer un trigger sur serveur-test :
   - Nom : `CPU élevé sur {HOST.NAME}`
   - Gravité : Haute
   - Expression : CPU avg(5m) > 80
2. L'alerte apparaîtra dans Supervision → Problèmes

Durée : 10 minutes

Exercice 5 : Solution 💡

```
1. Configuration → Hôtes → serveur-test
2. Déclencheurs → Créer un déclencheur

3. Remplir:
   Nom: CPU élevé sur {HOST.NAME}
   Gravité: Haute

4. Expression → Ajouter:
   - Item: CPU utilization
   - Function: avg
   - Last of (T): 5m
   - Result: > 80

5. Ajouter (l'expression)
6. Ajouter (le trigger)

Test:
# Sur serveur-test, stresser le CPU
sudo apt install stress -y
stress --cpu 4 --timeout 300s

# Dans Zabbix
Supervision → Problèmes
# L'alerte apparaît après 5 minutes
```

Exercice 6 : Configuration email

Objectif : Configurer l'envoi d'emails (simulation)

Note : Pour Gmail, créer un "mot de passe d'application"
(Google Account → Security → 2-Step Verification → App passwords)

Consignes :

1. Administration → Types de média → Email
2. Configurer SMTP Gmail :
   - Serveur : smtp.gmail.com:587
   - STARTTLS
   - Identifiants
3. Tester l'envoi
4. Assigner l'email à l'utilisateur Admin
5. Créer une action "Alerte CPU"

Durée : 20 minutes

Revenir au sommaire

Exercice 6 : Solution 💡

```
1. Administration → Types de média → Email

2. Configuration:
   Serveur SMTP: smtp.gmail.com
   Port: 587
   SMTP helo: gmail.com
   Courriel SMTP: votre-email@gmail.com
   Sécurité de connexion: STARTTLS
   Authentification: Nom d'utilisateur et mot de passe
   Nom d'utilisateur: votre-email@gmail.com
   Mot de passe: [mot de passe application]

3. Test → Envoyer à: votre-email@gmail.com → Test
   Vérifier réception

4. Administration → Utilisateurs → Admin → Média → Ajouter:
   Type: Email
   Envoyer à: votre-email@gmail.com
   Actif: 1-7, 00:00-24:00
   Gravité: toutes
   Ajouter → Mettre à jour

5. Configuration → Actions → Créer une action:
   Nom: Alerte CPU
   Conditions: Gravité >= Haute
   Operations: Envoyer à Admin via Email
   Ajouter
```

Exercice 7 : Dashboard personnalisé 📊

Objectif : Créer un tableau de bord

Consignes :

1. Supervision → Tableaux de bord → Créer un tableau de bord
2. Nom : `Mon serveur test`
3. Ajouter widget Graph : CPU
4. Ajouter widget Graph : RAM
5. Ajouter widget Graph : Disque
6. Enregistrer

Durée : 15 minutes

Exercice 7 : Solution 💡

```
1. Supervision → Tableaux de bord → Créer un tableau de bord

2. Nom: Mon serveur test

3. Ajouter un widget:
   Type: Graph
   Nom: CPU
   Hôte: serveur-test
   Item: CPU utilization
   Ajouter

4. Ajouter un widget:
   Type: Graph
   Nom: Mémoire
   Hôte: serveur-test
   Item: Memory utilization
   Ajouter

5. Ajouter un widget:
   Type: Graph
   Nom: Disque
   Hôte: serveur-test
   Item: Disk space usage /
   Ajouter

6. Appliquer
7. Réorganiser les widgets par drag & drop
8. Enregistrer

# Résultat: Vue d'ensemble graphique de votre serveur !
```

Projet fil rouge : Monitoring complet 🎯

Objectif : Mettre en place un monitoring complet de production

Scénario : Vous gérez 3 serveurs :

- 1 serveur web (Nginx)
- 1 serveur base de données (MySQL)
- 1 serveur applicatif

Tâches :

1. Installer les agents sur les 3 serveurs
2. Les ajouter dans Zabbix avec templates appropriés
3. Créer des triggers pour :
   - CPU > 80%
   - RAM > 90%
   - Disque > 85%
4. Configurer les alertes email
5. Créer un dashboard global

Durée : 1-2 heures

Checklist de validation ✅

Votre installation Zabbix est opérationnelle si :

- ☐ Serveur Zabbix installé et actif
- ☐ Interface web accessible
- ☐ Au moins 1 agent installé et vert (ZBX)
- ☐ Données collectées visibles (Latest data)
- ☐ 1 trigger créé et fonctionnel
- ☐ Configuration email testée
- ☐ 1 dashboard personnalisé créé

Si tous les points sont cochés : BRAVO ! 🎉

Récapitulatif des exercices ✅

Compétences acquises :

- ✅ Installation complète de Zabbix
- ✅ Configuration serveur et agents
- ✅ Ajout de hosts à surveiller
- ✅ Création de triggers d'alerte
- ✅ Configuration notifications email
- ✅ Création de dashboards
- ✅ Utilisation des templates

Vous savez maintenant déployer un monitoring professionnel !

Bravo ! 🎉

Vous maîtrisez maintenant Zabbix pour la supervision avancée.

Prochaine étape (BONUS) : Module 6 - Optimisation & Performances

Module 6 (BONUS)

Optimisation & Performances

Aller plus loin dans l'optimisation système

Revenir au sommaire

Quand optimiser ? 🤔

Ne pas optimiser prématurément !

Citation célèbre :

> "Premature optimization is the root of all evil" - Donald Knuth

Optimiser SEULEMENT si :

- Problème de performance identifié
- Goulet d'étranglement localisé
- Impact business réel
- Données de monitoring disponibles

Démarche :

1. **Mesurer** (monitoring)
2. **Identifier** le problème
3. **Optimiser** la zone critique
4. **Re-mesurer** l'impact

sysctl : tuning du noyau ⚙️

sysctl : modifier les paramètres du noyau Linux à chaud

```
# Voir tous les paramètres
sysctl -a

# Voir un paramètre spécifique
sysctl vm.swappiness

# Modifier temporairement
sudo sysctl vm.swappiness=10

# Appliquer depuis /etc/sysctl.conf
sudo sysctl -p
```

Modifications temporaires = perdues au redémarrage
Modifications permanentes = dans `/etc/sysctl.conf`

Optimisation mémoire : swappiness 💾

vm.swappiness : tendance à utiliser le swap (0-100)

- **100** : swap agressif (par défaut anciennes versions)
- **60** : valeur par défaut moderne
- **10** : swap minimal (serveurs avec beaucoup de RAM)
- **0** : pas de swap sauf urgence absolue

Pour un serveur avec 8 Go+ de RAM :

```
# Temporaire
sudo sysctl vm.swappiness=10

# Permanent
echo "vm.swappiness=10" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

Optimisation réseau : paramètres TCP

Pour un serveur web haute charge :

```
sudo nano /etc/sysctl.conf
```

```
# Augmenter la file d'attente de connexions
net.core.somaxconn = 4096

# Augmenter les backlog SYN
net.ipv4.tcp_max_syn_backlog = 8096

# Réduire le timeout FIN_WAIT
net.ipv4.tcp_fin_timeout = 30

# Activer la réutilisation des sockets TIME_WAIT
net.ipv4.tcp_tw_reuse = 1

# Augmenter les buffers réseau
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
```

```
sudo sysctl -p
```

Limites fichiers ouverts 📂

Problème courant : "Too many open files"

```
# Voir la limite actuelle
ulimit -n

# Augmenter temporairement
ulimit -n 65535
```

Permanent : `/etc/security/limits.conf`

```
* soft nofile 65535
* hard nofile 65535
```

Pour systemd services : dans le fichier .service

```
[Service]
LimitNOFILE=65535
```

Tuned : profiles d'optimisation automatiques 🎚️

tuned : daemon qui applique des profils d'optimisation (Red Hat)

```
# Installer
sudo dnf install tuned

# Démarrer
sudo systemctl enable --now tuned

# Voir les profils disponibles
tuned-adm list

# Profils courants:
# - throughput-performance : serveurs haute performance
# - latency-performance : faible latence
# - network-latency : optimisé réseau
# - virtual-guest : VM invitée
```

Utiliser tuned 🔧

```
# Voir le profil actif
tuned-adm active

# Appliquer un profil
sudo tuned-adm profile throughput-performance

# Recommandation automatique
sudo tuned-adm recommend

# Désactiver
sudo tuned-adm off
```

Avantage : optimisation automatique sans éditer sysctl manuellement !

Nginx : optimisations de base 🚀

Fichier : `/etc/nginx/nginx.conf`

```
# Processus = nombre de CPUs
worker_processes auto;

# Connexions par worker
events {
    worker_connections 1024;
    use epoll;
}

http {
    # Keepalive
    keepalive_timeout 65;
    keepalive_requests 100;

    # Compression
    gzip on;
    gzip_types text/plain text/css application/json application/javascript;
    gzip_min_length 1000;
}
```

Nginx : cache FastCGI (PHP) 💨

Accélérer PHP avec cache FastCGI :

```
# Dans http
fastcgi_cache_path /var/cache/nginx levels=1:2 keys_zone=APPCACHE:100m inactive=60m;
fastcgi_cache_key "$scheme$request_method$host$request_uri";

# Dans location ~ \.php$
fastcgi_cache APPCACHE;
fastcgi_cache_valid 200 60m;
fastcgi_cache_bypass $http_pragma $http_authorization;
add_header X-Cache-Status $upstream_cache_status;
```

Résultat : PHP exécuté 1x, puis servi depuis le cache pendant 60min

MySQL : optimisations essentielles 🗄️

Fichier : `/etc/mysql/my.cnf` **ou** `/etc/mysql/mysql.conf.d/mysqld.cnf`

```
[mysqld]
# Buffer pool = 70-80% de la RAM disponible
innodb_buffer_pool_size = 2G

# Logs
innodb_log_file_size = 256M
innodb_flush_log_at_trx_commit = 2

# Connexions
max_connections = 200

# Query cache (si MySQL < 8.0)
query_cache_size = 64M
query_cache_type = 1
```

```
sudo systemctl restart mysql
```

MySQL : identifier les requêtes lentes 🐌

Activer le slow query log :

```
sudo nano /etc/mysql/my.cnf
```

```
[mysqld]
slow_query_log = 1
slow_query_log_file = /var/log/mysql/slow.log
long_query_time = 2
log_queries_not_using_indexes = 1
```

```
sudo systemctl restart mysql

# Analyser les requêtes lentes
sudo mysqldumpslow /var/log/mysql/slow.log
```

MySQLTuner : script d'analyse

MySQLTuner : script qui analyse votre MySQL et recommande des optimisations

```
# Télécharger
wget https://raw.githubusercontent.com/major/MySQLTuner-perl/master/mysqltuner.pl
chmod +x mysqltuner.pl

# Exécuter
sudo ./mysqltuner.pl

# Le script affiche:
# - Statistiques actuelles
# - Recommandations d'optimisation
# - Paramètres à ajuster
```

Suivre les recommandations en ajustant my.cnf

Cache applicatif : Redis ⚡

Redis : cache en mémoire clé-valeur

```
# Installer
sudo apt install redis-server

# Configurer en mémoire uniquement
sudo nano /etc/redis/redis.conf
# maxmemory 256mb
# maxmemory-policy allkeys-lru

# Redémarrer
sudo systemctl restart redis-server
```

Utilisation : cache de sessions, résultats de requêtes DB, pages HTML...

Exemple en PHP : remplacer les sessions fichier par Redis

Checklist optimisation serveur web ✅

Pour un serveur web classique (Nginx + PHP + MySQL) :

☐ **Système**

- vm.swappiness = 10
- Limites fichiers ouverts = 65535
- Profile tuned : throughput-performance

☐ **Nginx**

- worker_processes auto
- gzip activé
- keepalive configuré
- Cache FastCGI pour PHP

☐ **MySQL**

- innodb_buffer_pool_size ajusté (70% RAM)
- Slow query log activé
- MySQLTuner exécuté

Récapitulatif du Module 6 (BONUS) ✅

Ce que vous avez appris :

- ✅ Tuning mémoire (swappiness)
- ✅ Optimisation réseau (TCP)
- ✅ Limites fichiers ouverts
- ✅ Profiles tuned automatiques
- ✅ Optimisation Nginx (workers, gzip, cache)
- ✅ Optimisation MySQL (buffer pool, slow queries)
- ✅ MySQLTuner pour l'analyse
- ✅ Cache Redis

Important : toujours mesurer avant/après !

Félicitations ! 🎉

Vous avez complété la formation **Administration Serveur Linux** !

Compétences acquises :

- Supervision système (top, htop, iotop, sar)
- Monitoring avancé (Zabbix)
- Optimisation performances (sysctl, tuned)
- Configuration NTP/DHCP

Prochaines étapes :

- Pratiquer sur des serveurs réels
- Explorer d'autres outils (Prometheus, Ansible)
- Se former aux conteneurs (Docker, Kubernetes)

QCM - Optimisation & Performances (BONUS) ✅

Module 6 : Validation des connaissances

Revenir au sommaire

Question 1 🎯

Quand faut-il optimiser un système ?

A) Dès l'installation
B) Tous les mois systématiquement
C) Seulement si un problème de performance est identifié
D) Jamais, le système s'optimise tout seul

Réponse : C

Ne pas optimiser prématurément. Optimiser seulement si un problème réel est identifié et mesuré.

Question 2 💾

Que signifie vm.swappiness=10 ?

A) Utiliser 10% de swap
B) Swap agressif
C) Swap minimal, préférer la RAM
D) Désactiver complètement le swap

Réponse : C

swappiness=10 minimise l'utilisation du swap, en privilégiant la RAM. Recommandé pour serveurs avec RAM suffisante.

Question 3 🔧

Comment appliquer les changements dans /etc/sysctl.conf ?

A) `sudo sysctl -a`
B) `sudo sysctl -p`
C) `sudo systemctl reload sysctl`
D) Redémarrer le serveur

Réponse : B

`sudo sysctl -p` relit et applique /etc/sysctl.conf sans redémarrage.

Question 4 🚀

Quelle valeur recommandée pour worker_processes dans Nginx ?

A) 1
B) 2
C) 4
D) auto

Réponse : D

`worker_processes auto` adapte automatiquement au nombre de CPUs disponibles.

Question 5 🗄️

Quelle valeur recommandée pour innodb_buffer_pool_size dans MySQL ?

A) 128 Mo
B) 10-20% de la RAM
C) 70-80% de la RAM disponible
D) 100% de la RAM

Réponse : C

70-80% de la RAM disponible pour MySQL est recommandé (laisser de la place pour l'OS).

Score 🎯

Barème :

- 5/5 : Excellent ! ⭐⭐⭐
- 4/5 : Très bien ! ⭐⭐
- 3/5 : Bien ⭐
- < 3 : Réviser le module

Revenir au sommaire

Fin du QCM BONUS ✅

Passez aux exercices pratiques pour expérimenter !

Revenir au sommaire

Exercices - Optimisation & Performances (BONUS)

Module 6 : Mise en pratique

Revenir au sommaire

Exercice 1 : Optimiser la mémoire 💾

Objectif : Configurer swappiness

Consignes :

1. Vérifier la valeur actuelle de swappiness
2. La passer à 10 temporairement
3. Vérifier le changement
4. Rendre la modification permanente dans /etc/sysctl.conf
5. Recharger la configuration

Durée : 10 minutes

Exercice 1 : Solution 💡

```
# 1. Valeur actuelle
sysctl vm.swappiness
# ou
cat /proc/sys/vm/swappiness

# 2. Modifier temporairement
sudo sysctl vm.swappiness=10

# 3. Vérifier
sysctl vm.swappiness

# 4. Permanent
echo "vm.swappiness=10" | sudo tee -a /etc/sysctl.conf

# 5. Recharger
sudo sysctl -p

# Vérification finale
sysctl vm.swappiness
```

Exercice 2 : Augmenter les limites fichiers 📂

Objectif : Passer la limite à 65535 fichiers ouverts

Consignes :

1. Vérifier la limite actuelle : `ulimit -n`
2. Éditer /etc/security/limits.conf
3. Ajouter les lignes pour * soft/hard nofile
4. Redémarrer la session (logout/login)
5. Vérifier la nouvelle limite

Durée : 10 minutes

Exercice 2 : Solution 💡

```
# 1. Limite actuelle
ulimit -n

# 2. Éditer limits.conf
sudo nano /etc/security/limits.conf

# 3. Ajouter à la fin:
* soft nofile 65535
* hard nofile 65535

# 4. Enregistrer et fermer

# 5. Logout puis login
exit
# Se reconnecter

# 6. Vérifier
ulimit -n
# Devrait afficher 65535
```

Exercice 3 : Optimiser Nginx 🚀

Objectif : Activer gzip et ajuster les workers

Consignes :

1. Éditer /etc/nginx/nginx.conf
2. Configurer worker_processes à auto
3. Activer gzip avec types appropriés
4. Tester la configuration
5. Recharger Nginx
6. Vérifier avec curl que gzip est actif

Durée : 15 minutes

Exercice 3 : Solution 💡

```
# 1. Éditer la config
sudo nano /etc/nginx/nginx.conf

# 2-3. Modifier:
user www-data;
worker_processes auto;

events {
    worker_connections 1024;
}

http {
    # Gzip
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml application/xml application/xml+rss text/javascript;
    gzip_min_length 1000;

    # ... reste de la config
}

# 4. Tester
sudo nginx -t

# 5. Recharger
sudo systemctl reload nginx

# 6. Vérifier gzip
curl -H "Accept-Encoding: gzip" -I http://localhost
# Chercher: Content-Encoding: gzip
```

Exercice 4 : Tuned (Red Hat/CentOS uniquement)

Objectif : Appliquer un profil d'optimisation

Prérequis : RHEL/CentOS/AlmaLinux/Rocky

Consignes :

1. Installer tuned
2. Lister les profils disponibles
3. Voir le profil actif
4. Appliquer le profil throughput-performance
5. Vérifier l'application

Durée : 10 minutes

Revenir au sommaire

Exercice 4 : Solution 💡

```
# 1. Installation
sudo dnf install tuned -y

# Démarrer
sudo systemctl enable --now tuned

# 2. Lister les profils
tuned-adm list

# 3. Profil actif
tuned-adm active

# 4. Appliquer le profil
sudo tuned-adm profile throughput-performance

# 5. Vérifier
tuned-adm active
# Devrait afficher: Current active profile: throughput-performance

# Voir les changements appliqués
tuned-adm profile_info
```

Exercice 5 : Optimiser MySQL avec MySQLTuner 🗄️

Objectif : Analyser et optimiser MySQL

Consignes :

1. Télécharger MySQLTuner
2. Rendre exécutable
3. Exécuter le script
4. Noter les recommandations principales
5. Appliquer 2-3 recommandations dans my.cnf
6. Redémarrer MySQL
7. Relancer MySQLTuner pour voir l'amélioration

Durée : 20 minutes

Exercice 5 : Solution 💡

```
# 1-2. Télécharger et rendre exécutable
wget https://raw.githubusercontent.com/major/MySQLTuner-perl/master/mysqltuner.pl
chmod +x mysqltuner.pl

# 3. Exécuter
sudo ./mysqltuner.pl

# Le script demande les identifiants MySQL root
# Puis affiche les recommandations

# 4. Exemple de recommandations:
# - Augmenter innodb_buffer_pool_size
# - Ajuster max_connections
# - Activer slow_query_log

# 5. Appliquer
sudo nano /etc/mysql/my.cnf

# Ajouter/modifier:
[mysqld]
innodb_buffer_pool_size = 1G
max_connections = 200
slow_query_log = 1
slow_query_log_file = /var/log/mysql/slow.log

# 6. Redémarrer
sudo systemctl restart mysql

# 7. Re-tester
sudo ./mysqltuner.pl
# Les warnings devraient avoir diminué
```

Projet BONUS : Benchmark avant/après ⚡

Objectif : Mesurer l'impact des optimisations

Outils : Apache Bench (ab)

Étapes :

1. Installer Apache Bench : `sudo apt install apache2-utils`
2. Tester Nginx AVANT optimisations :
```
ab -n 1000 -c 10 http://localhost/
```
3. Noter le "Requests per second"
4. Appliquer les optimisations Nginx (exercice 3)
5. Re-tester APRÈS :
```
ab -n 1000 -c 10 http://localhost/
```
6. Comparer les résultats

Objectif : Gain de +20% en requests/sec

Projet BONUS : Solution 💡

```
# 1. Installer Apache Bench
sudo apt install apache2-utils -y

# 2. Benchmark AVANT
ab -n 1000 -c 10 http://localhost/
# Noter: Requests per second: XXX [#/sec]

# 3. Optimiser Nginx (voir exercice 3)
# - worker_processes auto
# - gzip on
# - keepalive_timeout 65

# 4. Benchmark APRÈS
ab -n 1000 -c 10 http://localhost/
# Noter: Requests per second: YYY [#/sec]

# 5. Calculer le gain
# Gain % = ((YYY - XXX) / XXX) * 100

# Exemple:
# Avant: 500 req/sec
# Après: 650 req/sec
# Gain: +30%
```

Checklist optimisation complète ✅

Système optimisé si :

- ☐ vm.swappiness = 10
- ☐ Limites fichiers = 65535
- ☐ Paramètres réseau TCP ajustés (somaxconn, syn_backlog)
- ☐ Nginx : worker_processes auto + gzip
- ☐ MySQL : buffer_pool ajusté + slow_query activé
- ☐ MySQLTuner exécuté et recommandations appliquées
- ☐ Benchmarks avant/après documentés

Récapitulatif des exercices BONUS ✅

Compétences acquises :

- ✅ Configuration sysctl (swappiness, réseau)
- ✅ Limites fichiers ouverts
- ✅ Optimisation Nginx (workers, gzip)
- ✅ Profils tuned (Red Hat)
- ✅ Analyse MySQL avec MySQLTuner
- ✅ Benchmarking avec Apache Bench

Important : Toujours mesurer l'impact réel des optimisations !

Revenir au sommaire

Bravo ! 🎉

Vous connaissez maintenant les bases de l'optimisation système.

Pour aller plus loin :

- Profiling avancé (perf, flamegraphs)
- Cache distribué (Redis, Memcached, Varnish)
- Load balancing et haute disponibilité
- Monitoring APM (Application Performance Monitoring)

Module BONUS

Optimisation Réseau sous Linux

Comprendre et optimiser la pile réseau

Revenir au sommaire

"Optimiser le réseau" sur Linux 🤔

C'est quoi concrètement ?

C'est l'ensemble des techniques qui visent à :

- 🔻 Réduire la latence
- 🔺 Augmenter le débit
- 🧠 Réduire la charge CPU
- 🛡️ Améliorer la stabilité

"Optimiser le réseau" sur Linux 🤔

C'est quoi concrètement ? (suite)

C'est l'ensemble des techniques qui visent à :

- 📦 Éviter la perte de paquets
- ⚙️ Adapter le comportement du noyau au type d'usage

Revenir au sommaire

En résumé 💡

On ne "rend pas Internet plus rapide"

On rend la machine plus efficace pour envoyer, recevoir et traiter le trafic.

Les 6 grands piliers 🏛️

Optimisation réseau sous Linux

1. 🧠 Optimisation de la pile TCP/IP
2. 📡 Optimisation de la carte réseau (NIC)
3. ⚖️ File d'attente & congestion
4. 🔐 Pare-feu & filtrage
5. 🧱 Virtualisation & conteneurs
6. 📊 Supervision et diagnostic

Revenir au sommaire

1️⃣ Pile TCP/IP 🧠

Basé sur le protocole TCP

Objectif :

- Éviter la congestion
- Mieux gérer les files d'attente
- Accélérer les échanges

Outils :

- `sysctl`
- `/proc/sys/net/*`

1️ Pile TCP/IP - Exemples 🧠

Exemples de réglages :

- Taille des buffers d'envoi/réception
- Nombre de connexions simultanées
- Algorithme de congestion (BBR, CUBIC...)

```
# Voir les paramètres réseau actuels
sysctl -a | grep net.ipv4
```

1 Optimisation TCP - Configuration

```
# Éditer la configuration
sudo nano /etc/sysctl.conf
```

```
# Augmenter les buffers TCP
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
net.ipv4.tcp_rmem = 4096 87380 16777216
net.ipv4.tcp_wmem = 4096 65536 16777216

# Augmenter le backlog
net.core.somaxconn = 4096
net.ipv4.tcp_max_syn_backlog = 8192
```

1 Optimisation TCP - Configuration (suite)

```
# Réduire les timeouts
net.ipv4.tcp_fin_timeout = 30
net.ipv4.tcp_keepalive_time = 300
net.ipv4.tcp_keepalive_probes = 5
net.ipv4.tcp_keepalive_intvl = 15

# Réutiliser les sockets TIME_WAIT
net.ipv4.tcp_tw_reuse = 1

# Algorithme de congestion BBR (2025)
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
```

1 Appliquer les changements

```
# Appliquer immédiatement
sudo sysctl -p

# Vérifier l'algorithme actif
sysctl net.ipv4.tcp_congestion_control

# Voir les algorithmes disponibles
cat /proc/sys/net/ipv4/tcp_available_congestion_control
```

2️ Carte réseau (NIC) 📡

Matériel : Network Interface Card

Objectif :

- Réduire l’overhead CPU
- Améliorer la gestion des interruptions

Techniques :

- **Offloading** (TSO, GRO, LRO)
- **Multiqueue**
- **IRQ Affinity**

Outil principal :

- `ethtool`

2 Offloading avec ethtool

```
# Voir la configuration actuelle
sudo ethtool -k eth0

# Activer le TSO (TCP Segmentation Offload)
sudo ethtool -K eth0 tso on

# Activer le GRO (Generic Receive Offload)
sudo ethtool -K eth0 gro on

# Activer le GSO (Generic Segmentation Offload)
sudo ethtool -K eth0 gso on
```

2 Statistiques réseau

```
# Voir les statistiques de la carte
sudo ethtool -S eth0

# Voir la vitesse et le mode duplex
sudo ethtool eth0

# Régler la taille du ring buffer
sudo ethtool -G eth0 rx 4096 tx 4096
```

2 Multiqueue et IRQ Affinity

Multiqueue : distribuer le trafic sur plusieurs CPU

```
# Voir le nombre de queues
ls /sys/class/net/eth0/queues/

# Voir l'affectation des IRQ
cat /proc/interrupts | grep eth0

# Affecter une IRQ à un CPU spécifique
echo "2" > /proc/irq/125/smp_affinity_list
```

3 File d'attente & congestion ⚖️

Objectif :

- Éviter le **bufferbloat**
- Donner la priorité au trafic important

Algorithmes recommandés (2025) :

- `fq` (Fair Queue)
- `fq_codel` (Fair Queue Controlled Delay)
- `cake` (Common Applications Kept Enhanced)

Outil principal :

- `tc` (Traffic Control)

3️⃣ Qu'est-ce que le bufferbloat ? 🤔

Bufferbloat = latence excessive causée par des buffers trop grands

Symptômes :

- Latence élevée quand le réseau est chargé
- Jeux en ligne qui "lag" pendant un téléchargement
- Vidéoconférence saccadée

Solution : Utiliser un algorithme moderne comme `fq_codel` ou `cake`

3 Configurer fq_codel

```
# Voir la discipline actuelle
tc qdisc show dev eth0

# Installer fq_codel (moderne, par défaut sur kernel récent)
sudo tc qdisc replace dev eth0 root fq_codel
```

3 Configurer cake (plus avancé)

```
# Installer cake (peut nécessiter modules kernel)
sudo tc qdisc replace dev eth0 root cake bandwidth 100Mbit

# Avec priorité pour le trafic interactif
sudo tc qdisc replace dev eth0 root cake bandwidth 100Mbit besteffort

# Voir les statistiques cake
tc -s qdisc show dev eth0
```

4️⃣ Pare-feu & filtrage 🔐

Objectif : Sécurité sans tuer les performances

Technologies (2025) :

- `iptables` (classique)
- `nftables` (moderne, recommandé)

Revenir au sommaire

4 Optimisations pare-feu

Bonnes pratiques :

- ✅ Éviter les règles inutiles
- ✅ Ordre des règles optimal (plus fréquentes en premier)
- ✅ Bypass pour les flux internes (localhost)
- ✅ Utiliser les ensembles (ipset) pour les listes d’IP
- ✅ Activer connection tracking sélectivement

4 Exemple optimisé avec nftables

```
# Installer nftables
sudo apt install nftables

# Éditer la configuration
sudo nano /etc/nftables.conf
```

4 Configuration nftables optimisée

```
#!/usr/sbin/nft -f

flush ruleset

table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;

        # Accepter loopback sans filtrage
        iif lo accept

        # Connection tracking
        ct state established,related accept
        ct state invalid drop

        # SSH
        tcp dport 22 accept

        # HTTP/HTTPS
        tcp dport { 80, 443 } accept
    }
}
```

4 Appliquer nftables

```
# Activer nftables
sudo systemctl enable --now nftables

# Recharger la configuration
sudo nft -f /etc/nftables.conf

# Voir les règles actives
sudo nft list ruleset
```

5 Virtualisation & conteneurs

Avec Docker / Podman

Problèmes fréquents :

- NAT (Network Address Translation)
- Bridges virtuels
- Surcoût réseau

Optimisations possibles :

- Réseau `host`
- `macvlan`
- Bypass du bridge

Revenir au sommaire

5 Docker : mode réseau host

```
# Lancer un conteneur en mode host
docker run --network host nginx

# Plus de NAT, performances maximales
# ATTENTION : le conteneur partage le réseau de l'hôte
```

5 Docker : ajuster le MTU

```
# Éditer le daemon Docker
sudo nano /etc/docker/daemon.json
```

```
{
  "mtu": 1450,
  "default-address-pools": [
    {
      "base": "172.80.0.0/16",
      "size": 24
    }
  ]
}
```

5 Redémarrer Docker

```
# Redémarrer le daemon
sudo systemctl restart docker

# Vérifier le MTU
docker network inspect bridge | grep Mtu
```

5 Podman : réseau slirp4netns vs netavark

Podman 2025 utilise `netavark` par défaut (plus performant)

```
# Vérifier le backend réseau
podman info | grep -i network

# Mode host avec Podman
podman run --network host nginx

# Créer un réseau macvlan
podman network create -d macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  -o parent=eth0 macvlan-net
```

6 Supervision et diagnostic 📊

Principe fondamental :

On n’optimise jamais sans mesurer !

6 Outils essentiels

Commandes de base :

- `ip` : configuration réseau moderne
- `ss` : statistiques sockets (remplace netstat)
- `iftop` : monitoring bande passante en temps réel
- `iperf3` : tests de débit
- `tcpdump` : capture de paquets

6 Utiliser ip

```
# Afficher toutes les interfaces
ip addr show

# Afficher les statistiques
ip -s link show eth0

# Afficher la table de routage
ip route show

# Afficher les voisins (ARP)
ip neigh show
```

6 Utiliser ss

```
# Afficher toutes les connexions TCP
ss -tan

# Afficher les statistiques résumées
ss -s

# Connexions établies avec détails
ss -tiep

# Processus écoutant sur les ports
sudo ss -tlnp
```

6 Utiliser iftop

```
# Installer iftop
sudo apt install iftop

# Lancer iftop
sudo iftop -i eth0

# Options utiles:
# -P : afficher les ports
# -n : ne pas résoudre les noms
sudo iftop -i eth0 -P -n
```

6 Utiliser iperf3

```
# Installer iperf3
sudo apt install iperf3

# Serveur (machine A)
iperf3 -s

# Client (machine B) - test TCP
iperf3 -c 192.168.1.100

# Test UDP avec 1 Gbps
iperf3 -c 192.168.1.100 -u -b 1G

# Test parallèle (10 connexions)
iperf3 -c 192.168.1.100 -P 10
```

6 Utiliser tcpdump

```
# Capturer sur eth0
sudo tcpdump -i eth0

# Capturer et sauvegarder
sudo tcpdump -i eth0 -w capture.pcap

# Filtrer par port
sudo tcpdump -i eth0 port 80

# Filtrer par IP
sudo tcpdump -i eth0 host 192.168.1.100

# Voir le contenu des paquets (ASCII)
sudo tcpdump -i eth0 -A port 80
```

Cas pratique : Serveur web 🖥️

Optimisations typiques

Pour Nginx haute performance :

- TCP backlog augmenté
- KeepAlive optimisé
- Buffers TCP élargis
- Multi-core network queues

Cas pratique : Serveur web (config)

```
# Optimisations système
sudo nano /etc/sysctl.conf
```

```
# Backlog élevé
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 20480

# Buffers
net.core.rmem_max = 268435456
net.core.wmem_max = 268435456
net.ipv4.tcp_rmem = 4096 87380 134217728
net.ipv4.tcp_wmem = 4096 65536 134217728
```

Cas pratique : Serveur web (nginx)

```
# /etc/nginx/nginx.conf
user www-data;
worker_processes auto;
worker_rlimit_nofile 65535;

events {
    worker_connections 4096;
    use epoll;
    multi_accept on;
}

http {
    keepalive_timeout 65;
    keepalive_requests 1000;
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
}
```

Cas pratique : Conteneurs 📦

Problèmes courants

- Latence ajoutée par le bridge Docker
- NAT lent sur forte charge
- MTU inadapté

Cas pratique : Conteneurs (solutions)

Solutions :

1. **Réseau host** pour performances maximales
2. **Ajustement MTU** selon le réseau
3. **Désactivation iptables automatique** si non nécessaire
4. **Utiliser macvlan** pour intégration directe

```
# Désactiver iptables Docker
sudo nano /etc/docker/daemon.json
```

```
{
  "iptables": false,
  "ip-forward": false
}
```

Cas pratique : Jeux en ligne 🎮

Objectifs

- ⬇️ **Latence minimale**
- ⬇️ **Jitter réduit**

Actions recommandées :

- Utiliser `fq_codel` ou `cake`
- Priorité au trafic UDP
- Désactivation du bufferbloat

Revenir au sommaire

Cas pratique : Jeux (configuration)

```
# Installer cake avec priorité jeux
sudo tc qdisc replace dev eth0 root cake bandwidth 100Mbit diffserv3

# Priorité UDP (gaming)
sudo iptables -t mangle -A POSTROUTING -p udp -j DSCP --set-dscp-class EF
```

Cas pratique : Stockage réseau ☁️

NFS / Samba

Objectifs :

- Débit maximal
- TCP window large
- Jumbo Frames (si supporté)

Cas pratique : Stockage réseau (config)

```
# Activer Jumbo Frames (MTU 9000)
sudo ip link set eth0 mtu 9000

# Optimisations TCP pour gros transferts
sudo sysctl -w net.ipv4.tcp_window_scaling=1
sudo sysctl -w net.core.rmem_max=536870912
sudo sysctl -w net.core.wmem_max=536870912

# Configuration NFS optimisée
# /etc/exports
/data 192.168.1.0/24(rw,sync,no_subtree_check,no_root_squash)
```

Cas pratique : Stockage réseau (montage)

```
# Montage NFS optimisé
sudo mount -t nfs -o \
  rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,tcp \
  192.168.1.100:/data /mnt/nfs
```

Plan de cours complet 🎓

Chapitre 1 : Ce qu'on cherche à optimiser

- Débit
- Latence
- CPU
- Stabilité
- Sécurité

Revenir au sommaire

Plan de cours complet 🎓

Chapitre 2 : Architecture réseau Linux

- Pile TCP/IP
- Carte réseau
- Files d'attente
- Routage
- Pare-feu

Plan de cours complet 🎓

Chapitre 3 : Outils de base

- ip
- ss
- ethtool
- iperf3
- tc

Plan de cours complet 🎓

Chapitre 4 : Optimisation TCP avec sysctl

- Buffers
- Connexions
- Congestion

Revenir au sommaire

Plan de cours complet 🎓

Chapitre 5 : Optimisation carte réseau

- Offload
- Interruptions
- Multiqueue

Plan de cours complet 🎓

Chapitre 6 : Cas des conteneurs

- Bridge vs Host
- NAT vs direct
- MTU

Revenir au sommaire

Plan de cours complet 🎓

Chapitre 7 : Mesure & validation

- Tests avant / après
- Graphes
- Interprétation

Revenir au sommaire

Checklist d'optimisation complète ✅

☐ **Pile TCP/IP**

- Buffers élargis (rmem/wmem)
- Algorithme BBR activé
- Backlog augmenté
- Réutilisation TIME_WAIT

☐ **Carte réseau**

- Offloading activé (TSO, GRO)
- Ring buffers optimisés
- Multiqueue configuré

Checklist d'optimisation complète ✅

☐ **File d'attente**

- fq_codel ou cake installé
- Bufferbloat éliminé

☐ **Pare-feu**

- nftables ou iptables optimisé
- Règles ordonnées efficacement
- Connection tracking optimisé

Checklist d'optimisation complète ✅

☐ **Conteneurs**

- MTU ajusté
- Mode réseau adapté
- NAT optimisé ou bypassé

☐ **Supervision**

- iperf3 testé régulièrement
- ss et ip utilisés pour diagnostic
- tcpdump pour troubleshooting

Récapitulatif du module 🎯

Ce que vous avez appris :

- ✅ Les 6 piliers de l'optimisation réseau
- ✅ Tuning TCP/IP avec sysctl
- ✅ Optimisation carte réseau (ethtool)
- ✅ Gestion des files d'attente (tc, fq_codel, cake)
- ✅ Optimisation pare-feu (nftables)

Récapitulatif du module 🎯

Ce que vous avez appris (suite) :

- ✅ Optimisation conteneurs (Docker/Podman)
- ✅ Outils de diagnostic (ip, ss, iftop, iperf3, tcpdump)
- ✅ Cas pratiques (serveur web, jeux, stockage)

Points clés à retenir 💡

1. Toujours mesurer avant d'optimiser
2. Optimiser selon le cas d'usage
3. Tester après chaque modification
4. Documenter les changements
5. Ne pas optimiser prématurément

Ressources complémentaires 📚

Documentation officielle :

- `man sysctl`
- `man tc`
- `man ethtool`
- `man iperf3`

Ressources en ligne :

- kernel.org/doc/Documentation/networking/
- nftables.org
- netfilter.org

Félicitations ! 🎉

Vous maîtrisez maintenant l'optimisation réseau sous Linux !

Compétences acquises :

- Tuning avancé de la pile TCP/IP
- Optimisation matérielle
- Gestion des performances réseau
- Diagnostic et troubleshooting

Prochaines étapes :

- Pratiquer sur des serveurs réels
- Explorer eBPF pour l'optimisation avancée
- Se former à XDP (eXpress Data Path)

QCM - Optimisation Réseau Linux (BONUS) ✅

Module BONUS : Validation des connaissances

Question 1 🎯

Que signifie "optimiser le réseau" sous Linux ?

A) Rendre Internet plus rapide
B) Rendre la machine plus efficace pour traiter le trafic réseau
C) Augmenter la vitesse de la connexion Internet
D) Installer plus de cartes réseau

Réponse : B

Optimiser le réseau ne change pas la vitesse d'Internet, mais améliore la façon dont la machine traite le trafic.

Question 2

Quel est l'outil principal pour modifier les paramètres TCP/IP ?

A) ifconfig
B) netstat
C) sysctl
D) ping

Réponse : C

`sysctl` permet de modifier les paramètres du noyau Linux, notamment la pile TCP/IP.

Revenir au sommaire

Question 3 🌐

Quel algorithme de congestion TCP est recommandé en 2025 ?

A) Reno
B) Tahoe
C) BBR
D) Vegas

Réponse : C

BBR (Bottleneck Bandwidth and RTT) est l'algorithme moderne développé par Google, optimisé pour les réseaux actuels.

Question 4

Quel outil permet d'optimiser les paramètres de la carte réseau ?

A) sysctl
B) ethtool
C) iptables
D) netcat

Réponse : B

`ethtool` est l'outil dédié à la configuration et l'optimisation des cartes réseau (NIC).

Revenir au sommaire

Question 5 🐢

Qu’est-ce que le bufferbloat ?

A) Un virus réseau
B) Une latence excessive causée par des buffers trop grands
C) Un protocole de sécurité
D) Un type de pare-feu

Réponse : B

Le bufferbloat est une latence excessive causée par des buffers réseau surdimensionnés.

Question 6 ⚖️

Quel algorithme de file d'attente est moderne et recommandé ?

A) pfifo
B) fq_codel
C) bfifo
D) red

Réponse : B

`fq_codel` (Fair Queue Controlled Delay) est un algorithme moderne qui combat le bufferbloat.

Question 7 🔐

Quel est le système de pare-feu moderne recommandé en 2025 ?

A) ipchains
B) iptables (classique)
C) nftables
D) firewalld seulement

Réponse : C

`nftables` est le successeur moderne d'iptables, avec de meilleures performances et une syntaxe améliorée.

Question 8 🐳

Quel mode réseau Docker offre les meilleures performances ?

A) bridge (par défaut)
B) none
C) host
D) overlay

Réponse : C

Le mode `host` élimine le NAT et offre les meilleures performances (mais moins d'isolation).

Revenir au sommaire

Question 9

Quel outil permet de tester le débit réseau entre deux machines ?

A) ping
B) traceroute
C) iperf3
D) wget

Réponse : C

`iperf3` est l'outil standard pour tester les performances réseau (débit, latence).

Revenir au sommaire

Question 10 🔍

Quelle commande remplace netstat dans les systèmes modernes ?

A) ip
B) ss
C) ifconfig
D) route

Réponse : B

`ss` (socket statistics) remplace netstat et est plus rapide et moderne.

Question 11 🚀

Que signifie TSO dans le contexte de l'offloading réseau ?

A) TCP Socket Optimization
B) TCP Segmentation Offload
C) Total System Offload
D) Transport Security Offload

Réponse : B

TSO (TCP Segmentation Offload) décharge le CPU en faisant segmenter les paquets par la carte réseau.

Question 12 📈

Quelle valeur de net.core.somaxconn est recommandée pour un serveur web haute charge ?

A) 128 (défaut)
B) 512
C) 4096 ou plus
D) 50

Réponse : C

Pour un serveur web haute charge, somaxconn doit être augmenté à 4096 ou plus.

Revenir au sommaire

Question 13 🔌

Quel MTU est utilisé pour les Jumbo Frames ?

A) 1500 (standard)
B) 1450
C) 9000
D) 65535

Réponse : C

Les Jumbo Frames utilisent un MTU de 9000 octets (au lieu de 1500), utile pour le stockage réseau.

Question 14 🚦

Que permet la commande "tc" sous Linux ?

A) Gérer les tâches cron
B) Contrôler le trafic et les files d'attente réseau
C) Tester la connectivité
D) Créer des conteneurs

Réponse : B

`tc` (Traffic Control) gère les files d'attente réseau et le contrôle du trafic.

Question 15 📏

Quelle est la première étape avant d'optimiser un système ?

A) Appliquer toutes les optimisations disponibles
B) Mesurer les performances actuelles
C) Redémarrer le serveur
D) Installer de nouveaux outils

Réponse : B

Toujours mesurer avant d'optimiser ! On ne peut améliorer que ce qu'on mesure.

Question 16 ⏱️

Quel paramètre sysctl permet de réutiliser les sockets TIME_WAIT ?

A) net.ipv4.tcp_reuse
B) net.ipv4.tcp_tw_reuse
C) net.ipv4.tcp_recycle
D) net.ipv4.tcp_socket_reuse

Réponse : B

`net.ipv4.tcp_tw_reuse` permet de réutiliser les sockets en état TIME_WAIT.

Question 17

Quel backend réseau utilise Podman par défaut en 2025 ?

A) CNI
B) slirp4netns
C) netavark
D) bridge

Réponse : C

Podman utilise désormais `netavark` par défaut, plus performant que slirp4netns.

Question 18 💾

Pour un serveur de stockage NFS, quel paramètre est important ?

A) Latence minimale uniquement
B) TCP window large pour débit maximal
C) Compression des données
D) Chiffrement systématique

Réponse : B

Pour NFS, un TCP window large maximise le débit des gros transferts de fichiers.

Question 19

Quel outil permet de visualiser la bande passante en temps réel ?

A) ping
B) iftop
C) top
D) htop

Réponse : B

`iftop` affiche la bande passante en temps réel par connexion.

Revenir au sommaire

Question 20 🎯

Quelle est la bonne approche pour l'optimisation réseau ?

A) Appliquer tous les réglages trouvés sur Internet
B) Mesurer, optimiser, re-mesurer
C) Optimiser sans mesurer, ça marche toujours
D) Ne jamais toucher aux paramètres par défaut

Réponse : B

La démarche scientifique : mesurer avant, optimiser, mesurer après pour valider l'amélioration.

Score 🎯

Barème :

- 18-20/20 : Expert ! ⭐⭐⭐⭐
- 15-17/20 : Excellent ! ⭐⭐⭐
- 12-14/20 : Très bien ! ⭐⭐
- 10-11/20 : Bien ⭐
- < 10 : Réviser le module

Revenir au sommaire

Fin du QCM Optimisation Réseau ✅

Félicitations ! Vous maîtrisez maintenant l'optimisation réseau sous Linux !

Passez aux exercices pratiques pour expérimenter avec des serveurs réels.

Revenir au sommaire

Exercices - Optimisation Réseau Linux (BONUS) 🎯

Module BONUS : Mise en pratique

Revenir au sommaire

Exercice 1 : Optimiser TCP avec sysctl 🧠

Objectif : Configurer les paramètres TCP pour haute performance

Consignes :

1. Voir les paramètres TCP actuels
2. Éditer /etc/sysctl.conf
3. Augmenter les buffers TCP (rmem/wmem)
4. Augmenter somaxconn à 4096
5. Activer BBR si disponible
6. Appliquer les changements
7. Vérifier l'application

Durée : 15 minutes

Exercice 1 : Solution 💡

```
# 1. Voir paramètres actuels
sysctl -a | grep -E "net.core.rmem_max|net.ipv4.tcp_congestion"

# 2. Éditer
sudo nano /etc/sysctl.conf

# 3-5. Ajouter:
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
net.ipv4.tcp_rmem = 4096 87380 16777216
net.ipv4.tcp_wmem = 4096 65536 16777216
net.core.somaxconn = 4096
net.ipv4.tcp_max_syn_backlog = 8192
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr

# 6. Appliquer
sudo sysctl -p
```

Exercice 1 : Solution (suite) 💡

```
# 7. Vérifier
sysctl net.ipv4.tcp_congestion_control
sysctl net.core.somaxconn

# Vérifier les algorithmes disponibles
cat /proc/sys/net/ipv4/tcp_available_congestion_control
```

Exercice 2 : Optimiser la carte réseau

Objectif : Activer les optimisations matérielles avec ethtool

Consignes :

1. Identifier votre interface réseau (ip a)
2. Voir la configuration actuelle avec ethtool
3. Activer TSO (TCP Segmentation Offload)
4. Activer GRO (Generic Receive Offload)
5. Activer GSO (Generic Segmentation Offload)
6. Vérifier l'activation
7. Rendre permanent dans /etc/network/interfaces ou systemd

Durée : 15 minutes

Exercice 2 : Solution 💡

```
# 1. Identifier l'interface
ip addr show
# Exemple: eth0, ens33, enp0s3...

# 2. Voir config actuelle
sudo ethtool -k eth0 | grep -E "tcp-segmentation|generic-receive"

# 3-5. Activer les offloads
sudo ethtool -K eth0 tso on
sudo ethtool -K eth0 gro on
sudo ethtool -K eth0 gso on
```

Exercice 2 : Solution (suite) 💡

```
# 6. Vérifier
sudo ethtool -k eth0 | grep -E "tcp-segmentation|generic-receive|generic-segmentation"

# 7. Rendre permanent (systemd)
sudo nano /etc/systemd/network/10-eth0.link
```

Exercice 2 : Configuration systemd 💡

Contenu du fichier `/etc/systemd/network/10-eth0.link` :

```
[Match]
MACAddress=XX:XX:XX:XX:XX:XX

[Link]
TCPSegmentationOffload=yes
GenericReceiveOffload=yes
GenericSegmentationOffload=yes
```

Exercice 3 : Configurer fq_codel ⚖️

Objectif : Éliminer le bufferbloat avec fq_codel

Consignes :

1. Voir la discipline de file d'attente actuelle
2. Installer fq_codel sur l'interface principale
3. Vérifier l'installation
4. Tester la latence avec ping avant/après
5. Rendre permanent

Durée : 15 minutes

Exercice 3 : Solution 💡

```
# 1. Voir la discipline actuelle
tc qdisc show dev eth0

# 2. Installer fq_codel
sudo tc qdisc replace dev eth0 root fq_codel

# 3. Vérifier
tc qdisc show dev eth0
# Devrait afficher: qdisc fq_codel ...

# 4. Tester la latence (en charge)
ping -c 10 8.8.8.8
# Faire un téléchargement simultanément
# wget http://speedtest.tele2.net/100MB.zip
# Observer si la latence reste stable
```

Exercice 3 : Solution (suite) 💡

```
# 5. Rendre permanent
sudo nano /etc/rc.local
# Ajouter avant exit 0 :
# #!/bin/bash
# tc qdisc replace dev eth0 root fq_codel

# Rendre exécutable
sudo chmod +x /etc/rc.local
```

Exercice 4 : Configurer nftables 🔐

Objectif : Mettre en place un pare-feu performant

Consignes :

1. Installer nftables
2. Créer une configuration de base
3. Autoriser localhost sans filtrage
4. Autoriser SSH, HTTP, HTTPS
5. Drop par défaut le reste
6. Activer le service
7. Tester les connexions

Durée : 20 minutes

Exercice 4 : Solution 💡

```
# 1. Installer
sudo apt install nftables -y

# 2. Éditer la configuration
sudo nano /etc/nftables.conf
```

Exercice 4 : Solution (config) 💡

```
#!/usr/sbin/nft -f

flush ruleset

table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;

        # Loopback
        iif lo accept

        # Connection tracking
        ct state established,related accept
        ct state invalid drop

        # SSH
        tcp dport 22 accept

        # HTTP/HTTPS
        tcp dport { 80, 443 } accept

        # ICMP (ping)
        icmp type echo-request accept
    }

    chain forward {
        type filter hook forward priority 0; policy drop;
    }

    chain output {
        type filter hook output priority 0; policy accept;
    }
}
```

Exercice 4 : Solution (activation) 💡

```
# 6. Activer
sudo systemctl enable --now nftables

# Recharger la config
sudo nft -f /etc/nftables.conf

# Vérifier
sudo nft list ruleset

# 7. Tester
# SSH doit fonctionner
ssh user@localhost

# HTTP doit fonctionner
curl http://localhost

# Autres ports doivent être bloqués
telnet localhost 3306
# Connection refused ou timeout
```

Exercice 5 : Tester avec iperf3

Objectif : Mesurer le débit réseau avant/après optimisations

Consignes :

1. Installer iperf3 sur 2 machines (ou 2 VMs)
2. Mesurer le débit AVANT optimisations
3. Appliquer les optimisations TCP (exercice 1)
4. Appliquer les optimisations NIC (exercice 2)
5. Mesurer le débit APRÈS optimisations
6. Comparer les résultats
7. Documenter le gain

Durée : 20 minutes

Exercice 5 : Solution 💡

```
# 1. Installer sur les 2 machines
sudo apt install iperf3 -y

# 2. Machine A (serveur)
iperf3 -s

# Machine B (client) - Test AVANT
iperf3 -c 192.168.1.100 -t 30
# Noter: [ ID] Interval   Transfer  Bitrate

# 3-4. Appliquer les optimisations (ex 1 et 2)
# Sur les deux machines !

# 5. Machine A (serveur)
iperf3 -s
```

Exercice 5 : Solution (suite) 💡

```
# Machine B (client) - Test APRÈS
iperf3 -c 192.168.1.100 -t 30
# Noter: [ ID] Interval   Transfer  Bitrate

# Test avec plusieurs connexions parallèles
iperf3 -c 192.168.1.100 -P 10 -t 30

# 6-7. Comparer
# Exemple:
# AVANT: 800 Mbits/sec
# APRÈS: 940 Mbits/sec
# GAIN: +17.5%
```

Exercice 6 : Optimiser Docker réseau 🐳

Objectif : Améliorer les performances réseau des conteneurs

Consignes :

1. Lancer un conteneur Nginx en mode bridge (défaut)
2. Tester avec ab (Apache Bench)
3. Relancer en mode host
4. Tester à nouveau
5. Comparer les performances
6. Ajuster le MTU Docker si nécessaire

Durée : 20 minutes

Exercice 6 : Solution 💡

```
# 1. Mode bridge (défaut)
docker run -d -p 8080:80 --name nginx-bridge nginx

# 2. Tester
sudo apt install apache2-utils -y
ab -n 10000 -c 100 http://localhost:8080/
# Noter: Requests per second

# Arrêter
docker stop nginx-bridge && docker rm nginx-bridge

# 3. Mode host
docker run -d --network host --name nginx-host nginx
```

Exercice 6 : Solution (suite) 💡

```
# 4. Tester à nouveau
ab -n 10000 -c 100 http://localhost/
# Noter: Requests per second

# 5. Comparer
# Mode bridge: ~5000 req/sec (exemple)
# Mode host: ~8000 req/sec (exemple)
# Gain: +60%

# 6. Ajuster MTU
sudo nano /etc/docker/daemon.json
# Ajouter: { "mtu": 1450 }

# Redémarrer Docker
sudo systemctl restart docker
```

Exercice 7 : Monitoring réseau en temps réel

Objectif : Surveiller le trafic réseau

Consignes :

1. Installer iftop
2. Surveiller eth0 en temps réel
3. Installer et utiliser ss pour voir les connexions
4. Utiliser tcpdump pour capturer le trafic HTTP
5. Analyser une capture avec Wireshark (optionnel)

Durée : 15 minutes

Exercice 7 : Solution 💡

```
# 1-2. Installer et utiliser iftop
sudo apt install iftop -y
sudo iftop -i eth0 -P -n
# -P: afficher les ports
# -n: ne pas résoudre les noms
# Appuyer sur 'q' pour quitter

# 3. Utiliser ss
# Voir toutes les connexions TCP
ss -tan

# Voir les processus écoutant
sudo ss -tlnp

# Statistiques résumées
ss -s
```

Exercice 7 : Solution (suite) 💡

```
# 4. Capturer avec tcpdump
# Capturer le trafic HTTP pendant 60 secondes
sudo tcpdump -i eth0 port 80 -w capture.pcap -c 1000

# Voir le contenu (ASCII)
sudo tcpdump -i eth0 port 80 -A

# Filtrer par IP
sudo tcpdump -i eth0 host 192.168.1.100

# 5. Analyser avec Wireshark (sur poste local)
# Copier capture.pcap vers le poste local
# Ouvrir avec: wireshark capture.pcap
```

Projet BONUS : Serveur web haute performance 🚀

Objectif : Configurer un serveur Nginx optimisé pour haute charge

Étapes complètes :

1. Installer Nginx
2. Appliquer TOUTES les optimisations système (TCP, NIC, fq_codel)
3. Optimiser Nginx (workers, keepalive, buffers)
4. Configurer nftables
5. Benchmark avec ab : 100 000 requêtes, 500 connexions simultanées
6. Objectif : > 10 000 req/sec

Durée : 45 minutes

Projet BONUS : Solution (1/4) 💡

```
# 1. Installer Nginx
sudo apt update
sudo apt install nginx -y

# 2. Optimisations système
sudo nano /etc/sysctl.conf
# Ajouter les paramètres TCP ci-dessous
```

Projet BONUS : Configuration sysctl 💡

Contenu à ajouter dans `/etc/sysctl.conf` :

```
# TCP
net.core.rmem_max = 268435456
net.core.wmem_max = 268435456
net.ipv4.tcp_rmem = 4096 87380 134217728
net.ipv4.tcp_wmem = 4096 65536 134217728
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 20480
net.ipv4.tcp_fin_timeout = 30
net.ipv4.tcp_tw_reuse = 1
```

Projet BONUS : Solution (2/4) 💡

Ajouter aussi dans `/etc/sysctl.conf` :

```
# Congestion
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
```

Projet BONUS : Appliquer les optimisations 💡

```
# Appliquer sysctl
sudo sysctl -p

# Optimiser NIC
sudo ethtool -K eth0 tso on gro on gso on

# fq_codel
sudo tc qdisc replace dev eth0 root fq_codel
```

Projet BONUS : Solution (3/4) 💡

```
# 3. Optimiser Nginx
sudo nano /etc/nginx/nginx.conf
# Utiliser la configuration des slides suivantes
```

Projet BONUS : Configuration Nginx (partie 1) 💡

```
user www-data;
worker_processes auto;
worker_rlimit_nofile 100000;
pid /run/nginx.pid;

events {
    worker_connections 10000;
    use epoll;
    multi_accept on;
}

http {
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    keepalive_requests 10000;
    types_hash_max_size 2048;
```

Projet BONUS : Configuration Nginx (partie 2) 💡

```
    # Gzip
    gzip on;
    gzip_vary on;
    gzip_types text/plain text/css application/json application/javascript text/xml application/xml text/javascript;

    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    access_log off;
    error_log /var/log/nginx/error.log crit;

    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
}
```

Projet BONUS : Tester et appliquer 💡

```
# Tester la configuration
sudo nginx -t

# Recharger Nginx
sudo systemctl reload nginx
```

Projet BONUS : Benchmark final 💡

```
# 4. nftables (voir exercice 4)

# 5. Benchmark final
ab -n 100000 -c 500 http://localhost/

# Résultats attendus:
# Requests per second: > 10000 [#/sec]
# Time per request: < 50 [ms] (mean)
# Failed requests: 0

# Si échec, vérifier:
ulimit -n  # Doit être >= 65535
sudo systemctl status nginx
dmesg | tail  # Erreurs système
```

Checklist optimisation réseau complète ✅

Système optimisé si :

- ☐ **TCP/IP** : Buffers élargis, BBR activé, backlog augmenté
- ☐ **NIC** : TSO/GRO/GSO activés
- ☐ **Queue** : fq_codel ou cake installé
- ☐ **Pare-feu** : nftables optimisé
- ☐ **Conteneurs** : MTU ajusté, mode réseau optimal
- ☐ **Monitoring** : iperf3, ss, iftop maîtrisés
- ☐ **Benchmarks** : avant/après documentés

Récapitulatif des exercices ✅

Compétences acquises :

- ✅ Tuning TCP/IP avec sysctl
- ✅ Optimisation carte réseau (ethtool)
- ✅ Gestion des files d'attente (tc, fq_codel)
- ✅ Pare-feu moderne (nftables)
- ✅ Tests de performance (iperf3, ab)
- ✅ Optimisation conteneurs Docker
- ✅ Monitoring réseau (iftop, ss, tcpdump)

Récapitulatif des exercices ✅

Gains typiques observés :

- 🚀 +15-20% débit réseau (TCP optimisé)
- 🎯 -40% latence sous charge (fq_codel)
- 💪 +60% performance conteneurs (mode host)
- ⚡ +30% requêtes/sec serveur web

Important : Toujours mesurer avant/après !

Revenir au sommaire

Félicitations ! 🎉

Vous maîtrisez maintenant l'optimisation réseau avancée sous Linux !

Pour aller plus loin :

- eBPF et XDP (eXpress Data Path)
- DPDK (Data Plane Development Kit)
- Load balancing avancé (HAProxy, Traefik)
- Tuning kernel avancé (IRQ affinity, NUMA)
- Réseau haute performance 10G/40G/100G

Ressources :

- kernel.org/doc/Documentation/networking/
- brendangregg.com/blog/ (performance)
- netdevconf.org (conférences)

✅ QCM de validation finale

Évaluation des connaissances Unix/Linux

Testez vos compétences !

Revenir au sommaire

Instructions

Format du QCM :

- 50 questions
- 4 réponses possibles par question
- 1 seule bonne réponse
- Durée recommandée : 60 minutes

Notation :

- 80% et + : Excellent ⭐⭐⭐
- 60-79% : Bien ⭐⭐
- 40-59% : Moyen ⭐
- Moins de 40% : À revoir

Conseil : Lisez attentivement chaque question !

Question 1

Quelle commande permet de voir le répertoire courant ?

A) `ls`

B) `pwd`

C) `cd`

D) `whereis`

Revenir au sommaire

Question 2

Quelle permission est représentée par le chiffre 7 en octal ?

A) `r--`

B) `rw-`

C) `rwx`

D) `r-x`

Question 3

Quel utilisateur a l'UID 0 ?

A) `admin`

B) `root`

C) `system`

D) `sudo`

Question 4

Quelle commande permet de changer le propriétaire d'un fichier ?

A) `chmod`

B) `chown`

C) `chgrp`

D) `usermod`

Revenir au sommaire

Question 5

Quel signal envoie la commande `kill` par défaut ?

A) SIGKILL (9)

B) SIGTERM (15)

C) SIGHUP (1)

D) SIGSTOP (19)

Question 6

Quel fichier contient les mots de passe chiffrés ?

A) `/etc/passwd`

B) `/etc/shadow`

C) `/etc/group`

D) `/etc/security`

Question 7

Quelle est la commande pour voir les processus en temps réel ?

A) `ps`

B) `top`

C) `pgrep`

D) `kill`

Revenir au sommaire

Question 8

Quel est le PID du premier processus du système ?

A) 0

B) 1

C) 10

D) 100

Revenir au sommaire

Question 9

Quelle commande permet de monter un système de fichiers ?

A) `mount`

B) `umount`

C) `fsck`

D) `df`

Revenir au sommaire

Question 10

Où sont stockées les configurations de montage automatique au boot ?

A) `/etc/mount`

B) `/etc/fstab`

C) `/etc/filesystems`

D) `/boot/mount.conf`

Question 11

Quelle commande permet de voir l'utilisation du disque par répertoire ?

A) `df`

B) `du`

C) `fdisk`

D) `lsblk`

Question 12

Quelle adresse IP est réservée pour localhost ?

A) `192.168.1.1`

B) `10.0.0.1`

C) `127.0.0.1`

D) `0.0.0.0`

Question 13

Quel port utilise le protocole SSH par défaut ?

A) 21

B) 22

C) 23

D) 80

Question 14

Quelle commande permet de voir les connexions réseau actives ?

A) `ping`

B) `netstat` ou `ss`

C) `ifconfig`

D) `route`

Revenir au sommaire

Question 15

Quel gestionnaire de paquets est utilisé sur Debian/Ubuntu ?

A) `yum`

B) `dnf`

C) `apt`

D) `pacman`

Question 16

Quelle commande met à jour la liste des paquets disponibles ?

A) `apt upgrade`

B) `apt update`

C) `apt install`

D) `apt search`

Question 17

Quel fichier contient le shebang d'un script bash ?

A) La dernière ligne

B) La première ligne

C) N'importe où dans le fichier

D) Dans un fichier séparé

Question 18

Quelle est la syntaxe correcte pour une boucle for en bash ?

A) `for i in {1..5}; do echo $i; done`

B) `for (i=1; i<=5; i++) { echo $i }`

C) `foreach i in 1..5: echo $i`

D) `for i = 1 to 5 do echo $i end`

Question 19

Quelle commande permet de planifier des tâches récurrentes ?

A) `at`

B) `cron`

C) `systemd`

D) `timer`

Revenir au sommaire

Question 20

Dans un crontab, que signifie `0 2 * * *` ?

A) Toutes les 2 minutes

B) À 2h du matin tous les jours

C) Tous les 2 jours à minuit

D) Le 2 de chaque mois

Revenir au sommaire

Question 21

Où sont généralement stockés les logs système ?

A) `/log`

B) `/var/log`

C) `/etc/log`

D) `/home/log`

Revenir au sommaire

Question 22

Quelle commande permet de voir les logs systemd en temps réel ?

A) `tail -f /var/log/syslog`

B) `journalctl -f`

C) `dmesg -w`

D) `logwatch`

Question 23

Qu’est-ce que SELinux ?

A) Un gestionnaire de paquets

B) Un système de contrôle d’accès obligatoire (MAC)

C) Un éditeur de texte

D) Un protocole réseau

Question 24

Quelle commande permet de désactiver temporairement SELinux ?

A) `selinux disable`

B) `setenforce 0`

C) `systemctl stop selinux`

D) `chmod 000 /etc/selinux`

Question 25

Quel outil bloque les IP après plusieurs tentatives de connexion échouées ?

A) `iptables`

B) `fail2ban`

C) `ufw`

D) `firewalld`

Question 26

Quel est le protocole pour transférer des fichiers de manière sécurisée ?

A) FTP

B) HTTP

C) SCP ou SFTP

D) Telnet

Question 27

Où sont stockées les clés SSH par défaut ?

A) `/etc/ssh/`

B) `~/.ssh/`

C) `/var/ssh/`

D) `/home/ssh/`

Question 28

Quelle commande génère une paire de clés SSH ?

A) `ssh-key`

B) `ssh-gen`

C) `ssh-keygen`

D) `keygen`

Question 29

Quel serveur web est connu pour ses performances et sa faible consommation mémoire ?

A) Apache

B) Nginx

C) IIS

D) Tomcat

Question 30

Où sont stockés les fichiers de configuration de Nginx sur Debian/Ubuntu ?

A) `/etc/apache2/`

B) `/etc/nginx/`

C) `/var/www/nginx/`

D) `/usr/local/nginx/`

Question 31

Quelle commande teste la configuration de Nginx sans redémarrer ?

A) `nginx test`

B) `nginx -t`

C) `systemctl test nginx`

D) `nginx --check`

Question 32

Quel protocole permet d’obtenir des certificats SSL gratuits avec Let’s Encrypt ?

A) HTTP

B) FTP

C) ACME

D) SSL

Question 33

Quelle commande permet de sauvegarder une base MySQL ?

A) `mysql-backup`

B) `mysqldump`

C) `mysql-save`

D) `mysqlexport`

Question 34

Quelle est la différence entre `systemctl start` et `systemctl enable` ?

A) Aucune différence

B) `start` démarre maintenant, `enable` démarre au boot

C) `start` est permanent, `enable` est temporaire

D) `enable` démarre maintenant, `start` au boot

Question 35

Quel fichier contient la configuration principale de systemd ?

A) `/etc/systemd/system.conf`

B) `/etc/systemd/systemd.conf`

C) `/etc/init.d/systemd`

D) `/lib/systemd/systemd.conf`

Question 36

Quelle commande permet de voir les services qui ont échoué ?

A) `systemctl failed`

B) `systemctl --failed`

C) `systemctl list-failed`

D) `systemctl status --failed`

Question 37

Que signifie LVM ?

A) Linux Virtual Memory

B) Logical Volume Manager

C) Linux Volume Management

D) Local Virtual Machine

Question 38

Quelle commande permet de créer un lien symbolique ?

A) `link`

B) `ln -s`

C) `symlink`

D) `mklink`

Revenir au sommaire

Question 39

Quelle est la différence entre `>` et `>>` en redirection ?

A) Aucune différence

B) `>` écrase, `>>` ajoute à la fin

C) `>>` écrase, `>` ajoute à la fin

D) `>` est pour stdout, `>>` pour stderr

Revenir au sommaire

Question 40

Quelle variable d'environnement contient le répertoire personnel ?

A) `$USER`

B) `$PWD`

C) `$HOME`

D) `$PATH`

Question 41

Comment afficher les 10 premières lignes d'un fichier ?

A) `tail fichier`

B) `head fichier`

C) `cat fichier | head`

D) `top fichier`

Question 42

Quelle commande recherche un fichier par son nom ?

A) `search`

B) `find`

C) `locate`

D) B et C sont corrects

Revenir au sommaire

Question 43

Quelle commande permet de compresser un fichier avec gzip ?

A) `gzip fichier`

B) `compress fichier`

C) `zip fichier`

D) `tar fichier`

Question 44

Que fait la commande `chmod +x script.sh` ?

A) Rend le script lisible

B) Rend le script exécutable

C) Rend le script modifiable

D) Supprime le script

Revenir au sommaire

Question 45

Quelle commande permet de voir l'espace disque disponible ?

A) `du`

B) `df`

C) `disk`

D) `free`

Revenir au sommaire

Question 46

Quel est le répertoire temporaire standard ?

A) `/temp`

B) `/tmp`

C) `/var/tmp`

D) B et C sont corrects

Revenir au sommaire

Question 47

Quelle commande affiche les messages du noyau ?

A) `kernel`

B) `dmesg`

C) `syslog`

D) `messages`

Revenir au sommaire

Question 48

Quelle est la commande pour redémarrer le système ?

A) `restart`

B) `reboot`

C) `shutdown -r now`

D) B et C sont corrects

Revenir au sommaire

Question 49

Qu’est-ce qu’un daemon ?

A) Un virus

B) Un processus en arrière-plan

C) Un fichier de configuration

D) Un utilisateur système

Revenir au sommaire

Question 50

Quelle est la meilleure pratique pour travailler en tant qu'administrateur ?

A) Toujours se connecter en root

B) Utiliser sudo pour les tâches admin

C) Désactiver root

D) Partager le mot de passe root

Revenir au sommaire

RÉPONSES

Les réponses sont sur les slides suivantes

Ne trichez pas ! 😉

Revenir au sommaire

Réponses 1-5

Réponse 1 : B) `pwd`

- `pwd` : Print Working Directory
- `ls` : liste les fichiers
- `cd` : change de répertoire

Réponse 2 : C) `rwx`

- 7 = 4 (read) + 2 (write) + 1 (execute)
- 6 = rw-, 5 = r-x, 4 = r–

Réponse 3 : B) `root`

- L'utilisateur `root` a toujours l'UID 0
- C'est le super-utilisateur avec tous les privilèges

Réponse 4 : B) `chown`

- `chown` : change owner (propriétaire)
- `chmod` : change les permissions

Réponse 5 : B) SIGTERM (15)

- SIGTERM permet au processus de se terminer proprement
- L (9) termine immédiatement

Réponses 6-10

Réponse 6 : B) `/etc/shadow`

- `/etc/shadow` : mots de passe chiffrés (accessible uniquement par root)
- `/etc/passwd` : informations utilisateurs

Réponse 7 : B) `top`

- `top` : affichage en temps réel des processus
- `ps` : snapshot à un instant T

Réponse 8 : B) 1

- Le PID 1 est toujours `init` ou `systemd`
- C'est le processus parent de tous les autres

Réponse 9 : A) `mount`

- `mount` : monte un système de fichiers
- `umount` : démonte

Réponse 10 : B) `/etc/fstab`

- `/etc/fstab` : File System TABle
- Contient les montages automatiques au démarrage

Réponses 11-15

Réponse 11 : B) `du`

- `du` : Disk Usage (par fichier/répertoire)
- `df` : Disk Free (par partition)

Réponse 12 : C) `127.0.0.1`

- `127.0.0.1` est l'adresse de loopback (localhost)
- `::1` est l'équivalent IPv6

Réponse 13 : B) 22

- Port 22 : SSH (Secure Shell)
- Port 21 : FTP, Port 23 : Telnet

Réponse 14 : B) `netstat` **ou** `ss`

- `ss` : Socket Statistics (moderne)
- `netstat` : ancien mais toujours utilisé

Réponse 15 : C) `apt`

- `apt` : Advanced Package Tool (Debian/Ubuntu)
- `dnf` : RHEL/Fedora

Réponses 16-20

Réponse 16 : B) `apt update`

- `apt update` : met à jour la **liste** des paquets
- `apt upgrade` : installe les **mises à jour**

Réponse 17 : B) La première ligne

- Le shebang est `#!/bin/bash`
- Doit être sur la première ligne

Réponse 18 : A) `for i in {1..5}; do echo $i; done`

- Syntaxe Bash standard
- `do` et `done` encadrent le corps

Réponse 19 : B) `cron`

- `cron` : tâches récurrentes
- `at` : tâche ponctuelle unique

Réponse 20 : B) À 2h du matin tous les jours

- Format cron : `minute heure jour mois jour_semaine`
- `0 2 * * *` = minute 0, heure 2, tous les jours

Réponses 21-25

Réponse 21 : B) `/var/log`

- `/var/log` : répertoire standard des logs
- Fichiers : `syslog` , `auth.log` , `kern.log`

Réponse 22 : B) `journalctl -f`

- `journalctl -f` : follow (comme `tail -f`)
- Affiche les logs systemd en temps réel

Réponse 23 : B) Un système de contrôle d'accès obligatoire (MAC)

- SELinux = Security-Enhanced Linux
- Trois modes : enforcing, permissive, disabled

Réponse 24 : B) `setenforce 0`

- `setenforce 0` : passe en mode permissive
- `setenforce 1` : passe en mode enforcing

Réponse 25 : B) `fail2ban`

- `fail2ban` : bannit les IP après X échecs
- Protection contre le brute-force

MINISTÈRE DES ARMÉES
Liberté
Égalité
Fraternité

Revenir au sommaire

Réponses 26-30

Réponse 26 : C) SCP ou SFTP

- SCP : Secure Copy (basé sur SSH)
- SFTP : SSH File Transfer Protocol

Réponse 27 : B) `~/.ssh/`

- `~/.ssh/` : répertoire personnel de l'utilisateur
- `id_rsa` , `id_ed25519` : clés privées

Réponse 28 : C) `ssh-keygen`

- `ssh-keygen -t ed25519` : recommandé en 2025
- Génère une paire de clés

Réponse 29 : B) Nginx

- Nginx : architecture asynchrone, très performant
- Apache : plus ancien, multi-processus

Réponse 30 : B) `/etc/nginx/`

- `/etc/nginx/nginx.conf` : config principale
- `/etc/nginx/sites-available/` : sites disponibles

Réponses 31-35

Réponse 31 : B) `nginx -t`

- `nginx -t` : test de syntaxe
- Toujours tester avant de recharger

Réponse 32 : C) ACME

- ACME : Automatic Certificate Management Environment
- Utilisé par Certbot (Let's Encrypt)

Réponse 33 : B) `mysqldump`

- `mysqldump` : export d'une base en SQL
- `mysqldump -u root -p database > backup.sql`

Réponse 34 : B) `start` **démarre maintenant,** `enable` **démarre au boot**

- `start` : démarre immédiatement
- `enable` : active le démarrage au boot

Réponse 35 : A) `/etc/systemd/system.conf`

- Configuration globale de systemd
- Services dans `/etc/systemd/system/`

MINISTÈRE DES ARMÉES
Liberté
Égalité
Fraternité

Revenir au sommaire

Réponses 36-40

Réponse 36 : B) `systemctl --failed`

- Liste tous les services en état "failed"
- Utile pour le dépannage

Réponse 37 : B) Logical Volume Manager

- LVM : gestion flexible du stockage
- Structure : PV → VG → LV

Réponse 38 : B) `ln -s`

- `ln -s source destination` : lien symbolique
- `ln source destination` : lien dur

Réponse 39 : B) `>` **écrase,** `>>` **ajoute à la fin**

- `>` : redirection (écrase)
- `>>` : redirection en ajout (append)

Réponse 40 : C) `$HOME`

- `$HOME` : répertoire personnel (`/home/user`)
- `$USER` : nom de l'utilisateur

MINISTÈRE DES ARMÉES
Liberté Égalité Fraternité

Revenir au sommaire

Réponses 41-45

Réponse 41 : B) `head fichier`

- `head` : 10 premières lignes par défaut
- `tail` : 10 dernières lignes

Réponse 42 : D) B et C sont corrects

- `find` : recherche en temps réel
- `locate` : recherche dans une base de données

Réponse 43 : A) `gzip fichier`

- `gzip` : compression (crée `fichier.gz`)
- `gunzip` : décompression

Réponse 44 : B) Rend le script exécutable

- `+x` : ajoute le droit d'exécution
- Nécessaire pour lancer `./script.sh`

Réponse 45 : B) `df`

- `df` : Disk Free (par partition)
- `df -h` : format lisible

Réponses 46-50

Réponse 46 : D) B et C sont corrects

- `/tmp` : effacé au redémarrage
- `/var/tmp` : persistant entre redémarrages

Réponse 47 : B) `dmesg`

- `dmesg` : affiche le buffer du noyau
- Utile pour diagnostiquer les problèmes matériels

Réponse 48 : D) B et C sont corrects

- `reboot` : redémarre immédiatement
- `shutdown -r now` : idem

Réponse 49 : B) Un processus en arrière-plan

- Daemon : service en arrière-plan
- Exemples : `sshd` , `nginx` , `cron`

Réponse 50 : B) Utiliser sudo pour les tâches admin

- Principe du moindre privilège
- `sudo` offre la traçabilité

Correction du QCM 🎓

Comptez vos bonnes réponses :

- **40-50 points (80%+)** : Excellent ! ⭐⭐⭐ Vous maîtrisez Unix/Linux !
- **30-39 points (60-79%)** : Bien ! ⭐⭐ Bonnes bases, quelques révisions recommandées
- **20-29 points (40-59%)** : Moyen ⭐ Revoir les modules où vous avez des lacunes
- **< 20 points (< 40%)** : À revoir Reprenez la formation depuis le début

Points à réviser selon votre score 📚

Si vous avez raté les questions 1-10 :

- Revoir le Module 1 (Introduction)
- Commandes de base
- Système de fichiers

Si vous avez raté les questions 11-20 :

- Revoir les Modules 2-4
- Utilisateurs, permissions
- Processus, stockage

Si vous avez raté les questions 21-30 :

- Revoir les Modules 5-6
- Réseau, pare-feu
- Paquets, maintenance

Points à réviser (suite) 📚

Si vous avez raté les questions 31-40 :

- Revoir les Modules 7-8
- Scripts, automatisation
- Sécurité système

Si vous avez raté les questions 41-50 :

- Revoir le Module 9
- Services serveurs
- Bonnes pratiques

Félicitations ! 🎉

Vous avez complété la formation Unix/Linux - Initiation !

Compétences acquises :

- ✅ Utilisation de la ligne de commande
- ✅ Administration système
- ✅ Gestion réseau
- ✅ Scripting et automatisation
- ✅ Sécurité
- ✅ Services serveurs

Prochaines étapes :

- Pratiquer régulièrement
- Mettre en place un projet personnel
- Envisager une certification **(LPIC-1, RHCSA)**
- Continuer à apprendre !

Merci et bon courage ! 💪

La communauté Linux est là pour vous aider !

Continuez à apprendre, pratiquer et partager.

L'open source est une aventure collaborative.

Revenir au sommaire